

José Ramón Martínez Saavedra

Post-Quantum Cryptography

Mathematical Foundations for the Quantum Age

March 30, 2026

Springer Nature

Preface

This book grew out of lecture notes for a seminar on post-quantum cryptography that I teach within the Master of Quantum Science and Technology at the Università degli Studi di Bari Aldo Moro, Italy. The notes were originally meant to fit a compact course module, but they kept expanding. My students—most of them physicists by training—asked questions that existing references could not answer in their language. Why does adding noise to a linear system make it cryptographically hard? What is the geometric intuition behind lattice reduction? How does a worst-case to average-case reduction actually work, and why should a physicist care? The answers to those questions, accumulated over several iterations of the course, became the core of this book.

Why this book exists. The post-quantum cryptography literature sits uncomfortably between two communities that rarely read each other’s textbooks. Computer science treatments assume no quantum mechanics and develop cryptographic reductions in a formalism that is foreign to physicists. Quantum information texts, conversely, rarely engage with the practical machinery of public-key cryptography or the details of the NIST standardisation process. The result is a gap: a graduate student in quantum technologies who wants to understand ML-KEM or SLH-DSA at a mathematical level must assemble knowledge from half a dozen sources written for different audiences with different conventions. This book stands at the intersection. It develops the quantum-mechanical context that makes the threat concrete, the algebraic and geometric foundations that underpin the new algorithms, and the cryptographic formalism needed to reason about their security—all within a single, self-contained treatment.

Who this book is for. The primary audience is graduate students in quantum science and technology programmes who need to understand post-quantum cryptography as part of their professional formation. The book is also intended for physicists and engineers entering the cybersecurity field, computer science students who wish to engage seriously with the quantum aspects of cryptographic security, and practitioners in industry or government who are responsible for planning and executing post-quantum migration. Readers will benefit most if they bring curiosity about both the physics and the mathematics; the book aims to reward that curiosity with rigour rather than hand-waving.

Prerequisites. We assume a working knowledge of linear algebra (vector spaces, norms, inner products), basic abstract algebra (groups, rings, fields, modular arithmetic), discrete probability, and computational complexity at the level of asymptotic notation and the polynomial/exponential distinction. For the quantum-mechanical content in Part I, familiarity with Dirac notation, unitary evolution, and measurement is expected. No prior knowledge of cryptography is assumed; classical cryptographic systems are developed from scratch in Chapter 3.

How to read this book. The book is organised in three parts. Part I (Chapters 1–4, Foundations) should be read sequentially, as each chapter builds on the previous one. Part II (Chapters 5–11, Post-Quantum Cryptographic Primitives) begins with three core chapters: Chapter 5 surveys the post-quantum landscape, Chapter 6 develops lattice theory, and Chapter 7 builds lattice-based encryption culminating in ML-KEM. These three chapters are prerequisites for the rest of Part II. After them, Chapters 8–11—covering lattice-based signatures, code-based cryptography, hash-based signatures, and other approaches—can be read in any order according to the reader’s interest. Part III (Chapters 12–15, Practice and Future) addresses implementation, migration, advanced primitives, and open problems; it assumes familiarity with at least one cryptographic family from Part II. A reader focused on the NIST standards can proceed efficiently through Chapters 1–7, then Chapter 8, and then jump to Part III. A reader with a background in coding theory may prefer to continue from Chapter 5 directly to Chapter 9.

A note on tools used. AI-assisted tools were used during the preparation of this manuscript for structural organisation, $\text{\LaTeX}$ formatting, and code generation. The author reviewed and takes full responsibility for all technical content, mathematical proofs, and security claims.

Acknowledgments. I owe a debt to the students of the Master of Quantum Science and Technology at Bari, whose sharp questions and refusal to accept vague answers shaped every chapter of this book. I am grateful to my colleagues at Quside Technologies for many conversations on the practical side of quantum-safe security, and for the environment that made writing possible alongside the demands of industry.

Bari, Month 2026

José Ramón Martínez Saavedra

Contents

Notation and Conventions	xix
--------------------------------	-----

Part I Foundations

1 The Quantum Threat to Modern Cryptography	3
1.1 The Cryptographic Infrastructure of the Digital Age	3
1.1.1 Scale and Economic Exposure	3
1.1.2 The Asymmetric Bottleneck	4
1.1.3 What Exactly Breaks	4
1.2 Harvest Now, Decrypt Later	5
1.2.1 The Threat Model	5
1.2.2 Data Sensitivity Lifetimes	5
1.2.3 Evidence of Active Harvesting	6
1.2.4 Impact on Signatures vs. Encryption	7
1.3 From Shor’s Algorithm to NIST Standards: A Historical Arc	7
1.3.1 The Theoretical Foundation (1994–2005)	7
1.3.2 Building Momentum (2006–2016)	8
1.3.3 The NIST Standardisation Process (2017–2024)	9
1.3.4 Cautionary Tales	9
1.4 Scope and Organization of This Book	10
1.4.1 Prerequisites	10
1.4.2 Structure of the Book	10
1.4.3 Chapter Dependencies	11
1.4.4 What This Book Does Not Cover	12
Problems	12
2 Mathematical Preliminaries	15
2.1 Groups, Rings, and Fields	15
2.1.1 Groups	15
2.1.2 Rings	16
2.1.3 Fields	18
2.2 Modular Arithmetic and the Chinese Remainder Theorem	19

2.2.1	Modular Arithmetic	19
2.2.2	The Chinese Remainder Theorem	19
2.2.3	The Number Theoretic Transform	20
2.3	Probability and Statistical Distance	21
2.3.1	Discrete Probability	21
2.3.2	The Leftover Hash Lemma	22
2.3.3	Discrete Gaussian Distributions	23
2.4	Computational Complexity	24
2.4.1	Complexity Classes	25
2.4.2	Reductions	25
2.4.3	Average-Case vs. Worst-Case Complexity	26
2.4.4	Quantum Complexity and Cryptography	26
2.5	Security Definitions and Reductions	27
2.5.1	Security Games and Advantages	27
2.5.2	Security of Encryption: IND-CPA and IND-CCA2	27
2.5.3	Security of Signatures: EUF-CMA	29
2.5.4	Security Reductions	29
2.5.5	The Random Oracle Model and QROM	30
	Problems	31
3	Classical Cryptographic Systems	33
3.1	Symmetric Cryptography	33
3.1.1	Block Ciphers and AES	33
3.1.2	Authenticated Encryption: AES-GCM	35
3.1.3	Hash Functions: SHA-2 and SHA-3	35
3.1.4	Message Authentication Codes	36
3.2	Public-Key Cryptography	36
3.2.1	The Key Distribution Problem	36
3.2.2	One-Way Trapdoor Functions	37
3.2.3	RSA	37
3.2.4	Diffie–Hellman Key Exchange	38
3.2.5	Elliptic Curve Cryptography	38
3.3	Digital Signatures	39
3.3.1	Syntax and Security	40
3.3.2	RSA Signatures and RSA-PSS	40
3.3.3	Schnorr Signatures and the Fiat–Shamir Heuristic	40
3.3.4	ECDSA	41
3.4	Public Key Infrastructure	42
3.4.1	Certificate Authorities and Chains of Trust	42
3.4.2	X.509 Certificates	42
3.4.3	Failures and Lessons	43
3.5	The TLS Protocol	43
3.5.1	The TLS 1.3 Handshake	43
3.5.2	Cipher Suites	44
3.5.3	Where Each Primitive Is Used	44
3.6	The Computational Hardness Foundation	46

3.6.1	The Integer Factorisation Problem	46
3.6.2	The Discrete Logarithm Problem	47
3.6.3	The Elliptic Curve Discrete Logarithm Problem	47
3.6.4	The Monoculture Risk	48
	Problems	48
4	Quantum Algorithms and Cryptanalysis	51
4.1	The Quantum Computing Model	51
4.1.1	Qubits, Gates, and Circuits	51
4.1.2	The Quantum Fourier Transform	52
4.1.3	Computational Models	52
4.2	Shor's Algorithm	52
4.2.1	The Period-Finding Problem	53
4.2.2	Reduction of Factoring to Period Finding	53
4.2.3	The Quantum Circuit	54
4.2.4	Extension to the Discrete Logarithm Problem	55
4.3	Grover's Algorithm and Amplitude Amplification	56
4.3.1	The Unstructured Search Problem	56
4.3.2	The Grover Iteration	56
4.3.3	Optimality: The BBBV Lower Bound	57
4.3.4	Amplitude Amplification	58
4.3.5	Impact on Symmetric Cryptography	58
4.4	Quantum Resource Estimates	59
4.4.1	Logical Resource Requirements	59
4.4.2	Physical Overhead: Surface Codes	59
4.4.3	Concrete Estimates	59
4.4.4	Current Hardware vs. Requirements	60
4.5	What Quantum Computers Cannot Do	60
4.5.1	NP-Complete Problems	60
4.5.2	Lattice Problems	61
4.5.3	The Polynomial/Exponential Divide as Design Criterion	61
	Problems	62

Part II Post-Quantum Cryptographic Primitives

5	The Post-Quantum Landscape	67
5.1	Algorithm Families at a Glance	67
5.1.1	Lattice-Based Cryptography	67
5.1.2	Code-Based Cryptography	68
5.1.3	Hash-Based Signatures	68
5.1.4	Multivariate Cryptography	68
5.1.5	Isogeny-Based Cryptography	68
5.1.6	Comparative Overview	69
5.2	The NIST Standardization Process	69
5.2.1	Timeline	69
5.2.2	Evaluation Criteria	70

5.2.3	Why Multiple Winners?	70
5.3	PQC vs QKD: Complementary, Not Competing	71
5.3.1	Security Paradigms	72
5.3.2	Infrastructure and Scalability	72
5.3.3	The Signature Gap	72
5.3.4	When to Use Which	73
5.4	Hybrid Classical/Post-Quantum Approaches	73
5.4.1	Composite KEMs	74
5.4.2	Dual Signatures	75
5.4.3	X.509 Hybrid Certificates	75
5.5	Evaluating a PQC Candidate	76
5.5.1	Security	76
5.5.2	Performance	77
5.5.3	Implementation Quality	77
5.5.4	Maturity	78
5.5.5	Flexibility	78
	Problems	79
6	Lattice Theory	81
6.1	Definitions and Basic Properties	81
6.1.1	The Physics Connection	82
6.1.2	Determinant and Volume	82
6.1.3	Successive Minima	83
6.1.4	The Dual Lattice	83
6.1.5	Good Bases vs. Bad Bases	84
6.1.6	High-Dimensional Phenomena	85
6.2	Fundamental Computational Problems	86
6.2.1	The Shortest Vector Problem	86
6.2.2	The Closest Vector Problem	87
6.2.3	Approximation Variants	87
6.2.4	Additional Problems	88
6.3	Lattice Basis Reduction	88
6.3.1	Gram–Schmidt Orthogonalisation	89
6.3.2	The LLL Algorithm	89
6.3.3	BKZ: The Practical Workhorse	90
6.3.4	The Core-SVP Hardness Model	91
6.4	Worst-Case to Average-Case Reductions	92
6.4.1	Why Average-Case Hardness Matters	92
6.4.2	Ajtai’s Breakthrough	92
6.4.3	Proof Intuition	93
6.5	Computational Complexity of Lattice Problems	93
6.5.1	Classical Complexity	93
6.5.2	Quantum Complexity	94
6.5.3	Concrete Security Estimates	95
6.5.4	Summary: Why Lattices Survive Quantum Computers	96
	Problems	96

7	Learning With Errors and Lattice-Based Encryption	99
7.1	The LWE Problem	99
7.1.1	Parameters	100
7.1.2	The Error Distribution	101
7.2	Hardness of LWE	101
7.2.1	Regev’s Quantum Reduction	101
7.2.2	Classical Reductions	102
7.2.3	Concrete Hardness	102
7.3	LWE-Based Encryption	103
7.4	Ring-LWE: Efficiency Through Structure	103
7.4.1	The Polynomial Ring R_q	104
7.4.2	Ring-LWE Definition	105
7.4.3	The Number Theoretic Transform	105
7.4.4	Security of Structured Variants	106
7.5	Module-LWE: The Middle Ground	106
7.6	The Fujisaki–Okamoto Transform	107
7.6.1	Construction	107
7.7	ML-KEM (CRYSTALS-Kyber)	108
7.7.1	Construction	108
7.7.2	Parameter Sets	110
7.7.3	Performance	111
7.7.4	Decryption Failure Probability	111
	Problems	112
8	Lattice-Based Digital Signatures	113
8.1	The Challenge of Lattice-Based Signatures	113
8.1.1	Why “Encrypt in Reverse” Fails	113
8.1.2	The Information Leakage Problem	114
8.1.3	Two Solutions	114
8.2	The Fiat-Shamir Framework	114
8.2.1	Interactive Identification Protocols	115
8.2.2	The Fiat-Shamir Transform	115
8.2.3	Security: ROM and QROM	115
8.3	Fiat-Shamir with Aborts	116
8.3.1	The Protocol	116
8.3.2	Why Rejection Sampling is Necessary	116
8.3.3	The Rejection Sampling Lemma	117
8.3.4	Uniform Signature Distribution	117
8.4	ML-DSA (CRYSTALS-Dilithium)	117
8.4.1	Algebraic Setting	118
8.4.2	Key Generation	118
8.4.3	Signing	119
8.4.4	Verification	120
8.4.5	Parameter Sets	120
8.4.6	Hints and Compression	121
8.4.7	Security	121

8.5	NTRU Lattices and the GPV Framework	121
8.5.1	NTRU Lattices	122
8.5.2	The GPV Hash-and-Sign Framework	122
8.5.3	Preimage Sampling	123
8.5.4	Security of GPV Signatures	123
8.6	Falcon	124
8.6.1	Construction Overview	124
8.6.2	Key Generation	124
8.6.3	Signing: The Falcon Tree	125
8.6.4	Verification	126
8.6.5	Parameters and Security	126
8.6.6	The Constant-Time Challenge	127
8.7	ML-DSA vs Falcon: A Comparative Analysis	128
8.7.1	Size vs. Simplicity	128
8.7.2	Performance Characteristics	129
8.7.3	Deployment Considerations	129
8.7.4	The Broader Landscape	130
	Problems	130
9	Code-Based Cryptography	133
9.1	Error-Correcting Codes	133
9.1.1	Linear Codes	133
9.1.2	Minimum Distance and Error Correction	134
9.1.3	Syndrome Decoding	134
9.1.4	The Physics Connection	135
9.2	Goppa Codes	135
9.2.1	Definition and Parameters	136
9.2.2	Patterson's Decoding Algorithm	136
9.2.3	Why Goppa Codes Are Suitable for Cryptography	137
9.3	The Syndrome Decoding Problem	137
9.3.1	Formal Definition and Hardness	137
9.3.2	Information Set Decoding	138
9.3.3	Improved ISD Variants	139
9.3.4	Quantum Speedup for ISD	139
9.4	The McEliece Cryptosystem	140
9.4.1	Construction	140
9.4.2	Security Analysis	141
9.4.3	Classic McEliece: The NIST Candidate	141
9.4.4	The Key Size Problem	142
9.5	HQC: Hamming Quasi-Cyclic	142
9.5.1	Quasi-Cyclic Codes	143
9.5.2	HQC Construction	143
9.5.3	Parameters and NIST Standardisation	144
9.6	BIKE: Bit Flipping Key Encapsulation	145
9.6.1	QC-MDPC Codes	145
9.6.2	BIKE Construction	146

9.6.3	The Decoding Failure Challenge	146
9.7	Comparative Analysis	147
	Problems	148
10	Hash-Based Signatures	151
10.1	One-Time Signatures	151
10.1.1	The Lamport Construction	151
10.1.2	Winternitz OTS (WOTS+)	153
10.2	Merkle's Tree-Based Signatures	154
10.2.1	The Merkle Tree Construction	154
10.2.2	Authentication Paths	155
10.2.3	Key Generation Cost	156
10.3	Stateful Schemes: XMSS and LMS	156
10.3.1	XMSS	157
10.3.2	Multi-Tree XMSS (XMSS^{MT})	157
10.3.3	LMS	157
10.3.4	The State Management Challenge	158
10.4	FORS: Few-Time Signatures from Random Subsets	158
10.4.1	Construction	159
10.4.2	Few-Time Security	159
10.5	SLH-DSA (SPHINCS+)	160
10.5.1	The Hypertree Construction	160
10.5.2	Removing State: Randomised Hashing	161
10.5.3	Signing and Verification	161
10.5.4	Parameter Sets	161
10.6	Security Analysis	163
10.6.1	Minimal Assumptions	163
10.6.2	Grover's Impact	163
10.6.3	Multi-Target Attacks	164
10.6.4	The Quantum Random Oracle Model	164
	Problems	165
11	Multivariate, Isogeny-Based, and Other Approaches	167
11.1	Multivariate Polynomial Cryptography	167
11.1.1	The MQ Problem	167
11.1.2	The Oil-Vinegar Construction	168
11.1.3	A Pattern of Breaks	168
11.1.4	UOV: The Survivor	170
11.2	Isogeny-Based Cryptography	170
11.2.1	Elliptic Curves and Isogenies	170
11.2.2	Supersingular Isogeny Graphs	171
11.2.3	SIDH/SIKE: Construction and Collapse	172
11.3	Group Actions and CSIDH	173
11.3.1	Class Group Actions	174
11.3.2	Why CSIDH Survives the SIKE Attack	174
11.3.3	Challenges	175

11.4 Lessons Learned	175
11.4.1 What Makes an Assumption Trustworthy?	175
11.4.2 Structured vs. Unstructured Problems	176
11.4.3 The Danger of Auxiliary Information	177
11.4.4 Diversification and Crypto-Agility	177
Problems	178

Part III Practice and Future

12 Implementation and Side-Channel Resistance	183
12.1 The Constant-Time Imperative	183
12.1.1 Why Timing Variations Leak Secrets	183
12.1.2 Constant-Time Programming Patterns	184
12.1.3 Verification Tools	185
12.2 Side Channels in Lattice Cryptography	185
12.2.1 Timing Attacks on the NTT	186
12.2.2 Power and Electromagnetic Analysis	186
12.2.3 Cache-Timing Attacks	187
12.2.4 Masking Countermeasures for the NTT	187
12.3 Fault Injection Attacks	188
12.3.1 Fault Models	188
12.3.2 Attacks on ML-DSA (Dilithium)	189
12.3.3 Attacks on ML-KEM (Kyber)	189
12.3.4 Countermeasures	189
12.4 Random Number Generation	190
12.4.1 DRBG Requirements	190
12.4.2 Hardware and Quantum Random Number Generation	191
12.4.3 Deterministic Key Generation	191
12.5 Key and Ciphertext Sizes	192
12.5.1 Impact on TLS	192
12.5.2 Storage and Compression	193
12.6 Testing and Validation	193
12.6.1 Known-Answer Tests	193
12.6.2 FIPS 140-3 and the CMVP	194
12.6.3 Formal Verification	194
13 Hardware Security Analysis of Post-Quantum Cryptography	197
13.1 The Entropy-to-Cryptography Pipeline	197
13.2 Side-Channel Analysis from First Principles	199
13.2.1 The Physics of Leakage	199
13.2.2 What a Power Trace Looks Like	200
13.2.3 Simple Power Analysis	201
13.2.4 Differential and Correlation Power Analysis	201
13.2.5 Electromagnetic Emanation	203
13.2.6 Microarchitectural Side Channels	203
13.2.7 Threat Model Summary	204

13.3	Countermeasure Foundations	204
13.3.1	Boolean Masking	205
13.3.2	Domain-Oriented Masking	206
13.3.3	Arithmetic Masking	206
13.3.4	The A2B/B2A Conversion Problem	207
13.3.5	Shuffling	208
13.3.6	Fault Injection Countermeasures	208
13.3.7	Zeroisation	209
13.4	ML-KEM: Attack Surfaces and Hardening	210
13.4.1	Decapsulation: A Guided Tour of the Attack Surface	210
13.4.2	Masking Cost Summary	211
13.4.3	The Implicit Rejection Mechanism	212
13.5	ML-DSA: Attack Surfaces and Hardening	212
13.5.1	The Rejection Sampling Timing Channel	212
13.5.2	Deterministic vs. Hedged Mode	213
13.5.3	Non-Linear Rounding and Norm Checks	213
13.5.4	Fault Injection on SampleInBall	214
13.5.5	The 23-Bit Cost Scaling Problem	214
13.5.6	Hint Generation and Secret Leakage	214
13.5.7	Countermeasure Summary for ML-DSA	215
13.6	SLH-DSA: Attack Surfaces and Hardening	215
13.6.1	The Sheer Volume Problem	215
13.6.2	WOTS+ Chains as CPA Targets	216
13.6.3	Masking Order: DOM-1 vs. DOM-2	216
13.6.4	Fault Injection on Address and Leaf Index	216
13.6.5	Countermeasure Summary	217
13.7	Cross-Component Interactions	217
13.7.1	The DRBG as Systemic Vulnerability	217
13.7.2	The Reseed Integrity Problem	218
13.7.3	The Circular Dependency	218
13.7.4	Shared Keccak Core Contention	219
13.7.5	Frequent Reseeding as a Side-Channel Countermeasure	220
13.7.6	Comparative Hardening Cost	220
13.8	The Certification Landscape	221
13.8.1	FIPS 140-3	221
13.8.2	Common Criteria	221
13.8.3	BSI Guidelines	222
13.8.4	Certification Mapping	222
13.8.5	The Practical Path	222
13.8.6	What Certification Does Not Cover	223
	Problems	224

14 Migration Strategies	227
14.1 Hybrid Modes	227
14.1.1 Composite KEMs	227
14.1.2 Dual Signatures	228
14.1.3 X.509 Hybrid Certificates	229
14.1.4 Performance Impact	229
14.2 TLS Migration	230
14.2.1 TLS 1.3 with Post-Quantum Key Exchange	230
14.2.2 Chrome and Cloudflare Experiments	230
14.2.3 X25519Kyber768	231
14.2.4 Certificate Chain Sizes and Latency	231
14.3 Code Signing and Software Supply Chain	232
14.3.1 Firmware Updates	232
14.3.2 Package Managers and Repository Signing	232
14.3.3 The Signature Size Challenge	232
14.4 Blockchain and Cryptocurrencies	233
14.4.1 ECDSA Vulnerability	233
14.4.2 Migration Challenges	234
14.4.3 Frozen Funds Risk	234
14.5 Organizational Roadmap	235
14.5.1 Phase 1: Cryptographic Inventory	235
14.5.2 Phase 2: Risk Assessment	236
14.5.3 Phase 3: Prioritize	236
14.5.4 Phase 4: Test	237
14.5.5 Phase 5: Deploy	237
14.5.6 Phase 6: Verify and Maintain	238
14.5.7 CNSA 2.0 Timeline	238
14.5.8 European Guidance	239
14.6 Timeline and Urgency	239
14.6.1 Mosca’s Inequality	239
14.6.2 Sector-Specific Deadlines	240
14.6.3 Cost of Delay	241
Problems	241
15 Advanced Cryptographic Primitives	243
15.1 Fully Homomorphic Encryption	243
15.1.1 The Concept: Computing on Encrypted Data	243
15.1.2 Gentry’s Blueprint: Somewhat HE Plus Bootstrapping	244
15.1.3 LWE-Based FHE: BGV, BFV, CKKS, and TFHE	245
15.1.4 Noise Growth and the Depth Barrier	246
15.1.5 Bootstrapping: The Noise Reset	246
15.1.6 Practical FHE Today	247
15.2 Zero-Knowledge Proofs	247
15.2.1 The Concept: Proving Without Revealing	248
15.2.2 Interactive Proofs and the Fiat–Shamir Heuristic	249
15.2.3 Lattice-Based Zero-Knowledge: Stern-Type Protocols	249

15.2.4	Post-Quantum SNARKs and STARKs	250
15.2.5	Applications of Zero-Knowledge	250
15.3	Secure Multiparty Computation	251
15.3.1	The Millionaires' Problem	251
15.3.2	Shamir Secret Sharing	251
15.3.3	Garbled Circuits	252
15.3.4	MPC from Lattice-Based Oblivious Transfer	252
15.4	Lattice-Based Advanced Constructions	253
15.4.1	Identity-Based Encryption	253
15.4.2	Attribute-Based Encryption	254
15.4.3	Threshold Signatures and Distributed Key Generation	254
15.4.4	Functional Encryption	255
	Problems	256
16	Open Problems and Research Directions	259
16.1	Quantum Cryptanalysis	259
16.1.1	The Quantum Algorithm Toolkit	260
16.1.2	Quantum Speedups for Sieving and Enumeration	260
16.1.3	Quantum Walks on Lattice Graphs	261
16.1.4	The SVP Holy Grail	262
16.2	Physical Side-Channel Research	262
16.2.1	Electromagnetic Analysis of PQC	263
16.2.2	Deep Learning for Side-Channel Attacks	263
16.2.3	Physics-Informed Countermeasures	264
16.2.4	Quantum Side Channels	264
16.3	Hardware Optimization	265
16.3.1	FPGA and ASIC Implementations of NTT	265
16.3.2	Gaussian Sampling Hardware	265
16.3.3	Energy-Efficient PQC for IoT	266
16.4	Cryptographic Agility	266
16.4.1	Algorithm-Agnostic Design	267
16.4.2	Protocol Negotiation	267
16.4.3	Lessons from SHA-1 and MD5	268
16.5	Preparing for New Breaks	268
16.5.1	What if Module-LWE Breaks?	268
16.5.2	Defence in Depth	269
16.5.3	Monitoring Cryptanalysis	269
16.6	The 50-Year Horizon	270
16.6.1	The Quantum Internet	270
16.6.2	Post-Quantum Plus Quantum: The Full Stack	271
16.6.3	Information-Theoretic vs. Computational Security	271
16.7	Concrete Open Problems	272

A	Proof Supplements	277
A.1	Ajtai’s Worst-Case to Average-Case Reduction	277
A.2	Regev’s LWE Hardness Reduction	278
A.3	Correctness of the Fujisaki–Okamoto Transform	280
A.4	The Rejection Sampling Lemma	281
A.5	Security of Merkle’s Signature Scheme	283
B	Reference Implementations	285
B.1	LWE Encryption (Educational)	285
B.2	ML-KEM with liboqs	286
B.3	ML-DSA with liboqs	287
B.4	SLH-DSA with liboqs	288
B.5	Hybrid TLS with Post-Quantum KEM	289
B.6	Performance Benchmarks	290
	Solutions	293
	References	300
	Index	303

Notation and Conventions

Symbol	Meaning
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$	Natural, integer, rational, real, complex numbers
$\mathbb{F}_q$	Finite field with q elements
$\mathbb{Z}_q$	Integers modulo q
R_q	Polynomial ring $\mathbb{Z}_q[x]/(x^n + 1)$
a, b	Column vectors (bold lowercase)
A, B	Matrices (bold uppercase)
$\ \mathbf{v}\ $	Euclidean norm of $\mathbf{v}$
$\langle \mathbf{u}, \mathbf{v} \rangle$	Inner product of $\mathbf{u}$ and $\mathbf{v}$
$\mathbf{A}^\top$	Matrix transpose
$x \xleftarrow{\$} S$	x sampled uniformly at random from set S
$x \leftarrow D$	x sampled according to distribution D
$D_{\mathbb{Z}, \sigma}$	Discrete Gaussian over $\mathbb{Z}$ with parameter σ
$U(\mathbb{Z}_q^n)$	Uniform distribution over $\mathbb{Z}_q^n$
$\text{negl}(\lambda)$	Negligible function in security parameter λ
$\mathcal{L}(\mathbf{B})$	Lattice generated by basis $\mathbf{B}$
$\mathcal{L}^*$	Dual lattice
$\lambda_i(\mathcal{L})$	i -th successive minimum of $\mathcal{L}$
$\det(\mathcal{L})$	Determinant of lattice $\mathcal{L}$
λ	Security parameter
KeyGen, Enc, Dec	Key generation, encryption, decryption
Sign, Verify	Signature generation, verification
Encaps, Decaps	Key encapsulation, decapsulation
pk, sk	Public key, secret key
$H, \mathcal{H}$	Hash function, hash function family
$\mathcal{O}(f(n))$	Asymptotic upper bound
$\text{poly}(n)$	Some polynomial in n

Part I

Foundations

Chapter 1

The Quantum Threat to Modern Cryptography

Abstract The advent of large-scale quantum computers poses an existential threat to the cryptographic infrastructure underpinning modern digital communications. This chapter quantifies the scope of that infrastructure, introduces the “harvest now, decrypt later” attack model that makes the threat an immediate concern, traces the historical arc from Shor’s algorithm in 1994 to the NIST post-quantum standards finalised in 2024, and outlines the structure of the remainder of this book.

1.1 The Cryptographic Infrastructure of the Digital Age

Every digital interaction of economic or strategic value—from credit-card transactions to diplomatic cables—depends on a small number of public-key cryptographic primitives. The scale of this dependence is difficult to overstate. Before we can understand what a quantum computer threatens, we need to appreciate the sheer size of the system at risk.

1.1.1 Scale and Economic Exposure

The Transport Layer Security (TLS) protocol negotiates roughly 10^{10} new sessions per day worldwide; each session begins with a public-key handshake that establishes a shared secret. The certificate ecosystem backing those handshakes comprises over 5×10^9 active X.509 certificates, every one of which binds a public key to an identity via a digital signature. The annual volume of electronic commerce protected by these mechanisms exceeds USD 5×10^{12} .

Beneath this surface, essentially all inter-bank settlement, government communications, software-update authentication, and critical-infrastructure control traffic rely on the same mathematical foundations: the presumed hardness of integer factorisation (RSA [57]), the discrete logarithm problem in finite fields (Diffie–Hellman key exchange [15]), or the elliptic-curve discrete logarithm problem (ECDSA, ECDH).

1.1.2 The Asymmetric Bottleneck

The numbers above describe the infrastructure at risk. The next question is which part of it is vulnerable—and the answer is surprisingly narrow.

Symmetric-key algorithms such as AES are not directly threatened by Shor’s algorithm; Grover’s algorithm [25] provides at most a quadratic speedup, which is mitigated by doubling key lengths. The vulnerability concentrates in the *asymmetric* component of every protocol: key exchange, digital signatures, and public-key encryption. This asymmetric bottleneck means that a single algorithmic advance—Shor’s algorithm—compromises the entirety of the public-key infrastructure without touching the symmetric layer.

A useful analogy. Think of the global cryptographic infrastructure as a building. Symmetric ciphers are the walls: solid, well-understood, and quantum-resistant with modest reinforcement. Public-key algorithms are the load-bearing columns: few in number, but if any one fails the entire structure collapses. Shor’s algorithm is a threat to the columns, not the walls.

1.1.3 What Exactly Breaks

Having identified public-key cryptography as the single point of failure, we can be precise about the mathematics at stake. The vulnerability of current public-key cryptography rests on two number-theoretic problems.

The first is **integer factorisation**: given $N = pq$ for large primes p, q , find p and q . RSA encryption and RSA signatures rely on the hardness of this problem. The best known classical algorithm, the General Number Field Sieve, runs in sub-exponential time $e^{O(n^{1/3} \log^{2/3} n)}$ where $n = \log N$.

The second is the **discrete logarithm**: given g, h in a cyclic group G of order q , find x such that $g^x = h$. Diffie–Hellman key exchange, the Digital Signature Algorithm (DSA), and all elliptic-curve protocols depend on this problem.

Shor’s algorithm [58, 59] solves both problems in polynomial time on a quantum computer, specifically in $O(n^3)$ quantum gate operations for an n -bit input. The algorithm reduces factorisation and discrete logarithm computation to instances of the *hidden subgroup problem* over abelian groups, which quantum Fourier sampling solves efficiently.

Example 1.1 (Concrete Resource Estimates) Gidney and Ekerå [22] estimated that factoring a 2048-bit RSA modulus requires approximately 20×10^6 noisy physical qubits and roughly 8 hours of computation using surface-code error correction. As of 2025, the largest superconducting processors contain on the order of 10^3 qubits with error rates near 10^{-3} —roughly four orders of magnitude short in qubit count and nine

orders of magnitude short in error rate. The gap is large but finite; roadmaps from multiple vendors project 10^4 – 10^5 physical qubits by 2030.

1.2 Harvest Now, Decrypt Later

The resource estimates above might suggest comfort: a cryptographically relevant quantum computer (CRQC) is years away, perhaps decades. But an adversary need not wait for a quantum computer to begin exploiting its future existence. The “harvest now, decrypt later” (HNDL) attack model captures the essential timing mismatch, and it transforms the quantum threat from a distant concern into a present one.

1.2.1 The Threat Model

The core idea is disarmingly simple. Encrypted traffic traverses networks that adversaries can monitor. Storage is cheap. A rational adversary will record high-value ciphertexts today, archive them, and decrypt them the moment a CRQC becomes available. This logic leads to the following formalisation.

Definition 1.1 (HNDL Attack Model) An adversary with access to encrypted network traffic at time t_0 records ciphertexts at low marginal cost. At a future time $t_0 + \Delta$, a CRQC becomes available. The adversary retroactively decrypts all stored ciphertexts whose underlying keys were established via quantum-vulnerable key exchange.

The HNDL model implies a simple inequality: migration to post-quantum cryptography must be complete before

$$t_{\text{migrate}} \leq t_{\text{CRQC}} - t_{\text{sensitivity}}, \quad (1.1)$$

where $t_{\text{sensitivity}}$ is the required secrecy lifetime of the data and t_{CRQC} is the date a CRQC becomes operational. If data must remain confidential for 25 years and a CRQC arrives in 2035, migration should have been complete by 2010—a deadline already passed.

1.2.2 Data Sensitivity Lifetimes

The inequality (1.1) makes the urgency concrete, but only once we know how long different categories of data must remain secret. The variation across sectors is dramatic. Table 1.1 summarises representative sensitivity lifetimes; the rightmost column applies the HNDL inequality assuming a CRQC in 2035.

The table reveals that for the most sensitive categories—state secrets, medical records, long-lived commercial data—the migration deadline has already passed under even moderate assumptions about when a CRQC will arrive.

Table 1.1 Approximate data sensitivity lifetimes by sector

Sector / data type	Sensitivity	Deadline if CRQC in 2035
Classified intelligence	50+ years	Already past
Strategic military plans	25–50 years	Already past
Medical records	Patient lifetime (~50 yr)	Already past
Trade secrets, M&A data	10–20 years	2015–2025
Financial transactions	5–10 years	2025–2030
Consumer web browsing	1–3 years	2032–2034

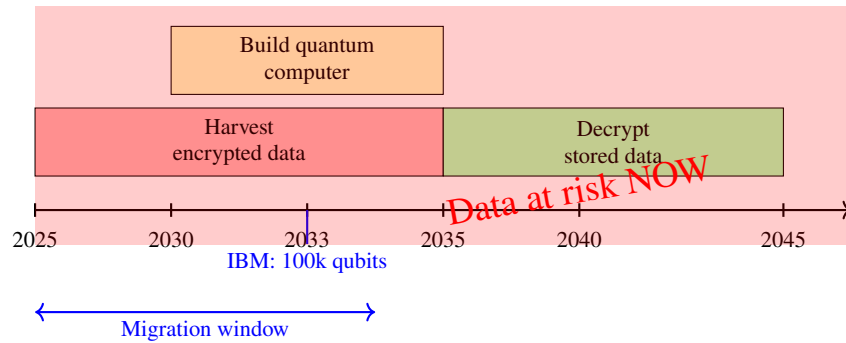
**Fig. 1.1** The harvest now, decrypt later timeline aligned with quantum-computing roadmaps. The overlap between the harvesting window and the migration window underscores the urgency of early action.

Figure 1.1 illustrates the HNDL attack timeline aligned with projected quantum-computing roadmaps, making the overlap between the harvesting window and the migration window visually explicit.

1.2.3 Evidence of Active Harvesting

The HNDL model is not hypothetical. Publicly documented programs confirm that state-level actors routinely intercept and store encrypted communications at scale. The economic case for harvesting is straightforward: storage costs roughly USD 10 per terabyte per year, while the intelligence value of decrypted diplomatic, military, or corporate communications is immense. Even a conservative estimate of daily international backbone traffic suggests that targeted collection of high-value encrypted flows is well within the budget of a major intelligence service.

The asymmetry of the HNDL threat. Protecting against HNDL requires acting *before* the quantum computer exists, on the basis of a probabilistic estimate of when one will arrive. If the estimate is too optimistic (migration happens too

late), the damage is irreversible—decrypted secrets cannot be re-encrypted. If the estimate is too pessimistic (migration happens early), the cost is merely the engineering effort of deploying new algorithms. The asymmetry strongly favours early action.

1.2.4 Impact on Signatures vs. Encryption

The HNDL threat does not affect all cryptographic primitives equally, and the distinction matters for migration planning.

Encryption and key exchange are vulnerable to HNDL in the most direct sense: ciphertexts recorded today can be decrypted retroactively the moment a CRQC exists. This makes key exchange the more urgent migration target. Digital signatures, by contrast, face a different timeline. A signature verified today does not need to resist future quantum attack *unless* the signature must remain valid for a long period—software supply-chain signatures, long-lived certificates, and legal documents all fall into this category. The threat to signatures is forward-looking: once a CRQC exists, new forgeries become possible, but previously verified signatures are not retroactively invalidated.

This distinction explains why early hybrid deployments—Google’s CECpq2 experiment in 2016, Cloudflare’s post-quantum key agreement in 2019—focused on key exchange before signatures. The engineering community, correctly reading the threat model, prioritised the primitive with the retroactive risk.

The following section traces how the research community arrived at the countermeasures now available to address these threats.

1.3 From Shor’s Algorithm to NIST Standards: A Historical Arc

The transition from theoretical vulnerability to deployed countermeasures spans three decades. Understanding this timeline clarifies both the pace of the field and the maturity of the solutions now available. The story divides naturally into three periods: a decade of theoretical groundwork, a decade of building research momentum, and an eight-year standardisation effort that culminated in the first federal standards.

1.3.1 The Theoretical Foundation (1994–2005)

The story begins in 1994, when Peter Shor presented his factoring and discrete-logarithm algorithm at FOCS [58]. The full journal version appeared in 1997 [59]. At that date,

quantum computers were purely theoretical—no one had demonstrated even a two-qubit gate with useful fidelity—and most of the cryptographic community treated Shor’s result as an elegant curiosity with no practical relevance.

Two years later, in 1996, the landscape shifted in two important ways. Lov Grover discovered his search algorithm [25], providing a quadratic speedup for unstructured search. Unlike Shor’s algorithm, Grover’s does not break any widely deployed cryptosystem; it merely halves the effective key length of symmetric ciphers. The same year, Ajtai proved the first worst-case to average-case reduction for lattice problems—a result whose significance for cryptography would only become clear a decade later, but which laid the theoretical foundation for lattice-based constructions.

The period from 1997 to 2005 saw scattered but prescient efforts to build public-key cryptography on alternative mathematical foundations. McEliece’s 1978 code-based cryptosystem, long regarded as impractical due to its large keys, received renewed attention. Lattice-based schemes appeared in several forms: NTRU in 1998, the Ajtai–Dwork encryption scheme, and various constructions building on Ajtai’s reduction. Hash-based signature schemes, tracing back to Lamport (1979) and Merkle (1979), were revisited with modern efficiency improvements. By the mid-2000s the term “post-quantum cryptography” had entered the literature, though the community working on it remained small and largely disconnected from mainstream cryptographic practice.

1.3.2 Building Momentum (2006–2016)

The first *PQCrypto* workshop, held in 2006, marked a turning point. For the first time, researchers working on code-based, lattice-based, hash-based, and multivariate cryptography gathered under a common banner, and the workshop established a dedicated, self-aware research community.

The most consequential theoretical advance of this period came from Regev, who in 2005 introduced the Learning With Errors (LWE) problem and proved a quantum worst-case to average-case reduction. This result provided the theoretical cornerstone for modern lattice-based encryption: it meant that breaking an LWE-based cryptosystem would require solving worst-case lattice problems, a far stronger security guarantee than ad hoc constructions could offer. In 2010, Lyubashevsky, Peikert, and Regev introduced Ring-LWE, enabling efficient lattice-based constructions with quasi-linear operations and bringing lattice cryptography into the range of practical deployment.

Momentum accelerated on the policy side as well. In 2015, the NSA’s Information Assurance Directorate announced that it would transition to quantum-resistant algorithms, signalling government-level urgency to an audience that had until then viewed the quantum threat as distant. The following year, NIST formally issued a call for post-quantum cryptographic algorithm submissions, initiating the most consequential cryptographic standardisation process since the AES competition. By the November 2017 deadline, 82 submissions had arrived—a testament to the depth of research that the preceding decade had produced.

Table 1.2 Timeline of the NIST PQC standardisation process

Date	Phase	Milestone
Dec 2016	Call	82 submissions received
Jan 2019	Round 2	26 candidates advance
Jul 2020	Round 3	15 candidates: 7 finalists + 8 alternates
Jul 2022	Selection	4 algorithms selected for standardisation
Aug 2024	Publication	FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA)

1.3.3 The NIST Standardisation Process (2017–2024)

With 82 candidate algorithms in hand, NIST began a multi-round public evaluation that would last eight years. Table 1.2 summarises the key milestones.

Each round involved extensive public cryptanalysis, performance benchmarking, and implementation review. The process was deliberately open: any researcher could submit attacks or analyses, and several candidates fell to newly discovered vulnerabilities between rounds. The four algorithms that emerged in July 2022 represent the survivors of this sustained scrutiny.

CRYSTALS-Kyber (now ML-KEM, FIPS 203 [50]) is a lattice-based key encapsulation mechanism built on Module-LWE, selected as the primary KEM standard. **CRYSTALS-Dilithium** (now ML-DSA, FIPS 204 [49]) is a lattice-based digital signature scheme built on Module-LWE and Module-SIS, selected as the primary signature standard. **SPHINCS+** (now SLH-DSA, FIPS 205 [51]) is a hash-based signature scheme offering conservative, well-understood security at the cost of larger signatures. **FALCON**, a lattice-based signature scheme based on NTRU lattices offering compact signatures but more complex implementation, is expected to be standardised as FIPS 206 in 2025.

Two features of this outcome deserve emphasis. First, three of the four selected algorithms are lattice-based, confirming the central role of lattice theory in post-quantum cryptography. Second, the process took eight years from call to publication—a timeline that underscores the difficulty of building confidence in new cryptographic assumptions.

1.3.4 Cautionary Tales

The standardisation process did not only produce winners; it also eliminated candidates whose underlying assumptions turned out to be weaker than believed. Two cases are particularly instructive.

SIKE, a finalist based on supersingular isogeny graphs, was broken in 2022 by a devastating classical attack. Castryck and Decru showed that private keys could be recovered in minutes on a laptop—not by a quantum computer, but by exploiting the algebraic structure that the scheme’s designers had believed to be intractable. **Rainbow**,

a multivariate signature scheme, fell the same year to a key-recovery attack by Beullens that exploited the structure of the central map.

These breaks are a healthy reminder that cryptographic confidence is built incrementally through sustained cryptanalysis, not through existence proofs alone. The lattice-based schemes have survived over 25 years of scrutiny; the isogeny-based and multivariate schemes had shorter track records. The lesson for the rest of this book is clear: mathematical elegance is necessary but not sufficient. Every construction we study must be evaluated not only for its provable security guarantees but also for the depth and duration of the cryptanalytic effort it has withstood.

1.4 Scope and Organization of This Book

The preceding sections have established the threat and sketched the countermeasures now available. The rest of this book develops those countermeasures in full mathematical detail. This section describes the scope, prerequisites, and structure of the chapters ahead.

1.4.1 Prerequisites

The book is written for graduate students in mathematics, physics, computer science, or electrical engineering. We assume familiarity with linear algebra (vector spaces, basis transformations, eigenvalues, norms, inner products), abstract algebra at an introductory level (groups, rings, fields, and modular arithmetic), basic discrete and continuous probability (expectation, variance, tail bounds), and algorithms (asymptotic notation O , Ω , Θ and the distinction between polynomial and exponential running time). Part I additionally requires comfort with quantum mechanics: Dirac notation, unitary evolution, measurement, and entanglement.

No prior knowledge of cryptography is assumed. Readers with a physics background will find lattice geometry and quantum algorithms particularly natural; readers with a computer science background will be more comfortable with reductions and complexity arguments. Both perspectives are needed, and the book aims to bridge them.

1.4.2 Structure of the Book

The book is organised in three parts that move from foundations through mathematical depth to practical deployment.

Part I: Foundations (Chapters 1–5) establishes the context and the classical and quantum tools needed for everything that follows. This chapter (Chapter 1) has motivated the transition to post-quantum cryptography. Chapter 2 reviews the mathematical prerequisites: number theory, algebra, and probability. Chapter 3 develops classical public-key

cryptography—RSA, Diffie–Hellman, elliptic curves—with enough detail to understand precisely what Shor’s algorithm breaks. Chapter 4 presents the quantum algorithms relevant to cryptography: Shor’s algorithm, Grover’s algorithm, and quantum random walks. Chapter 5 surveys the post-quantum landscape, comparing the main families of quantum-resistant schemes and the criteria by which they are evaluated.

Part II: Post-Quantum Cryptographic Families (Chapters 6–11) is the mathematical core of the book. Chapter 6 develops lattice theory from first principles: definitions, computational problems, basis reduction, and the worst-case to average-case reductions that underpin lattice-based security. Chapter 7 introduces the Learning With Errors problem and builds lattice-based encryption, culminating in ML-KEM (FIPS 203). Chapter 8 covers lattice-based signatures: the Fiat–Shamir with Aborts paradigm (ML-DSA) and hash-then-sign over NTRU lattices (FALCON). Chapter 9 treats code-based cryptography, from the McEliece cryptosystem to the BIKE and HQC candidates. Chapter 10 develops hash-based signatures from Lamport and Merkle trees through SPHINCS+ (SLH-DSA). Chapter 11 surveys multivariate, isogeny-based, and group-based approaches—including the schemes that did not survive the NIST process and why they failed.

Part III: Deployment and Open Problems (Chapters 12–16) addresses the gap between algorithmic specification and real-world deployment. Chapter 12 discusses implementation: constant-time code, side-channel resistance, and performance engineering. Chapter 14 covers the migration challenge: hybrid modes, crypto-agility, and transition planning. Chapter 15 introduces advanced primitives built on post-quantum assumptions—fully homomorphic encryption, attribute-based encryption, and zero-knowledge proofs. Chapter 16 identifies open problems and research directions at the frontier of the field.

1.4.3 Chapter Dependencies

Chapter dependencies. Chapters 2–4 are prerequisites for the rest of the book. Within Part II, Chapter 6 is required before Chapters 7 and 8. Chapters 9, 10, and 11 can be read independently. Part III assumes familiarity with at least one family from Part II.

A reader interested primarily in the NIST standards may proceed directly from Chapter 5 to Chapter 6 and then to Chapters 7–8, deferring or omitting the remaining families. A reader with a background in coding theory may prefer to start with Chapter 9 after completing Part I.

1.4.4 What This Book Does Not Cover

Three important topics lie outside our scope, and stating them upfront prevents false expectations. Quantum key distribution (QKD), an information-theoretically secure approach to key exchange that requires a quantum channel, is complementary to PQC rather than a competitor; we occasionally contrast the two approaches but do not develop QKD in detail. Quantum error correction, while essential for building CRQCs, is not needed to understand the cryptographic countermeasures. Post-quantum TLS and protocol engineering are discussed at the level of principles in Chapter 14, but we do not provide implementation-level protocol specifications—that territory belongs to IETF drafts and engineering manuals, not a mathematics textbook.

Problems

1.1 A government agency encrypts classified documents with RSA-2048. The documents have a 30-year sensitivity period. Assuming a cryptographically relevant quantum computer becomes available in 2035, determine the latest date by which the agency must complete its migration to post-quantum cryptography. Justify your answer with a concrete threat model based on (1.1).

1.2 Grover’s algorithm provides a quadratic speedup for unstructured search.

- (a) Explain why Grover’s algorithm reduces the effective security of AES-128 to approximately 64 bits against a quantum adversary.
- (b) A system designer proposes using AES-256 to achieve 128-bit post-quantum security. Is this sufficient? Discuss any caveats related to the cost model for quantum computation (logical qubits, gate depth, parallelism).
- (c) Why does Grover’s quadratic speedup *not* pose the same existential threat to symmetric ciphers that Shor’s exponential speedup poses to RSA?

1.3 Consider an organisation that processes financial transactions with an average sensitivity lifetime of 7 years. The organisation estimates that deploying post-quantum cryptography across its infrastructure will take 3 years.

- (a) Using the HNDL inequality (1.1), determine the latest date the migration must *begin* if a CRQC is expected by 2035.
- (b) Suppose the CRQC timeline is uncertain, with estimates ranging from 2033 to 2045. How should the organisation weigh the cost of early migration against the risk of late migration? Frame your answer in terms of the asymmetry discussed in Sect. 1.2.

1.4 Gidney and Ekerå [22] estimate that factoring an n -bit RSA modulus requires approximately $3n + 0.002n \log_2 n$ logical qubits using their optimised circuit.

- (a) Compute the logical qubit count for RSA-2048 and RSA-4096.
- (b) Assuming a surface-code overhead of ~ 5000 physical qubits per logical qubit at the required error rate, estimate the total physical qubit count for each key size.

- (c) If quantum hardware scales at a rate of $2\times$ more physical qubits every 2 years from a 2025 baseline of 1000 qubits, in what year would factoring RSA-2048 become feasible under this (highly simplified) model?

Chapter 2

Mathematical Preliminaries

Abstract This chapter establishes the algebraic, probabilistic, and complexity-theoretic foundations required throughout the book. We develop groups, rings, and fields with emphasis on the structures central to post-quantum cryptography— $\mathbb{Z}_q$, polynomial rings $\mathbb{Z}_q[x]/(f(x))$, and finite fields $\mathbb{F}_q$ —then cover modular arithmetic, the Chinese Remainder Theorem, discrete probability, computational complexity including quantum classes, and the formal security definitions (IND-CPA, IND-CCA2, EUF-CMA) that govern modern cryptographic design. Readers comfortable with abstract algebra and complexity theory may skim and return as needed; others should work through the chapter carefully.

2.1 Groups, Rings, and Fields

Post-quantum cryptographic schemes operate over algebraic structures that are considerably richer than the integers modulo a prime used in classical Diffie–Hellman or RSA. We develop the relevant definitions from scratch, emphasising the structures that will reappear throughout the book.

2.1.1 Groups

Every cryptographic scheme needs a setting in which to compute—a set of objects with operations that behave predictably. Groups are the simplest such setting: a single operation with just enough structure to support inverses and cancellation. Classical Diffie–Hellman lives in a cyclic group; the additive groups $\mathbb{Z}_q$ that dominate lattice-based cryptography are the first examples we meet.

Definition 2.1 (Group) A *group* $(G, \cdot)$ is a set G together with a binary operation $\cdot : G \times G \rightarrow G$ satisfying:

1. **Associativity:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ for all $a, b, c \in G$.

2. **Identity:** There exists $e \in G$ such that $e \cdot a = a \cdot e = a$ for all $a \in G$.
 3. **Inverses:** For each $a \in G$, there exists $a^{-1} \in G$ with $a \cdot a^{-1} = a^{-1} \cdot a = e$.

If additionally $a \cdot b = b \cdot a$ for all $a, b \in G$, the group is *abelian*. The *order* of G , denoted $|G|$, is the cardinality of G .

Example 2.1 (Additive Groups) The integers $(\mathbb{Z}, +)$ form an infinite abelian group. For any positive integer q , the set $\mathbb{Z}_q = \{0, 1, \dots, q-1\}$ with addition modulo q forms a finite abelian group of order q . These additive groups are the default setting for lattice-based cryptography.

Among all groups, those generated by a single element are the most tractable—and the most cryptographically ubiquitous. The discrete logarithm problem, which underpins classical public-key cryptography, is meaningful only in cyclic groups. Understanding cyclicity will also clarify why roots of unity exist in $\mathbb{Z}_q^*$, a fact we exploit when we introduce the Number Theoretic Transform later in this chapter.

Definition 2.2 (Cyclic Group) A group G is *cyclic* if there exists a *generator* $g \in G$ such that every element of G can be written as g^k for some integer k . We write $G = \langle g \rangle$. Every cyclic group of order n is isomorphic to $(\mathbb{Z}_n, +)$.

A natural question follows: what constraints does the size of a group impose on its internal structure? Lagrange’s theorem provides a sharp answer, and its consequences ripple through all of number-theoretic cryptography—from Euler’s theorem to the structure of $\mathbb{Z}_q^*$.

Theorem 2.1 (Lagrange’s Theorem) If H is a subgroup of a finite group G , then $|H|$ divides $|G|$. In particular, the order of every element divides $|G|$.

Proof. The cosets $aH = \{ah : h \in H\}$ for $a \in G$ partition G into subsets of equal size $|H|$. The number of distinct cosets, called the *index* $[G : H]$, satisfies $|G| = [G : H] \cdot |H|$. $\square$

Lagrange’s theorem has an immediate cryptographic consequence: in $\mathbb{Z}_q^*$ (the multiplicative group of units modulo q), every element a satisfies $a^{|\mathbb{Z}_q^*|} \equiv 1 \pmod{q}$. This leads directly to Euler’s theorem in Sect. 2.2.

2.1.2 Rings

Groups give us one operation; cryptography usually demands two. Encryption requires both addition (to inject noise) and multiplication (to mix secrets with public data), and these two operations must interact coherently through distributivity. A ring is precisely the structure that provides this. The ring $\mathbb{Z}_q$ is the arithmetic workhorse of lattice-based schemes, and the quotient polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ will reappear as the algebraic backbone of ML-KEM in Chapter 7 and ML-DSA in Chapter 8.

Definition 2.3 (Ring) A *ring* $(R, +, \cdot)$ is a set R with two binary operations such that:

1. $(R, +)$ is an abelian group (with identity 0).
2. Multiplication is associative and distributes over addition.
3. There exists a multiplicative identity $1 \in R$ (we consider only unital rings).

A ring is *commutative* if $a \cdot b = b \cdot a$ for all $a, b \in R$.

Example 2.2 (The Ring $\mathbb{Z}_q$) The set $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ with addition and multiplication modulo q is a commutative ring. An element $a \in \mathbb{Z}_q$ has a multiplicative inverse if and only if $\gcd(a, q) = 1$. The set of invertible elements $\mathbb{Z}_q^* = \{a \in \mathbb{Z}_q : \gcd(a, q) = 1\}$ forms a group under multiplication of order $\varphi(q)$ (Euler's totient function).

Polynomial rings are central to efficient lattice-based cryptography. We introduce them in stages.

Definition 2.4 (Polynomial Ring) The *polynomial ring* $\mathbb{Z}_q[x]$ consists of all polynomials in the variable x with coefficients in $\mathbb{Z}_q$. Addition and multiplication follow the usual rules, with coefficient arithmetic performed modulo q .

Polynomial rings by themselves are infinite objects—not directly suited to computation or cryptography. We tame them by imposing a reduction rule: divide by a fixed polynomial $f(x)$ and keep only the remainder. The result is a finite ring whose elements are polynomials of bounded degree, and whose multiplication wraps around in a controlled way. This quotient construction is the algebraic mechanism that compresses lattice-based keys from matrices to single polynomials, cutting key sizes by a factor of n .

Definition 2.5 (Quotient Polynomial Ring) Given a polynomial $f(x) \in \mathbb{Z}_q[x]$ of degree n , the *quotient ring* $\mathbb{Z}_q[x]/(f(x))$ consists of polynomials of degree less than n , with multiplication defined by polynomial multiplication followed by reduction modulo $f(x)$. Formally, two polynomials $a(x), b(x) \in \mathbb{Z}_q[x]$ are identified if $f(x)$ divides $a(x) - b(x)$.

The most important quotient ring in post-quantum cryptography is $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, where n is a power of two. Elements of R_q are polynomials $a(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ with $a_i \in \mathbb{Z}_q$. The reduction by $x^n + 1$ imposes $x^n \equiv -1$, so multiplication produces a *negacyclic convolution*.

Example 2.3 (Multiplication in R_q) Let $R_q = \mathbb{Z}_7[x]/(x^4 + 1)$. Consider $a(x) = 2 + 3x + x^2$ and $b(x) = 1 + 4x + 2x^3$. Their product in $\mathbb{Z}_7[x]$ is

$$a(x) \cdot b(x) = 2 + 11x + 8x^2 + 13x^3 + 6x^4 + 2x^5.$$

Reducing modulo $x^4 + 1$ using $x^4 \equiv -1$ and $x^5 \equiv -x$ gives

$$(2 - 6) + (11 - 2)x + 8x^2 + 13x^3 = -4 + 9x + 8x^2 + 13x^3 \equiv 3 + 2x + x^2 + 6x^3 \pmod{7}.$$

Why $x^n + 1$? The polynomial $x^n + 1$ (with n a power of two) is the $2n$ -th cyclotomic polynomial. It is irreducible over $\mathbb{Z}$ (so R_q has useful algebraic properties), and the negacyclic structure enables fast multiplication via the Number Theoretic Transform (Sect. 2.2). The schemes ML-KEM and ML-DSA (Chaps. 7–8) use $n = 256$ and $q = 3329$.

2.1.3 Fields

Rings lack one property that many algorithms take for granted: not every non-zero element can be divided. In $\mathbb{Z}_6$, the element 2 has no multiplicative inverse—it is a zero divisor, since $2 \cdot 3 = 0$. A field eliminates this pathology by requiring that every non-zero element be invertible. Fields govern two distinct strands of post-quantum cryptography: finite fields $\mathbb{F}_{p^k}$ underpin code-based schemes such as Classic McEliece (Chapter 9), while the distinction between “ring but not field” shapes the algebraic properties of R_q that lattice-based schemes exploit.

Definition 2.6 (Field) A *field* $(F, +, \cdot)$ is a commutative ring in which every non-zero element has a multiplicative inverse. Equivalently, $(F \setminus \{0\}, \cdot)$ is an abelian group.

A remarkable fact about finite fields is that their existence is entirely determined by cardinality: for each prime power there is exactly one, and no finite field of any other size exists. The construction mirrors the quotient polynomial rings we just defined—but now the modular polynomial must be irreducible, guaranteeing that every non-zero element has an inverse.

Theorem 2.2 (Finite Fields) *For every prime power p^k , there exists a unique (up to isomorphism) finite field of order p^k , denoted $\mathbb{F}_{p^k}$. When $k = 1$, we have $\mathbb{F}_p \cong \mathbb{Z}_p$. When $k > 1$, the field $\mathbb{F}_{p^k}$ is constructed as $\mathbb{F}_p[x]/(g(x))$ for an irreducible polynomial $g(x) \in \mathbb{F}_p[x]$ of degree k .*

Example 2.4 (The Field $\mathbb{F}_4$) The polynomial $g(x) = x^2 + x + 1$ is irreducible over $\mathbb{F}_2$. The field $\mathbb{F}_4 = \mathbb{F}_2[x]/(x^2 + x + 1)$ has elements $\{0, 1, \alpha, \alpha + 1\}$ where $\alpha^2 = \alpha + 1$. Code-based cryptography (Chap. 9) often operates over such binary extension fields.

An important distinction: $\mathbb{Z}_q$ is a field if and only if q is prime. When q is composite, $\mathbb{Z}_q$ contains zero divisors (elements $a \neq 0$ such that $ab = 0$ for some $b \neq 0$). Similarly, the quotient ring $\mathbb{Z}_q[x]/(f(x))$ is a field only when $f(x)$ is irreducible over $\mathbb{Z}_q$. The ring R_q used in lattice-based cryptography is typically *not* a field—for most choices of q , the polynomial $x^n + 1$ splits modulo q —but this is by design: the splitting enables the NTT, the fast multiplication algorithm we turn to next.

2.2 Modular Arithmetic and the Chinese Remainder Theorem

The previous section defined the algebraic structures; now we need the computational machinery to work inside them. Modular arithmetic is the engine that drives all concrete operations in $\mathbb{Z}_q$ and R_q —from key generation to decryption—and the Chinese Remainder Theorem provides a decomposition principle that turns expensive computations into independent, parallel sub-problems.

2.2.1 Modular Arithmetic

A subtle but consequential choice faces every implementer of lattice-based cryptography: how to represent elements of $\mathbb{Z}_q$. We can use the *standard* range $\{0, 1, \dots, q-1\}$ or the *centred* range $\{-\lfloor q/2 \rfloor, \dots, \lfloor (q-1)/2 \rfloor\}$. The centred representation is essential in lattice-based cryptography, where we measure the “size” of elements—noise coefficients in LWE must be small in absolute value, and centring is what gives “small” a meaning.

Definition 2.7 (Modular Reduction) For $a \in \mathbb{Z}$ and $q > 0$, define $a \bmod q = a - \lfloor a/q \rfloor \cdot q \in \{0, 1, \dots, q-1\}$. The centred reduction $[a]_q$ maps a to the unique representative in $(-q/2, q/2]$.

The extended Euclidean algorithm computes $\gcd(a, q)$ together with Bézout coefficients s, t satisfying $sa + tq = \gcd(a, q)$. When $\gcd(a, q) = 1$, the coefficient s reduced modulo q is the multiplicative inverse $a^{-1} \bmod q$.

Lagrange’s theorem, applied to the multiplicative group $\mathbb{Z}_q^*$, yields a result that underpins RSA and that we will reference when discussing why Shor’s algorithm destroys classical public-key cryptography in Chapter 1.

Theorem 2.3 (Euler’s Theorem) If $\gcd(a, q) = 1$, then $a^{\varphi(q)} \equiv 1 \pmod{q}$, where $\varphi(q) = |\mathbb{Z}_q^*|$ is Euler’s totient function.

Proof. The map $x \mapsto ax$ is a bijection on $\mathbb{Z}_q^*$. Multiplying all elements of $\mathbb{Z}_q^*$, we get $\prod_{x \in \mathbb{Z}_q^*} (ax) = a^{\varphi(q)} \prod_{x \in \mathbb{Z}_q^*} x \equiv \prod_{x \in \mathbb{Z}_q^*} x \pmod{q}$. Cancelling gives $a^{\varphi(q)} \equiv 1$. $\square$

When $q = p$ is prime, $\varphi(p) = p-1$ and we recover *Fermat’s little theorem*: $a^{p-1} \equiv 1 \pmod{p}$ for all $a \not\equiv 0$.

2.2.2 The Chinese Remainder Theorem

Modular arithmetic confines us to a single modulus at a time, but many cryptographic constructions involve composite moduli or require decomposing a large computation into smaller ones. The Chinese Remainder Theorem (CRT) provides exactly this capability: it shows that arithmetic modulo a product of coprime integers can be split into

independent arithmetic modulo each factor. Beyond its classical applications, the CRT is the conceptual ancestor of the Number Theoretic Transform, which we introduce next—and which dominates the runtime of every deployed lattice-based scheme.

Theorem 2.4 (Chinese Remainder Theorem (CRT)) *Let $q_1, q_2, \dots, q_k$ be pairwise coprime positive integers, and let $Q = \prod_{i=1}^k q_i$. The map*

$$\phi : \mathbb{Z}_Q \longrightarrow \mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2} \times \dots \times \mathbb{Z}_{q_k}, \quad a \longmapsto (a \bmod q_1, a \bmod q_2, \dots, a \bmod q_k) \quad (2.1)$$

is a ring isomorphism. In particular, the system $x \equiv a_i \pmod{q_i}$ for $i = 1, \dots, k$ has a unique solution modulo Q .

Proof sketch. The map ϕ is a ring homomorphism by the compatibility of modular reduction with addition and multiplication. Both $\mathbb{Z}_Q$ and the product ring have cardinality Q , so it suffices to show injectivity. If $\phi(a) = \phi(b)$, then $q_i \mid (a - b)$ for all i . Since the q_i are pairwise coprime, $Q \mid (a - b)$, hence $a \equiv b \pmod{Q}$. $\square$

The explicit reconstruction formula is:

$$x = \sum_{i=1}^k a_i Q_i (Q_i^{-1} \bmod q_i) \bmod Q, \quad Q_i = Q/q_i. \quad (2.2)$$

Example 2.5 (CRT Computation) Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 1 \pmod{7}$. Here $Q = 105$, $Q_1 = 35$, $Q_2 = 21$, $Q_3 = 15$. We compute $35^{-1} \equiv 2 \pmod{3}$, $21^{-1} \equiv 1 \pmod{5}$, $15^{-1} \equiv 1 \pmod{7}$. Then $x = 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 1 \cdot 15 \cdot 1 = 140 + 63 + 15 = 218 \equiv 8 \pmod{105}$.

2.2.3 The Number Theoretic Transform

The CRT showed how to decompose a ring of integers into a product of smaller rings. The same idea applies to polynomial rings—and when it does, the payoff is dramatic. Multiplying two polynomials of degree n naïvely costs $O(n^2)$ operations; the NTT reduces this to $O(n \log n)$ by converting each polynomial into a vector of pointwise evaluations, much as the fast Fourier transform does for signals. This single optimisation makes lattice-based cryptography practical at the key sizes required for security.

The CRT isomorphism extends from integers to polynomials. When q is prime and $q \equiv 1 \pmod{2n}$, the polynomial $x^n + 1$ splits completely modulo q as

$$x^n + 1 \equiv \prod_{i=0}^{n-1} (x - \omega^{2i+1}) \pmod{q}, \quad (2.3)$$

where ω is a primitive $2n$ -th root of unity modulo q . The CRT then gives a ring isomorphism

$$R_q = \mathbb{Z}_q[x]/(x^n + 1) \cong \mathbb{Z}_q \times \mathbb{Z}_q \times \dots \times \mathbb{Z}_q \quad (n \text{ copies}). \quad (2.4)$$

This is the *Number Theoretic Transform* (NTT). Under the NTT, polynomial multiplication reduces to n independent multiplications in $\mathbb{Z}_q$, yielding an $O(n \log n)$ algorithm instead of the naïve $O(n^2)$.

NTT in practice. ML-KEM uses $n = 256$ and $q = 3329$. Since $3329 = 13 \cdot 256 + 1$, we have $q \equiv 1 \pmod{512}$, so $x^{256} + 1$ splits completely modulo 3329. A primitive 512-th root of unity is $\omega = 17$. The NTT converts each polynomial to its “evaluation” representation in $\mathbb{Z}_q^{256}$; multiplication becomes pointwise, and inverse NTT recovers the coefficient form. This is the analogue of the FFT for finite fields, and it dominates the implementation cost of lattice-based schemes.

2.3 Probability and Statistical Distance

Cryptographic security is inherently probabilistic: keys are chosen at random, adversaries are randomised algorithms, and security is defined in terms of the inability to distinguish distributions. We establish the probabilistic tools needed throughout the book.

2.3.1 Discrete Probability

If algebra gives cryptography its structure, probability gives it its security. Keys are sampled at random, adversaries are randomised algorithms, and the central question in every security proof is whether two probability distributions can be told apart. We need a precise language for randomness and a quantitative measure of how “different” two distributions are.

We write $x \xleftarrow{\$} S$ to denote sampling x uniformly at random from a finite set S . More generally, $x \leftarrow D$ denotes sampling from a distribution D . The *support* of a distribution D over a set S is $\text{supp}(D) = \{x \in S : D(x) > 0\}$.

The question we most often face in security proofs is: given two distributions, how well can any algorithm distinguish between them? Statistical distance captures this exactly.

Definition 2.8 (Statistical Distance) The *statistical distance* (or *total variation distance*) between two distributions D_1 and D_2 over a countable set S is

$$\Delta(D_1, D_2) = \frac{1}{2} \sum_{x \in S} |D_1(x) - D_2(x)| = \max_{T \subseteq S} \left| \Pr_{x \leftarrow D_1} [x \in T] - \Pr_{x \leftarrow D_2} [x \in T] \right|. \quad (2.5)$$

The second equality shows that statistical distance captures the maximum advantage any (even unbounded) distinguisher can achieve. If $\Delta(D_1, D_2) \leq \varepsilon$, then replacing

D_1 by D_2 in any experiment changes the outcome probability by at most ε . This makes statistical distance the canonical measure for “closeness” of distributions in cryptographic proofs.

Saying that two distributions are “close” is useful only if we can quantify what “close enough” means as the security parameter grows. Cryptography draws the line at *negligible* functions—quantities that shrink so fast that no polynomial-time adversary can exploit them, no matter how many samples it collects.

Definition 2.9 (Negligible Function) A function $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible* if for every positive polynomial $p(\cdot)$, there exists n_0 such that $f(n) < 1/p(n)$ for all $n \geq n_0$. We write $f(n) = \text{negl}(n)$. Intuitively, a negligible quantity vanishes faster than any inverse polynomial.

Three properties make statistical distance the tool of choice in cryptographic proofs. First, Δ is a *metric*: it is non-negative, symmetric, and satisfies the triangle inequality. Second, post-processing never increases distance—for any (possibly randomised) function f , $\Delta(f(D_1), f(D_2)) \leq \Delta(D_1, D_2)$. This *data-processing inequality* ensures that if an adversary cannot distinguish D_1 from D_2 , then it cannot distinguish their images under any computation either. Third, the triangle inequality extends to chains of distributions via the *hybrid argument*: $\Delta(D_1, D_k) \leq \sum_{i=1}^{k-1} \Delta(D_i, D_{i+1})$. Security proofs routinely introduce intermediate “hybrid” distributions, bounding the total distinguishing advantage as the sum of small steps—a technique we will use repeatedly from Chapter 7 onward.

2.3.2 The Leftover Hash Lemma

Statistical distance tells us *how* to measure closeness; the leftover hash lemma (LHL) tells us *when* closeness holds. The situation arises constantly in lattice-based cryptography: we have a source with high entropy (a secret vector $\mathbf{s}$), and we multiply it by a public random matrix $\mathbf{A}$ to produce a value that should look uniformly random. The LHL guarantees that the output is indeed close to uniform, provided the source has enough entropy relative to the output size. This is the engine behind the security proof of basic LWE-based encryption in Chapter 7.

To state the lemma precisely, we first need the notion of a universal hash family—a collection of functions that spread collisions as evenly as possible.

Definition 2.10 (Universal Hash Family) A family $\mathcal{H} = \{h : X \rightarrow Y\}$ is *2-universal* if for all distinct $x_1, x_2 \in X$,

$$\Pr_{h \leftarrow \mathcal{H}} [h(x_1) = h(x_2)] \leq \frac{1}{|Y|}.$$

Theorem 2.5 (Leftover Hash Lemma [39]) Let $\mathcal{H}$ be a 2-universal family from X to Y , and let D be a distribution on X with min-entropy $H_\infty(D) \geq \log |Y| + 2 \log(1/\varepsilon)$. Then

$$\Delta((h, h(x)), (h, u)) \leq \varepsilon, \quad (2.6)$$

where $h \xleftarrow{\$} \mathcal{H}$, $x \leftarrow D$, and $u \xleftarrow{\$} Y$.

Informally: if the source has enough entropy relative to the output size, then the hash output is statistically close to uniform—even given the hash function itself. In the LWE setting, the matrix $\mathbf{A}$ plays the role of the hash function and the secret $\mathbf{s}$ plays the role of the high-entropy source.

2.3.3 Discrete Gaussian Distributions

Lattice-based cryptography hides secrets behind noise, and the character of that noise determines both security and correctness. Discrete Gaussian distributions are the single most important distribution family in this setting. They appear in the definition of LWE, in the Gaussian sampling needed for worst-case to average-case security reductions (Chapter 6), and in the signing algorithms for lattice-based signatures (Chapter 8). Their tails decay exponentially—fast enough that decryption almost always succeeds, yet wide enough that the noise masks the secret.

We build up the definition in two stages: the continuous Gaussian function, then its restriction to integer points.

Definition 2.11 (Gaussian Function) For $s > 0$ and $\mathbf{c} \in \mathbb{R}^n$, the *Gaussian function* centred at $\mathbf{c}$ with parameter s is

$$\rho_{s,\mathbf{c}}(\mathbf{x}) = \exp\left(-\pi \frac{\|\mathbf{x} - \mathbf{c}\|^2}{s^2}\right). \quad (2.7)$$

When $\mathbf{c} = \mathbf{0}$, we write simply ρ_s .

The continuous Gaussian assigns a weight to every point in $\mathbb{R}^n$; in cryptography we sample only integer points. Normalising the Gaussian function over the integer lattice (or any lattice) yields the discrete Gaussian distribution.

Definition 2.12 (Discrete Gaussian Distribution) The *discrete Gaussian distribution* over $\mathbb{Z}^n$ with parameter $s > 0$ and centre $\mathbf{c}$ is

$$D_{\mathbb{Z}^n,s,\mathbf{c}}(\mathbf{x}) = \frac{\rho_{s,\mathbf{c}}(\mathbf{x})}{\sum_{\mathbf{y} \in \mathbb{Z}^n} \rho_{s,\mathbf{c}}(\mathbf{y})}, \quad \mathbf{x} \in \mathbb{Z}^n. \quad (2.8)$$

More generally, for a lattice $\mathcal{L}$, the discrete Gaussian $D_{\mathcal{L},s,\mathbf{c}}$ restricts the support to $\mathcal{L}$.

Theorem 2.6 (Tail Bound for Discrete Gaussians) For $x \leftarrow D_{\mathbb{Z},s}$ (the one-dimensional discrete Gaussian centred at zero),

$$\Pr[|x| > t \cdot s] \leq 2 \exp(-\pi t^2) \quad (2.9)$$

for all $t > 0$. In particular, $|x| \leq s\sqrt{\ln(2n/\delta)/\pi}$ with probability at least $1 - \delta$ for each coordinate of a sample from $D_{\mathbb{Z}^n,s}$.

This tail bound is essential for setting parameters: the “noise” terms in LWE-based schemes must remain small enough that decryption succeeds, so we need s large enough for security but small enough that the tail probability of a decryption failure is negligible.

A critical question arises when sampling from a discrete Gaussian: how wide must the distribution be before the lattice structure becomes invisible? If the width parameter s is too small, the samples cluster near a few lattice points and leak structural information. The smoothing parameter $\eta_\varepsilon(\mathcal{L})$ marks the threshold above which the discrete Gaussian “smooths out” the lattice, making samples statistically close to continuous. This parameter is the linchpin of the worst-case to average-case reductions in Chapter 6.

Definition 2.13 (Smoothing Parameter) The *smoothing parameter* $\eta_\varepsilon(\mathcal{L})$ of a lattice $\mathcal{L}$ (for $\varepsilon > 0$) is the smallest $s > 0$ such that

$$\rho_{1/s}(\mathcal{L}^* \setminus \{\mathbf{0}\}) \leq \varepsilon, \quad (2.10)$$

where $\mathcal{L}^*$ is the dual lattice.

Intuitively, $\eta_\varepsilon(\mathcal{L})$ is the Gaussian width above which the discrete Gaussian $D_{\mathcal{L},s}$ “smooths out” the lattice structure: samples look statistically close to a continuous Gaussian. The smoothing parameter appears in the worst-case to average-case reductions of Chap. 6 and in Gaussian sampling algorithms for lattice-based signatures (Chap. 8). The key property is:

Theorem 2.7 (Smoothing Property [46]) For $s \geq \eta_\varepsilon(\mathcal{L})$ and any $\mathbf{c} \in \mathbb{R}^n$,

$$D_{\mathcal{L},s,\mathbf{c}} \bmod \mathcal{P}(\mathcal{L}) \approx_\varepsilon U(\mathcal{P}(\mathcal{L})), \quad (2.11)$$

where $\mathcal{P}(\mathcal{L})$ is the fundamental parallelepiped and U denotes the uniform distribution.

With the probabilistic toolkit established—statistical distance, negligible functions, the leftover hash lemma, and discrete Gaussians—we have the analytic machinery to state and prove security results. What we still lack is a formal model of what adversaries can and cannot compute, which is the subject of the next section.

2.4 Computational Complexity

The algebraic and probabilistic tools of the previous sections give us the language to *state* cryptographic constructions; complexity theory gives us the language to argue that they are *secure*. Every security claim in this book ultimately rests on the assumption that some computational problem cannot be solved efficiently—and “efficiently” must be defined with care, since quantum computers change what counts as efficient.

2.4.1 Complexity Classes

We assume familiarity with Turing machines at the level of a first course in theoretical computer science and define the classes that will appear in our security statements. The classical classes P, NP, and BPP are standard; the quantum class BQP is the one that matters most for this book, since the entire motivation for post-quantum cryptography is the gap between what BPP and BQP can solve.

Definition 2.14 (Basic Complexity Classes) The class P consists of decision problems solvable by a deterministic Turing machine in time $\text{poly}(n)$, where n is the input size. The class NP enlarges this to problems whose “yes” instances have a certificate (witness) verifiable in $\text{poly}(n)$ time; coNP is its complement class, and a problem in $\text{NP} \cap \text{coNP}$ admits short certificates for both answers, making it unlikely to be NP-complete. Introducing randomness, BPP captures problems solvable by a probabilistic Turing machine in $\text{poly}(n)$ time with error probability at most $1/3$, amplifiable to $\text{negl}(n)$ by repetition. Finally, BQP is the quantum analogue of BPP: problems solvable by a quantum computer in $\text{poly}(n)$ time with bounded error.

The known inclusions are:

$$\text{P} \subseteq \text{BPP} \subseteq \text{BQP} \subseteq \text{PSPACE}, \quad \text{P} \subseteq \text{NP} \subseteq \text{PSPACE}. \quad (2.12)$$

Whether $\text{P} = \text{NP}$ and whether $\text{NP} \subseteq \text{BQP}$ are major open questions. For cryptography, the central concern is the relationship between NP and BQP: if $\text{NP} \not\subseteq \text{BQP}$, then NP-hard problems remain intractable even for quantum adversaries.

2.4.2 Reductions

Complexity classes tell us *where* problems live; reductions tell us *how hard* they are relative to one another. The idea is simple but powerful: if we can transform any instance of problem A into an instance of problem B efficiently, then B is at least as hard as A . This comparative reasoning is the foundation of every security proof in modern cryptography—including the lattice-based proofs in Chapters 6–8.

Definition 2.15 (Karp Reduction) A *Karp (many-one) reduction* from problem A to problem B , written $A \leq_p B$, is a polynomial-time computable function f such that for every instance x :

$$x \in A \iff f(x) \in B. \quad (2.13)$$

If $A \leq_p B$ and A is hard, then B is at least as hard as A .

A more general notion is the *Turing reduction* (or *Cook reduction*), where the algorithm for A may make polynomially many adaptive queries to an oracle for B . Cryptographic security reductions are typically Turing reductions: the reduction algorithm is given oracle access to an adversary and uses its output to solve the underlying hard problem.

Reductions create a hierarchy of hardness. At the top sit the problems to which *every* problem in NP can be reduced—the hardest problems in that class.

Definition 2.16 (NP-Hardness) A problem B is *NP-hard* if every problem in NP reduces to B . If additionally $B \in \text{NP}$, then B is *NP-complete*.

2.4.3 Average-Case vs. Worst-Case Complexity

NP-hardness is a powerful guarantee, but it has a subtle limitation: it says that *some* instances are hard, not that *most* are. A problem can be NP-hard yet easy on almost all random inputs—which would be useless for cryptography, where keys and challenges are generated at random. What we need is *average-case* hardness: the assurance that a problem remains intractable on typical instances.

Definition 2.17 (Average-Case Hardness) A problem Π with input distribution D is *average-case hard* if no polynomial-time algorithm solves a non-negligible fraction of instances drawn from D .

In general, worst-case hardness does not imply average-case hardness: a problem may have a few extremely hard instances but be easy on average. For classical cryptographic assumptions (factoring, discrete logarithm), we *assume* average-case hardness without proof. The breakthrough of lattice-based cryptography is that for problems such as LWE and SIS, worst-case hardness *provably implies* average-case hardness via reductions [1, 56]. This worst-case to average-case connection is developed in Chap. 6.

2.4.4 Quantum Complexity and Cryptography

Everything in the previous subsections applies to classical computation. Quantum computers change the landscape in a precise and dramatic way: Shor’s algorithm [59] places integer factorisation and the discrete logarithm problem in BQP, shattering RSA, Diffie–Hellman, and elliptic curve cryptography. Chapter 1 analyses this threat in detail; here we establish the complexity-theoretic picture. The essential question for post-quantum cryptography is: which hard problems survive in BQP?

The bottom four rows of Table 2.1 are the problems on which post-quantum cryptographic schemes are built. Their resistance to quantum attack is the entire premise of this book. But knowing that a problem is hard is only half the story—we still need to define what “breaking a scheme” means and to connect scheme security to problem hardness through formal reductions.

Table 2.1 Classical vs. quantum complexity of cryptographically relevant problems

Problem	Classical	Quantum
Integer factorisation	Sub-exponential (NFS)	$\text{poly}(n)$ (Shor)
Discrete logarithm	Sub-exponential	$\text{poly}(n)$ (Shor)
Approx-SVP ($\gamma = \text{poly}(n)$)	$\text{poly}(n)$ (LLL)	$\text{poly}(n)$
Approx-SVP ($\gamma = O(1)$)	$2^{\Theta(n)}$	$2^{\Theta(n)}$
LWE (decision)	$2^{\Theta(n)}$	$2^{\Theta(n)}$
Syndrome decoding	$2^{\Theta(n)}$	$2^{\Theta(n)}$

2.5 Security Definitions and Reductions

Algebra provides the structures, probability provides the tools, and complexity theory provides the adversary model. What remains is the most important ingredient of all: a precise definition of what it means for a cryptographic scheme to be “secure.” Without formal definitions, security claims are just opinions. The definitions in this section apply equally to classical and post-quantum schemes; the only difference is whether the adversary is a classical or quantum polynomial-time machine.

2.5.1 Security Games and Advantages

Modern cryptography defines security not by listing attacks a scheme resists, but by specifying a *game*—a structured interaction between a *challenger* (who runs the scheme honestly) and an *adversary* $\mathcal{A}$ (who tries to break it). The adversary’s *advantage* is the probability that it “wins” the game minus the trivial success probability (typically $1/2$ for a distinguishing game, or 0 for a search game). A scheme is secure if no efficient adversary achieves more than a negligible advantage.

Definition 2.18 (Asymptotic Security) A scheme is *secure* (under a given definition) if for every probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the advantage $\text{Adv}(\mathcal{A})$ is negligible in the security parameter λ :

$$\text{Adv}(\mathcal{A}) \leq \text{negl}(\lambda).$$

2.5.2 Security of Encryption: IND-CPA and IND-CCA2

What does it mean for an encryption scheme to “not leak information”? The answer depends on how powerful we assume the adversary to be. The weakest standard notion, IND-CPA, models a passive eavesdropper who can encrypt messages of its choice but cannot tamper with ciphertexts. The stronger IND-CCA2 models an active attacker who can also submit ciphertexts for decryption—a realistic threat in networked protocols.

Every lattice-based encryption scheme in this book targets one of these two notions, and the Fujisaki–Okamoto transform (Chapter 7) bridges the gap between them.

Definition 2.19 (IND-CPA Security) A public-key encryption scheme ($\text{KeyGen}, \text{Enc}, \text{Dec}$) is *IND-CPA secure* (indistinguishable under chosen-plaintext attack) if for every PPT adversary $\mathcal{A}$, the following advantage is negligible:

$$\text{Adv}_{\mathcal{A}}^{\text{ind-cpa}}(\lambda) = \left| \Pr[\mathcal{A} \text{ wins IND-CPA game}] - \frac{1}{2} \right|. \quad (2.14)$$

The game proceeds as follows:

1. The challenger generates $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ and gives pk to $\mathcal{A}$.
2. $\mathcal{A}$ submits two equal-length messages m_0, m_1 .
3. The challenger chooses $b \xleftarrow{\$} \{0, 1\}$ and sends $c^* = \text{Enc}(pk, m_b)$ to $\mathcal{A}$.
4. $\mathcal{A}$ outputs a guess b' . It wins if $b' = b$.

IND-CPA captures the minimal requirement: an eavesdropper who sees only public keys and ciphertexts cannot determine which of two messages was encrypted. Since $\mathcal{A}$ holds the public key, it can encrypt any message it wishes—hence “chosen plaintext.”

IND-CPA is necessary but insufficient for most real-world deployments. In practice, adversaries often interact with the decryption side of a system—probing it with crafted ciphertexts and observing error messages, timing variations, or protocol-level responses. IND-CCA2 captures this stronger threat model.

Definition 2.20 (IND-CCA2 Security) A scheme is *IND-CCA2 secure* (indistinguishable under adaptive chosen-ciphertext attack) if it remains IND-CPA secure even when the adversary has access to a decryption oracle $\text{Dec}(sk, \cdot)$ throughout the game, with the restriction that it may not query the challenge ciphertext c^* .

IND-CCA2 is the standard security target for deployed encryption. It models an adversary who can submit ciphertexts for decryption (e.g., by observing error messages or timing behaviour) and still cannot extract information about the challenge plaintext. The Fujisaki–Okamoto transform [19], used in ML-KEM, upgrades an IND-CPA scheme to IND-CCA2 security.

CPA vs. CCA2 in practice. An IND-CPA-secure scheme may be completely broken under CCA2 attack. For example, textbook LWE encryption (Chap. 7) is IND-CPA secure but malleable: an adversary can modify a ciphertext in a predictable way. The Fujisaki–Okamoto transform eliminates this malleability by binding the encryption randomness to a hash of the message, so any modification is detectable.

2.5.3 Security of Signatures: EUF-CMA

Encryption protects confidentiality; digital signatures protect authenticity. The security question is different: instead of asking whether an adversary can *learn* something, we ask whether it can *forge* something—produce a valid signature on a message it has never seen signed. The standard formalisation, EUF-CMA, gives the adversary generous power: it may request signatures on any messages it chooses before attempting its forgery. This is the security target for ML-DSA and SLH-DSA (Chapters 8 and ??).

Definition 2.21 (EUF-CMA Security) A signature scheme (KeyGen , Sign , Verify) is *EUF-CMA secure* (existentially unforgeable under chosen-message attack) if for every PPT adversary $\mathcal{A}$, the following advantage is negligible:

$$\text{Adv}_{\mathcal{A}}^{\text{euf-cma}}(\lambda) = \Pr[\mathcal{A} \text{ outputs valid forgery}] . \quad (2.15)$$

The game:

1. The challenger generates $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ and gives pk to $\mathcal{A}$.
2. $\mathcal{A}$ adaptively queries a signing oracle $\text{Sign}(sk, \cdot)$ on messages $m_1, \dots, m_Q$ of its choice.
3. $\mathcal{A}$ outputs a pair (m^*, σ^*) . It wins if $\text{Verify}(pk, m^*, \sigma^*) = 1$ and $m^* \notin \{m_1, \dots, m_Q\}$.

EUF-CMA is the standard security target for digital signatures. The adversary may see signatures on any messages it chooses but must produce a valid signature on a *new* message.

2.5.4 Security Reductions

Security definitions tell us *what* to prove; reductions tell us *how* to prove it. A *security reduction* connects the dots between a computational hardness assumption and a scheme’s security definition: it shows that any adversary breaking the scheme can be transformed into an algorithm solving the underlying hard problem. Since the hard problem is assumed intractable, the adversary cannot exist. The quality of this connection—how much security is “lost” in the transformation—has direct consequences for parameter selection, as we will see repeatedly in Chapters 7–8.

Definition 2.22 (Security Reduction Structure) A reduction from the hardness of problem Π to the security of scheme Σ is a PPT algorithm $\mathcal{R}$ (the *reducer*) that, given oracle access to any adversary $\mathcal{A}$ breaking Σ with advantage ε , solves Π with advantage at least $\varepsilon' = \varepsilon/L$, where L is the *security loss*.

The structure of a security proof follows a three-step pattern. First, we assume that some adversary $\mathcal{A}$ breaks the scheme with non-negligible advantage. Second, we construct a reduction algorithm $\mathcal{R}^{\mathcal{A}}$ that uses $\mathcal{A}$ as a subroutine, feeding it a simulated view of the cryptographic game while secretly embedding an instance of the hard

problem. Third, we show that whenever $\mathcal{A}$ succeeds, $\mathcal{R}^{\mathcal{A}}$ extracts a solution to the hard problem—contradicting the hardness assumption.

The loss factor L determines the *tightness* of the reduction. A *tight* reduction has $L = \text{poly}(\lambda)$; a *loose* reduction may have L that depends on the number of queries. Tight reductions are preferred because they translate concrete hardness assumptions directly into concrete security guarantees without inflating parameters.

2.5.5 The Random Oracle Model and QROM

Security reductions need a model for hash functions, and two options exist. In the *standard model*, hash functions are concrete objects with fixed code, and proofs must work for any instantiation. Standard-model proofs are rare and often impractical. The alternative—and the dominant paradigm for practical scheme design—is to treat hash functions as idealised random functions. This abstraction, the random oracle model, enables clean and modular proofs. Its quantum extension, the QROM, is essential for post-quantum security because a quantum adversary can evaluate hash functions on superpositions of inputs.

Definition 2.23 (Random Oracle Model (ROM)) In the *random oracle model*, hash functions are modelled as truly random functions $H : \{0, 1\}^* \rightarrow \{0, 1\}^k$, accessible only via oracle queries. All parties—including the adversary—interact with the same oracle.

The ROM enables clean, modular security proofs: the reduction can “program” the random oracle (choose its outputs on specific inputs) and observe the adversary’s queries. Real-world implementations instantiate the oracle with a concrete hash function (e.g., SHA-3). While a proof in the ROM does not automatically imply security with a concrete hash function, it provides strong heuristic evidence and is the standard model for practical scheme design.

Definition 2.24 (Quantum Random Oracle Model (QROM)) In the *quantum random oracle model*, the adversary may query the hash function in *quantum superposition*: given a quantum state $\sum_x \alpha_x |x\rangle |0\rangle$, the oracle returns $\sum_x \alpha_x |x\rangle |H(x)\rangle$.

The QROM is essential for post-quantum security proofs, and transitioning from ROM to QROM is not merely a matter of changing notation—several classical proof techniques break down fundamentally. In the ROM, the reduction can silently record the adversary’s hash queries; in the QROM, quantum queries cannot be observed without disturbing the quantum state, a consequence of the no-cloning theorem. Lazy sampling—choosing random oracle outputs on the fly as queries arrive—requires substantial reworking, since a quantum adversary queries exponentially many inputs in superposition with a single oracle call. Even the standard technique of rewinding the adversary, used routinely in classical proofs to extract information, becomes delicate in the quantum setting and requires the *Watrous rewinding lemma*.

The Fujisaki–Okamoto transform and the Fiat–Shamir transform [17]—both used in NIST PQC standards—have been proven secure in the QROM, but with additional technical conditions compared to their ROM proofs. We revisit these in Chaps. 7 and 8.

The machinery is now in place. We have the algebraic structures ($\mathbb{Z}_q$, R_q , finite fields), the probabilistic tools (statistical distance, discrete Gaussians), the complexity-theoretic framework (worst-case and average-case hardness, BQP), and the security definitions (IND-CPA, IND-CCA2, EUF-CMA) that will govern every construction in this book. The next chapter introduces the quantum computational model and Shor’s algorithm in detail, making precise the threat that motivates everything that follows.

Problems

2.1 Let $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $q = 17$ and $n = 4$. Compute the product of $a(x) = 3x^3 + x + 2$ and $b(x) = 2x^2 + x + 1$ in R_q . Verify your answer by checking that the result has degree less than 4 and all coefficients lie in $\{0, 1, \dots, 16\}$.

2.2 Use the Chinese Remainder Theorem to solve:

$$x \equiv 3 \pmod{5}, \quad x \equiv 4 \pmod{7}, \quad x \equiv 2 \pmod{11}.$$

Verify your answer by checking all three congruences.

2.3 Let $q = 17$ and $n = 4$. Verify that $\omega = 2$ is a primitive 8th root of unity modulo 17 (i.e., $2^8 \equiv 1 \pmod{17}$ and $2^4 \not\equiv 1 \pmod{17}$). List the roots of $x^4 + 1$ modulo 17.

2.4 Let D_1 be the uniform distribution on $\{0, 1, 2, 3, 4\}$ and let D_2 assign probabilities $(0.3, 0.2, 0.2, 0.2, 0.1)$. Compute $\Delta(D_1, D_2)$. If a security proof replaces D_1 by D_2 in one step, by how much does the adversary’s advantage change?

2.5 Explain why a polynomial-time reduction from Problem A to Problem B , combined with a known lower bound for Problem A , provides evidence for the hardness of Problem B . Now suppose the reduction is *loose* with security loss $L = 2^{40}$. If Problem A has 128-bit hardness, what concrete security can we claim for a scheme based on Problem B ?

2.6 Consider an encryption scheme where $\text{Enc}(pk, m) = (r, m \oplus H(r))$ for $r \xleftarrow{\$} \{0, 1\}^k$ and H is a random oracle.

- (a) Prove that this scheme is IND-CPA secure in the random oracle model.
- (b) Is this scheme IND-CCA2 secure? If not, describe a concrete attack.

2.7 Conceptual. Explain why a security proof in the classical ROM may fail in the QROM. Give a concrete example of a proof technique that relies on recording the adversary’s hash queries, and explain what goes wrong when the adversary can make quantum superposition queries.

Chapter 3

Classical Cryptographic Systems

Abstract Before understanding what quantum computers threaten, one must understand what exists to be broken. This chapter develops the mathematical foundations of symmetric and asymmetric cryptography—from AES and SHA-3 through RSA, Diffie–Hellman, and elliptic curves—and traces their deployment through the TLS 1.3 protocol that protects virtually all internet communications. The chapter concludes by isolating the three computational hardness assumptions on which the entire edifice rests, setting the stage for the quantum attacks of Chapter 1.

3.1 Symmetric Cryptography

Modern cryptography rests on three equally critical pillars: *confidentiality* (only authorised parties read the data), *integrity* (any tampering is detectable), and *authentication* (the identity of the sender is verifiable). Symmetric cryptography addresses the first two using a single shared secret key.

Figure 3.1 shows how these components are organised into a layered architecture: user-facing applications rely on protocols, which in turn invoke cryptographic primitives built atop mathematical structures whose security rests on unproven—but widely believed—hardness assumptions. The quantum threat strikes at the bottom of this stack.

The three pillars are illustrated in Figure 3.2, which also highlights the distinct adversarial models each pillar defends against and the looming quantum threat that cuts across all of them.

3.1.1 Block Ciphers and AES

Confidentiality requires a reversible transformation: a way to scramble data so that only the keyholder can unscramble it. The simplest useful abstraction is a function that encrypts one fixed-size chunk at a time—a *block cipher*.

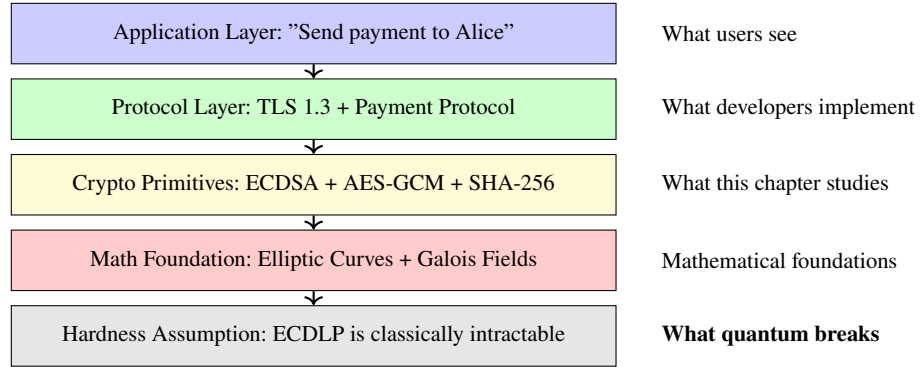


Fig. 3.1 The cryptographic stack: from user action to mathematical foundation. Each layer depends on the one below it; a quantum break at the bottom propagates upward.

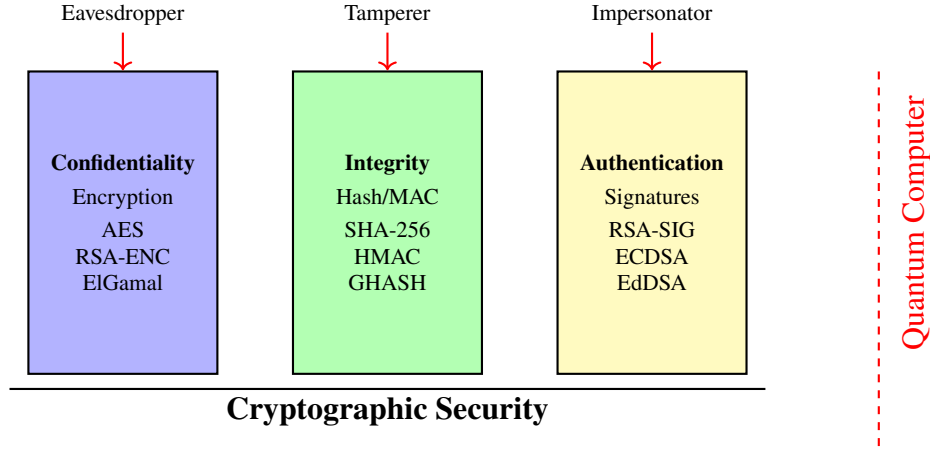


Fig. 3.2 The three pillars of cryptographic security and the adversaries each defends against. The dashed line represents the quantum threat, which undermines the asymmetric primitives in confidentiality and authentication.

Definition 3.1 (Block Cipher) A *block cipher* is a pair of deterministic algorithms (Enc, Dec) operating on fixed-length blocks, with a key $k \in \{0, 1\}^\kappa$:

$$\text{Enc}_k : \{0, 1\}^n \rightarrow \{0, 1\}^n, \quad \text{Dec}_k : \{0, 1\}^n \rightarrow \{0, 1\}^n, \quad \text{Dec}_k(\text{Enc}_k(m)) = m.$$

For every fixed key k , the map Enc_k is a permutation on $\{0, 1\}^n$.

The *Advanced Encryption Standard* (AES) operates on 128-bit blocks with key sizes $\kappa \in \{128, 192, 256\}$. Its internal structure consists of N_r rounds (10, 12, or 14 depending on key size), each applying four transformations: **SubBytes** (non-linear S-box substitution), **ShiftRows** (byte permutation), **MixColumns** (linear diffusion over $\text{GF}(2^8)$), and **AddRoundKey** (XOR with round key). After 20 years of intensive cryptanalysis, no attack against full AES is faster than brute-force key search.

3.1.2 Authenticated Encryption: AES-GCM

A block cipher alone provides confidentiality for a single block, but real messages span many blocks and face threats beyond eavesdropping—an active adversary can flip bits, reorder blocks, or splice fragments from different messages. A *mode of operation* extends encryption to arbitrary-length messages, and the most important modes go further: they bind confidentiality and integrity into a single primitive called *authenticated encryption with associated data* (AEAD). The dominant AEAD construction in practice is *Galois/Counter Mode* (GCM).

Definition 3.2 (AEAD Scheme) An *authenticated encryption* scheme provides $\text{Enc}_k(N, A, M) \rightarrow (C, T)$ and $\text{Dec}_k(N, A, C, T) \rightarrow M$ or $\perp$, where N is a nonce, A is associated data authenticated but not encrypted, M is the plaintext, C is the ciphertext, and T is an authentication tag. Decryption returns $\perp$ if any bit of (A, C) has been modified.

AES-GCM encrypts in counter mode (CTR) and computes the tag T using a universal hash function GHASH over $\text{GF}(2^{128})$. It dominates internet traffic: the cipher suite TLS_AES_256_GCM_SHA384 is the most widely deployed in TLS 1.3.

3.1.3 Hash Functions: SHA-2 and SHA-3

Authenticated encryption protects data in transit, but many cryptographic operations need something different: a way to produce a short, unique fingerprint of an arbitrary-length message. Key derivation, digital signatures, and integrity checks all depend on *cryptographic hash functions*—deterministic, efficiently computable maps that behave, for all practical purposes, like random oracles.

Definition 3.3 (Cryptographic Hash Function) A *cryptographic hash function* $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ satisfies three properties:

1. **Preimage resistance:** given y , it is infeasible to find x with $H(x) = y$.
2. **Second preimage resistance:** given x , it is infeasible to find $x' \neq x$ with $H(x') = H(x)$.
3. **Collision resistance:** it is infeasible to find any pair (x, x') with $x \neq x'$ and $H(x) = H(x')$.

The **SHA-2** family (SHA-256, SHA-384, SHA-512) uses a Merkle–Damgård construction with Davies–Meyer compression. **SHA-3** (Keccak) uses a sponge construction over a 1600-bit permutation, offering a completely independent design lineage. Both families are considered secure; SHA-3 provides a hedge against a future structural break in Merkle–Damgård.

3.1.4 Message Authentication Codes

Hash functions detect accidental corruption, but they cannot detect malicious tampering: an adversary who modifies a message can simply recompute the hash. What we need is a keyed variant—a function whose output can only be produced or verified by someone who holds a shared secret. This is a *message authentication code* (MAC).

Definition 3.4 (MAC) A *message authentication code* is a triple (Gen, Tag, Verify) where $\text{Tag}_k(m)$ produces a tag and $\text{Verify}_k(m, t)$ outputs accept or reject. Security requires that no efficient adversary, even one who can obtain tags on chosen messages, can produce a valid tag on a new message (*existential unforgeability*).

HMAC, defined as $\text{HMAC}_k(m) = H((k \oplus \text{opad}) \| H((k \oplus \text{ipad}) \| m))$, is the standard keyed-hash MAC. It appears prominently in TLS 1.3 key derivation (HKDF).

Quantum impact on symmetric primitives. Grover’s algorithm gives a quadratic speedup for brute-force key search, reducing the effective security of AES-128 from 2^{128} to 2^{64} operations. The standard mitigation is to double the key length: AES-256 retains 128-bit security against quantum adversaries. Hash functions similarly require doubled output lengths. Symmetric cryptography survives the quantum transition largely intact.

3.2 Public-Key Cryptography

3.2.1 The Key Distribution Problem

Before 1976, all cryptography was symmetric, leading to a fundamental paradox: to communicate securely, two parties first had to exchange a secret key over a channel that was already secure. At scale, this is impractical.

Example 3.1 (The Key Distribution Nightmare) If n parties each wish to communicate securely with every other, they require $\binom{n}{2}$ distinct shared keys. For $n = 1,000$ this means 499,500 keys, each needing secure physical distribution. Adding a single new party requires 1,000 additional key exchanges.

Historical solutions—diplomatic pouches, pre-distributed code books, key distribution centres—were expensive, unscalable, and fragile. The emerging internet of the 1970s demanded something fundamentally different.

3.2.2 One-Way Trapdoor Functions

The key distribution problem seems inescapable if encryption and decryption require the same key. The breakthrough insight of the 1970s was that the two operations need not be symmetric: if computing a function is easy but inverting it is hard—unless one holds a secret “trapdoor”—then the function itself can serve as a public key.

Definition 3.5 (One-Way Trapdoor Function) A function $f : X \rightarrow Y$ is a *one-way trapdoor function* if:

1. **Easy to compute:** given x , computing $f(x)$ is efficient.
2. **Hard to invert:** given $y = f(x)$, finding x is computationally infeasible.
3. **Trapdoor:** there exists secret information t that makes inversion efficient.

In 1976, Diffie and Hellman proposed separating the key into a public component (for encryption or verification) and a private component (for decryption or signing) [15]. This idea—*public-key cryptography*—resolved the key distribution problem and made the internet possible.

3.2.3 RSA

Diffie and Hellman described the *concept* of public-key cryptography in 1976, but did not provide a concrete system. The first practical realisation came two years later from Rivest, Shamir, and Adleman [57], who found a trapdoor function hiding in elementary number theory.

The design intuition is this: multiplying two large primes is trivial, but recovering them from the product is—as far as anyone knows—extraordinarily hard. RSA exploits this asymmetry. The product $n = pq$ is made public; anyone can use it to encrypt. But computing the decryption exponent requires knowing $\phi(n) = (p - 1)(q - 1)$, which is easy if one knows p and q and apparently intractable otherwise. The trapdoor is the factorisation itself.

Definition 3.6 (RSA Key Generation)

1. Choose two large primes p, q (each ≈ 1024 bits for RSA-2048).
2. Compute $n = pq$ and $\phi(n) = (p - 1)(q - 1)$.
3. Choose e with $\gcd(e, \phi(n)) = 1$; the standard choice is $e = 65537$.
4. Compute $d \equiv e^{-1} \pmod{\phi(n)}$.
5. Public key: (n, e) . Private key: d .

Encryption and decryption are modular exponentiations:

$$\text{Enc}(m) = m^e \bmod n, \quad \text{Dec}(c) = c^d \bmod n. \quad (3.1)$$

Theorem 3.1 (RSA Correctness) For any $m \in \mathbb{Z}_n^*$, decryption recovers the plaintext:

$$(m^e)^d = m^{ed} = m^{1+k\phi(n)} = m \cdot (m^{\phi(n)})^k \equiv m \pmod{n}, \quad (3.2)$$

where the last step follows from Euler's theorem: $m^{\phi(n)} \equiv 1 \pmod{n}$ whenever $\gcd(m, n) = 1$.

Textbook RSA as described above is *insecure* in practice: it is deterministic (identical plaintexts yield identical ciphertexts) and malleable (given $c = m^e$, an adversary can compute $(2m)^e = 2^e \cdot c$). Real deployments use *Optimal Asymmetric Encryption Padding* (OAEP), which randomises the input and achieves chosen-ciphertext security in the random oracle model.

3.2.4 Diffie–Hellman Key Exchange

RSA solves encryption and signatures, but it does not directly address the original key distribution problem: how can two parties who have never met agree on a shared secret, communicating only over a public channel? The Diffie–Hellman (DH) protocol, published in the same landmark 1976 paper that introduced public-key cryptography [15], provides an elegant answer using modular exponentiation.

Definition 3.7 (Diffie–Hellman Protocol) Let p be a large prime and g a generator of the multiplicative group $\mathbb{Z}_p^*$.

1. Alice chooses a secret $a \xleftarrow{\$} \{2, \dots, p-2\}$ and sends $A = g^a \bmod p$.
2. Bob chooses a secret $b \xleftarrow{\$} \{2, \dots, p-2\}$ and sends $B = g^b \bmod p$.
3. Both compute the shared secret $K = g^{ab} \bmod p$: Alice computes B^a and Bob computes A^b .

An eavesdropper observes $(g, p, g^a \bmod p, g^b \bmod p)$ but must compute $g^{ab} \bmod p$ —the *Computational Diffie–Hellman* (CDH) problem, which is believed to be as hard as the discrete logarithm. Note that DH provides no authentication: without additional measures, it is vulnerable to man-in-the-middle attacks.

3.2.5 Elliptic Curve Cryptography

Both RSA and Diffie–Hellman achieve security through the difficulty of number-theoretic problems in $\mathbb{Z}_n^*$ or $\mathbb{Z}_p^*$, but they pay a steep price in key size: 3072-bit keys for 128-bit security. In the mid-1980s, Koblitz [33] and Miller [47] independently discovered that the discrete logarithm problem is dramatically harder in the group of points on an elliptic curve, allowing the same security from keys roughly ten times shorter. Elliptic curve cryptography (ECC) replaces the multiplicative group $\mathbb{Z}_p^*$ with this richer algebraic structure.

Definition 3.8 (Elliptic Curve over a Finite Field) An *elliptic curve* E over $\mathbb{F}_p$ (with $p > 3$) is the set of points $(x, y) \in \mathbb{F}_p \times \mathbb{F}_p$ satisfying the Weierstrass equation

Table 3.1 Equivalent security levels for RSA/DLP and ECC key sizes (in bits)

Security (bits)	RSA modulus	DLP ($\mathbb{Z}_p^*$)	ECC key
80	1 024	1 024	160
128	3 072	3 072	256
256	15 360	15 360	512

$$y^2 = x^3 + ax + b, \quad 4a^3 + 27b^2 \neq 0, \quad (3.3)$$

together with a distinguished *point at infinity* O . The set $E(\mathbb{F}_p)$ forms an abelian group under a chord-and-tangent addition law, with O as the identity element.

The group law is defined geometrically: to add two distinct points P and Q , draw the line through them; it intersects E at a third point R' , and we define $P + Q$ as the reflection of R' across the x -axis. Point doubling ($P + P$) uses the tangent line at P . Translated into field arithmetic over $\mathbb{F}_p$, these operations are efficient.

Scalar multiplication $kP = P + P + \dots + P$ (k times) is computed in $O(\log k)$ group operations via double-and-add, but recovering k from the pair (P, kP) is the *elliptic curve discrete logarithm problem* (ECDLP).

The dramatic key-size advantage (Table 3.1) has made ECC the dominant choice for modern protocols: TLS 1.3 mandates ECDHE key exchange (typically over Curve25519 or P-256) and supports ECDSA signatures.

The hybrid approach. In practice, public-key cryptography is never used to encrypt bulk data directly—it is too slow. Instead, modern protocols employ a *hybrid* design: an asymmetric algorithm (ECDHE, RSA) establishes a shared symmetric key, and a fast symmetric cipher (AES-GCM) encrypts the actual data. This division of labour—asymmetric for trust establishment, symmetric for throughput—pervades every protocol discussed in this chapter.

3.3 Digital Signatures

Public-key encryption and key exchange solve confidentiality, but a critical piece of the security triad remains: how does one prove, to anyone, that a given message was produced by a specific party and has not been altered? Symmetric MACs (Section 3.1.4) provide integrity between two parties who share a key, but they cannot convince a third party—both parties can produce the same tag. Digital signatures solve this by binding the authentication to an asymmetric key pair: only the private-key holder can sign, but anyone with the public key can verify. This property—*non-repudiation*—makes digital signatures the foundation of software distribution, financial transactions, and, as we shall see in Section 3.4, the certificate chains that underpin internet trust.

3.3.1 Syntax and Security

Before examining concrete schemes, we fix the interface that every signature scheme must implement and the security property it must satisfy.

Definition 3.9 (Digital Signature Scheme) A *signature scheme* is a triple $(\text{KeyGen}, \text{Sign}, \text{Verify})$:

- $\text{KeyGen}() \rightarrow (pk, sk)$: generates a key pair.
- $\text{Sign}(sk, m) \rightarrow \sigma$: produces a signature on message m .
- $\text{Verify}(pk, m, \sigma) \rightarrow \{0, 1\}$: outputs 1 if and only if σ is a valid signature on m under pk .

Definition 3.10 (EUF-CMA Security) A signature scheme is *existentially unforgeable under chosen-message attack* (EUF-CMA) if no efficient adversary, even after obtaining signatures on polynomially many messages of its choice, can produce a valid signature on a new message.

EUF-CMA is the standard security notion for signatures in practice; all schemes discussed below target this level of security.

3.3.2 RSA Signatures and RSA-PSS

The RSA trapdoor function lends itself naturally to signatures: because only the private-key holder can compute d -th powers modulo n , a modular exponentiation with d serves as proof of identity. The textbook RSA signature on message m is $\sigma = H(m)^d \bmod n$, verified by checking $\sigma^e \equiv H(m) \pmod{n}$. As with RSA encryption, the textbook version is insecure due to malleability. The standard secure variant is *RSA-PSS* (Probabilistic Signature Scheme), which incorporates randomised padding and is provably EUF-CMA secure in the random oracle model under the RSA assumption.

3.3.3 Schnorr Signatures and the Fiat–Shamir Heuristic

RSA-PSS derives its security from the hardness of factoring. An entirely different design path begins not with a trapdoor function but with an *identification protocol*—an interactive proof that one knows a secret—and then compiles it into a signature scheme. Schnorr signatures are the cleanest example of this approach, and the construction paradigm they introduce—the Fiat–Shamir heuristic—recurs throughout post-quantum cryptography.

Definition 3.11 (Schnorr Signature) Let $G = \langle g \rangle$ be a group of prime order q in which the discrete logarithm is hard. The signer’s key pair is $(pk, sk) = (g^x, x)$.

1. **Sign:** choose $r \xleftarrow{\$} \mathbb{Z}_q$; compute $R = g^r$, $e = H(R||m)$, and $s = r - ex \bmod q$. Output $\sigma = (e, s)$.

2. **Verify:** compute $R' = g^s \cdot pk^e$ and accept if $H(R' || m) = e$.

Schnorr signatures are derived from a three-move identification protocol (commit R , challenge e , respond s) via the *Fiat–Shamir heuristic* [17]: the verifier’s random challenge is replaced by the hash $H(R || m)$. This transform converts any honest-verifier zero-knowledge proof of knowledge of a discrete logarithm into a non-interactive signature scheme. The Fiat–Shamir heuristic will appear again in the construction of lattice-based signatures (Chapter 8).

3.3.4 ECDSA

Just as elliptic curves shrank Diffie–Hellman key sizes by an order of magnitude, they do the same for signatures. ECDSA combines the Schnorr-style “commit-challenge-respond” structure with the compact group arithmetic of elliptic curves, producing signatures that are both shorter and faster to verify than their RSA counterparts.

Definition 3.12 (ECDSA) Let $E(\mathbb{F}_p)$ be an elliptic curve with base point G of order n . The signer’s key pair is $(pk, sk) = (Q = dG, d)$.

1. **Sign:** choose ephemeral $k \xleftarrow{\$} \{1, \dots, n-1\}$; compute $(x_1, y_1) = kG$; set $r = x_1 \bmod n$; compute $s = k^{-1}(H(m) + rd) \bmod n$. Output $\sigma = (r, s)$.
2. **Verify:** compute $u_1 = H(m)s^{-1} \bmod n$, $u_2 = rs^{-1} \bmod n$; accept if the x -coordinate of $u_1G + u_2Q$ equals $r \bmod n$.

ECDSA is the workhorse signature algorithm in TLS 1.3, code signing, and cryptocurrency (Bitcoin, Ethereum). A critical implementation requirement is that the ephemeral scalar k must be generated with high-quality randomness: reuse of k across two signatures reveals the private key d by simple algebra. This vulnerability caused the 2010 PlayStation 3 private key extraction.

Quantum vulnerability of signatures. Every signature scheme in this section—RSA-PSS, Schnorr, ECDSA—derives its security from the hardness of factoring or discrete logarithms. Shor’s algorithm breaks all of them. Forging a signature allows an adversary to impersonate any server, sign malicious software as trusted vendors, or create fraudulent certificates. Post-quantum signature schemes (Chapters 8 and ??) must preserve the EUF-CMA guarantee using quantum-resistant assumptions.

3.4 Public Key Infrastructure

Public-key cryptography solves key distribution but introduces a new problem: how does one verify that a given public key genuinely belongs to its claimed owner? *Public Key Infrastructure* (PKI) answers this question at internet scale.

3.4.1 Certificate Authorities and Chains of Trust

The answer is delegation: a small number of universally trusted entities vouch for the keys of others, who in turn vouch for still others. PKI organises this delegation hierarchically. A small number of *root Certificate Authorities* (CAs) are pre-installed in every browser and operating system. Root CAs sign the certificates of *intermediate CAs*, which in turn sign *end-entity certificates* for individual domains. This produces a *certificate chain*:

$$\text{Root CA} \xrightarrow{\text{signs}} \text{Intermediate CA} \xrightarrow{\text{signs}} \text{End-entity (e.g., github.com)}.$$

Verification proceeds bottom-up: the relying party checks each signature in the chain until reaching a root that it trusts.

3.4.2 X.509 Certificates

A chain of trust is only as precise as the documents it links. The standard format for these documents is the X.509 certificate, whose fields have been refined over three decades to encode exactly the information a relying party needs to decide whether to trust a public key.

Definition 3.13 (X.509 Certificate) An X.509 certificate binds a public key to an identity. The *Subject* field names the entity being certified (e.g., CN=github.com), while the *Issuer* field identifies the CA that vouches for this binding—together, they answer *who* holds the key and *who* says so. The *Public key* field contains the subject’s actual cryptographic key (RSA, ECDSA, or another algorithm), which is the whole point of the certificate: to distribute this key authentically. The *Validity period* limits the window of trust by specifying start and expiry dates, ensuring that compromised or outdated keys eventually expire even if revocation mechanisms fail. The *Signature* field carries the issuer’s digital signature over all other fields, making any tampering detectable—this is the cryptographic glue that holds the chain together. Finally, the *Extensions* encode policy constraints: whether the key may be used for signing, encryption, or both; alternative names under which the subject operates; and whether the certificate may itself issue further certificates. Extensions transform X.509 from a simple key–identity binding into a flexible policy language for large-scale trust management.

Certificate chain validation verifies, for each certificate C_i in the chain: (i) the current time falls within the validity period; (ii) C_{i+1} has signed C_i ; (iii) C_i has not been revoked (checked via OCSP or CRL); and (iv) key-usage extensions permit the intended operation. The chain terminates at a self-signed root certificate present in the local trust store.

3.4.3 Failures and Lessons

The X.509 model is elegant in theory, but it places extraordinary trust in certificate authorities—a trust that has, on occasion, been catastrophically misplaced.

Example 3.2 (DigiNotar Collapse, 2011) Iranian attackers compromised the Dutch CA DigiNotar and issued fraudulent certificates for *.google.com, enabling man-in-the-middle surveillance of Iranian citizens. DigiNotar declared bankruptcy within weeks; the Dutch government, whose sites relied on DigiNotar certificates, faced a crisis. The incident accelerated the deployment of *Certificate Transparency* logs, which provide a publicly auditable record of all issued certificates.

PKI currently secures over 200 million active certificates serving more than 4 billion users. Its entire trust hierarchy depends on digital signatures (RSA or ECDSA). When a quantum computer can forge these signatures, any adversary can fabricate certificates for arbitrary domains, impersonate certificate authorities, and collapse the chain of trust entirely.

3.5 The TLS Protocol

Transport Layer Security (TLS) weaves together every primitive discussed so far—symmetric encryption, hash functions, digital signatures, key exchange, and certificates—into a single protocol that secures virtually all internet communications. TLS 1.3 (RFC 8446, 2018) is the current standard.

3.5.1 The TLS 1.3 Handshake

The protocol achieves its security guarantees through a carefully choreographed exchange called the *handshake*, which establishes a shared symmetric key, authenticates the server, and protects against downgrade and replay attacks—all within a single network round trip. A full TLS 1.3 handshake (1-RTT) proceeds as follows.

1. **ClientHello (Client → Server).** The client sends supported cipher suites, a random nonce, and—crucially—an ECDHE key share (e.g., its X25519 public key).

2. **ServerHello (Server → Client).** The server selects a cipher suite and responds with its own ECDHE key share and random nonce. At this point, both sides can compute the ECDHE shared secret.
3. **Key derivation.** Both sides derive multiple traffic secrets from the shared secret using HKDF:

$$\text{Handshake Secret} = \text{HKDF-Extract}(\text{ECDHE shared secret}), \quad (3.4)$$

$$\text{Traffic keys} = \text{HKDF-Expand}(\text{Handshake Secret}, \text{transcript hash}). \quad (3.5)$$

From this point, all subsequent handshake messages are encrypted.

4. **Certificate (Server → Client, encrypted).** The server sends its certificate chain. Unlike TLS 1.2, the certificate is transmitted under encryption, preventing passive observers from identifying the server.
5. **CertificateVerify (Server → Client, encrypted).** The server signs the handshake transcript with its private key, proving possession of the key certified in the previous step. The signature covers all negotiated parameters, preventing downgrade and replay attacks.
6. **Finished messages (both directions, encrypted).** Each side sends an HMAC over the complete handshake transcript, providing key confirmation.
7. **Application data.** Bulk data is encrypted with AES-256-GCM (or ChaCha20-Poly1305) using the derived application traffic keys.

Figure 3.3 visualises this message flow, distinguishing plaintext, encrypted, and signed messages.

3.5.2 Cipher Suites

The handshake begins with a negotiation: client and server must agree on which algorithms to use. TLS 1.3 dramatically simplified this step compared to its predecessors. Only five AEAD cipher suites are defined; the most common is:

TLS_AES_256_GCM_SHA384

which specifies AES-256-GCM for encryption and SHA-384 for the HKDF-based key schedule. The key exchange algorithm (e.g., X25519 or P-256) and signature algorithm (e.g., ECDSA or RSA-PSS) are negotiated separately through extensions.

3.5.3 Where Each Primitive Is Used

With the handshake and cipher suites in place, we can now map every cryptographic primitive from this chapter to its role in TLS—and, critically, assess which components fall to quantum attack. Table 3.2 provides this summary.

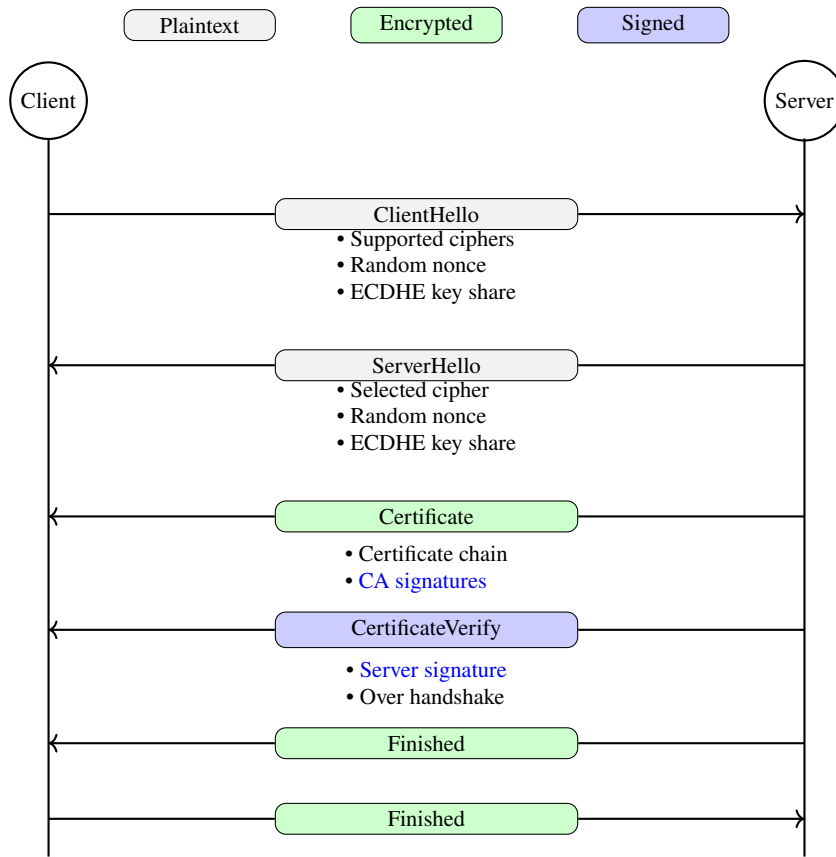


Fig. 3.3 TLS 1.3 handshake message flow. After the initial plaintext exchange, all subsequent messages are encrypted. The signed CertificateVerify message authenticates the server.

Table 3.2 Cryptographic primitives in a TLS 1.3 handshake and their quantum vulnerability

Primitive	Role in TLS 1.3	Type	Quantum threat
ECDHE (X25519)	Key exchange	Asymmetric	Shor (broken)
ECDSA / RSA-PSS	Certificate signatures	Asymmetric	Shor (broken)
ECDSA / RSA-PSS	CertificateVerify	Asymmetric	Shor (broken)
AES-256-GCM	Bulk encryption	Symmetric	Grover (mitigated)
SHA-384 / HMAC	Key derivation, Finished	Symmetric	Grover (mitigated)

The critical observation is that every asymmetric operation—key exchange, certificate chain verification, and handshake authentication—falls to Shor’s algorithm. The symmetric components survive with increased key lengths, but without functioning public-key cryptography, the symmetric keys cannot be established securely in the first place.

The post-quantum migration challenge. Replacing TLS’s asymmetric components with post-quantum alternatives is not merely a cryptographic exercise—it is a coordination problem of enormous scale. Every browser, server, operating system, and IoT device must be updated. Post-quantum key encapsulation mechanisms (e.g., ML-KEM, Chapter 7) produce larger public keys and ciphertexts, increasing handshake sizes from ~4 KB to ~15 KB. At the scale of 300 billion daily HTTPS connections, these increases have significant bandwidth implications.

3.6 The Computational Hardness Foundation

The entire public-key infrastructure described above—RSA, Diffie–Hellman, ECDSA, TLS, PKI—rests on exactly three computational hardness assumptions. This mathematical monoculture is both the field’s greatest success and its most critical vulnerability.

3.6.1 The Integer Factorisation Problem

We begin with the assumption that underpins RSA. The security argument is simple: anyone can multiply two primes, but no one knows how to efficiently reverse the operation.

Definition 3.14 (Integer Factorisation Problem (IFP)) Given a composite integer $n = pq$, where p and q are large primes, find the factors p and q .

The asymmetry is dramatic: multiplication $n = pq$ runs in $O((\log n)^2)$ time, while the best classical factoring algorithm—the *General Number Field Sieve* (GNFS)—runs in sub-exponential time:

$$T_{\text{GNFS}}(n) = \exp\left(c (\log n)^{1/3} (\log \log n)^{2/3}\right), \quad c \approx 1.923. \quad (3.6)$$

For RSA-2048, this amounts to roughly 2^{112} operations—well beyond reach.

The security of RSA relies on a slightly stronger assumption: not merely that factoring n is hard, but that computing e -th roots modulo n (the *RSA problem*) is hard. It is known that factoring reduces to the RSA problem, but the converse is open.

3.6.2 The Discrete Logarithm Problem

Where RSA relies on the difficulty of factoring, Diffie–Hellman and DSA rely on a different one-way phenomenon: modular exponentiation is easy, but extracting the exponent—computing a discrete logarithm—appears to be hard.

Definition 3.15 (Discrete Logarithm Problem (DLP)) Given a cyclic group $G = \langle g \rangle$ of order q and an element $h \in G$, find the integer $x \in \{0, \dots, q-1\}$ such that $g^x = h$.

In the multiplicative group $\mathbb{Z}_p^*$, forward exponentiation ($g^x \bmod p$) costs $O(\log x)$ multiplications, while the best classical discrete-log algorithm (index calculus) is sub-exponential. For groups of prime order embedded in $\mathbb{Z}_p^*$, the DLP underpins the Diffie–Hellman protocol and DSA.

Knowing that discrete logarithms are hard does not, by itself, suffice for proving protocols secure. The DLP says that recovering a from g^a is infeasible, but Diffie–Hellman key exchange never asks an adversary to find a —it asks whether the adversary can compute g^{ab} from the pair (g^a, g^b) . This is a potentially easier task: perhaps g^{ab} can be computed without finding either a or b . Conversely, some protocols require an even stronger guarantee: not just that g^{ab} cannot be *computed*, but that it cannot even be *distinguished* from a random group element. These operational differences motivate two refinements of the DLP.

Definition 3.16 (CDH and DDH Assumptions) Let $G = \langle g \rangle$ be a cyclic group of prime order q .

1. **Computational Diffie–Hellman (CDH):** Given (g, g^a, g^b) for random $a, b \xleftarrow{\$} \mathbb{Z}_q$, it is infeasible to compute g^{ab} .
2. **Decisional Diffie–Hellman (DDH):** The distributions (g^a, g^b, g^{ab}) and (g^a, g^b, g^c) are computationally indistinguishable for random $a, b, c \xleftarrow{\$} \mathbb{Z}_q$.

These assumptions form a strict hierarchy: if one can solve the DLP, one can certainly compute g^{ab} (just recover a and exponentiate), so $\text{DLP} \Rightarrow \text{CDH}$. Similarly, if one can compute g^{ab} , one can distinguish it from a random element, so $\text{CDH} \Rightarrow \text{DDH}$. The converses are not known to hold, meaning DDH is the weakest—and therefore most conservative—assumption of the three. In practice, the security of Diffie–Hellman key exchange relies on CDH (the adversary must compute the shared secret), while the semantic security of ElGamal encryption relies on DDH (the adversary must not even learn partial information about the plaintext).

3.6.3 The Elliptic Curve Discrete Logarithm Problem

The DLP, CDH, and DDH can be stated in any cyclic group. When the group is $\mathbb{Z}_p^*$, sub-exponential index-calculus algorithms exist, forcing the use of multi-thousand-bit primes. Elliptic curve groups resist these algorithms entirely, which is why they deliver equivalent security at a fraction of the key size. The formal hardness assumption is the natural analogue of the DLP, transplanted to the curve.

Definition 3.17 (Elliptic Curve Discrete Logarithm Problem (ECDLP)) Given an elliptic curve E over $\mathbb{F}_p$, a base point $G \in E(\mathbb{F}_p)$ of order n , and a point $P = kG$, find the integer k .

The best classical algorithm for ECDLP is Pollard’s rho method, which runs in $O(\sqrt{n})$ time—fully exponential in the key length. There is no analogue of index calculus for generic elliptic curves, which is why 256-bit ECC achieves 128-bit security whereas 3072-bit RSA is needed for the same level (Table 3.1). This efficiency advantage has made ECC the default for internet-scale cryptography.

3.6.4 The Monoculture Risk

Three problems, three different-looking mathematical structures—yet beneath the surface, they are uncomfortably alike. All three—IFP, DLP, ECDLP—share a deep structural property: they reduce to instances of the *hidden subgroup problem* (HSP) over abelian groups. Shor’s algorithm solves the HSP in polynomial time on a quantum computer [59], which means:

Theorem 3.2 (Quantum Collapse of Classical Assumptions) *A sufficiently large quantum computer running Shor’s algorithm solves:*

1. IFP (factoring an n -bit integer) in $O(n^3)$ quantum gates.
2. DLP in $\mathbb{Z}_p^*$ in $O((\log p)^3)$ quantum gates.
3. ECDLP over $E(\mathbb{F}_p)$ in $O((\log p)^3)$ quantum gates.

The mathematical monoculture means that a single algorithmic advance—Shor’s algorithm—simultaneously demolishes factoring, discrete logarithms, and elliptic curve discrete logarithms. Every protocol examined in this chapter collapses: RSA encryption and signatures, Diffie–Hellman key exchange, ECDSA, the certificate chains of PKI, and the asymmetric core of TLS.

Why “bigger keys” cannot help. Against classical attacks, doubling the RSA modulus roughly squares the attack cost; this scaling has kept RSA viable for decades. Against Shor’s algorithm, the attack cost grows only polynomially with key size. No feasible key length can bridge an exponential-to-polynomial collapse. The only path forward is to replace the underlying mathematical problems with ones that resist quantum attack—the subject of the remainder of this book.

Problems

3.1 Construct an RSA key pair with the small primes $p = 61$ and $q = 53$.

- (a) Compute n , $\phi(n)$, and choose a suitable public exponent e . Find d .
- (b) Encrypt and decrypt the message $m = 42$ to verify correctness.
- (c) Explain why this key pair would be trivially broken and estimate the security of RSA-2048 using (3.6).

3.2 Alice and Bob execute Diffie–Hellman with $p = 23$ and $g = 5$. Alice chooses $a = 6$ and Bob chooses $b = 15$.

- (a) Compute the public values A and B , and verify that both parties derive the same shared secret K .
- (b) An eavesdropper Eve observes (p, g, A, B) . For this toy example, find K by brute force. Explain why this approach is infeasible for 2048-bit primes.

3.3 Enumerate every cryptographic operation in a TLS 1.3 handshake using the cipher suite TLS_AES_256_GCM_SHA384 with X25519 key exchange and ECDSA certificates. For each operation, state whether it is vulnerable to Shor’s algorithm, Grover’s algorithm, or neither, and justify your answer.

3.4 In ECDSA (Definition 3.12), suppose two messages $m_1 \neq m_2$ are signed using the *same* ephemeral key k . Show that an adversary who observes the two signatures $\sigma_1 = (r, s_1)$ and $\sigma_2 = (r, s_2)$ can recover the long-term private key d . Derive the formula explicitly and explain the practical consequences.

3.5 Consider the Schnorr identification protocol (the interactive version of Definition 3.11).

- (a) Describe the three-move interaction (commitment, challenge, response) and argue that the protocol is honest-verifier zero-knowledge.
- (b) Explain how the Fiat–Shamir heuristic [17] transforms this interactive protocol into a non-interactive signature scheme.
- (c) Why does Fiat–Shamir require a collision-resistant hash function? What happens if the hash function is weak?

Chapter 4

Quantum Algorithms and Cryptanalysis

Abstract This chapter provides a precise treatment of the two quantum algorithms that reshape the cryptographic landscape: Shor’s algorithm, which renders integer factorization and discrete logarithms tractable, and Grover’s algorithm, which provides a quadratic speedup for unstructured search. We analyze the quantum resources required to break deployed cryptographic parameters and establish the boundary between quantum-vulnerable and quantum-resistant problems.

4.1 The Quantum Computing Model

We assume the reader is familiar with the postulates of quantum mechanics and restrict ourselves to the computational essentials needed for the algorithms that follow. Standard references for the full circuit model include Nielsen and Chuang [53].

4.1.1 Qubits, Gates, and Circuits

A *qubit* is a two-dimensional quantum system with computational basis states $|0\rangle$ and $|1\rangle$. An n -qubit register lives in the Hilbert space $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$; a general state is

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1. \quad (4.1)$$

Computation proceeds by applying *unitary gates* to this register. A universal gate set consists of the single-qubit Hadamard H , the phase gate T , and the two-qubit controlled-NOT (CNOT):

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}, \quad \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (4.2)$$

Any n -qubit unitary can be decomposed into a circuit of $O(\text{poly}(n))$ gates from this set (the Solovay–Kitaev theorem). Measurement in the computational basis yields outcome x with probability $|\alpha_x|^2$ and collapses the state to $|x\rangle$.

4.1.2 The Quantum Fourier Transform

The *Quantum Fourier Transform* (QFT) over $\mathbb{Z}_M$ maps computational basis states as

$$\text{QFT} |x\rangle = \frac{1}{\sqrt{M}} \sum_{y=0}^{M-1} e^{2\pi i xy/M} |y\rangle. \quad (4.3)$$

When $M = 2^m$, the QFT decomposes into a product of $m(m-1)/2$ controlled-rotation gates and m Hadamard gates, giving a circuit of depth $O(m^2) = O(\log^2 M)$. This exponential improvement over the classical Fast Fourier Transform—which requires $O(M \log M)$ operations—is the engine behind Shor’s algorithm.

The physical intuition is direct: the QFT acts as a *computational diffraction grating*. Just as an optical grating converts spatial periodicity into sharp peaks in the frequency domain, the QFT converts a periodic quantum state into sharp peaks at multiples of the reciprocal period. This analogy will become precise in Sect. 4.2.

4.1.3 Computational Models

The *quantum circuit model* described above is equivalent in computational power to the adiabatic model and the measurement-based (cluster-state) model. All results in this chapter are stated in the circuit model. We measure algorithmic cost by the number of elementary gates (the *gate complexity*) and the number of qubits (the *space complexity*).

With the computational model in place, we can state precisely what quantum algorithms achieve—and the first result is the one that upended public-key cryptography.

4.2 Shor’s Algorithm

In 1994, Peter Shor demonstrated that a quantum computer can factor integers and compute discrete logarithms in polynomial time [58, 59]. The result is not a brute-force parallel search—a widespread misconception—but an elegant reduction of factoring to period finding, followed by quantum interference to extract the period.

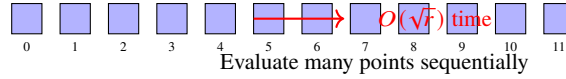
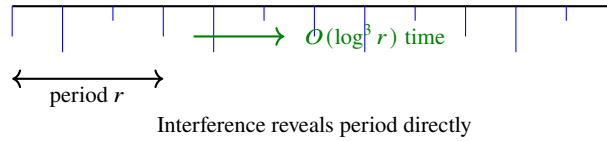
Classical Period Finding:**Quantum Period Finding:**

Fig. 4.1 Classical sampling vs. quantum interference for period finding. The classical approach evaluates f at many points sequentially, requiring $O(\sqrt{r})$ time. The quantum approach uses interference to reveal the period directly in $O(\log^3 r)$ time.

4.2.1 The Period-Finding Problem

Definition 4.1 (Period Finding) Given a function $f: \mathbb{Z} \rightarrow S$ with the promise that there exists a smallest positive integer r such that $f(x) = f(x + r)$ for all x , find r .

Classically, period finding requires $\Omega(\sqrt{r})$ evaluations of f (via birthday-type arguments). The quantum algorithm solves it in $O(\log^3 r)$ operations, an exponential improvement. Figure 4.1 contrasts the two approaches.

4.2.2 Reduction of Factoring to Period Finding

Theorem 4.1 (Factoring Reduces to Order Finding [59]) Let $N = pq$ be a product of two distinct odd primes. The following probabilistic procedure returns a non-trivial factor of N with probability at least $1/2$:

1. Choose a random $a \in \{2, \dots, N-1\}$ with $\gcd(a, N) = 1$.
2. Find the order r of a modulo N , i.e., the period of $f(x) = a^x \bmod N$.
3. If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then

$$\gcd(a^{r/2} - 1, N) \quad \text{or} \quad \gcd(a^{r/2} + 1, N) \quad (4.4)$$

is a non-trivial factor of N .

Proof sketch. Since $a^r \equiv 1 \pmod{N}$, we have $(a^{r/2})^2 - 1 \equiv 0 \pmod{N}$, so N divides $(a^{r/2} - 1)(a^{r/2} + 1)$. If neither factor is divisible by N (guaranteed by the condition $a^{r/2} \not\equiv \pm 1$), then N shares a non-trivial factor with each. A counting argument over the Chinese Remainder Theorem shows that a random a satisfies both conditions with probability at least $1/2$. $\square$

Algorithm 1 Quantum Order Finding (core of Shor’s algorithm)**Require:** Integer N , base a with $\gcd(a, N) = 1$ **Ensure:** Order r of a modulo N

- 1: Initialise $|0\rangle^{\otimes m} |0\rangle^{\otimes n}$
- 2: Apply $H^{\otimes m}$ to the first register: $\frac{1}{\sqrt{2^m}} \sum_{x=0}^{2^m-1} |x\rangle |0\rangle$
- 3: Compute modular exponentiation into the second register: $\frac{1}{\sqrt{2^m}} \sum_{x=0}^{2^m-1} |x\rangle |a^x \bmod N\rangle$
- 4: Apply $\text{QFT}_{2^m}^\dagger$ to the first register
- 5: Measure the first register to obtain y
- 6: Use the continued-fractions algorithm on $y/2^m$ to extract r

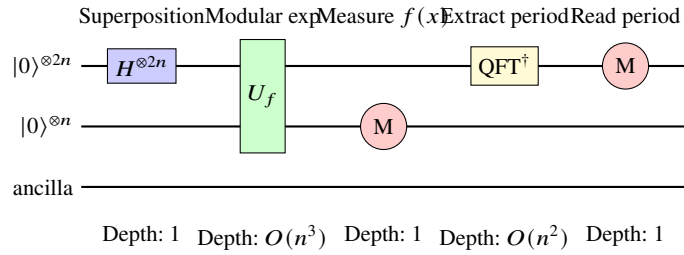


Fig. 4.2 High-level quantum circuit for Shor’s algorithm with depth annotations. The modular exponentiation U_f dominates the circuit depth at $O(n^3)$ gates, while the inverse QFT contributes $O(n^2)$.

The entire classical part—choosing a , computing GCDs, testing conditions—runs in $O(\log^2 N)$ time. The bottleneck is step 2: finding the order r . This is where the quantum computer intervenes.

4.2.3 The Quantum Circuit

Let $n = \lceil \log_2 N \rceil$ and $m = 2n$. Shor’s algorithm uses two quantum registers:

The high-level circuit structure is shown in Figure 4.2. The key insight is interference. After step 3, the first register is entangled with periodic structure: for each value $f_0 = a^{x_0} \bmod N$, the amplitudes are non-zero only at $x_0, x_0 + r, x_0 + 2r, \dots$. The inverse QFT in step 4 converts this periodic superposition into sharp peaks at multiples of $2^m/r$, exactly as a diffraction grating produces Bragg peaks at multiples of the reciprocal lattice spacing.

Theorem 4.2 (Correctness of Quantum Order Finding [59]) *The measurement outcome y in step 5 satisfies $|y/2^m - j/r| \leq 1/2^{m+1}$ for some integer $0 \leq j < r$, with probability $\Omega(1/\log \log r)$. The continued-fractions algorithm recovers r from $y/2^m$ whenever $\gcd(j, r) = 1$, which occurs with probability $\Omega(1/\log \log r)$. Repeating $O(\log \log N)$ times yields r with high probability.*

The total gate complexity is dominated by modular exponentiation, which requires $O(n^3)$ gates using standard reversible arithmetic. The qubit count is $3n + O(\log n)$ logical qubits.

Example 4.1 (Factoring $N = 15$ with $a = 7$) We trace the algorithm step by step:

1. The function $f(x) = 7^x \bmod 15$ has values $1, 7, 4, 13, 1, 7, 4, 13, \dots$ with period $r = 4$.
2. The quantum circuit with $m = 8$ creates a superposition over all $x \in \{0, \dots, 255\}$, computes $7^x \bmod 15$ in the second register, and applies $\text{QFT}_{256}^\dagger$.
3. Measurement of the first register yields $y \in \{0, 64, 128, 192\}$ (peaks at multiples of $256/4 = 64$).
4. Continued fractions on $y/256$: for $y = 64$, we get $64/256 = 1/4$, giving $r = 4$.
5. Classical post-processing: $r = 4$ is even, $7^{4/2} = 49 \equiv 4 \pmod{15}$, and $4 \not\equiv -1$. Thus $\gcd(48, 15) = 3$ and $\gcd(50, 15) = 5$.

Result: $15 = 3 \times 5$.

4.2.4 Extension to the Discrete Logarithm Problem

Shor's algorithm extends to the *discrete logarithm problem* (DLP) in any finite abelian group [59]. Given a group $G = \langle g \rangle$ of order q and an element $h = g^s$, we seek the exponent s . The quantum approach uses a two-register generalisation:

1. Prepare $\frac{1}{q} \sum_{x,y=0}^{q-1} |x\rangle |y\rangle |g^x h^y\rangle$.
2. The function $f(x, y) = g^x h^y = g^{x+sy}$ has a two-dimensional periodicity structure: $f(x, y) = f(x', y')$ if and only if $x + sy \equiv x' + sy' \pmod{q}$.
3. Applying $\text{QFT}_q^{\otimes 2}$ to the first two registers and measuring yields a pair (u, v) satisfying $u + sv \equiv 0 \pmod{q}$, from which $s \equiv -uv^{-1} \pmod{q}$.

For *elliptic curve* groups, the same approach applies with the group operation replaced by point addition. The gate complexity for the ECDLP on a curve over $\mathbb{F}_p$ is $O(n^3)$ where $n = \lceil \log_2 p \rceil$, making ECDSA-256 and ECDH-256 vulnerable to a quantum computer with approximately 2,500 logical qubits.

The diffraction analogy. Shor's algorithm is a computational diffraction experiment. The modular exponentiation creates a state with hidden periodicity—analogue to a crystal with unknown lattice spacing. The QFT acts as a diffraction grating, producing sharp peaks in the “momentum” (frequency) basis at multiples of the reciprocal period. Measurement reads off one of these Bragg peaks, and classical post-processing (continued fractions) extracts the period. The exponential speedup comes from the fact that the QFT circuit has depth $O(n^2)$, while a classical DFT over the same space would require $O(2^n \cdot n)$ operations.

Shor's algorithm settles the fate of RSA, finite-field Diffie–Hellman, and elliptic-curve cryptography: all fall in polynomial time. But what about problems that lack the algebraic periodicity Shor exploits? The next section addresses the best quantum strategy for the hardest remaining case—pure brute-force search.

Algorithm 2 Grover's Algorithm

Require: Oracle O_f with $O_f |x\rangle = (-1)^{f(x)} |x\rangle$, search space size $N = 2^n$

Ensure: Solution x^* with $f(x^*) = 1$

1: Initialise $|\psi\rangle = H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$

2: Set $k = \lfloor \frac{\pi}{4} \sqrt{N} \rfloor$

3: **for** $i = 1$ to k **do**

4: Apply oracle: $|\psi\rangle \leftarrow O_f |\psi\rangle$

5: Apply diffusion: $|\psi\rangle \leftarrow D |\psi\rangle$

6: **end for**

7: Measure $|\psi\rangle$ in the computational basis to obtain x

8: **if** $f(x) = 1$ **then return** x **else repeat**

4.3 Grover's Algorithm and Amplitude Amplification

While Shor's algorithm provides an *exponential* speedup for structured algebraic problems, Grover's algorithm [25] provides a *quadratic* speedup for the most general computational task: unstructured search. The distinction is critical for understanding which parts of cryptography survive the quantum era.

4.3.1 The Unstructured Search Problem

Definition 4.2 (Unstructured Search) Given oracle access to a function $f: \{0, 1\}^n \rightarrow \{0, 1\}$ where exactly one input x^* satisfies $f(x^*) = 1$, find x^* .

Classically, any deterministic algorithm must evaluate f at least $N - 1$ times in the worst case (where $N = 2^n$), and any randomised algorithm requires $\Omega(N)$ evaluations on average. Grover's algorithm finds x^* using only $O(\sqrt{N})$ quantum queries.

4.3.2 The Grover Iteration

The algorithm operates in the two-dimensional subspace spanned by the target state $|x^*\rangle$ and the uniform superposition over non-target states. Define:

- The *oracle operator*: $O_f |x\rangle = (-1)^{f(x)} |x\rangle$ (phase flip on the target).
- The *diffusion operator*: $D = 2 |s\rangle \langle s| - I$, where $|s\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle$ is the uniform superposition.

The *Grover operator* $G = D \cdot O_f$ is a rotation in the two-dimensional subspace by angle 2θ , where $\theta = \arcsin(1/\sqrt{N})$.

After k iterations, the amplitude of $|x^*\rangle$ is

$$\alpha_k = \sin((2k + 1)\theta), \quad (4.5)$$

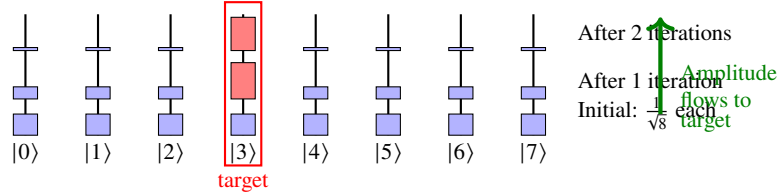
Amplitude Evolution:

Fig. 4.3 Grover's algorithm amplifies the target state's amplitude through constructive interference. Non-target amplitudes decrease while the marked state's amplitude grows with each iteration.

which reaches its maximum $\alpha_k \approx 1$ when $k \approx \frac{\pi}{4} \sqrt{N}$. The geometric picture is a rotation: each Grover iteration rotates the state vector by 2θ towards $|x^*\rangle$ in the target/non-target plane, and $\pi/(4\theta) \approx \frac{\pi}{4} \sqrt{N}$ rotations bring it to the target. Figure 4.3 illustrates how the target state's amplitude grows with each iteration.

The physics analogy is resonance: the Grover iteration is a driven oscillation that builds up amplitude at the target state through constructive interference while suppressing non-target amplitudes through destructive interference.

Example 4.2 (Breaking a 128-bit Key) Consider applying Grover's algorithm to search for a 128-bit AES key. Classical brute force requires $2^{128} \approx 3.4 \times 10^{38}$ evaluations—utterly infeasible. Grover's algorithm reduces this to $\frac{\pi}{4} \cdot 2^{64} \approx 1.4 \times 10^{19}$ oracle calls, where each call implements a reversible AES circuit that checks whether a candidate key produces the known plaintext–ciphertext pair. The upshot: AES-128 provides only 64-bit security against a quantum adversary.

4.3.3 Optimality: The BBBV Lower Bound

A natural question is whether a smarter quantum algorithm could do better than $O(\sqrt{N})$. The answer is no.

Theorem 4.3 (BBBV Lower Bound [3]) *Any quantum algorithm that finds a marked item in an unstructured database of size N with probability at least $2/3$ must make $\Omega(\sqrt{N})$ quantum queries to the oracle.*

Proof sketch. The argument proceeds by a *hybrid method*. Consider two oracles O_f and $O_{f'}$ that differ on a single input x^* . After k queries, the states produced by the two oracles differ by at most $O(k/\sqrt{N})$ in trace distance (because each query can shift the state by at most $O(1/\sqrt{N})$ when the marked item has amplitude $O(1/\sqrt{N})$). For the algorithm to distinguish the two oracles with constant probability, we need $k = \Omega(\sqrt{N})$. $\square$

Table 4.1 Grover’s impact on symmetric primitives

Primitive	Classical security	Quantum security	Post-quantum safe?
AES-128	2^{128}	2^{64}	No
AES-256	2^{256}	2^{128}	Yes
SHA-256 (preimage)	2^{256}	2^{128}	Yes
SHA-256 (collision)	2^{128}	2^{85}	Marginal
SHA-384 (collision)	2^{192}	2^{128}	Yes

This optimality result has a profound consequence: the quadratic speedup for unstructured search is the best quantum mechanics can offer. No future algorithmic breakthrough will improve upon it in the oracle model.

4.3.4 Amplitude Amplification

The BBBV bound closes the door on beating $O(\sqrt{N})$ for unstructured search, but it leaves open a powerful generalisation. Grover’s algorithm generalises to *amplitude amplification*: given any quantum algorithm $\mathcal{A}$ that produces a “good” outcome with probability p , we can boost the success probability to near 1 using $O(1/\sqrt{p})$ repetitions of $\mathcal{A}$ and $\mathcal{A}^{-1}$, rather than the classical $O(1/p)$. This is a general-purpose tool that appears throughout quantum algorithm design.

4.3.5 Impact on Symmetric Cryptography

With Grover’s quadratic speedup established—and its optimality proven—we can now quantify its impact on deployed symmetric primitives. The effect is a uniform halving of the effective security level:

The mitigation is straightforward: double the key length. AES-256 retains 128-bit security against quantum adversaries, and SHA-384 or SHA-512 provide adequate collision resistance. This stands in sharp contrast to public-key cryptography, where no increase in key size can restore security against Shor’s algorithm—RSA with a million-bit modulus is still broken in polynomial time.

The algorithmic picture is now clear: Shor breaks public-key primitives and Grover halves symmetric key lengths. What remains is a hardware question—how close are we to machines that can actually execute these algorithms?

4.4 Quantum Resource Estimates

Understanding *when* quantum computers will threaten deployed cryptography requires translating algorithmic gate counts into physical hardware requirements. The gap between logical algorithms and physical reality is vast.

4.4.1 Logical Resource Requirements

For an n -bit cryptographic target, the logical resources for Shor’s algorithm are:

- **Logical qubits:** $3n + O(\log n)$ for the standard textbook circuit, reducible to $2n + O(\log n)$ with space-optimised arithmetic [22].
- **Toffoli gates:** $O(n^3)$, dominated by modular exponentiation.
- **T-gates:** Each Toffoli decomposes into 7 T-gates in the Clifford+T basis, yielding $O(n^3)$ T-gates total.

4.4.2 Physical Overhead: Surface Codes

Fault-tolerant quantum computation using the surface code introduces a multiplicative overhead in physical qubits. Each logical qubit is encoded in a patch of $d \times d$ physical qubits, where d is the *code distance* determined by the target logical error rate p_L and the physical error rate p_{phys} :

$$p_L \approx 0.1 \left(\frac{p_{\text{phys}}}{p_{\text{thresh}}} \right)^{\lfloor (d+1)/2 \rfloor}, \quad (4.6)$$

where $p_{\text{thresh}} \approx 1\%$ is the surface code threshold. For $p_{\text{phys}} = 10^{-3}$, a code distance $d = 27$ suffices for the logical error rates needed to factor RSA-2048.

T-gate execution requires *magic state distillation*, which consumes additional physical qubits—typically 10–20% overhead on top of the data qubits.

4.4.3 Concrete Estimates

Table 4.2 summarises the best known resource estimates, primarily from the landmark analysis of Gidney and Ekerå [22].

The physical qubit counts assume superconducting qubits with $p_{\text{phys}} = 10^{-3}$, surface code with distance $d \approx 27$, and a reaction time (classical control latency) of 10 μs , yielding a logical clock speed of approximately 10 kHz.

Table 4.2 Quantum resources to break deployed cryptographic parameters [22]

Target	Logical qubits	T-gates	Physical qubits	Wall-clock time
RSA-2048	~4,000	$\sim 2.7 \times 10^{12}$	~20 million	~8 hours
RSA-4096	~8,000	$\sim 2.2 \times 10^{13}$	~40 million	~30 hours
ECC-256	~2,500	$\sim 1.3 \times 10^{11}$	~14 million	~4 hours

Table 4.3 Quantum hardware gap: current capabilities vs. cryptanalytic requirements

Metric	Current (2025)	Required (RSA-2048)
Physical qubits	~ 1,000	~ 20,000,000
Logical qubits	0 (demonstrated)	~ 4,000
Error rate	10^{-3} – 10^{-2}	$< 10^{-3}$ (threshold)
Coherence time	Milliseconds	Hours

4.4.4 Current Hardware vs. Requirements

As of 2025, the largest demonstrated quantum processors have on the order of 10^3 physical qubits with error rates of 10^{-3} – 10^{-2} . No system has demonstrated a single *logical* qubit with the quality needed for Shor’s algorithm. The gap is stark:

The gap is roughly four orders of magnitude in qubit count alone. However, progress is non-linear—error correction improves super-exponentially with decreasing physical error rate—and roadmaps from major hardware vendors project 10^5 – 10^6 physical qubits by the early 2030s. This motivates the urgency of the post-quantum transition discussed in Chap. 5.

4.5 What Quantum Computers Cannot Do

The previous sections established the devastating power of quantum algorithms against structured algebraic problems. Equally important for the design of post-quantum cryptography is understanding where quantum computers do *not* provide exponential advantages.

4.5.1 NP-Complete Problems

No quantum algorithm is known to solve NP-complete problems in polynomial time. The best known quantum speedup for NP-complete problems is the quadratic improvement from Grover’s algorithm, which reduces 2^n to $2^{n/2}$ —still exponential. The prevailing conjecture in complexity theory is that $\text{NP} \not\subseteq \text{BQP}$, i.e., quantum computers cannot efficiently solve all NP problems.

This has a direct consequence: cryptographic problems reducible to NP-complete problems are not threatened by an exponential quantum speedup. The lattice problems at the heart of most NIST-selected schemes are not NP-complete, but they exhibit the same quantum resistance—for related but distinct reasons.

4.5.2 Lattice Problems

The best known quantum algorithms for lattice problems provide only a modest constant-factor improvement in the exponent:

$$T_{\text{classical}}(\text{SVP}, n) = 2^{0.292n+o(n)}, \quad T_{\text{quantum}}(\text{SVP}, n) = 2^{0.265n+o(n)}. \quad (4.7)$$

The speedup factor is $2^{0.027n}$ —polynomial in 2^n , hence the complexity remains exponential. Four structural reasons explain why no Shor-like breakthrough exists for lattices:

1. **No hidden periodicity.** Shor’s algorithm exploits the periodicity of modular exponentiation ($a^{x+r} \equiv a^x \pmod{N}$). Lattice problems have no comparable periodic structure for the QFT to detect.
2. **No hidden subgroup structure.** The generalisation of Shor’s algorithm to the *hidden subgroup problem* over abelian groups succeeds because the QFT over abelian groups is efficient. Lattice problems do not naturally embed into this framework.
3. **Geometric, not algebraic.** The core of lattice cryptography is a geometric optimisation (finding short vectors), not an algebraic identity. Quantum computers have not demonstrated exponential advantages for geometric problems.
4. **Grover is insufficient.** A Grover search over the $2^{O(n)}$ candidate vectors still yields an exponential cost $2^{O(n/2)}$.

4.5.3 The Polynomial/Exponential Divide as Design Criterion

The central design criterion for post-quantum cryptography is the *polynomial vs. exponential divide*: we select problems where the best known quantum algorithm achieves at most a polynomial speedup over the best classical algorithm, as opposed to problems where quantum algorithms provide an exponential speedup.

The table reveals the pattern: problems in the first three rows are quantum-vulnerable (exponential speedup), while problems in the last three rows are quantum-resistant (at most polynomial speedup). The entire post-quantum cryptographic programme—lattice-based, code-based, hash-based, and multivariate schemes—is built on problems from the second category. The mathematical justification for each family appears in Chaps. 6–??.

Table 4.4 The quantum speedup landscape and its cryptographic implications

Problem	Classical	Quantum	Speedup type
Integer factoring	Sub-exp. (GNFS)	$\text{poly}(n)$ (Shor)	Exponential
Discrete logarithm	Sub-exp./exp.	$\text{poly}(n)$ (Shor)	Exponential
ECDLP	$O(2^{n/2})$	$\text{poly}(n)$ (Shor)	Exponential
Unstructured search	$O(2^n)$	$O(2^{n/2})$ (Grover)	Quadratic
Lattice SVP	$2^{0.292n}$	$2^{0.265n}$	Polynomial
Code-based decoding	$2^{0.097n}$	$2^{0.06n}$	Polynomial

Problems

4.1 Walk through Shor's algorithm to factor $N = 15$ using the base $a = 11$. Show each step: compute $f(x) = 11^x \bmod 15$ for $x = 0, 1, \dots$, identify the period r , verify the conditions of Theorem 4.1, and extract the factors via GCD computation.

4.2 Grover's algorithm achieves a quadratic speedup for unstructured search.

- Explain why AES-128 offers only 64-bit security against a quantum adversary, while AES-256 retains 128-bit security.
- A student proposes using Grover's algorithm to search for RSA-2048 private keys (search space $\sim 2^{2048}$). Show that this requires approximately 2^{1024} quantum operations and explain why this is infeasible despite the quadratic speedup.
- Why does Shor's algorithm, rather than Grover's, pose the real threat to RSA?

4.3 Using the estimates from Gidney and Ekerå [22] (Table 4.2), answer the following:

- The surface code with physical error rate $p_{\text{phys}} = 10^{-3}$ and code distance $d = 27$ encodes each logical qubit in $2d^2 = 1,458$ physical qubits. Compute the total physical qubit count for factoring RSA-2048 (approximately 4,000 logical qubits), ignoring magic state distillation overhead.
- Current quantum processors have $\sim 1,000$ physical qubits. By what factor must qubit counts increase to reach the RSA-2048 threshold?
- If physical qubit counts double every two years, estimate the year by which factoring RSA-2048 becomes feasible. Comment on the reliability of this extrapolation.

4.4 The BBBV theorem (Theorem 4.3) states that $\Omega(\sqrt{N})$ quantum queries are necessary for unstructured search.

- Explain why this implies that no quantum algorithm can break AES-256 with fewer than $\sim 2^{128}$ oracle queries.
- Why does the BBBV bound not apply to Shor's algorithm? (Hint: consider the role of structure in the oracle.)

4.5 The polynomial/exponential divide is the design criterion for post-quantum cryptography (Sect. 4.5.3). For each of the following problems, classify the quantum speedup as exponential or polynomial and state whether the problem is suitable as a PQC foundation:

- (a) Integer factoring.
- (b) Shortest Vector Problem in dimension n .
- (c) Syndrome decoding of a random linear code.
- (d) Solving a system of multivariate quadratic equations over $\mathbb{F}_2$.

Part II
Post-Quantum Cryptographic Primitives

Chapter 5

The Post-Quantum Landscape

Abstract With the quantum threat established, this chapter surveys the five families of mathematical problems proposed as quantum-resistant foundations for cryptography. We trace the NIST standardization process from its 2016 call for proposals through the selection of ML-KEM, ML-DSA, and SLH-DSA as federal standards, examine the complementary—not competing—roles of post-quantum cryptography and quantum key distribution, and discuss the hybrid classical/post-quantum strategies that will dominate the transition period. The chapter concludes with a framework for evaluating any PQC candidate.

5.1 Algorithm Families at a Glance

Post-quantum cryptography draws on five families of mathematical problems, each with a distinct history, security profile, and set of engineering trade-offs. No single family dominates across all criteria; the diversity is a feature, not a deficiency [6].

5.1.1 Lattice-Based Cryptography

Lattice-based schemes—the subject of Chaps. 6–8—derive their security from the hardness of finding short vectors in high-dimensional lattices. The Learning With Errors (LWE) and Module-LWE problems underpin the NIST standards ML-KEM [50] and ML-DSA [49]. Lattice-based cryptography offers uniquely strong *worst-case to average-case* security reductions (Sect. 6.4), moderate key sizes, and efficient implementations. Its algebraic structure also enables advanced constructions such as fully homomorphic encryption [20]. The main concern is that lattice problems have been studied intensively for only three decades, and the rich algebraic structure that enables advanced applications also provides a wider attack surface than, say, hash-based schemes.

5.1.2 Code-Based Cryptography

Code-based cryptography, inaugurated by McEliece in 1978 [42], rests on the hardness of decoding a random linear code. The generic decoding problem has resisted algorithmic progress for over four decades, making code-based schemes among the most conservatively secure PQC candidates. Classic McEliece, the oldest unbroken public-key scheme, was a NIST round-4 finalist; HQC, a Hamming Quasi-Cyclic code-based KEM, was selected for standardization in 2024. The principal drawback is key size: Classic McEliece public keys exceed 250 KB at the 128-bit security level, which limits deployment in bandwidth-constrained environments.

5.1.3 Hash-Based Signatures

Hash-based signatures rely solely on the security of the underlying hash function—no number-theoretic or algebraic assumption is required. Lamport one-time signatures [36] and Merkle trees [44] date from the late 1970s; the modern SPHINCS+ construction (standardized as SLH-DSA [51]) extends this to a stateless many-time signature scheme. Hash-based signatures are the “insurance policy” of PQC: even if every algebraic assumption fails simultaneously, hash functions would need to be broken as well. The trade-off is large signatures (8–50 KB for SLH-DSA) and slower signing compared to lattice-based alternatives.

5.1.4 Multivariate Cryptography

Multivariate schemes are built on the NP-hardness of solving systems of multivariate quadratic equations over finite fields. They can produce very compact signatures (tens of bytes) and fast verification, but public keys tend to be large and the design space is riddled with structural attacks. Rainbow, a prominent NIST round-3 finalist, was broken in 2022 by Beullens [7] on a single laptop—a sobering reminder that compact parameters alone do not imply security. Research continues on schemes such as UOV (Unbalanced Oil and Vinegar), selected by NIST for additional signature evaluation, but no multivariate KEM has survived serious cryptanalysis.

5.1.5 Isogeny-Based Cryptography

Isogeny-based cryptography uses maps between elliptic curves as the hard mathematical object. SIKE (Supersingular Isogeny Key Encapsulation) [31] offered the smallest keys of any PQC candidate but was catastrophically broken in 2022 by Castryck and Decru [12], who exploited auxiliary torsion-point information to recover secret isogenies

Table 5.1 Comparative overview of PQC algorithm families

Family	Hard problem	Key sizes	Speed	Study (yrs)	NIST status
Lattice	LWE/MODULE-LWE	Moderate	Fast	~30	ML-KEM, ML-DSA
Code	Syndrome decoding	Large	Fast	~45	HQC (round 4)
Hash	Hash function security	N/A*	Slow sign	~45	SLH-DSA
Multivariate	MQ problem	Large	Fast ver.	~35	Under evaluation
Isogeny	Isogeny computation	Small	Slow	~15	SIKE broken

*Hash-based schemes are signature-only; key sizes depend on the specific construction.

in polynomial time. The CSIDH family [13], which avoids torsion-point auxiliary data, remains unbroken but is too slow for most practical applications. Isogeny-based cryptography is the youngest and least mature family; its long-term viability remains an open question.

5.1.6 Comparative Overview

Table 5.1 summarises the key characteristics of each family. Readers should note that “years of study” refers to the age of the underlying hard problem, not any individual scheme. How these families were tested, compared, and ultimately winnowed down to a handful of federal standards is the subject of the next section.

5.2 The NIST Standardization Process

Five families of hard problems, dozens of candidate schemes, and no consensus on which to trust—this was the state of affairs in 2016. Choosing post-quantum standards required more than academic debate; it required a structured, adversarial, multi-year evaluation. The NIST Post-Quantum Cryptography Standardization Process became that evaluation—the largest coordinated cryptographic competition in history, deliberately modelled on the successful AES and SHA-3 processes but unprecedented in scope [6].

5.2.1 Timeline

The process unfolded over nine years:

1. **2015 – Catalyst.** The NSA announces that it will transition to quantum-resistant algorithms, signalling to the world that the threat is taken seriously at the highest levels.
2. **2016 – Call for proposals.** NIST publishes a formal call for post-quantum public-key encryption, key-encapsulation, and digital-signature algorithms. Submission requirements include: complete specification, reference implementation, performance benchmarks, and a security analysis.
3. **2017 – Round 1.** Eighty-two submissions are received from teams spanning academia, industry, and government laboratories worldwide; 69 are deemed complete and accepted for evaluation.
4. **2019 – Round 2.** After two years of public cryptanalysis and three dedicated PQC conferences, NIST narrows the field to 26 candidates. Several submissions are withdrawn after attacks; others are merged.
5. **2020 – Round 3.** Fifteen candidates survive: 7 finalists (expected to be standardized) and 8 alternates (backup candidates and schemes meriting further study).
6. **2022 – Winners announced.** NIST selects CRYSTALS-Kyber (KEM), CRYSTALS-Dilithium (signature), and SPHINCS+ (signature) for standardization. Falcon is selected as an additional signature standard. A 4th round is opened for four KEM candidates, including Classic McEliece and HQC.
7. **2024 – Standards published.** Three Federal Information Processing Standards are released: FIPS 203 (ML-KEM, formerly Kyber) [50], FIPS 204 (ML-DSA, formerly Dilithium) [49], and FIPS 205 (SLH-DSA, formerly SPHINCS+) [51]. HQC is selected from the 4th round as an additional KEM standard, providing algorithmic diversity beyond lattices. A separate call for additional signature schemes continues.

5.2.2 Evaluation Criteria

NIST evaluated candidates across multiple dimensions, recognizing that theoretical security alone is insufficient for a deployable standard. Table 5.2 summarises the criteria and their approximate weighting.

The multi-criterion evaluation prevented single-point optimization and exposed unexpected weaknesses. Rainbow, for instance, offered tiny 66-byte signatures and fast verification, but its large public keys (158 KB) and eventual complete break in 2022 [7] illustrated why a holistic assessment is essential.

5.2.3 Why Multiple Winners?

NIST deliberately selected algorithms from different mathematical families to provide *algorithmic diversity*. If a breakthrough attack compromises all lattice-based schemes—however unlikely—SLH-DSA (hash-based) and HQC (code-based) remain secure. This portfolio strategy mirrors engineering best practice: no single point of failure.

Table 5.2 NIST evaluation criteria for PQC candidates

Criterion	Description	Approx. weight
Security	Resistance to classical and quantum attacks; quality of security reductions; parameter margins against future cryptanalysis	High
Performance	Speed on diverse platforms (server, desktop, mobile, embedded)	Medium
Key/ciphertext size	Bandwidth and storage impact; compatibility with existing protocols	Medium
Implementation	Side-channel resistance; simplicity; misuse resistance	Medium–Low
Flexibility	Parameter choices; suitability for advanced protocols; crypto-agility	Low
Maturity	Years of public cryptanalytic study; diversity of analysis	Low

The naming convention. During standardization, NIST renamed the selected algorithms:

- CRYSTALS-Kyber → ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism) [50].
- CRYSTALS-Dilithium → ML-DSA (Module-Lattice-Based Digital Signature Algorithm) [49].
- SPHINCS+ → SLH-DSA (Stateless Hash-Based Digital Signature Algorithm) [51].

The literature uses both names interchangeably; this book follows the FIPS naming throughout.

5.3 PQC vs QKD: Complementary, Not Competing

The NIST process settled the question of *which* algorithms to standardize, but it did not settle a different and surprisingly persistent question: do we even need PQC, or does quantum physics itself offer a better solution? A persistent misconception frames Post-Quantum Cryptography (PQC) and Quantum Key Distribution (QKD) as competitors. This is a false dichotomy. The two technologies solve different problems, operate on different principles, and have different failure modes. Most critically, QKD *cannot* provide digital signatures, making PQC mandatory regardless of QKD deployment.

Table 5.3 Infrastructure comparison: QKD versus PQC

Requirement	QKD	PQC
Physical channel	Dedicated fiber or free-space link	Any classical channel
Distance limit	~100–200 km (fiber)	Unlimited
Trusted repeaters	Required for long distance	Not needed
Quantum hardware	Single-photon sources and detectors	None
Cost per link	\$100 000+	\$0 (software update)
Deployment time	Months (fiber installation)	Minutes (software update)
Scalability	$\binom{n}{2}$ quantum channels for n parties	Any-to-any over existing networks

5.3.1 Security Paradigms

The fundamental distinction is between *information-theoretic* and *computational* security:

- **QKD** derives security from the laws of quantum mechanics (the no-cloning theorem, measurement disturbance). No amount of computational power helps an eavesdropper; security is guaranteed by physics.
- **PQC** derives security from the assumed computational hardness of mathematical problems (lattice problems, syndrome decoding, hash inversion). Security holds as long as the adversary’s computational resources remain bounded.

Neither paradigm is inherently superior. QKD wins when the adversary has unlimited computation but must obey physics; PQC wins when the application requires signatures, internet-scale deployment, or a software-only solution.

5.3.2 Infrastructure and Scalability

The physical nature of QKD imposes severe deployment constraints. Table 5.3 summarises the key differences.

For $n = 1000$ parties, QKD requires 499 500 dedicated quantum channels (or trusted relay nodes, which weaken the security model). PQC works over the existing internet infrastructure with no additional hardware.

5.3.3 The Signature Gap

QKD distributes shared secret keys between two parties. It cannot, even in principle, provide *digital signatures*—the public verifiability, non-repudiation, and transferability that underpin software updates, TLS certificates, financial transactions, and legal documents all require public-key signatures. Moreover, QKD itself requires an authen-

Table 5.4 Technology applicability by use case

Application	QKD viable?	PQC required?	Recommendation
Web browsing (TLS)	No	Yes	PQC only
Software signing	No	Yes	PQC only
Cloud / IoT / mobile	No	Yes	PQC only
Government backbone	Maybe	Yes	QKD + PQC hybrid
Critical infrastructure	Maybe	Yes	Evaluate case-by-case
Nuclear / diplomatic	Yes	Yes	QKD + PQC hybrid

ticated classical channel to prevent man-in-the-middle attacks, and that authentication ultimately depends on digital signatures or pre-shared keys.

The implication is stark: *you cannot deploy QKD without also deploying PQC* (for signatures and authentication), but you can deploy PQC without QKD.

5.3.4 When to Use Which

QKD is justified when *all* of the following hold: the data value is extreme, the link is point-to-point between fixed facilities, the distance is within fiber reach, the budget supports dedicated quantum infrastructure, and the threat model includes adversaries with potentially unbounded future computation. In practice, this means a small number of government and critical-infrastructure links.

For the remaining 99.9% of cryptographic applications—web browsing, email, software signing, cloud services, mobile devices, IoT, blockchain—PQC is the only viable technology. Table 5.4 provides a concise summary, and the decision tree in Figure 5.1 formalises the evaluation: QKD is warranted only when every branch of the tree leads to “yes.”

The engineering reality. QKD is elegant physics with rigorous security proofs, but engineering reality is unforgiving. PQC is less exotic, yet it solves the actual problem for the vast majority of use cases: a software update that deploys in minutes, works over existing networks, supports signatures, and costs nothing beyond development effort. The two technologies are complementary layers, not competing alternatives.

5.4 Hybrid Classical/Post-Quantum Approaches

PQC is necessary; QKD is complementary. But even within PQC, a practical concern remains: the new algorithms are young, and trusting them *exclusively* on day one is a leap

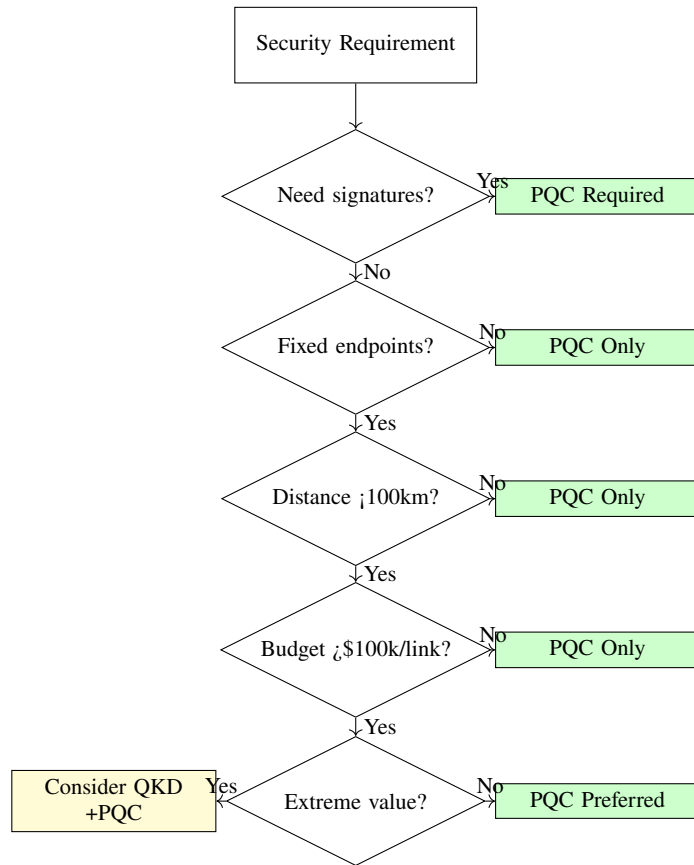


Fig. 5.1 Decision tree for evaluating QKD viability. Only when every constraint—signatures not required, fixed endpoints, short distance, sufficient budget, and extreme data value—is satisfied does QKD merit consideration, and even then it must be paired with PQC.

of faith that many organisations are unwilling to take. The prudent strategy during the transition is *hybrid deployment*: combining a classical algorithm (e.g., ECDH, ECDSA) with a post-quantum algorithm so that the system remains secure as long as *at least one* of the two underlying assumptions holds. This defense-in-depth approach hedges against both premature quantum computers and undiscovered weaknesses in new PQC schemes.

5.4.1 Composite KEMs

A *composite KEM* runs two independent key encapsulations—one classical, one post-quantum—and combines the resulting shared secrets via a key derivation function:

$$K_{\text{final}} = \text{KDF}(K_{\text{classical}} \parallel K_{\text{PQC}} \parallel \text{context}). \quad (5.1)$$

If either $K_{\text{classical}}$ or K_{PQC} is indistinguishable from random, then so is K_{final} (under standard assumptions on the KDF). This construction is already deployed: Google’s CECPQ2 experiment combined X25519 with NTRU-HRSS in Chrome, and Cloudflare has deployed X25519 + ML-KEM to millions of websites with negligible overhead.

The cost of a composite KEM is approximately the sum of the two individual KEM costs in computation and bandwidth. For ML-KEM-768 combined with X25519, the additional ciphertext overhead is roughly 1 KB and the additional computation is sub-millisecond on modern hardware—a modest price for hedged security.

5.4.2 Dual Signatures

Hybrid signatures are more complex than hybrid KEMs because the verification semantics must be unambiguous. Two principal strategies exist:

1. **Concatenated signatures.** Both a classical signature σ_C and a PQC signature σ_P are attached to the message; the verifier accepts if and only if *both* signatures verify. Security requires only one assumption to hold. The downside is doubled signature size and doubled verification time.
2. **Nested signatures.** The PQC signature signs the message concatenated with the classical signature: $\sigma_P = \text{Sign}_{\text{PQC}}(m \parallel \sigma_C)$. This binds the two signatures more tightly but introduces an asymmetric dependency.

In both cases, the combined signature is valid only if neither component can be stripped by an attacker—the protocol must be designed to reject messages with missing components.

5.4.3 X.509 Hybrid Certificates

The Web PKI depends on X.509 certificates, which contain a single public key and a single signature from the issuing CA. Hybrid certificates embed both a classical and a post-quantum key pair, along with dual signatures from the CA. Several approaches are under development:

- **Composite keys (IETF draft):** A single “composite” public key contains both an ECDSA and an ML-DSA key; the corresponding composite signature contains both signatures. Existing relying parties that do not understand composite keys reject the certificate gracefully (fail-closed).
- **Alternative signatures (X.509 extension):** The certificate carries a classical signature in the standard field and a PQC signature in an extension. Legacy clients verify only the classical signature; updated clients verify both. This provides backward compatibility at the cost of a weaker security guarantee for legacy clients.

- **Dual certificate chains:** Two parallel certificate chains—one classical, one post-quantum—are maintained. The client negotiates which chain to use via TLS extensions. This approach is the simplest conceptually but doubles the CA’s operational burden.

Transition is the hard part. The mathematical security of hybrid constructions is well-understood; the difficulty lies in engineering the transition. Every protocol, library, hardware module, and certificate chain must be updated. Organizations that invest in *crypto-agility*—the ability to swap algorithms without redesigning systems—will navigate this transition far more smoothly than those with hard-coded cryptographic choices.

5.5 Evaluating a PQC Candidate

The preceding sections surveyed the families, the standardization gauntlet, and the hybrid strategies that hedge our bets during the transition. But one question recurs every time a new scheme appears on the ePrint archive or a vendor markets a “quantum-safe” product: *how do we know it is any good?* Enthusiasm and conservatism are both dangerous here—the former leads to premature deployment of fragile constructions, the latter to paralysis. What we need is a systematic evaluation framework, and the five criteria below—adapted from NIST’s own methodology—provide one that applies to any PQC candidate, whether it is a FIPS standard or a preprint posted last week.

5.5.1 Security

Security is the first and heaviest criterion, but it is not a single number. Three dimensions matter, and they do not always point in the same direction.

The most informative signal is the quality of a scheme’s *security reduction*: what exactly must an attacker break in order to compromise the cryptosystem? Lattice-based schemes such as ML-KEM enjoy worst-case to average-case reductions (Sect. 6.4), meaning that breaking any random instance is at least as hard as solving the hardest instance of a well-studied lattice problem. This is a powerful guarantee—it turns every failed attack on the worst case into a blanket assurance for the average case. Most code-based and multivariate schemes lack this luxury; their security rests on average-case hardness, which is weaker in principle even if it has held up well in practice for decades.

Equally important is the *parameter margin* between the concrete security estimate and the target security level. A scheme operating at exactly 128 bits of security is walking a tightrope: a single algorithmic improvement—a better sieving constant, a new lattice reduction technique—can push it below the threshold overnight. Schemes with 15–20% headroom absorb such improvements gracefully, without forcing emergency

re-parameterization. Finally, the *cryptanalytic history* of the underlying hard problem deserves weight. Syndrome decoding has resisted attacks for over four decades from hundreds of independent groups; supersingular isogeny problems had barely a decade of scrutiny before the SIKE collapse [12] vindicated exactly this concern. Longevity alone proves nothing, but a problem that has survived sustained, diverse assault provides a qualitatively different kind of confidence than one that has merely been neglected.

5.5.2 Performance

Security without deployability is an academic exercise. Performance determines whether a scheme can actually protect real systems, and it must be evaluated across three interrelated dimensions.

Speed—of key generation, encapsulation or signing, and decapsulation or verification—varies enormously across families and across platforms. A scheme that runs in microseconds on a server with AVX-512 instructions may take seconds on an ARM Cortex-M4 microcontroller. Benchmarks on a single platform are misleading; what matters is the speed profile across the range of hardware the scheme will actually encounter. Closely linked to speed are the *sizes* of public keys, secret keys, ciphertexts, and signatures. These are not abstract numbers: they translate directly into bandwidth consumption, storage requirements, and protocol round-trips. ML-KEM-768 public keys are 1 184 bytes; Classic McEliece public keys exceed 261 000 bytes. That three-order-of-magnitude difference determines whether a scheme fits inside a TLS handshake, a DNSSEC record, or the flash memory of a smart card. The third dimension, often overlooked, is *memory footprint*. Embedded devices and smart cards may have only a few kilobytes of RAM available for cryptographic operations. Schemes that require large intermediate buffers—Falcon’s FFT-based signing is the canonical example—may deliver excellent speed and compact outputs while remaining entirely unsuitable for constrained environments.

5.5.3 Implementation Quality

A mathematically secure scheme that is routinely implemented insecurely is, for all practical purposes, broken. Implementation quality addresses the gap between theoretical security and what happens when real engineers write real code on real hardware.

The most critical dimension is *side-channel resistance*: does the algorithm lend itself to constant-time implementation, or does securing it against timing and power-analysis attacks require heroic countermeasures? ML-KEM’s core operations are largely linear algebra over small moduli—additions, multiplications, and NTT transforms that can be implemented in constant time with straightforward effort. Falcon, by contrast, requires floating-point discrete Gaussian sampling during signing, and making that constant-time across different floating-point units is notoriously difficult. A related concern is *misuse resistance*: how badly does the scheme fail when something goes wrong? Nonce reuse,

bad randomness, and parameter misuse are not hypothetical—they occur regularly in deployed systems. An IND-CCA2 KEM such as ML-KEM, hardened by the Fujisaki–Okamoto transform [19], degrades gracefully; a CPA-secure scheme may leak the secret key entirely on ciphertext manipulation. Finally, raw *simplicity* deserves consideration. Fewer lines of code and simpler control flow mean fewer places for bugs to hide. This is a practical consideration that theoretical analyses, focused on asymptotic security, almost always neglect—yet it is precisely the dimension where deployed systems most often fail.

5.5.4 Maturity

Maturity is the criterion that cannot be bought, accelerated, or simulated. It is the difference between a scheme that has survived a decade of scrutiny from cryptanalysts, side-channel experts, formal-methods researchers, and production engineers—and one that has survived only the review of its own authors.

The relevant metric is not merely the number of years since publication, but the *diversity and intensity* of the analysis the scheme has received. A scheme examined only by algebraic cryptanalysts may harbour implementation vulnerabilities that a side-channel expert would spot immediately; one studied only in academia may contain engineering assumptions that shatter on contact with production workloads. ML-KEM and ML-DSA have benefited from seven years of public scrutiny within the NIST process, adversarial implementation in dozens of libraries, and large-scale deployment experiments by Google, Cloudflare, and others. That operational history surfaces classes of bugs—memory safety issues, platform-specific timing leaks, interoperability failures—that no amount of paper analysis can anticipate. Classic McEliece and SLH-DSA carry even longer pedigrees, their underlying problems dating to the late 1970s. At the other extreme, a newly proposed scheme, however elegant, carries a maturity debt that only time and diverse adversarial attention can repay.

5.5.5 Flexibility

The final criterion looks forward rather than backward: how well does a scheme adapt to requirements that do not yet exist?

Parameter scaling is the most immediate concern. A scheme that offers a smooth continuum of security levels—ML-KEM, for instance, provides three parameter sets (512, 768, 1024) covering NIST levels I, III, and V—can be tuned to the needs of each deployment without fundamental redesign. A scheme that works at a single security level, or whose parameter space has unexplored gaps, is far less useful in a world where different applications demand different trade-offs. Beyond parameters, the *algebraic structure* of the underlying mathematics determines whether a scheme can participate in advanced protocols. Lattice-based cryptography supports threshold signatures, multi-party computation, and fully homomorphic encryption; hash-based signatures, by

Table 5.5 Evaluation framework applied to selected PQC algorithms

	ML-KEM	ML-DSA	SLH-DSA	McEliece	HQC
Reduction	Worst-case	Worst-case	Minimal [†]	Average-case	Average-case
Key sizes	Moderate	Moderate	Small	Very large	Moderate
Speed	Fast	Fast	Slow sign	Fast decaps	Moderate
Side-ch.	Good	Good	Excellent	Good	Good
Maturity	High	High	Very high	Very high	Moderate
Flexibility	High	High	Low	Low	Low

[†]SLH-DSA security reduces to hash-function properties, which are minimal assumptions.

design, offer no algebraic structure whatsoever. For an organisation building infrastructure that may need these capabilities in five or ten years, this distinction is decisive. The broadest version of flexibility is *crypto-agility*: how easily can the scheme be swapped out if a better alternative emerges or if a new attack renders it obsolete? Systems that abstract the cryptographic layer behind clean interfaces—rather than hard-coding algorithm-specific assumptions into protocols and data formats—can migrate with a configuration change. Those that do not will face the same painful, multi-year transition that the quantum threat is imposing on classical cryptography today.

Table 5.5 applies this framework to the three NIST-standardized algorithms and two notable alternatives. No single scheme dominates every column—which is precisely why NIST selected a portfolio rather than a single winner.

Problems

5.1 Construct a detailed comparison table of ML-KEM-768, Classic McEliece (kem/mceliece6960119), and HQC-192 as KEM candidates. Include: underlying hard problem, public key size (bytes), ciphertext size (bytes), shared-secret size, decapsulation failure probability, security reduction quality, and approximate years of cryptanalytic study of the underlying problem. Based on your table, which scheme would you recommend for (a) a resource-constrained IoT sensor with 32 KB of RAM, (b) a government classified network where long-term confidence outweighs bandwidth cost? Justify your answers.

5.2 Consider a composite KEM that combines X25519 (classical ECDH) with ML-KEM-768 (post-quantum). Let K_C denote the X25519 shared secret and K_P the ML-KEM shared secret, with $K_{\text{final}} = \text{HKDF}(K_C \parallel K_P)$.

- Argue that K_{final} is indistinguishable from random if *either* the CDH assumption or the MODULE-LWE assumption holds (but not necessarily both).
- Compute the total bandwidth overhead of the composite KEM (public keys + ciphertexts) compared to X25519 alone and to ML-KEM-768 alone.
- A colleague argues that the hybrid approach is pointless because “if we trust ML-KEM, we don’t need X25519.” Provide a counterargument grounded in the precautionary principle during cryptographic transitions.

5.3 The NIST PQC competition began with 82 submissions and ended with 4 selected algorithms. Research the fates of at least five eliminated candidates. For each, identify: (a) the algorithm family, (b) the reason for elimination (cryptanalytic break, poor performance, patent issues, merged with another submission, or withdrawn by authors), and (c) whether the underlying hard problem is still considered viable for future schemes.

5.4 A bank with 10 branch offices is evaluating quantum-safe options. The branches are distributed across a 300 km region. The bank needs to protect both inter-branch encrypted communications and customer-facing TLS connections.

- (a) Explain why QKD alone cannot secure the bank's customer-facing services.
- (b) Estimate the number of QKD links needed for full inter-branch connectivity and the approximate infrastructure cost, assuming \$200 000 per link and a maximum QKD distance of 100 km without trusted relays.
- (c) Propose a practical architecture that uses PQC for all services, with QKD optionally deployed for the highest-value links. Justify which links, if any, warrant QKD investment.

Chapter 6

Lattice Theory

Abstract Lattices—discrete subgroups of $\mathbb{R}^n$ —form the mathematical bedrock of the most important post-quantum cryptographic schemes. This chapter develops lattice theory from definitions familiar to physicists through crystallography, progresses to the fundamental computational problems (SVP, CVP, and their approximation variants), introduces lattice basis reduction algorithms that set the practical security baseline, and establishes the worst-case to average-case reductions that make lattice-based cryptography uniquely well-founded among all public-key paradigms.

6.1 Definitions and Basic Properties

In crystallography, a lattice describes the periodic arrangement of atoms in a crystal—a discrete set of points with translational symmetry. Cryptographic lattices are the same mathematical object, but embedded in hundreds of dimensions where geometric intuition breaks down and computational problems become exponentially hard [45].

Definition 6.1 (Lattice) A lattice $\mathcal{L} \subseteq \mathbb{R}^n$ is the set of all integer linear combinations of n linearly independent vectors $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^n$:

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^n a_i \mathbf{b}_i : a_i \in \mathbb{Z} \right\} = \{ \mathbf{B}\mathbf{x} : \mathbf{x} \in \mathbb{Z}^n \}, \quad (6.1)$$

where $\mathbf{B} = [\mathbf{b}_1 \mid \dots \mid \mathbf{b}_n] \in \mathbb{R}^{n \times n}$ is the *basis matrix*. The integer n is the *rank* (or *dimension*) of the lattice.

A lattice admits infinitely many bases: if $\mathbf{B}$ is a basis and $\mathbf{U} \in \mathbb{Z}^{n \times n}$ is any unimodular matrix ($\det \mathbf{U} = \pm 1$), then $\mathbf{B}' = \mathbf{B}\mathbf{U}$ generates the same lattice. This non-uniqueness of the basis is the source of cryptographic hardness, as we shall see in Sect. 6.2.

Table 6.1 Crystallographic and cryptographic lattice concepts

Crystallography	Crypto lattices	Key difference
3D Bravais lattice	n -dimensional lattice	$n \approx 500\text{--}1000$
Primitive unit cell	Good basis	Many possible bases
Reciprocal lattice	Dual lattice	Same hardness properties
Brillouin zone	Fundamental domain	Voronoi cell
Miller indices	Coefficient vector	Integer constraints
Bragg peaks	Lattice points	Discrete structure

6.1.1 The Physics Connection

Readers with a background in solid-state physics or crystallography will recognise many of the concepts below under different names. Table 6.1 summarises the correspondence; the key difference is dimension.

Example 6.1 (From NaCl to Cryptography) The face-centred cubic lattice of NaCl lives in $\mathbb{R}^3$. Finding the nearest atom to any point in space is a straightforward computation. Now imagine the same lattice embedded in $\mathbb{R}^{700}$: finding nearest points becomes exponentially hard. This dimension-induced hardness is the foundation of lattice-based cryptography.

6.1.2 Determinant and Volume

If a lattice can be described by many different bases, we need a quantity that is intrinsic to the lattice itself—something that does not change when we swap one basis for another. The determinant provides exactly this invariant: it captures how densely the lattice points pack the ambient space.

Definition 6.2 (Lattice Determinant) The *determinant* of a lattice $\mathcal{L}(\mathbf{B})$ is

$$\det(\mathcal{L}) = |\det(\mathbf{B})|. \quad (6.2)$$

It equals the n -dimensional volume of the *fundamental parallelepiped* $\mathcal{P}(\mathbf{B}) = \{\mathbf{B}\mathbf{x} : \mathbf{x} \in [0, 1)^n\}$ and is independent of the choice of basis.

Geometrically, $\det(\mathcal{L})$ measures the “density” of lattice points: the larger the determinant, the sparser the lattice. In a crystal, this corresponds to the volume of the unit cell.

6.1.3 Successive Minima

The determinant tells us about global density, but cryptography demands something more specific: how short can lattice vectors actually be? The answer determines the security margin of every lattice-based scheme, because an attacker who can find short vectors can break keys. The successive minima formalise this question by measuring, for each dimension, the radius at which a new linearly independent lattice vector first appears.

Definition 6.3 (Successive Minima) The i -th *successive minimum* $\lambda_i(\mathcal{L})$ of a lattice $\mathcal{L}$ is the smallest radius r such that the closed ball $\overline{B}(\mathbf{0}, r)$ contains i linearly independent lattice vectors:

$$\lambda_i(\mathcal{L}) = \inf\{r > 0 : \dim(\text{span}(\mathcal{L} \cap \overline{B}(\mathbf{0}, r))) \geq i\}. \quad (6.3)$$

In particular, $\lambda_1(\mathcal{L})$ is the length of a shortest non-zero lattice vector.

The successive minima are related to the determinant by *Minkowski's theorem*:

Theorem 6.1 (Minkowski's First Theorem) For any n -dimensional lattice $\mathcal{L}$,

$$\lambda_1(\mathcal{L}) \leq \sqrt{n} \det(\mathcal{L})^{1/n}. \quad (6.4)$$

For random lattices, the bound is essentially tight: the *Gaussian heuristic* predicts $\lambda_1(\mathcal{L}) \approx \sqrt{n/(2\pi e)} \det(\mathcal{L})^{1/n}$.

6.1.4 The Dual Lattice

Every lattice carries a shadow: a companion lattice whose points have integer inner products with all points of the original. Physicists know this construction as the reciprocal lattice, the object that governs diffraction patterns and Brillouin zones. In cryptography, the dual lattice matters because hardness results transfer across the duality—an adversary gains nothing by attacking the dual instead of the original.

Definition 6.4 (Dual Lattice) The *dual* (or *reciprocal*) lattice of $\mathcal{L}$ is

$$\mathcal{L}^* = \{\mathbf{y} \in \mathbb{R}^n : \langle \mathbf{x}, \mathbf{y} \rangle \in \mathbb{Z} \text{ for all } \mathbf{x} \in \mathcal{L}\}. \quad (6.5)$$

If $\mathcal{L} = \mathcal{L}(\mathbf{B})$, then $\mathcal{L}^* = \mathcal{L}((\mathbf{B}^{-1})^\top)$.

The key properties are $\det(\mathcal{L}^*) = 1/\det(\mathcal{L})$ and $\lambda_1(\mathcal{L}) \cdot \lambda_n(\mathcal{L}^*) \leq n$ (the *transference theorem*). The transference theorem makes the cryptographic consequence precise: short vectors in the primal lattice imply long vectors in the dual, and vice versa, so neither perspective offers an easier attack surface.

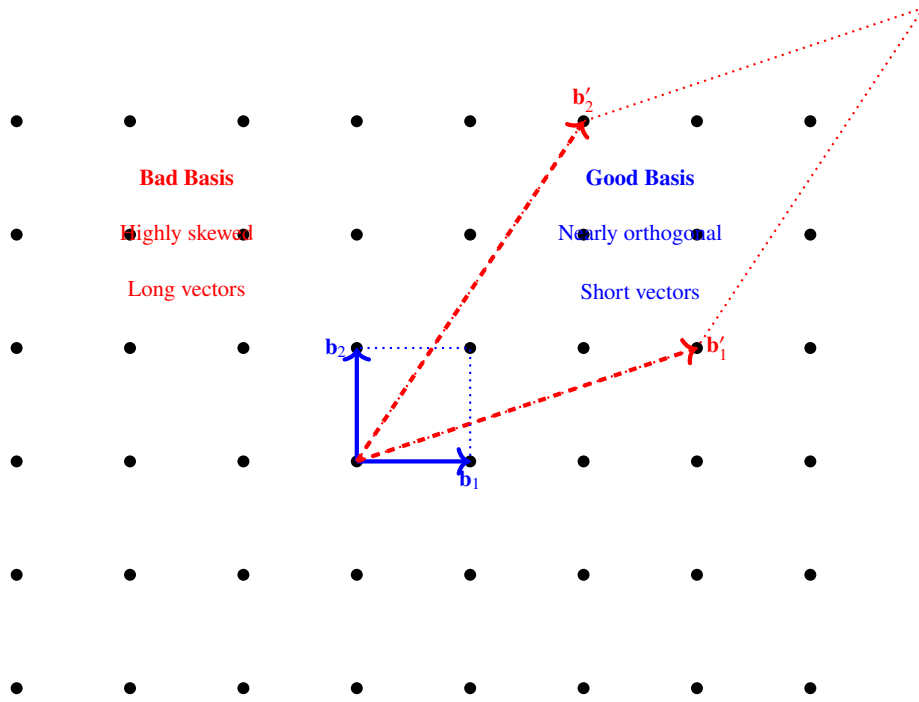


Fig. 6.1 Same lattice, vastly different bases. Finding the good basis from the bad one is the computational problem that underpins lattice-based cryptography.

6.1.5 Good Bases vs. Bad Bases

We have defined lattices, their determinants, their shortest vectors, and their duals. All of these are properties of the lattice itself, independent of how it is presented. But the *presentation* matters enormously in practice: the same lattice can be described by bases of vastly different quality, and this ambiguity is precisely what makes cryptography possible. A “good” basis reveals the lattice’s short vectors at a glance; a “bad” basis buries them under layers of near-cancellation. The private key is the good basis; the public key is the bad one.

Example 6.2 (Concrete Basis Comparison) Consider the 2D integer lattice $\mathbb{Z}^2$ with two bases:

Good basis: $\mathbf{B}_{\text{good}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. The shortest vector $(1, 0)$ has length 1 and is read off immediately.

Bad basis: $\mathbf{B}_{\text{bad}} = \begin{pmatrix} 97 & 89 \\ 89 & 81 \end{pmatrix}$. This generates the same lattice ($\det = 97 \cdot 81 - 89 \cdot 89 = 1$), but the basis vectors have length > 100 . The shortest vector is still $(1, 0)$, which equals $97\mathbf{b}_1 - 89\mathbf{b}_2$ —a non-trivial integer combination.

Figure 6.1 illustrates this contrast geometrically: two bases generate the same lattice, but one reveals the short vectors immediately while the other hides them.

To make the notion of “good” and “bad” precise, we need quantitative measures. Two are standard: one captures how far the basis vectors are from orthogonal, and the other measures how well the first basis vector approximates the true shortest vector.

Definition 6.5 (Basis Quality Measures) For a basis $\mathbf{B} = [\mathbf{b}_1 \mid \cdots \mid \mathbf{b}_n]$, define:

- **Orthogonality defect:** $\delta(\mathbf{B}) = \frac{\prod_{i=1}^n \|\mathbf{b}_i\|}{\det(\mathcal{L})}$ (equals 1 for an orthogonal basis).
- **Hermite factor:** $\gamma = \left(\frac{\|\mathbf{b}_1\|}{\det(\mathcal{L})^{1/n}} \right)^{1/n}$ (smaller is better; measures how well $\mathbf{b}_1$ approximates λ_1).

6.1.6 High-Dimensional Phenomena

Everything we have discussed so far—the determinant, the successive minima, the gap between good and bad bases—becomes qualitatively different when the dimension n grows from the single digits familiar to crystallographers to the hundreds required by cryptographers. High-dimensional Euclidean space is profoundly counter-intuitive, and these counter-intuitions are what make lattice problems hard.

Theorem 6.2 (Dimension Changes Everything) *In dimension n :*

1. The volume of the unit ball $V_n = \pi^{n/2} / \Gamma(n/2 + 1)$ converges to 0 as $n \rightarrow \infty$.
2. Almost all the volume concentrates near the surface: $V_n(r) / V_n(R) = (r/R)^n$.
3. Random unit vectors are nearly orthogonal: $\mathbb{E}[|\langle \mathbf{x}, \mathbf{y} \rangle|] = O(1/\sqrt{n})$.
4. The number of lattice vectors of length close to λ_1 grows as $\exp(\Theta(n))$.

The contrast with low-dimensional physics is worth stating explicitly, because it is the single most important fact in this chapter.

The dimension phase transition. In condensed-matter physics, lattice problems in 3D are polynomial-time computations—band structures, ground-state energies, defect calculations. In 500 dimensions, the same problems become exponentially hard. This dimension-induced phase transition from tractable to intractable is what protects cryptographic secrets.

The following table illustrates the concrete impact of dimension on the hardness of the shortest vector problem:

With these geometric foundations in place—definitions, determinants, successive minima, duality, and the high-dimensional phenomena that make everything hard—we can now state the computational problems that lattice-based cryptography actually relies on.

Table 6.2 Time to find the shortest vector in a random lattice

Dimension	Time (best known)	Feasible?
10	Seconds	Trivial
50	Hours	Yes
100	Years	Borderline
300	10^{30} years	No
700	10^{100} years	Never

6.2 Fundamental Computational Problems

The geometry of high-dimensional lattices would be a curiosity if it did not give rise to computational problems of extraordinary hardness. Two problems in particular—finding short vectors and finding close vectors—form the foundation of lattice-based cryptography. Unlike integer factorisation or the discrete logarithm, they have resisted efficient algorithms for decades, classical and quantum alike. Everything in the next four chapters ultimately reduces to the hardness of one of these two problems.

6.2.1 The Shortest Vector Problem

The most natural question one can ask about a lattice is also the hardest: given a basis, find the shortest non-zero vector. In low dimensions this is straightforward—geometrically obvious, even. In hundreds of dimensions, it becomes a needle-in-a-haystack problem where the haystack has exponentially many straws of nearly equal length.

Definition 6.6 (Shortest Vector Problem (SVP)) Given a basis $\mathbf{B}$ of a lattice $\mathcal{L}$, find a non-zero vector $\mathbf{v} \in \mathcal{L}$ of minimum Euclidean norm:

$$\mathbf{v} \in \mathcal{L} \setminus \{\mathbf{0}\} \quad \text{such that} \quad \|\mathbf{v}\| = \lambda_1(\mathcal{L}). \quad (6.6)$$

Why is SVP hard?

- **Exponentially many candidates:** a ball of radius R contains approximately $R^n / \det(\mathcal{L})$ lattice points.
- **No local structure:** gradient-descent methods cannot navigate a discrete set.
- **Basis dependence:** a good basis makes the problem trivial; a bad basis makes it exponentially hard.

Figure 6.2 visualises the problem: the green vector is the shortest non-zero lattice vector λ_1 , and the dashed circle marks the minimum distance from the origin.

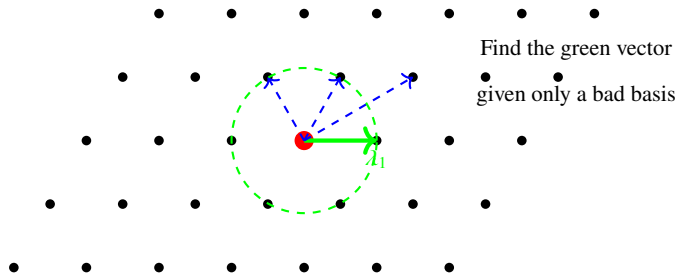


Fig. 6.2 The Shortest Vector Problem: find the shortest non-zero lattice vector, given only a bad basis as input.

6.2.2 The Closest Vector Problem

The Shortest Vector Problem asks us to find structure *within* the lattice. The Closest Vector Problem asks something subtly different: given a point that is *not* on the lattice, find the lattice point nearest to it. This is the decoding problem in disguise, and it will become the beating heart of Learning With Errors.

Definition 6.7 (Closest Vector Problem (CVP)) Given a basis $\mathbf{B}$ of a lattice $\mathcal{L}$ and a target point $\mathbf{t} \in \mathbb{R}^n$, find a lattice vector closest to $\mathbf{t}$:

$$\mathbf{v} \in \mathcal{L} \quad \text{such that} \quad \|\mathbf{v} - \mathbf{t}\| = \min_{\mathbf{x} \in \mathcal{L}} \|\mathbf{x} - \mathbf{t}\|. \quad (6.7)$$

CVP is the *decoding problem*: given a noisy observation $\mathbf{t} = \mathbf{v} + \mathbf{e}$ (lattice point plus small error), recover $\mathbf{v}$. This connection to decoding will reappear in Chap. 7, where the Learning With Errors problem is essentially a structured instance of CVP.

Theorem 6.3 (SVP–CVP Connection) *The two problems are intimately related [45]:*

1. Any γ -CVP oracle yields a (2γ) -SVP algorithm.
2. Both problems have the same quantum complexity up to polynomial factors.

6.2.3 Approximation Variants

Exact SVP and CVP are NP-hard, but cryptography does not need exact solutions—and neither do attackers. What matters is how *close* to exact one must get before the problem becomes tractable. This is controlled by the approximation factor γ , and the relationship between γ and computational complexity is where lattice-based security parameters come from.

Definition 6.8 (Approximate SVP and CVP)

$$\gamma\text{-SVP:} \quad \text{Find } \mathbf{v} \in \mathcal{L} \setminus \{\mathbf{0}\} \text{ with } \|\mathbf{v}\| \leq \gamma \cdot \lambda_1(\mathcal{L}). \quad (6.8)$$

$$\gamma\text{-CVP:} \quad \text{Find } \mathbf{v} \in \mathcal{L} \text{ with } \|\mathbf{v} - \mathbf{t}\| \leq \gamma \cdot \text{dist}(\mathbf{t}, \mathcal{L}). \quad (6.9)$$

Table 6.3 Complexity of approximate SVP as a function of the approximation factor γ

Approx. factor γ	Complexity	Best algorithm
1 (exact)	NP-hard	$2^{O(n)}$ sieving
$O(1)$	NP-hard	$2^{O(n)}$ sieving
$2^{O(\sqrt{\log n})}$	Unknown	Sub-exponential
$\text{poly}(n)$	P	LLL (polynomial)

The complexity landscape depends critically on γ :

6.2.4 Additional Problems

SVP and CVP are the problems an attacker must solve to break a scheme. But security *proofs* often reduce from different problems—ones whose worst-case hardness is easier to characterise. Two such problems recur throughout the literature.

The first generalises SVP from finding one short vector to finding an entire basis of short vectors:

Definition 6.9 (Shortest Independent Vectors Problem (SIVP)) Given a basis of an n -dimensional lattice $\mathcal{L}$, find n linearly independent vectors $\mathbf{v}_1, \dots, \mathbf{v}_n \in \mathcal{L}$ with $\max_i \|\mathbf{v}_i\| \leq \gamma \cdot \lambda_n(\mathcal{L})$.

The second converts the search problem into a decision problem—not “find a short vector” but “is there a short vector?” This formulation connects more naturally to the complexity classes NP and coNP:

Definition 6.10 (Decisional GapSVP (GapSVP)) Given a basis of $\mathcal{L}$ and a value d , distinguish between $\lambda_1(\mathcal{L}) \leq d$ and $\lambda_1(\mathcal{L}) > \gamma \cdot d$.

GapSVP is the decision version most commonly used in security reductions to LWE (see Chap. 7). It is known to lie in $\text{NP} \cap \text{coNP}$ for $\gamma = \sqrt{n}$ [1, 45].

We have now catalogued the problems. The next question is practical: how well can algorithms actually solve them? The answer comes from basis reduction—the art of transforming a bad basis into a better one—and it determines every security parameter in lattice-based cryptography.

6.3 Lattice Basis Reduction

The computational problems of the previous section are defined abstractly: “find a short vector,” “find the closest lattice point.” The primary algorithmic approach to all of them is the same—improve the basis until short vectors reveal themselves. *Basis reduction* algorithms transform a bad basis into a better one, producing shorter and

more orthogonal vectors. They are both the main tool for *solving* lattice problems and, consequently, the main tool for *attacking* lattice-based cryptosystems. Understanding their power—and its limits—is essential for setting secure parameters.

6.3.1 Gram–Schmidt Orthogonalisation

The quality of a basis is measured against its *Gram–Schmidt orthogonalisation* (GSO). Given $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n)$, define

$$\tilde{\mathbf{b}}_i = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \tilde{\mathbf{b}}_j, \quad \mu_{i,j} = \frac{\langle \mathbf{b}_i, \tilde{\mathbf{b}}_j \rangle}{\langle \tilde{\mathbf{b}}_j, \tilde{\mathbf{b}}_j \rangle}. \quad (6.10)$$

The vectors $\tilde{\mathbf{b}}_i$ are orthogonal but in general *not* lattice vectors. A well-reduced basis is one where the $\|\tilde{\mathbf{b}}_i\|$ decrease slowly—that is, no GSO vector is much shorter than its predecessor.

6.3.2 The LLL Algorithm

The Lenstra–Lenstra–Lovász (LLL) algorithm, published in 1982 [37], was the first polynomial-time lattice reduction algorithm. It remains the standard first step in any lattice attack.

What does “reduced” mean precisely? LLL defines two conditions on a basis: the Gram–Schmidt coefficients must be small (the basis is nearly size-reduced), and consecutive GSO vectors must not drop too sharply in norm (the Lovász condition). Together, these prevent the pathological near-cancellations that make bad bases bad.

Definition 6.11 (LLL-Reduced Basis) A basis $(\mathbf{b}_1, \dots, \mathbf{b}_n)$ is *LLL-reduced* with parameter $\delta \in (1/4, 1]$ if:

1. **Size-reduced:** $|\mu_{i,j}| \leq 1/2$ for all $1 \leq j < i \leq n$.
2. **Lovász condition:** $\|\tilde{\mathbf{b}}_i\|^2 \geq (\delta - \mu_{i,i-1}^2) \|\tilde{\mathbf{b}}_{i-1}\|^2$ for all $2 \leq i \leq n$.

Theorem 6.4 (LLL Guarantees [37]) *The LLL algorithm with $\delta = 3/4$ runs in polynomial time $O(n^5 d \log^3 B)$ (where B bounds the basis vector norms) and produces a basis whose first vector satisfies*

$$\|\mathbf{b}_1\| \leq 2^{(n-1)/2} \lambda_1(\mathcal{L}). \quad (6.11)$$

The exponential factor $2^{(n-1)/2}$ is the price of polynomial running time. For $n = 500$, this bound is astronomically loose: LLL finds a vector that is at most 2^{250} times the

Algorithm 3 The LLL Algorithm**Require:** Basis $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n)$, parameter $\delta \in (1/4, 1]$ **Ensure:** LLL-reduced basis

```

1: Compute GSO  $(\tilde{\mathbf{b}}_i, \mu_{i,j})$ 
2:  $k \leftarrow 2$ 
3: while  $k \leq n$  do
4:   for  $j = k - 1, k - 2, \dots, 1$  do                                 $\triangleright$  Size reduction
5:     if  $|\mu_{k,j}| > 1/2$  then
6:        $\mathbf{b}_k \leftarrow \mathbf{b}_k - \lfloor \mu_{k,j} \rfloor \mathbf{b}_j$ 
7:       Update  $\mu$  values
8:     end if
9:   end for
10:  if  $\|\tilde{\mathbf{b}}_k\|^2 \geq (\delta - \mu_{k,k-1}^2) \|\tilde{\mathbf{b}}_{k-1}\|^2$  then                 $\triangleright$  Lovász condition
11:     $k \leftarrow k + 1$ 
12:  else
13:    Swap  $\mathbf{b}_k \leftrightarrow \mathbf{b}_{k-1}$ 
14:    Update GSO
15:     $k \leftarrow \max(k - 1, 2)$ 
16:  end if
17: end while
18: return  $(\mathbf{b}_1, \dots, \mathbf{b}_n)$ 

```

shortest. This is why LLL alone does *not* break lattice-based cryptosystems, though it serves as a crucial preprocessing step.

6.3.3 BKZ: The Practical Workhorse

LLL finds short vectors in polynomial time, but its approximation quality degrades exponentially with dimension—far too loose to threaten cryptographic parameters. To do better, we need to invest more computation. The *Block Korkin–Zolotarev* (BKZ) algorithm of Schnorr and Euchner (1994) achieves this by applying exact SVP solvers to local blocks within the basis, then propagating the improvement globally. The block size β controls the tradeoff: larger blocks yield shorter vectors but cost exponentially more time. The core idea is simple enough to fit in a single paragraph:

BKZ- β in a nutshell. Slide a window of width β along the GSO, solve exact SVP within each projected block, and insert the short vector back into the basis. Repeat until no further progress.

The tradeoff is controlled entirely by β :

- $\beta = 2$ recovers LLL.
- $\beta = n$ recovers HKZ reduction (exact SVP, exponential time).

- For intermediate β , BKZ- β runs in time $T(\beta) \cdot \text{poly}(n)$, where $T(\beta)$ is the cost of the SVP oracle in dimension β .

To quantify BKZ's output, we need a single number that summarises how short the first basis vector is relative to the lattice determinant. The root Hermite factor serves this purpose—it is the parameter that practitioners use to translate block size into concrete security estimates.

Definition 6.12 (Root Hermite Factor) After BKZ- β reduction, the first basis vector satisfies

$$\|\mathbf{b}_1\| \approx \delta_0^n \cdot \det(\mathcal{L})^{1/n}, \quad (6.12)$$

where the *root Hermite factor* δ_0 depends on β approximately as

$$\delta_0 \approx \left(\frac{\beta}{2\pi e} (\pi\beta)^{1/\beta} \right)^{\frac{1}{2(\beta-1)}}. \quad (6.13)$$

Smaller δ_0 means stronger reduction.

In practice, BKZ- β with $\beta \geq 60$ already surpasses what LLL can achieve, and the transition to the cryptographically relevant regime occurs around $\beta \approx 300$ –400.

6.3.4 The Core-SVP Hardness Model

The root Hermite factor tells us what block size an attacker needs. To convert that block size into a security level measured in bits—the currency that NIST and every standards body uses—we need one more ingredient: the cost of the SVP oracle that BKZ calls as a subroutine. The *core-SVP model* makes this connection explicit.

Definition 6.13 (Core-SVP Cost) The cost of running BKZ- β is dominated by the cost of the SVP oracle in dimension β . Using the best known algorithms:

$$\text{Classical sieving: } T_{\text{class}}(\beta) = 2^{0.292\beta + o(\beta)}, \quad (6.14)$$

$$\text{Quantum sieving: } T_{\text{quant}}(\beta) = 2^{0.265\beta + o(\beta)}. \quad (6.15)$$

Given a lattice-based scheme with parameters requiring BKZ block size β to break, the security in bits is $\approx 0.292\beta$ (classical) or $\approx 0.265\beta$ (quantum). This model is used to set the parameters of ML-KEM and ML-DSA (Chaps. 7–8).

Basis reduction gives us a concrete understanding of what attackers can do. But how confident should we be that these problems are *truly* hard, rather than merely hard for current algorithms? The next section addresses this question with a result that has no analogue in classical cryptography: a proof that average-case hardness follows from worst-case hardness.

6.4 Worst-Case to Average-Case Reductions

Basis reduction algorithms tell us how hard lattice problems are *in practice*—for the best algorithms we know today. But cryptographic confidence demands something deeper. How do we know that a clever new algorithm will not suddenly make random lattice instances easy, the way Shor’s algorithm made random factoring instances easy? Classical cryptography has no answer to this question: we *believe* that random RSA moduli are hard to factor, but we cannot prove it. Lattice-based cryptography offers something stronger.

6.4.1 Why Average-Case Hardness Matters

The distinction between worst-case and average-case hardness is subtle but consequential. A cryptographic scheme generates its keys *at random*. Its security therefore depends on the hardness of random instances, not worst-case instances. Knowing that some lattice instances are exponentially hard is cold comfort if the random instances used in practice happen to be easy. A *worst-case to average-case reduction* eliminates this gap: it shows that if *any* algorithm breaks a non-negligible fraction of random instances, then the same algorithm can be used to solve *every* instance of a worst-case problem believed to be hard. This is a qualitatively different kind of security guarantee—one that RSA, Diffie–Hellman, and elliptic-curve cryptography simply do not have.

6.4.2 Ajtai’s Breakthrough

In 1996, Miklós Ajtai proved the result that made lattice-based cryptography possible [1]. He showed that a specific average-case problem—finding short integer solutions to a random system of linear equations modulo q —is at least as hard as the worst case of SIVP. The problem itself is deceptively simple:

Definition 6.14 (Short Integer Solutions (SIS)) Given a uniformly random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a bound β , find a non-zero vector $\mathbf{x} \in \mathbb{Z}^m$ with $\|\mathbf{x}\| \leq \beta$ such that

$$\mathbf{A}\mathbf{x} = \mathbf{0} \pmod{q}. \quad (6.16)$$

Theorem 6.5 (Ajtai [1], Micciancio–Regev [46]) *For appropriate parameters, solving SIS on a random $\mathbf{A}$ is at least as hard as solving γ -SIVP on every n -dimensional lattice (worst case), where $\gamma = \beta \cdot \text{poly}(n)$.*

The implication is remarkable: breaking a cryptosystem built on random SIS instances would require an algorithm capable of finding short vectors in *every* lattice—including the hardest ones. No other public-key family offers a comparable guarantee.

6.4.3 Proof Intuition

The reduction works by showing that a random matrix $\mathbf{A}$ encodes information about an arbitrary “target” lattice in a way that is invisible to the adversary. The key tools are:

1. **Gaussian sampling:** Sampling lattice points from a discrete Gaussian distribution $D_{\mathcal{L},s}$ with standard deviation s . When s is above the *smoothing parameter* $\eta_\varepsilon(\mathcal{L})$, the samples look statistically close to continuous Gaussians, hiding the lattice structure.
2. **The smoothing parameter:** $\eta_\varepsilon(\mathcal{L})$ is the smallest s such that $D_{\mathcal{L}^*,1/s}$ has mass at most $1 + \varepsilon$ at the origin. Intuitively, it is the scale at which the lattice “looks smooth” (continuous) to a statistical test.
3. **Rerandomisation:** By combining Gaussian samples from the target lattice with the SIS oracle’s output, the reduction transforms the adversary’s success into a SIVP solver.

A complete proof appears in Appendix A.1. We note that Regev [56] later proved an analogous reduction for LWE (Chap. 7), using a *quantum* reduction from GAPSVP to LWE.

Ajtai’s result and its descendants give lattice-based cryptography a theoretical foundation unmatched in public-key cryptography. But theoretical hardness reductions involve polynomial losses that can be large, and they say nothing about the *exact* complexity of the problems involved. The final section of this chapter surveys what we know—and what remains open—about the concrete complexity of lattice problems, both classically and quantumly.

6.5 Computational Complexity of Lattice Problems

6.5.1 Classical Complexity

We have seen that basis reduction algorithms provide upper bounds on how easy lattice problems are, and worst-case reductions provide lower bounds on how hard they must be. The full complexity picture fills in the space between these bounds. The results of three decades of study [45, 54] can be summarised as follows:

- **Exact SVP:** NP-hard under randomised reductions [1].
- **γ -GAPSVP for $\gamma = \sqrt{n}$:** in $\text{NP} \cap \text{coNP}$ (hence unlikely to be NP-hard).
- **γ -SVP for $\gamma = \text{poly}(n)$:** solvable in polynomial time via LLL.

The best known classical algorithms for exact or near-exact SVP are:

Table 6.4 Classical algorithms for SVP in dimension n

Algorithm	Time	Approx.	Memory
LLL [37]	$\text{poly}(n)$	$2^{O(n)}$	$\text{poly}(n)$
BKZ- β	$2^{O(\beta)} \cdot \text{poly}(n)$	$\gamma(\beta)^n$	$\text{poly}(n)$
Sieving	$2^{0.292n+o(n)}$	Exact	$2^{0.208n}$
Enumeration	$2^{O(n \log n)}$	Exact	$\text{poly}(n)$

6.5.2 Quantum Complexity

For a book on post-quantum cryptography, this subsection contains the most important single fact: quantum computers do not help much with lattice problems. The contrast with factoring is stark. Where Shor’s algorithm obliterates RSA by exploiting algebraic periodicity, the best known quantum algorithms for lattice problems provide only a modest polynomial speedup—far too small to threaten properly sized parameters.

Theorem 6.6 (Quantum Speedup for Lattice Problems) *The best known quantum algorithms for SVP in dimension n achieve:*

$$T_{\text{quant}} = 2^{0.265n+o(n)}, \quad (6.17)$$

compared to the classical $2^{0.292n+o(n)}$. The speedup factor is $2^{0.027n} \approx 1.019^n$ —polynomial, not exponential.

This stands in sharp contrast to factoring, where Shor’s algorithm provides an *exponential* speedup (from sub-exponential $e^{O(n^{1/3} \log^{2/3} n)}$ to polynomial $O(n^3)$). The natural question is: why? What is it about lattice problems that denies quantum computers the leverage they enjoy elsewhere?

Why no quantum breakthrough for lattices? Four structural reasons are commonly cited:

1. *No periodic structure:* Shor’s algorithm exploits the periodicity of modular exponentiation. Lattice problems lack comparable periodic structure.
2. *No hidden subgroup:* The best quantum algorithms for algebraic problems exploit the hidden subgroup framework. Lattice problems do not fit naturally into this framework.
3. *Geometric, not algebraic:* Lattice problems are fundamentally geometric. Quantum computers have not shown exponential advantages for geometric optimisation.
4. *Grover gives little:* The search space is too large for Grover’s quadratic speedup to be meaningful ($2^{0.146n}$ is still exponential).

Figure 6.3 compares the quantum advantage for lattice problems with the exponential speedup that Shor’s algorithm provides for factoring. The narrow gap between classical

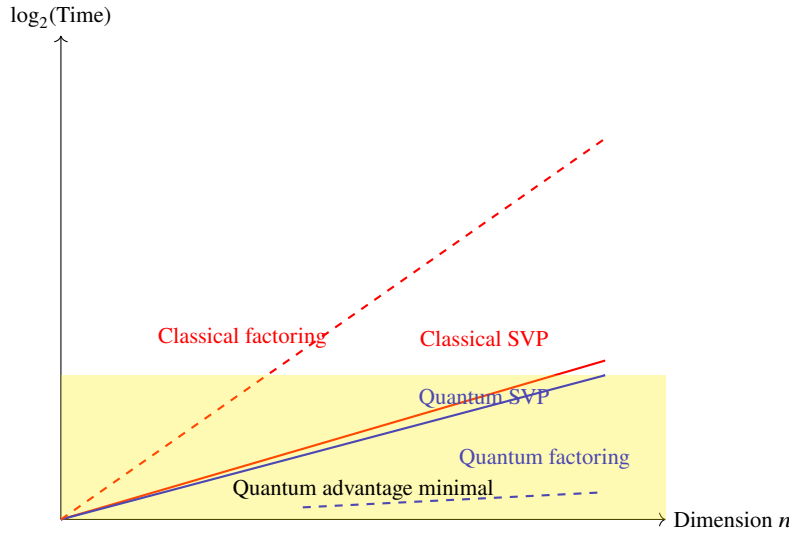


Fig. 6.3 Quantum speedup for lattice problems is polynomial (narrow gap between SVP curves), not exponential as for factoring (wide gap between factoring curves).

Table 6.5 Lattice dimensions required for NIST security levels

Dimension	Classical	Quantum	NIST Level
~ 400	2^{117}	2^{106}	Below I
~ 500	2^{146}	2^{133}	I (AES-128 equiv.)
~ 600	2^{175}	2^{159}	III (AES-192 equiv.)
~ 700	2^{204}	2^{186}	V (AES-256 equiv.)

and quantum SVP curves—in contrast to the chasm between classical and quantum factoring—is the geometric reason that lattice-based cryptography survives the quantum transition.

6.5.3 Concrete Security Estimates

Asymptotic hardness is necessary but not sufficient for parameter selection—we need numbers, not big- O notation. Translating complexity into deployable security levels requires fixing a cost model. Using the core-SVP model (Definition 6.13), we can map lattice dimensions to the NIST security levels that govern standardisation:

Example 6.3 (Breaking ML-KEM-768) ML-KEM-768 (Chap. 7) uses Module-LWE with effective lattice dimension 768. The best known classical attack requires approximately 2^{224} operations; the best known quantum attack, approximately 2^{203} . Even with all of Earth’s computing power operating in parallel, the classical attack would take

roughly 10^{50} years. Compare this with RSA-2048, which Shor's algorithm breaks in approximately 8 hours on a sufficiently large quantum computer [22].

6.5.4 Summary: Why Lattices Survive Quantum Computers

We can now see why NIST chose lattice-based schemes as the primary post-quantum standards. Lattice problems occupy a unique position in the complexity landscape, combining five properties that no other cryptographic family matches simultaneously. Average-case hardness reduces to worst-case hardness [1, 56], so random instances are provably no easier than the hardest ones. Exact SVP is NP-hard, anchoring the entire edifice to a complexity-theoretic pillar. No exponential quantum speedup is known—the best quantum algorithms save a constant in the exponent, not a polynomial in the input size. The problems have withstood over 25 years of sustained cryptanalytic attention without a breakthrough. And they are easy to state—find short vectors in a lattice—yet exponentially hard to solve.

This combination of theoretical depth and practical maturity is why lattices, not codes or isogenies, form the backbone of the post-quantum transition. The next chapter translates this hardness into practical cryptographic schemes via the Learning With Errors problem—a structured variant of CVP that turns lattice geometry into encryption and key exchange.

Problems

6.1 Consider the lattice $\mathcal{L} \subset \mathbb{R}^2$ with basis $\mathbf{b}_1 = (1, 0)$ and $\mathbf{b}_2 = (1/2, \sqrt{3}/2)$.

- Compute $\det(\mathcal{L})$ and identify this as a familiar physical lattice.
- Compute $\lambda_1(\mathcal{L})$ and $\lambda_2(\mathcal{L})$.
- Find a second basis for the same lattice and verify that the determinant is unchanged.
- Compute the dual lattice $\mathcal{L}^*$ and its basis.

6.2 Apply the LLL algorithm with $\delta = 3/4$ to the basis $\mathbf{b}_1 = (1, 1, 1)$, $\mathbf{b}_2 = (-1, 0, 2)$, $\mathbf{b}_3 = (3, 5, 6)$.

- Compute the Gram–Schmidt orthogonalisation.
- Perform each size-reduction and swap step.
- Verify that the output basis satisfies both LLL conditions.
- Compare $\|\mathbf{b}_1\|$ before and after reduction.

6.3 Using the core-SVP model, compute the classical and quantum security levels (in bits) for a lattice-based scheme that requires BKZ with block size $\beta = 400$ to break. How much does the quantum advantage matter in practical terms?

6.4 Minkowski's bound.

- (a) Prove that any lattice $\mathcal{L} \subset \mathbb{R}^n$ with $\det(\mathcal{L}) < 1$ contains a non-zero vector of norm at most $\sqrt{n}$. (Hint: use the pigeonhole principle on the hypercube $[-1/2, 1/2]^n$.)
- (b) For the lattice with basis $\mathbf{B} = 3\mathbf{I}_n$ (the $n \times n$ identity scaled by 3), compute $\lambda_1(\mathcal{L})$, $\det(\mathcal{L})$, and verify Minkowski's bound.

6.5 Let $\mathcal{L} \subset \mathbb{R}^2$ be the lattice with basis $\mathbf{b}_1 = (2, 1)$, $\mathbf{b}_2 = (0, 3)$.

- (a) Compute the dual basis and verify that $\langle \mathbf{b}_i, \mathbf{d}_j \rangle = \delta_{ij}$ (Kronecker delta).
- (b) Verify that $\det(\mathcal{L}) \cdot \det(\mathcal{L}^*) = 1$.
- (c) Find $\lambda_1(\mathcal{L})$ and $\lambda_1(\mathcal{L}^*)$. Verify the transference bound $\lambda_1(\mathcal{L}) \cdot \lambda_1(\mathcal{L}^*) \leq n$.

6.6 Root Hermite factor analysis.

A lattice-based scheme claims 128-bit classical security. Using (6.14), determine the minimum BKZ block size β needed by an attacker. Then use (6.13) to compute the root Hermite factor δ_0 . If the lattice dimension is $n = 768$ and $\det(\mathcal{L})^{1/n} = q^{k/n}$ with $q = 3329$ and $k = 3$, estimate $\|\mathbf{b}_1\|$ after BKZ- β reduction.

6.7 Research-level.

Ajtai's reduction shows that breaking random SIS instances is as hard as worst-case SIVP. Explain why this type of guarantee is not available for RSA (i.e., why factoring a random RSA modulus might in principle be easier than factoring the hardest composite). What are the practical implications for long-term cryptographic confidence?

Chapter 7

Learning With Errors and Lattice-Based Encryption

Abstract The Learning With Errors (LWE) problem transforms the geometric hardness of lattice problems into a practical cryptographic tool: solving noisy systems of linear equations over finite fields. This chapter develops LWE from its definition through its hardness guarantees, introduces the structured variants Ring-LWE and Module-LWE that enable practical efficiency, presents the Fujisaki–Okamoto transform that upgrades chosen-plaintext security to chosen-ciphertext security, and culminates in a complete description of ML-KEM—the NIST standard for post-quantum key encapsulation.

7.1 The LWE Problem

Every physicist knows how to solve systems of linear equations. The Learning With Errors problem adds a single twist: small noise in the right-hand side. This transforms a polynomial-time computation into a problem believed to be exponentially hard, even for quantum computers [56].

Definition 7.1 (Learning With Errors — Search Version) Let $n, q \in \mathbb{N}$ with $q \geq 2$, and let χ be a probability distribution over $\mathbb{Z}_q$ (the *error distribution*). The *search-LWE* problem is: given access to arbitrarily many independent samples

$$(\mathbf{a}_i, b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \bmod q), \quad \mathbf{a}_i \xleftarrow{\$} \mathbb{Z}_q^n, \quad e_i \leftarrow \chi, \quad (7.1)$$

recover the secret vector $\mathbf{s} \in \mathbb{Z}_q^n$.

Without the error terms ($e_i = 0$), the secret is recovered by Gaussian elimination on n samples. With errors drawn from a distribution of standard deviation $\sigma \ll q$, the problem becomes qualitatively different: the noise is too small to destroy the linear structure, yet large enough to prevent direct algebraic solution.

Definition 7.2 (Learning With Errors — Decision Version) Distinguish between the following two distributions over $\mathbb{Z}_q^n \times \mathbb{Z}_q$:

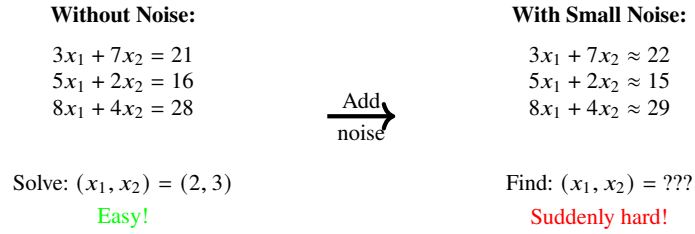


Fig. 7.1 The LWE problem: small noise transforms easy linear algebra into a hard problem.

Table 7.1 LWE parameter tradeoffs

Parameter	Typical range	Effect of increase
n (dimension)	512–1024	More security, larger keys
q (modulus)	2^{12} – 2^{15}	Easier implementation, slightly weaker security
σ (noise)	1.6–6.4	More security, higher decryption failure rate

1. LWE samples: $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e \bmod q)$ with $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$, $e \leftarrow \chi$.
2. Uniform samples: $(\mathbf{a}, u)$ with $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$, $u \xleftarrow{\$} \mathbb{Z}_q$.

Theorem 7.1 (Search–Decision Equivalence [56]) *For prime q and Gaussian error distribution with parameter $\sigma \geq 2\sqrt{n}$, search-LWE and decision-LWE are polynomial-time equivalent.*

This equivalence is essential for cryptographic applications, which typically rely on the *indistinguishability* guarantee of the decision version.

Figure 7.1 captures the essence of LWE in a single image: adding small noise to a system of linear equations transforms a trivial computation into a problem believed to be exponentially hard.

7.1.1 Parameters

The definition of LWE leaves three quantities free: the dimension n , the modulus q , and the noise width σ . These parameters are not independent—they pull against each other in a three-way tension that determines both the security and the usability of any scheme built on LWE.

The noise-to-modulus ratio $\alpha = \sigma/q$ is the key quantity: too small and the problem becomes easy (the noise can be rounded away); too large and decryption fails. Cryptographic parameters sit in a narrow sweet spot where the noise is large enough for security yet small enough for correctness.

7.1.2 The Error Distribution

The parameter σ controls the noise, but we have not yet said what *shape* the noise takes. The theoretical treatment of LWE uses a discrete Gaussian, while practical implementations substitute a simpler distribution that is easier to sample in constant time.

Definition 7.3 (Discrete Gaussian Distribution) The *discrete Gaussian* $D_{\mathbb{Z},\sigma}$ over the integers with parameter $\sigma > 0$ is the distribution assigning to each $x \in \mathbb{Z}$ a probability proportional to $\exp(-x^2/2\sigma^2)$. It is supported on integers in $\{x : |x| \leq B\}$ with negligible tail probability for $B = \Theta(\sigma\sqrt{\log n})$.

In the physics of thermal noise, σ plays the role of temperature: higher σ means more noise, stronger security, but also more decryption errors. The NIST standards use a *centred binomial distribution* β_η (sampling $\sum_{i=1}^\eta (a_i - b_i)$ for uniform bits a_i, b_i) as a practical substitute that is easier to implement in constant time.

7.2 Hardness of LWE

We have defined LWE and tuned its parameters, but definition is not destiny—many plausible-looking problems turn out to be easy. What makes LWE exceptional among cryptographic assumptions is the strength of its hardness evidence. Unlike RSA or discrete logarithms, whose security rests on decades of failed attacks but no formal worst-case guarantee, LWE enjoys a security guarantee unmatched in classical cryptography: solving random LWE instances is as hard as solving worst-case lattice problems.

7.2.1 Regev’s Quantum Reduction

The centrepiece of LWE hardness theory is Regev’s 2005 reduction, which established a remarkable connection: any algorithm that solves LWE efficiently can be transformed into a quantum algorithm that solves worst-case lattice problems.

Theorem 7.2 (Regev [56]) *For appropriate parameters ($q \leq \text{poly}(n)$, $\sigma \geq 2\sqrt{n}$), there is a quantum polynomial-time reduction from $\tilde{O}(n/\alpha)$ -GAP SVP to decision-LWE($n, q, D_{\mathbb{Z},\sigma}$) with $\alpha = \sigma/q$. That is, if decision-LWE can be solved efficiently, then a quantum algorithm can solve GAP SVP on every n -dimensional lattice.*

The qualifier “quantum” refers to the *reduction*, not the attacker. Even a purely classical LWE solver would, via this quantum reduction, yield a quantum algorithm for worst-case lattice problems.

Reduction intuition for physicists. Regev’s reduction exploits the duality between a lattice $\mathcal{L}$ and its dual $\mathcal{L}^*$ via the quantum Fourier transform—precisely the position–momentum duality familiar from quantum mechanics.

1. Start with an arbitrary target lattice $\mathcal{L}$ (a worst-case instance of GapSVP).
2. Use Gaussian sampling on $\mathcal{L}$ to produce states encoding the dual lattice $\mathcal{L}^*$.
3. Apply the quantum Fourier transform to convert dual-lattice information into LWE samples.
4. Feed these samples to the LWE oracle.
5. Use the oracle’s output to extract short vectors in $\mathcal{L}$, solving GapSVP .

The noise level σ maps to the approximation factor γ : more noise means a weaker (larger γ) lattice guarantee, but stronger LWE security.

A full proof sketch appears in Appendix A.2.

7.2.2 Classical Reductions

The word “quantum” in Regev’s reduction is a persistent source of confusion. Can we remove it?

Peikert (2009) and Brakerski et al. (2013) proved purely classical reductions from GapSVP to LWE, at the cost of a larger approximation factor or requiring a super-polynomial modulus. For cryptographic practice, the quantum reduction remains the tightest and most commonly cited.

7.2.3 Concrete Hardness

Worst-case reductions tell us that LWE is hard in principle, but they do not tell us which specific parameter choices yield 128-bit security. For that, we need to understand how the best known attacks actually perform.

The best known attacks on LWE reduce to lattice problems via the *primal* and *dual* attack strategies:

- **Primal attack:** Embed the LWE instance into a lattice and run BKZ to find the short error vector.
- **Dual attack:** Find a short vector in the dual lattice to distinguish LWE from uniform.

Both attacks require BKZ with block size β , giving security $\approx 0.292 \beta$ bits (classical) or $\approx 0.265 \beta$ bits (quantum), as established in Chap. 6.

7.3 LWE-Based Encryption

Hardness alone does not give us a cryptosystem—we need a construction that turns LWE samples into ciphertexts. Regev’s 2005 encryption scheme [56] remains the template for all modern LWE-based constructions. It encrypts a single bit $\mu \in \{0, 1\}$, and its simplicity makes the security argument almost transparent.

Definition 7.4 (Regev’s Encryption Scheme) Fix parameters (n, q, m, χ) with $m = O(n \log q)$.

- **KeyGen:** Sample $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$. For $i = 1, \dots, m$: sample $\mathbf{a}_i \xleftarrow{\$} \mathbb{Z}_q^n$, $e_i \leftarrow \chi$, compute $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \bmod q$. Set $\text{pk} = (\mathbf{A}, \mathbf{b})$ and $\text{sk} = \mathbf{s}$.
- **Enc(pk, μ):** Choose a random subset $S \subseteq [m]$. Output

$$\mathbf{c} = \left(\sum_{i \in S} \mathbf{a}_i, \sum_{i \in S} b_i + \mu \cdot \lfloor q/2 \rfloor \right) \bmod q. \quad (7.2)$$

- **Dec(sk, ($\mathbf{a}'$, b')):** Compute $v = b' - \langle \mathbf{a}', \mathbf{s} \rangle \bmod q$. Output 0 if $|v| < q/4$, else output 1.

Theorem 7.3 (Correctness) Let $(\mathbf{a}', b')$ be a ciphertext encrypting μ . Then

$$v = b' - \langle \mathbf{a}', \mathbf{s} \rangle = \sum_{i \in S} e_i + \mu \cdot \lfloor q/2 \rfloor. \quad (7.3)$$

Decryption succeeds whenever $|\sum_{i \in S} e_i| < q/4$.

Theorem 7.4 (Security) If decision-LWE(n, q, χ) is hard, Regev’s scheme is IND-CPA secure.

Proof sketch. The public key $(\mathbf{A}, \mathbf{b})$ consists of LWE samples. By the decision-LWE assumption, these are indistinguishable from uniform. If $\mathbf{b}$ is uniform, then the ciphertext carries no information about μ , so no adversary can distinguish encryptions of 0 from encryptions of 1. $\square$

Figure 7.2 illustrates the geometric picture behind Regev’s encryption. The ciphertext lands in a noise cloud around the secret point; with the secret key, decoding is straightforward, but without it one must solve LWE.

Regev’s scheme demonstrates feasibility but is not practical: public keys are $O(n^2 \log q)$ bits. The route to practical schemes proceeds through two steps: structured lattices (Sect. 7.4–7.5) and the Fujisaki–Okamoto transform (Sect. 7.6).

7.4 Ring-LWE: Efficiency Through Structure

Regev’s scheme proves that LWE can yield encryption, but the resulting public keys—megabytes of random matrix entries—are impractical for any real protocol. The question

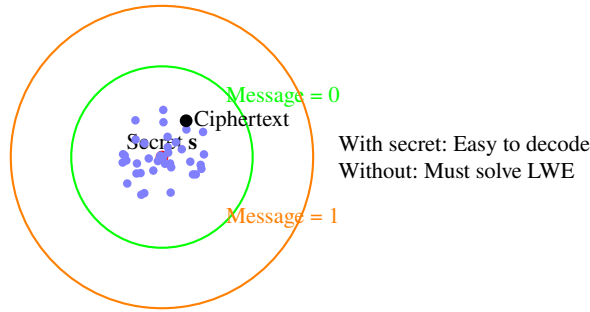


Fig. 7.2 LWE encryption geometry: messages are encoded as distances from the secret point, hidden within noise clouds.

is whether we can compress these keys without destroying the hardness guarantee. **RING-LWE** answers affirmatively: it replaces the unstructured random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ with multiplication in a polynomial ring, compressing keys by a factor of n .

7.4.1 The Polynomial Ring R_q

Definition 7.5 (The Cyclotomic Ring) Let n be a power of 2. Define

$$R = \mathbb{Z}[X]/(X^n + 1), \quad R_q = \mathbb{Z}_q[X]/(X^n + 1). \quad (7.4)$$

Elements of R_q are polynomials of degree $< n$ with coefficients in $\mathbb{Z}_q$.

The polynomial $X^n + 1$ is the $2n$ -th cyclotomic polynomial, irreducible over $\mathbb{Z}$ when n is a power of 2. This choice is motivated by three requirements:

1. **Security:** Ideal lattices in cyclotomic rings admit worst-case to average-case reductions [39].
2. **Fast arithmetic:** The Number Theoretic Transform (NTT) computes polynomial multiplication in $O(n \log n)$ operations when $q \equiv 1 \pmod{2n}$.
3. **Negacyclic structure:** Multiplication by a ring element corresponds to a negacyclic matrix, so a single polynomial encodes an $n \times n$ structured matrix:

$$a(X) \cdot b(X) \longleftrightarrow \begin{pmatrix} a_0 & -a_{n-1} & \cdots & -a_1 \\ a_1 & a_0 & \cdots & -a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & \cdots & a_0 \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_{n-1} \end{pmatrix}. \quad (7.5)$$

7.4.2 Ring-LWE Definition

With the ring R_q in hand, the Ring-LWE problem is the natural adaptation of LWE: replace vectors over $\mathbb{Z}_q$ with elements of R_q , and replace matrix–vector multiplication with polynomial multiplication.

Definition 7.6 (Ring Learning With Errors [39]) Let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ and χ be an error distribution over R . Given samples

$$(a_i, b_i = a_i \cdot s + e_i) \in R_q \times R_q, \quad a_i \xleftarrow{\$} R_q, \quad e_i \leftarrow \chi, \quad (7.6)$$

the search problem is to recover $s \in R_q$; the decision problem is to distinguish these from uniform pairs (a_i, u_i) .

Each RING-LWE sample provides n scalar LWE equations (one per coefficient), so a single sample suffices for encryption. This compresses public keys from $O(n^2)$ to $O(n)$ elements.

Theorem 7.5 (RING-LWE Hardness [39, 40]) *For the cyclotomic ring R_q and appropriate parameters, solving RING-LWE is at least as hard as solving γ -SVP on ideal lattices in the worst case, where $\gamma = \text{poly}(n)$.*

7.4.3 The Number Theoretic Transform

The key size advantage of Ring-LWE would be hollow if polynomial multiplication were slow. Naïve multiplication of two degree- n polynomials costs $O(n^2)$ operations, but the Number Theoretic Transform reduces this to $O(n \log n)$ —the same asymptotic speedup that the FFT brings to signal processing.

The NTT is the finite-field analogue of the FFT. For q prime with $q \equiv 1 \pmod{2n}$, a primitive $2n$ -th root of unity $\omega \in \mathbb{Z}_q$ exists, and every $a(X) \in R_q$ can be represented in the *NTT domain*:

$$\hat{a}_j = a(\omega^{2j+1}) \bmod q, \quad j = 0, 1, \dots, n-1. \quad (7.7)$$

In NTT domain, polynomial multiplication becomes component-wise: $\widehat{a \cdot b}_j = \hat{a}_j \cdot \hat{b}_j \bmod q$. The forward and inverse transforms each cost $O(n \log n)$ multiplications modulo q .

Example 7.1 (ML-KEM's NTT Parameters) ML-KEM uses $n = 256$ and $q = 3329$. Since $3329 = 13 \cdot 256 + 1 \equiv 1 \pmod{512}$, the NTT exists with root $\omega = 17$. A polynomial multiplication costs $256 \cdot 9 \approx 2300$ multiply-add operations—roughly $100\times$ faster than naïve $O(n^2)$ multiplication.

7.4.4 Security of Structured Variants

Efficiency gains always invite suspicion: the structure that makes Ring-LWE fast might also make it vulnerable. Adding algebraic structure raises a natural concern: does the structure enable new attacks?

After 15 years of intensive cryptanalysis, the answer is nuanced:

- No polynomial-time (classical or quantum) algorithm is known that exploits the ring structure.
- The best concrete attacks on RING-LWE and MODULE-LWE achieve only marginal improvements over generic lattice attacks.
- Sub-field attacks (2016) apply only when parameters are poorly chosen.

The cryptographic community’s pragmatic response to residual concerns about ideal lattice structure is Module-LWE.

7.5 Module-LWE: The Middle Ground

Ring-LWE compresses keys dramatically, but it does so by placing all security in a single polynomial ring—a single ideal lattice. If some future algorithm were to exploit the rich algebraic structure of ideal lattices, Ring-LWE would collapse entirely. The question that motivated the CRYSTALS team, and ultimately NIST’s selection, was whether one can reclaim most of Ring-LWE’s efficiency while hedging against this risk.

The answer is Module-LWE: instead of working with scalars in R_q , we work with short *vectors* of ring elements. The matrix $\mathbf{A}$ is now $k \times k$ over R_q rather than 1×1 (Ring-LWE) or $n \times n$ over $\mathbb{Z}_q$ (plain LWE). An attacker who can break the ring structure of one component still faces a lattice problem in the remaining $k - 1$ dimensions. For small k , the overhead relative to Ring-LWE is modest, and the resulting problem sits on a strictly harder security landscape.

Definition 7.7 (Module Learning With Errors) Work over R_q^k (vectors of k ring elements) for a small integer k :

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in R_q^{k \times k} \times R_q^k, \quad \mathbf{A} \xleftarrow{\$} R_q^{k \times k}, \quad \mathbf{s}, \mathbf{e} \leftarrow \chi^k. \quad (7.8)$$

MODULE-LWE interpolates between plain LWE ($k = n$, no ring structure) and RING-LWE ($k = 1$, maximum structure). For $k = 2, 3, 4$ it retains most of the efficiency of RING-LWE while relying on a weaker structural assumption. NIST selected MODULE-LWE as the foundation for both ML-KEM and ML-DSA.

The practical impact of this design choice is visible in key sizes. Table 7.2 shows that Module-LWE pays only a small premium over Ring-LWE while reducing public keys by three orders of magnitude compared to plain LWE.

The parameter k also provides a convenient “knob” for scaling security: ML-KEM-512 uses $k = 2$, ML-KEM-768 uses $k = 3$, and ML-KEM-1024 uses $k = 4$, all with the

Table 7.2 Key size comparison across LWE variants (≈ 128 -bit quantum security)

Variant	Public key	Ciphertext	Parameters
Plain LWE	~ 1.6 MB	~ 2 KB	$n \approx 500$
Module-LWE	1,184 B	1,088 B	$n = 256, k = 3$
Ring-LWE	~ 800 B	~ 768 B	$n \approx 512$

same ring dimension $n = 256$. This modularity is a significant engineering advantage—the same core implementation serves all security levels, with only the matrix dimension changing.

With Module-LWE providing the hard problem and Ring structure providing the efficiency, we have the algebraic foundation for a practical encryption scheme. But one critical piece remains: upgrading the security guarantee from passive eavesdropping resistance to full chosen-ciphertext security.

7.6 The Fujisaki–Okamoto Transform

We now have an efficient encryption scheme built on Module-LWE, but its security guarantee has a critical limitation. Regev’s encryption scheme (and its Ring/Module variants) achieve only IND-CPA security: they are secure against passive eavesdroppers. Real-world applications require IND-CCA2 security (resistance to chosen-ciphertext attacks), where the adversary can submit ciphertexts for decryption.

The Fujisaki–Okamoto (FO) transform [19] is a generic compiler that upgrades any IND-CPA encryption scheme into an IND-CCA2 key encapsulation mechanism (KEM).

7.6.1 Construction

The transform is elegant in its simplicity: it uses hash functions to bind the encryption randomness to the message, so that any tampering with the ciphertext becomes detectable.

Given an IND-CPA-secure public-key encryption scheme $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec})$:

Definition 7.8 (FO-Transformed KEM)

- **KeyGen'**: Run $(\text{pk}, \text{sk}') \leftarrow \text{KeyGen}$. Set $\text{sk} = (\text{sk}', \text{pk}, s)$ where $s \xleftarrow{\$} \{0, 1\}^{256}$ is a “failure secret.”
- **Encaps(pk)**: Sample a random message $m \xleftarrow{\$} \{0, 1\}^\ell$. Compute the shared key $K = H(m, \text{pk})$ and the randomness $r = G(m, \text{pk})$. Encrypt: $c = \text{Enc}(\text{pk}, m; r)$. Return (K, c) .
- **Decaps(sk, c)**: Decrypt $m' = \text{Dec}(\text{sk}', c)$. Recompute $r' = G(m', \text{pk})$ and $c' = \text{Enc}(\text{pk}, m'; r')$. If $c' = c$, return $K = H(m', \text{pk})$. Otherwise, return $K = H(s, c)$ (*implicit rejection*).

The re-encryption check ($c' = c$) is the key insight: it detects any ciphertext that was not honestly generated, preventing an adversary from learning useful information through decryption queries. Implicit rejection (returning a pseudorandom key instead of an error symbol) prevents timing side channels.

Theorem 7.6 (FO Security [19]) *In the (quantum) random oracle model, if Π is IND-CPA secure and δ -correct (decryption failure probability $\leq \delta$), then the FO-transformed KEM is IND-CCA2 secure with security loss proportional to δ .*

This is why the decryption failure probability of ML-KEM is engineered to be astronomically small ($< 2^{-140}$): the FO security proof requires it.

With the theoretical machinery complete—MODULE-LWE for hardness, ring arithmetic for speed, and the FO transform for CCA2 security—we can now assemble the full ML-KEM construction.

7.7 ML-KEM (CRYSTALS-Kyber)

ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism), standardised as FIPS 203 [50], welds these ingredients into a single, concrete construction—the first post-quantum KEM to become a NIST standard. What follows is the complete specification: every line of the algorithms maps directly to the theory developed above.

7.7.1 Construction

The construction consists of three algorithms. Key generation produces an MODULE-LWE instance; encapsulation encrypts a random message under it; decapsulation recovers the message and applies the FO consistency check. We present each algorithm with its design rationale.

Key generation.

The goal of key generation is to produce a public MODULE-LWE sample $(\mathbf{A}, \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e})$ that anyone can use for encryption, together with the secret $\mathbf{s}$ needed for decryption. Two design choices deserve attention. First, the matrix $\mathbf{A}$ is not transmitted explicitly; instead, a 32-byte seed ρ is sent, and both parties expand it deterministically using an extendable-output function (XOF). This reduces the public key from $k^2 \cdot n \cdot 12$ bits to $k \cdot n \cdot 12 + 256$ bits. Second, all arithmetic is performed in the NTT domain, so the secret and error vectors are transformed immediately after sampling—every subsequent operation is a pointwise multiplication rather than a full polynomial product.

Algorithm 4 ML-KEM.KeyGen**Require:** Parameters $(n = 256, k, q = 3329, \eta_1, \eta_2)$ **Ensure:** Public key pk , secret key sk

- 1: $\rho, \sigma \leftarrow \{0, 1\}^{256}$ ▷ Seed for $\mathbf{A}$ and noise
- 2: $\mathbf{A} \leftarrow \text{Sam}(\rho) \in R_q^{k \times k}$ ▷ Expand seed to matrix via XOF
- 3: $(\mathbf{s}, \mathbf{e}) \leftarrow \beta_{\eta_1}^k \times \beta_{\eta_1}^k$ ▷ Sample small secrets from CBD_{η_1}
- 4: $\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s}), \hat{\mathbf{e}} \leftarrow \text{NTT}(\mathbf{e})$
- 5: $\hat{\mathbf{t}} \leftarrow \hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}}$ ▷ Polynomial mult. in NTT domain
- 6: $\text{pk} \leftarrow (\hat{\mathbf{t}}, \rho), \text{sk} \leftarrow (\hat{\mathbf{s}}, \text{pk}, H(\text{pk}), z)$ with $z \xleftarrow{\$} \{0, 1\}^{256}$

Algorithm 5 ML-KEM.Encaps**Require:** Public key $\text{pk} = (\hat{\mathbf{t}}, \rho)$ **Ensure:** Shared key $K \in \{0, 1\}^{256}$, ciphertext c

- 1: $m \xleftarrow{\$} \{0, 1\}^{256}$
- 2: $(K, r) \leftarrow G(m \parallel H(\text{pk}))$ ▷ Derive key and randomness
- 3: $\mathbf{A} \leftarrow \text{Sam}(\rho)$
- 4: $(\mathbf{r}, \mathbf{e}_1, \mathbf{e}_2) \leftarrow \beta_{\eta_1}^k \times \beta_{\eta_2}^k \times \beta_{\eta_2}$ ▷ Encryption noise
- 5: $\mathbf{u} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}}^T \circ \text{NTT}(\mathbf{r})) + \mathbf{e}_1$ ▷ First ciphertext component
- 6: $v \leftarrow \text{NTT}^{-1}(\hat{\mathbf{t}}^T \circ \text{NTT}(\mathbf{r})) + \mathbf{e}_2 + \lceil q/2 \rceil \cdot \text{Decompress}(m)$
- 7: $c \leftarrow (\text{Compress}_{d_u}(\mathbf{u}), \text{Compress}_{d_v}(v))$
- 8: **return** (K, c)

Encapsulation.

Encapsulation is where the FO transform meets MODULE-LWE encryption. The encapsulator samples a random 256-bit message m , then derives *both* the shared key K and the encryption randomness r from m by hashing. This deterministic derivation of randomness is the crux of the FO transform: it ensures that anyone holding the secret key can re-derive r from the decrypted m' and verify that the ciphertext was honestly constructed. The encryption itself follows the standard MODULE-LWE template—multiply by the transposed public matrix, add noise, and encode the message in the high-order bits—but with an additional compression step that rounds ciphertext coefficients to fewer bits. This lossy compression is safe because the parameters guarantee that the rounding error is small relative to the decision boundary at $\lfloor q/4 \rfloor$.

Decapsulation.

Decapsulation is the most security-critical operation. It first performs standard MODULE-LWE decryption: subtract $\mathbf{s}^T \mathbf{u}$ from v to cancel the shared randomness, leaving (approximately) the encoded message plus accumulated noise. Rounding recovers the original message m' . But decryption alone is not enough—a passive decryption oracle would break CCA2 security. The FO check re-encrypts m' using the derived randomness r' and compares the result to the received ciphertext. If they match, the ciphertext is honest and the shared key is returned. If they differ, the ciphertext was tampered with or ma-

Algorithm 6 ML-KEM.Decaps**Require:** Secret key $\text{sk} = (\hat{s}, \text{pk}, h, z)$, ciphertext c **Ensure:** Shared key $K \in \{0, 1\}^{256}$

```

1:  $(\mathbf{u}, v) \leftarrow \text{Decompress}(c)$ 
2:  $m' \leftarrow \text{Compress}_1(v - \text{NTT}^{-1}(\hat{s}^T \circ \text{NTT}(\mathbf{u})))$  ▷ Decrypt
3:  $(K', r') \leftarrow G(m' \| h)$ 
4:  $c' \leftarrow \text{Enc}(\text{pk}, m'; r')$  ▷ FO re-encryption check
5: if  $c' = c$  then
6:   return  $K'$  ▷ Valid ciphertext
7: else
8:   return  $H(z \| c)$  ▷ Implicit rejection
9: end if

```

Table 7.3 ML-KEM parameter sets [50]

Param. set	k	η_1	η_2	d_u	d_v	PK (B)	CT (B)	NIST Level
ML-KEM-512	2	3	2	10	4	800	768	I (AES-128)
ML-KEM-768	3	2	2	10	4	1,184	1,088	III (AES-192)
ML-KEM-1024	4	2	2	11	5	1,568	1,568	V (AES-256)

licitiously constructed, and the algorithm returns a pseudorandom key derived from the secret z —the implicit rejection mechanism that denies the adversary any useful signal, including through timing.

Compression. The functions Compress_d and Decompress_d round coefficients from $\mathbb{Z}_q$ to d -bit values ($d < \lceil \log_2 q \rceil$), reducing ciphertext size at the cost of introducing small rounding errors. These errors are absorbed by the noise margin designed into the parameters.

7.7.2 Parameter Sets

Choosing parameters for a lattice-based scheme is an exercise in simultaneous optimisation under conflicting constraints. The designers of ML-KEM faced four tensions at once: security (measured by the BKZ block size needed to break the scheme) must exceed the target NIST level; the decryption failure probability must stay below 2^{-140} to satisfy the FO security proof; key and ciphertext sizes should remain small enough for deployment in TLS and other bandwidth-sensitive protocols; and all arithmetic must be efficient on commodity hardware, which favours NTT-friendly primes and power-of-two dimensions.

Table 7.3 shows the three standardised parameter sets.

All three parameter sets share $n = 256$ and $q = 3329$. The choice $q = 3329 = 13 \cdot 256 + 1$ satisfies $q \equiv 1 \pmod{512}$ (enabling the NTT), fits in 12 bits, and is the smallest prime meeting the correctness requirement $q > 12\eta\sqrt{n}$. Security is scaled via k , the module rank.

Table 7.4 ML-KEM-768 performance vs. classical algorithms (Intel Core i7, AVX2)

Operation	ML-KEM-768	X25519	RSA-2048
Key generation	35 μ s	45 μ s	~100 ms
Encapsulation	44 μ s	45 μ s	0.05 ms
Decapsulation	40 μ s	45 μ s	2 ms
Public key	1,184 B	32 B	256 B
Ciphertext	1,088 B	32 B	256 B

The compression parameters d_u and d_v illustrate a subtler tradeoff. Each bit removed from a ciphertext coefficient halves the ciphertext contribution from that component, but doubles the rounding noise. ML-KEM-512 uses $d_u = 10$ (dropping 2 bits per coefficient of $\mathbf{u}$) and $d_v = 4$ (a much more aggressive 8-bit reduction for v), because the scalar component v carries the encoded message and tolerates less noise than the vector component $\mathbf{u}$. Moving from ML-KEM-768 to ML-KEM-1024, the designers increased both d_u and d_v by one bit—the extra module dimension provides more noise margin, which can be traded for tighter compression. The noise parameters η_1 and η_2 follow a similar logic: ML-KEM-512 uses a wider centred binomial ($\eta_1 = 3$) to compensate for its smaller module dimension, while the higher security levels can afford the narrower $\eta_1 = 2$.

7.7.3 Performance

Parameters determine security and correctness, but the question practitioners care about most is speed. How does ML-KEM compare to the classical algorithms it aims to replace?

ML-KEM matches or outperforms X25519 in computation speed. The $\sim 30\times$ increase in public key and ciphertext size is the main cost of post-quantum security, but remains practical for all standard protocols including TLS (Chap. 14).

7.7.4 Decryption Failure Probability

Speed and key size are encouraging, but there is a subtlety that has no analogue in RSA or elliptic-curve cryptography: lattice-based encryption can *fail*. The rounding in compression and the error terms can occasionally cause decryption failure (the decrypted message differs from the encrypted one). The parameters are chosen so that the failure probability is at most 2^{-140} for ML-KEM-768—far below 2^{-128} , as required by the FO security proof (Theorem 7.6).

This extraordinarily low failure rate is not merely a convenience; it is a *security* requirement. An adversary who can provoke decryption failures can mount a *failure-boosting attack* to recover the secret key.

Problems

7.1 Consider LWE with $n = 4$, $q = 23$, and uniform ternary errors $e_i \in \{-1, 0, 1\}$. Let $\mathbf{s} = (3, 7, 11, 15)^T$.

- (a) Construct 6 LWE samples $(\mathbf{a}_i, b_i)$ using randomly chosen vectors $\mathbf{a}_i$.
- (b) Verify that the system $\mathbf{A}\mathbf{s} + \mathbf{e} \equiv \mathbf{b} \pmod{23}$ is consistent with your errors.
- (c) Attempt to recover $\mathbf{s}$ from the samples using only Gaussian elimination (ignoring the noise). Why does this fail?

7.2 Regev encryption by hand.

Let $n = 2$, $q = 17$, $\mathbf{s} = (3, 5)^T$. Given the public key samples:

$$(\mathbf{a}_1, b_1) = ((7, 11), 2), \quad (\mathbf{a}_2, b_2) = ((13, 4), 15), \quad (\mathbf{a}_3, b_3) = ((1, 9), 14),$$

encrypt $\mu = 1$ using the subset $S = \{1, 3\}$. Then decrypt and verify correctness.

7.3 NTT computation.

For $n = 4$ and $q = 17$, verify that $\omega = 2$ is a primitive 8-th root of unity modulo 17. Compute the NTT of the polynomial $a(X) = 1 + 3X + 5X^2 + 7X^3$ using the evaluation representation $\hat{a}_j = a(\omega^{2j+1}) \pmod{17}$ for $j = 0, 1, 2, 3$.

7.4 Compute the product of $a(X) = 3 + 2X + 5X^2 + X^3$ and $b(X) = 1 + 4X + 2X^2 + 3X^3$ in $R_q = \mathbb{Z}_{17}[X]/(X^4 + 1)$. Write out the corresponding negacyclic matrix multiplication and verify that both methods give the same result.

7.5 Fujisaki–Okamoto transform.

- (a) Explain why a CPA-secure encryption scheme is insufficient for use as a KEM in TLS. Give a concrete attack scenario.
- (b) Describe the role of the re-encryption check in the FO transform. What happens if an adversary submits a malformed ciphertext?
- (c) Why does ML-KEM use implicit rejection (returning a pseudorandom key) rather than explicit rejection (returning an error)? What side channel does this prevent?

7.6 ML-KEM parameter analysis.

ML-KEM-768 uses $n = 256$, $k = 3$, $q = 3329$, $\eta_1 = 2$, $d_u = 10$, $d_v = 4$.

- (a) Compute the public key size in bytes: $k \cdot n \cdot 12/8 + 32$ (polynomial coefficients at 12 bits each, plus a 32-byte seed). Verify it matches 1,184 B.
- (b) Compute the ciphertext size: $k \cdot n \cdot d_u/8 + n \cdot d_v/8$. Verify it matches 1,088 B.
- (c) The decryption noise has magnitude at most $\eta_1 \cdot k \cdot \eta_1 + \eta_2$. For correctness, this must be less than $\lfloor q/4 \rfloor$. Verify that this condition holds with a wide margin.

7.7 Research-level.

Regev’s original reduction from GapSVP to LWE is *quantum*. Discuss the implications: does this mean that the security of ML-KEM depends on a quantum assumption? What is the status of purely classical reductions, and what tradeoffs do they involve? (Hint: compare the approximation factors achieved by quantum vs. classical reductions.)

Chapter 8

Lattice-Based Digital Signatures

Abstract Constructing digital signatures from lattices requires fundamentally different techniques than encryption. This chapter develops two paradigms: the Fiat-Shamir-with-aborts approach underlying ML-DSA (CRYSTALS-Dilithium) and the hash-and-sign paradigm with Gaussian sampling used by Falcon. Both are NIST standards, and their contrasting designs illuminate deep tradeoffs in lattice-based cryptographic engineering.

8.1 The Challenge of Lattice-Based Signatures

For public-key encryption, the “encrypt in reverse” paradigm provides a natural route from lattice problems to cryptographic schemes: the secret key holder can decode a noisy codeword that is computationally infeasible for anyone else to decode. One might hope that a similar inversion—sign by decrypting, verify by encrypting—would yield lattice-based signatures just as easily. It does not.

8.1.1 Why “Encrypt in Reverse” Fails

In RSA, the algebraic symmetry between encryption ($m \mapsto m^e \bmod N$) and signing ($m \mapsto m^d \bmod N$) makes signatures almost free once encryption exists. Lattice-based encryption lacks this symmetry. The fundamental asymmetry is as follows:

- **Encryption** adds a small error $\mathbf{e}$ to a lattice point; the secret key holder removes it. The error is freshly sampled each time and does not depend on the secret key.
- **Signing** must produce a response that depends on both the message and the secret key. If that response involves a short vector $\mathbf{z} = \mathbf{s} \cdot c + \mathbf{y}$, where $\mathbf{s}$ is the secret, c the challenge, and $\mathbf{y}$ a random mask, then $\mathbf{z}$ is *correlated with* $\mathbf{s}$.

8.1.2 The Information Leakage Problem

Consider a naïve scheme where the signer outputs $\mathbf{z} = \mathbf{y} + c \mathbf{s}$ for a uniformly random mask $\mathbf{y}$ and a challenge c derived from the message. Each signature is a sample from the distribution $\mathbf{z} \sim U + c \mathbf{s}$, where U denotes the distribution of $\mathbf{y}$. After t signatures $(\mathbf{z}_1, c_1), \dots, (\mathbf{z}_t, c_t)$, an adversary forms the system

$$\begin{pmatrix} \mathbf{z}_1 \\ \vdots \\ \mathbf{z}_t \end{pmatrix} = \begin{pmatrix} \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_t \end{pmatrix} + \begin{pmatrix} c_1 \\ \vdots \\ c_t \end{pmatrix} \mathbf{s}. \quad (8.1)$$

Since $\mathbf{y}_i$ is bounded, the average $\frac{1}{t} \sum_i c_i^{-1} \mathbf{z}_i$ converges to $\mathbf{s}$ as t grows. More sophisticated attacks recover $\mathbf{s}$ from far fewer signatures by exploiting the distribution shift.

The core difficulty. A signature scheme must be *deterministic* in verification (anyone can check) yet must not leak the secret across multiple uses. For lattice schemes, this means the distribution of signatures must be *independent of the secret key*—a property that does not hold for the naïve construction above.

8.1.3 Two Solutions

The lattice signature literature has converged on two paradigms that solve the leakage problem in fundamentally different ways:

1. **Rejection sampling** (Sect. 8.3): the signer generates a candidate signature, then *rejects and restarts* with carefully controlled probability so that the output distribution is independent of $\mathbf{s}$. This is the approach of ML-DSA / CRYSTALS-Dilithium [16].
2. **Preimage sampling with trapdoors** (Sect. 8.5): the signer uses a secret “good basis” to sample from a discrete Gaussian centred at a message-dependent point. Because the Gaussian distribution is spherically symmetric, the output hides the trapdoor. This is the approach of Falcon [18].

Both solutions are provably secure under standard lattice assumptions, but they lead to schemes with markedly different engineering characteristics.

8.2 The Fiat-Shamir Framework

Most lattice-based signature schemes are constructed by applying the *Fiat-Shamir transform* [17] to an interactive identification protocol. We develop this framework in its general form before specialising to lattices.

8.2.1 Interactive Identification Protocols

A canonical identification scheme is a three-move protocol between a prover $\mathcal{P}$ (who knows the secret key) and a verifier $\mathcal{V}$ (who knows only the public key):

1. **Commit.** $\mathcal{P}$ samples randomness and sends a *commitment* w to $\mathcal{V}$.
2. **Challenge.** $\mathcal{V}$ sends a random *challenge* c to $\mathcal{P}$.
3. **Response.** $\mathcal{P}$ computes a *response* z using c , the secret key, and the commitment randomness, and sends z to $\mathcal{V}$.

The verifier accepts if (w, c, z) satisfies a public relation. Security requires:

- **Completeness:** an honest prover is always accepted.
- **Special soundness:** given two accepting transcripts (w, c, z) and (w, c', z') with $c \neq c'$, one can efficiently extract the secret key.
- **Honest-verifier zero-knowledge (HVZK):** transcripts can be simulated without the secret key.

8.2.2 The Fiat-Shamir Transform

The Fiat-Shamir transform removes interaction by replacing the verifier's random challenge with the output of a hash function applied to the commitment and the message:

$$c = H(w \parallel \mu), \quad (8.2)$$

where μ is the message to be signed. The resulting non-interactive scheme is:

- **KeyGen:** same as the identification scheme.
- **Sign(sk, μ):** sample commitment randomness, compute w ; set $c = H(w \parallel \mu)$; compute response z ; output $\sigma = (c, z)$ (or equivalently (w, z)).
- **Verify(pk, μ, σ):** recompute w from (c, z) using the public key; check $c = H(w \parallel \mu)$.

8.2.3 Security: ROM and QROM

In the *random oracle model* (ROM), where H is modelled as a truly random function, Fiat-Shamir signatures from a secure identification scheme satisfy existential unforgeability under chosen-message attacks (EU-CMA). The proof proceeds by programming the random oracle: the simulator embeds a challenge instance into the hash responses, then uses the forking lemma to extract a forgery into two accepting transcripts with the same commitment but different challenges, breaking special soundness.

In the post-quantum setting, the adversary can query H in *superposition*, requiring security in the *quantum random oracle model* (QROM). The classical forking lemma fails when the adversary can query H on superpositions of inputs, because measuring the query to “rewind” destroys the quantum state. Several techniques address this:

- **Unruh’s transform** (2015): adds a commitment to the signature, increasing size but enabling a QROM proof.
- **The measure-and-reprogram technique** of Don, Fehr, Majenz, and Schaffner (2019): directly proves Fiat-Shamir security in the QROM for schemes with large challenge spaces.
- **Deterministic commitments**: using a pseudorandom function (keyed by the secret key and the message) to derive the commitment randomness eliminates a subtle QROM attack vector and is the approach adopted by ML-DSA [49].

Both ML-DSA and Falcon have proofs of EU-CMA security in the QROM, establishing their soundness against quantum adversaries.

8.3 Fiat-Shamir with Aborts

The naïve lattice identification protocol leaks the secret key through the distribution of responses (Sect. 8.1.2). Lyubashevsky’s *Fiat-Shamir with aborts* framework [38] solves this by adding a rejection step.

8.3.1 The Protocol

Let $\mathbf{A} \in \mathbb{Z}_q^{k \times \ell}$ be the public matrix and $\mathbf{s} \in R_q^\ell$ the secret key, with public key $\mathbf{t} = \mathbf{A}\mathbf{s}$. The identification protocol proceeds as follows:

1. **Commit.** Sample $\mathbf{y} \xleftarrow{\$} S_{\gamma_1}^\ell$ (uniform over vectors with $\|\mathbf{y}\|_\infty < \gamma_1$). Compute $\mathbf{w} = \mathbf{A}\mathbf{y}$ and send the high-order bits $\mathbf{w}_1 = \text{HighBits}(\mathbf{w})$.
2. **Challenge.** Receive challenge $c \in C$, where C is a set of polynomials with small coefficients and bounded norm.
3. **Response.** Compute $\mathbf{z} = \mathbf{y} + c\mathbf{s}$.
4. **Reject.** If $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$, **abort and restart from step 1**.

The verifier checks that $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ and that $\text{HighBits}(\mathbf{A}\mathbf{z} - c\mathbf{t}) = \mathbf{w}_1$.

8.3.2 Why Rejection Sampling is Necessary

Without rejection, the response $\mathbf{z} = \mathbf{y} + c\mathbf{s}$ follows the distribution $U_{S_{\gamma_1}}$ shifted by $c\mathbf{s}$. Different secret keys produce different shifts, and an adversary observing multiple signatures can statistically distinguish (and eventually recover) the secret.

With rejection, the signer outputs $\mathbf{z}$ only when it falls in the “safe” region $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$. The key insight is that when γ_1 is sufficiently larger than $\|c\mathbf{s}\|_\infty \leq \beta$, the conditional distribution of $\mathbf{z}$ given acceptance is *nearly independent* of $\mathbf{s}$.

8.3.3 The Rejection Sampling Lemma

Theorem 8.1 (Rejection Sampling Lemma [38]) *Let V be a set of vectors with $\|\mathbf{v}\|_\infty \leq \beta$ for all $\mathbf{v} \in V$. Let $h : V \rightarrow [0, 1]$ be a probability function and $\sigma > 0$ a parameter. Sample $\mathbf{v} \sim h$ and $\mathbf{y} \sim D_\sigma^m$ (discrete Gaussian with parameter σ). Define $\mathbf{z} = \mathbf{y} + \mathbf{v}$. Then:*

$$\Pr \left[\text{output } \mathbf{z} \text{ with probability } \min \left(\frac{D_\sigma^m(\mathbf{z})}{M \cdot D_{\mathbf{v}, \sigma}^m(\mathbf{z})}, 1 \right) \right] \geq \frac{1}{M}, \quad (8.3)$$

where $M = \exp(12\beta/\sigma + 1/(2\sigma^2))$, and the output distribution is within statistical distance $2^{-100}/M$ of D_σ^m , independent of $\mathbf{v}$.

The constant M controls the expected number of repetitions before a signature is accepted. In ML-DSA, parameters are chosen so that $M \approx 4\text{--}7$, meaning the signer restarts on average 4–7 times per signature.

8.3.4 Uniform Signature Distribution

The rejection sampling lemma guarantees that the joint distribution of $(\mathbf{z}, c)$ in accepted signatures is (statistically close to) a *fixed* distribution that depends only on the public key and the message, not on the secret key. This is the lattice analogue of the perfect zero-knowledge property in Schnorr signatures: no matter which secret key produced the signature, the output “looks the same.”

The efficiency-security tradeoff. Making γ_1 larger (relative to β) reduces the rejection probability (fewer restarts, faster signing) but increases the signature size (the vector $\mathbf{z}$ has larger entries). Making γ_1 smaller tightens the bound, producing shorter signatures at the cost of more restarts. ML-DSA’s parameter sets (Sect. 8.4) represent carefully optimised operating points on this tradeoff curve.

8.4 ML-DSA (CRYSTALS-Dilithium)

ML-DSA (Module-Lattice-Based Digital Signature Algorithm), standardised as FIPS 204 [49], instantiates the Fiat-Shamir-with-aborts framework over module lattices. We present the complete scheme following the standard.

Algorithm 7 ML-DSA KeyGen**Require:** Security parameter (determining k, ℓ, η)**Ensure:** Public key $\text{pk} = (\mathbf{A}, \mathbf{t}_1)$, secret key $\text{sk} = (\mathbf{A}, \mathbf{t}_0, \mathbf{s}_1, \mathbf{s}_2)$

- 1: Sample seed $\rho \xleftarrow{\$} \{0, 1\}^{256}$
- 2: Expand $\mathbf{A} \in R_q^{k \times \ell}$ from ρ ▷ Public matrix
- 3: Sample $\mathbf{s}_1 \xleftarrow{\$} S_\eta^\ell, \mathbf{s}_2 \xleftarrow{\$} S_\eta^k$ ▷ Short secret vectors
- 4: $\mathbf{t} \leftarrow \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$
- 5: $(\mathbf{t}_1, \mathbf{t}_0) \leftarrow \text{Power2Round}(\mathbf{t})$ ▷ Split into high/low bits
- 6: **return** $\text{pk} = (\rho, \mathbf{t}_1), \text{sk} = (\rho, K, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$

8.4.1 Algebraic Setting

ML-DSA works over the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ with $q = 8\,380\,417$. Keys and operations involve matrices and vectors of elements from R_q . The module structure—vectors of ring elements rather than single ring elements—provides flexible parameterisation (Chap. 7).

Two hard problems underlie the security of ML-DSA:

- **Module-LWE:** given $(\mathbf{A}, \mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2)$, distinguish $\mathbf{t}$ from uniform, where $\mathbf{s}_1, \mathbf{s}_2$ have small coefficients.
- **Module-SIS:** given $\mathbf{A}$, find a short vector $\mathbf{z}$ such that $\mathbf{A}\mathbf{z} = \mathbf{0} \pmod{q}$.

8.4.2 Key Generation

The function `Power2Round` splits each coefficient of $\mathbf{t}$ into high-order bits $\mathbf{t}_1$ and low-order bits $\mathbf{t}_0$. Only $\mathbf{t}_1$ is included in the public key, compressing it significantly; $\mathbf{t}_0$ is retained in the secret key for use during signing.

The design of key generation reflects two competing pressures. On the one hand, the secret vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ must be short enough that the Module-LWE instance is hard—if their coefficients were too large, the noise would overwhelm the structure and make decoding trivial for an adversary with lattice reduction tools. On the other hand, they must be large enough that the rejection sampling during signing succeeds with reasonable probability; recall that the bound β on $\|c \cdot \mathbf{s}\|_\infty$ depends directly on the coefficient size η . The parameter η (either 2 or 4, depending on the security level) represents the carefully chosen balance point. Splitting $\mathbf{t}$ into high and low parts is not merely a compression trick: it is essential to the hint mechanism used during signing, which we describe next.

Algorithm 8 ML-DSA Sign

Require: Message μ , secret key $\text{sk} = (\rho, K, \text{tr}, s_1, s_2, t_0)$
Ensure: Signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$

- 1: $\mathbf{A} \leftarrow \text{ExpandA}(\rho)$
- 2: $\mu' \leftarrow H(\text{tr} \parallel \mu)$ ▷ Bind message to public key
- 3: $\kappa \leftarrow 0$
- 4: **repeat**
- 5: $\mathbf{y} \leftarrow \text{ExpandMask}(\rho', \kappa)$ ▷ Deterministic mask from K, μ
- 6: $\mathbf{w} \leftarrow \mathbf{A}\mathbf{y}$
- 7: $\mathbf{w}_1 \leftarrow \text{HighBits}(\mathbf{w})$ ▷ High-order bits of commitment
- 8: $\tilde{c} \leftarrow H(\mu' \parallel \mathbf{w}_1)$ ▷ Challenge hash
- 9: $c \leftarrow \text{SampleInBall}(\tilde{c})$ ▷ Challenge polynomial, $\|c\|_1 = \tau$
- 10: $\mathbf{z} \leftarrow \mathbf{y} + c \cdot \mathbf{s}_1$ ▷ Response
- 11: $\mathbf{r}_0 \leftarrow \text{LowBits}(\mathbf{w} - c \cdot \mathbf{s}_2)$
- 12: $\kappa \leftarrow \kappa + \ell$
- 13: **if** $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ **or** $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$ **then** ▷ Rejection check
- 14: **continue** ▷ Abort and restart
- 15: **end if**
- 16: $\mathbf{h} \leftarrow \text{MakeHint}(-c t_0, \mathbf{w} - c s_2 + c t_0)$ ▷ Hint for verification
- 17: **if** $\|c t_0\|_\infty \geq \gamma_2$ **or** number of 1's in $\mathbf{h} > \omega$ **then**
- 18: **continue**
- 19: **end if**
- 20: **until** signature produced
- 21: **return** $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$

8.4.3 Signing

The algorithm repays close reading, because each step reflects a deliberate design choice—not merely a mathematical convenience.

Consider first the generation of the mask $\mathbf{y}$. In a classical Schnorr signature, the randomness is freshly sampled; if an implementation ever reuses a nonce, the secret key is immediately recoverable. ML-DSA sidesteps this catastrophic failure mode by deriving $\mathbf{y}$ deterministically from the secret seed K and the message. Signing the same message twice always produces the same mask—and therefore the same signature—so there is no “nonce reuse” scenario to guard against. This deterministic derivation also simplifies the QROM security proof: the simulator can predict the signer’s randomness from the message, which is essential when the adversary can query the signing oracle on quantum superpositions of messages.

The two rejection conditions protect different secrets. The first check, $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$, ensures that the response $\mathbf{z}$ does not betray $\mathbf{s}_1$. Without it, large-magnitude entries in $\mathbf{z}$ would correlate with the secret, as we saw in Sect. 8.1.2. The second check, $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$, is subtler: it guards $\mathbf{s}_2$. The low-order bits of the commitment $\mathbf{w}$ shift when multiplied by the challenge c and the second secret vector, and if that shift is too large, the resulting signature leaks statistical information about $\mathbf{s}_2$ through the rounding behaviour. Both checks must pass simultaneously; failing either sends the signer back to step 1 with a fresh counter κ .

Finally, the hint vector $\mathbf{h}$ deserves attention because it solves a problem that arises nowhere in the abstract Fiat-Shamir framework. The signer commits to the high-order

Algorithm 9 ML-DSA Verify**Require:** Public key $\text{pk} = (\rho, \mathbf{t}_1)$, message μ , signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ **Ensure:** Accept or reject

```

1:  $\mathbf{A} \leftarrow \text{ExpandA}(\rho)$ 
2:  $\mu' \leftarrow H(\text{tr}\|\mu)$ 
3:  $c \leftarrow \text{SampleInBall}(\tilde{c})$ 
4:  $\mathbf{w}'_1 \leftarrow \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c \cdot \mathbf{t}_1 \cdot 2^d)$  ▷ Recover high bits
5: return  $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$  and  $\tilde{c} = H(\mu' \|\mathbf{w}'_1)$  and  $|\mathbf{h}| \leq \omega$ 

```

Table 8.1 ML-DSA parameter sets [49]. Sizes include encoding overhead.

Param. set	k	ℓ	η	τ	PK (B)	SK (B)	Sig (B)
ML-DSA-44	4	4	2	39	1 312	2 560	2 420
ML-DSA-65	6	5	4	49	1 952	4 032	3 309
ML-DSA-87	8	7	2	60	2 592	4 896	4 627

bits $\mathbf{w}_1$ of $\mathbf{A}\mathbf{y}$, but the verifier cannot compute $\mathbf{A}\mathbf{y}$ (since $\mathbf{y}$ is secret). The verifier instead computes $\mathbf{A}\mathbf{z} - c \mathbf{t}$, which *almost* equals $\mathbf{A}\mathbf{y} - c \mathbf{s}_2$ —close enough that the high-order bits agree in most coefficients, but off by one in a handful due to carries from the low-order part. The hint $\mathbf{h}$ is a sparse binary vector recording exactly which coefficients need correction. Because at most ω coefficients differ, encoding $\mathbf{h}$ costs far fewer bits than transmitting $\mathbf{w}_1$ directly. This is a small but consequential bandwidth saving that compounds across the millions of signatures a production system may generate.

8.4.4 Verification

Verification requires only one matrix-vector multiplication $\mathbf{A}\mathbf{z}$, making it fast. The check $\tilde{c} = H(\mu' \|\mathbf{w}'_1)$ ensures that the recovered commitment matches the challenge. The asymmetry between signing and verification is striking and deliberate: signing involves a rejection loop that may iterate several times, while verification is a single deterministic pass. In most deployment scenarios—TLS handshakes, certificate chains, software updates—signatures are created once and verified many times, so front-loading the computational cost into signing is the right engineering choice.

8.4.5 Parameter Sets

ML-DSA defines three parameter sets targeting NIST security levels I, III, and V:

The naming convention encodes the module dimensions: ML-DSA-44 uses a 4×4 module, ML-DSA-65 uses 6×5 , and ML-DSA-87 uses 8×7 . Larger modules provide higher security at the cost of larger keys and signatures.

8.4.6 Hints and Compression

The hint mechanism is one of ML-DSA’s most elegant engineering contributions, and it illustrates a general principle in lattice cryptography: rounding is cheap but its interaction with arithmetic is treacherous.

During signing, the signer computes $\mathbf{w} = \mathbf{A}\mathbf{y}$ and extracts its high-order bits $\mathbf{w}_1$. The verifier, who does not know $\mathbf{y}$, instead computes $\mathbf{A}\mathbf{z} - c\mathbf{t}$, which equals $\mathbf{A}\mathbf{y} - c\mathbf{s}_2$ up to the rounding decomposition. In most coefficients, the high-order bits agree. But wherever the low-order perturbation introduced by $c\mathbf{s}_2$ crosses a rounding boundary, the verifier’s high bits differ from the signer’s by exactly ± 1 . The hint $\mathbf{h}$ records which coefficients suffer this carry effect, allowing the verifier to patch its computation and recover the original $\mathbf{w}_1$. Because $\mathbf{h}$ is sparse—at most ω nonzero entries, typically far fewer—encoding it costs a fraction of what transmitting $\mathbf{w}_1$ directly would require.

Why not simply send $\mathbf{w}_1$ in the signature and skip the hint altogether? Because $\mathbf{w}_1$ has $k \cdot 256$ coefficients, each requiring several bits; encoding it would add hundreds of bytes to every signature. The hint, by contrast, needs only the positions of the nonzero entries—a list of at most ω indices—which can be encoded in roughly $\omega \cdot \lceil \log_2(k \cdot 256) \rceil$ bits. At ML-DSA-44’s parameters ($\omega = 80$), this saves over 600 bytes per signature compared to the naïve approach.

8.4.7 Security

The security of ML-DSA reduces to the hardness of Module-LWE and Module-SIS in the QROM. Specifically, if there exists a quantum adversary that produces an existential forgery under a chosen-message attack, then there exists a quantum algorithm that either solves Module-LWE with the scheme’s parameters, or solves Module-SIS—both of which reduce (via Chap. 6) to worst-case lattice problems.

ML-DSA’s Fiat-Shamir-with-aborts approach achieves security through careful engineering of the output distribution: rejection sampling forces every signature to look as though it were drawn from a fixed distribution, regardless of which secret key produced it. The second paradigm takes a different path entirely. Rather than masking the secret through rejection, it exploits a geometric property of Gaussian distributions—their rotational symmetry—to hide the trapdoor in the sampling process itself. This is mathematically cleaner but, as we shall see, far harder to implement safely.

8.5 NTRU Lattices and the GPV Framework

The second paradigm for lattice signatures follows the classical hash-and-sign approach: the signer uses a trapdoor to “invert” a hash function. The mathematical foundation combines NTRU lattices with the GPV (Gentry-Peikert-Vaikuntanathan) framework [21].

8.5.1 NTRU Lattices

The NTRU cryptosystem [26] works in the polynomial ring $R = \mathbb{Z}[X]/(X^n + 1)$ for n a power of 2, with computations modulo a prime q .

Definition 8.1 (NTRU Lattice) Given polynomials $f, g \in R$ with small coefficients and f invertible modulo q , define the public key $h = g \cdot f^{-1} \bmod q$. The *NTRU lattice* is the $2n$ -dimensional integer lattice

$$\mathcal{L}_h = \{(\mathbf{u}, \mathbf{v}) \in \mathbb{Z}^{2n} : \mathbf{u} \equiv h \cdot \mathbf{v} \pmod{q}\}. \quad (8.4)$$

Equivalently, in matrix form, $\mathcal{L}_h$ is generated by the rows of

$$\mathbf{B}_{\text{pub}} = \begin{pmatrix} q \mathbf{I}_n & \mathbf{0} \\ \mathbf{H} & \mathbf{I}_n \end{pmatrix}, \quad (8.5)$$

where $\mathbf{H}$ is the anti-circulant matrix corresponding to multiplication by h .

The pair (f, g) provides a *short basis* of $\mathcal{L}_h$:

$$\mathbf{B}_{\text{sec}} = \begin{pmatrix} f & g \\ F & G \end{pmatrix}, \quad (8.6)$$

where $F, G \in R$ satisfy the NTRU equation $fG - gF = q$. The rows of $\mathbf{B}_{\text{sec}}$ have norm $O(\sqrt{q})$, while the public basis $\mathbf{B}_{\text{pub}}$ has a row of norm q —an exponential quality gap in terms of the Gram-Schmidt norms.

The NTRU hardness assumption. Given only $h = g f^{-1} \bmod q$, it is computationally infeasible to recover the short pair (f, g) or any equivalently short basis of $\mathcal{L}_h$. This is closely related to the problem of finding short vectors in the NTRU lattice, and it has resisted cryptanalysis for over 25 years [26].

The exponential quality gap between the public and secret bases is what makes NTRU lattices useful for signatures. The public basis has a row of norm q —anyone can see it, but it is too “long” to sample short vectors efficiently. The secret basis, with rows of norm $O(\sqrt{q})$, is short enough to enable the Gaussian sampling that the GPV framework requires. This gap is the trapdoor: a structural advantage that the key holder possesses and the adversary does not, analogous to the role of the factorisation in RSA.

8.5.2 The GPV Hash-and-Sign Framework

Gentry, Peikert, and Vaikuntanathan [21] showed how to build signature schemes from any lattice equipped with a *trapdoor*—a short basis that enables efficient sampling of short lattice vectors near any target point.

Definition 8.2 (Hash-and-Sign Signature [21]) Let $\mathcal{L}$ be a lattice with public basis $\mathbf{B}_{\text{pub}}$ and trapdoor (short basis) $\mathbf{B}_{\text{sec}}$. Let H be a hash function mapping messages to points in $\mathbb{R}^n$.

- **KeyGen**: output $\text{pk} = \mathbf{B}_{\text{pub}}$, $\text{sk} = \mathbf{B}_{\text{sec}}$.
- **Sign**(sk, μ): compute $\mathbf{c} = H(\mu)$; use $\mathbf{B}_{\text{sec}}$ to sample $\mathbf{v} \sim D_{\mathcal{L}, \sigma, \mathbf{c}}$ (discrete Gaussian over $\mathcal{L}$ centred at $\mathbf{c}$); output $\sigma = \mathbf{v}$.
- **Verify**(pk, μ, σ): check that $\sigma \in \mathcal{L}$ and $\|\sigma - H(\mu)\| \leq B$ for an appropriate bound B .

8.5.3 Preimage Sampling

The core algorithmic challenge is sampling from $D_{\mathcal{L}, \sigma, \mathbf{c}}$ —the discrete Gaussian distribution over lattice points, centred at $\mathbf{c}$, with standard deviation σ . This distribution assigns probability proportional to $\exp(-\pi \|\mathbf{x} - \mathbf{c}\|^2 / \sigma^2)$ to each point $\mathbf{x} \in \mathcal{L}$.

Theorem 8.2 (GPV Preimage Sampling [21]) *Given a basis $\mathbf{B}$ of a lattice $\mathcal{L}$ and a Gaussian parameter $\sigma \geq \|\tilde{\mathbf{B}}\| \cdot \omega(\sqrt{\log n})$, where $\|\tilde{\mathbf{B}}\| = \max_i \|\tilde{\mathbf{b}}_i\|$ is the maximum Gram-Schmidt norm, there exists an efficient algorithm to sample from a distribution statistically close to $D_{\mathcal{L}, \sigma, \mathbf{c}}$ for any centre $\mathbf{c}$.*

The critical constraint is that σ must exceed the Gram-Schmidt norm of the basis. A shorter basis (smaller $\|\tilde{\mathbf{B}}\|$) allows a smaller σ , which in turn produces shorter signatures. This is why the trapdoor quality—measured by $\|\tilde{\mathbf{B}}\|$ —directly determines the scheme’s efficiency.

To see why this threshold matters for security (not just correctness), consider what happens when σ is too small. A Gaussian with parameter σ is concentrated within a ball of radius roughly $\sigma\sqrt{n}$. If σ is smaller than the longest Gram-Schmidt vector $\tilde{\mathbf{b}}_i$, the distribution cannot “wrap around” the fundamental domain in the i -th direction: samples cluster near the centre of that coordinate, and the resulting signature distribution is no longer spherically symmetric. An adversary collecting many signatures can detect this asymmetry and extract information about the secret basis direction by direction—precisely the kind of statistical leakage that the GPV framework is designed to prevent. The condition $\sigma \geq \|\tilde{\mathbf{B}}\| \cdot \omega(\sqrt{\log n})$ ensures that the Gaussian is wide enough to smooth over the lattice geometry in every direction, making the output statistically indistinguishable regardless of which short basis was used to produce it.

8.5.4 Security of GPV Signatures

Theorem 8.3 (GPV Security [21]) *If σ is sufficiently large relative to $\|\widetilde{\mathbf{B}_{\text{sec}}}\|$, then:*

1. *The distribution of signatures is statistically close to $D_{\mathcal{L}, \sigma, H(\mu)}$, which depends only on the public lattice and the message—not on which short basis was used. This ensures zero-knowledge.*
2. *Forging a signature requires finding a lattice vector close to $H(\mu)$, which is an instance of approximate CVP. The security reduces to the hardness of SIVP in the underlying lattice.*

The GPV framework is general: it applies to any lattice with an efficiently sampleable trapdoor. Falcon instantiates it using NTRU lattices, where the trapdoor is the short basis (f, g, F, G) .

8.6 Falcon

Falcon (Fast-Fourier Lattice-based Compact Signatures over NTRU) [18] combines the GPV hash-and-sign framework with NTRU lattices and a tree-based fast Fourier sampling algorithm to achieve the smallest signatures among all lattice-based schemes.

8.6.1 Construction Overview

The high-level structure follows the GPV template (Definition 8.2), with two crucial specialisations:

1. The underlying lattice is an NTRU lattice $\mathcal{L}_h$ (Definition 8.1), whose algebraic structure enables compact keys.
2. The discrete Gaussian sampling uses a *fast Fourier* decomposition of the basis, replacing the $O(n^2)$ Gram-Schmidt-based sampler with an $O(n \log n)$ tree-based algorithm.

8.6.2 Key Generation

Key generation produces an NTRU pair with controlled quality:

The NTRU equation $fG - gF = q$ is solved using the tower-of-fields approach: one works recursively in $\mathbb{Z}[X]/(X^{n/2} + 1)$, then $\mathbb{Z}[X]/(X^{n/4} + 1)$, and so on, solving a 2×2 system at each level. This yields F, G with entries of size $O(q)$, which are then reduced using lattice reduction in small dimension.

Key generation is by far the most expensive phase of Falcon—it may require hundreds of milliseconds or more, compared to microseconds for ML-DSA. This cost is acceptable because key generation is a one-time operation (or at worst infrequent), whereas signing and verification happen repeatedly over the key’s lifetime. The quality check on $\|\widetilde{\mathbf{B}}_{\text{sec}}\|$ is essential: a basis whose Gram-Schmidt norms are too large forces

Algorithm 10 Falcon KeyGen**Require:** Degree n (512 or 1024)**Ensure:** Public key h , secret key (f, g, F, G) and Falcon tree $\mathcal{T}$

```

1: repeat
2:   Sample  $f, g \xleftarrow{\$} D_{\mathbb{Z}^n, \sigma_{\text{kg}}}$  ▷ Short polynomials
3:   Check  $f$  is invertible mod  $q$  and mod  $X^n + 1$ 
4:   Solve NTRU equation: find  $F, G$  such that  $fG - gF = q$ 
5:   until  $\|\tilde{\mathbf{B}}_{\text{sec}}\|$  is below threshold
6:    $h \leftarrow g \cdot f^{-1} \bmod q$ 
7:   Precompute Falcon tree  $\mathcal{T}$  from  $(f, g, F, G)$  ▷ FFT of Gram-Schmidt data
8:   return  $\text{pk} = h, \text{sk} = \mathcal{T}$ 

```

Algorithm 11 Falcon Sign (simplified)**Require:** Message μ , Falcon tree $\mathcal{T}$, public key h **Ensure:** Signature s_2

```

1:  $\mathbf{c} \leftarrow H(\mu) \bmod q$  ▷ Hash to point
2:  $\mathbf{t} \leftarrow (\mathbf{c}, \mathbf{0}) \cdot \mathbf{B}_{\text{sec}}^{-1}$  ▷ Babai target in basis coords
3:  $\mathbf{z} \leftarrow \text{ffSampling}(\mathbf{t}, \mathcal{T})$  ▷ Tree-based discrete Gaussian
4:  $\mathbf{v} \leftarrow \mathbf{z} \cdot \mathbf{B}_{\text{sec}}$  ▷ Short lattice vector
5:  $s_2 \leftarrow \mathbf{v}[n:2n]$  ▷ Second component
6: if  $\|(s_1, s_2)\| > B$  then
7:   restart ▷ Rare: norm exceeded bound
8: end if
9: Compress and output  $s_2$ 

```

a wider Gaussian parameter σ , producing longer signatures that may exceed the verification bound. Rejecting poor-quality bases during key generation is cheaper than compensating for them in every subsequent signature.

The precomputed Falcon tree $\mathcal{T}$ deserves emphasis. Rather than storing the raw basis vectors and recomputing the Gram-Schmidt decomposition at each signing invocation, Falcon precomputes and stores the entire tree of FFT-domain Gram-Schmidt data. This trades secret key storage (the tree occupies roughly 40 KB for Falcon-512) for signing speed—a worthwhile exchange in most deployment scenarios.

8.6.3 Signing: The Falcon Tree

To sign a message μ , the signer hashes it to a point $\mathbf{c} = H(\mu) \in \mathbb{Z}_q^n$ and must find a short lattice vector $\mathbf{v}$ close to $(\mathbf{c}, \mathbf{0})$ in $\mathcal{L}_h$. Equivalently, find short $(s_1, s_2) \in R^2$ such that $s_1 + s_2 h \equiv \mathbf{c} \pmod{q}$. The signature is the short polynomial s_2 (from which s_1 can be recovered during verification).

The sampling uses the *Falcon tree*, a precomputed data structure that encodes the Gram-Schmidt decomposition of the NTRU basis in FFT form:

The key subroutine `ffSampling` exploits the recursive structure of the ring $R = \mathbb{Z}[X]/(X^n + 1)$. Since $X^n + 1 = (X^{n/2} + \zeta)(X^{n/2} - \zeta)$ over a suitable extension, the Gram-

Algorithm 12 Falcon Verify**Require:** Public key h , message μ , signature s_2 **Ensure:** Accept or reject

- 1: $\mathbf{c} \leftarrow H(\mu) \bmod q$
- 2: $s_1 \leftarrow \mathbf{c} - s_2 \cdot h \bmod q$
- 3: **return** $\|(s_1, s_2)\|^2 \leq B^2$

Table 8.2 Falcon parameter sets [18]

Param. set	n	q	PK (B)	Sig (B)	Security
Falcon-512	512	12 289	897	666	NIST Level I
Falcon-1024	1024	12 289	1 793	1 280	NIST Level V

Schmidt decomposition splits into two half-sized subproblems. Applied recursively, this yields a binary tree of depth $\log_2 n$, with a one-dimensional Gaussian sample at each leaf. The total cost is $O(n \log n)$ operations—the same complexity as an FFT.

The contrast with ML-DSA’s signing procedure is instructive. ML-DSA uses rejection sampling: generate a candidate, check whether it leaks information, and restart if it does. Falcon avoids rejection almost entirely (the norm check on line 6 triggers with probability less than 10^{-4} under typical parameters). Instead, the Gaussian sampling procedure *inherently* produces output that is independent of the secret basis, provided σ is large enough. This makes Falcon’s signing faster in terms of expected iterations—but each iteration is more expensive, requiring $O(n \log n)$ floating-point operations rather than the modular integer arithmetic of ML-DSA.

8.6.4 Verification

Verification is extremely efficient: one polynomial multiplication, one subtraction, and a norm check. No tree structure or Gaussian sampling is needed.

8.6.5 Parameters and Security

Falcon defines two parameter sets:

The signature sizes are the smallest among all NIST post-quantum signature candidates, roughly $2.5\times$ – $3.6\times$ smaller than ML-DSA at comparable security levels.

8.6.6 The Constant-Time Challenge

Falcon’s primary implementation difficulty is that the `ffSampling` algorithm requires high-precision floating-point arithmetic—specifically, IEEE 754 double-precision (53-bit significand) at minimum. This requirement creates a cascade of engineering problems that, taken together, make Falcon one of the hardest cryptographic primitives to implement safely.

The deepest problem is timing. On most processors, floating-point operations execute in constant time for “normal” operands but slow down dramatically when an operand is subnormal—a value so small that its leading significand bit is zero. The CPU either handles the subnormal in microcode (adding dozens of cycles) or raises a trap that the operating system resolves in software (adding thousands). During `ffSampling`, the Gram-Schmidt coefficients at deeper levels of the Falcon tree can become extremely small, especially for certain secret bases, and the intermediate products of these coefficients can land in the subnormal range. An attacker measuring signature generation time can detect these slowdowns and, from their pattern, reconstruct information about the secret tree—and therefore the secret basis. Multiple academic works have demonstrated precisely this attack against Falcon implementations.

The problem compounds across platforms. The IEEE 754 standard permits implementations to differ in their treatment of edge cases: flush-to-zero modes, rounding tie-breaking, and the handling of infinities and NaNs are all implementation-defined in certain contexts. A Falcon implementation that produces correct signatures on one CPU may produce subtly incorrect ones on another, or—worse—may produce signatures that are correct but whose timing profile reveals the secret key on one platform but not on another. Testing for constant-time behaviour requires not just code review but platform-specific timing analysis, often at the microarchitectural level.

Constrained devices present a separate barrier. Many embedded processors—ARM Cortex-M0/M3 microcontrollers, for example, which are the workhorses of IoT and smart-card applications—lack a hardware floating-point unit entirely. On these platforms, every floating-point operation must be emulated in software, multiplying the signing cost by one to two orders of magnitude and often exceeding the real-time budgets that embedded applications demand. ML-DSA, which uses only modular integer arithmetic, faces no such limitation.

Finally, the tree traversal itself poses a side-channel risk beyond arithmetic timing. The `ffSampling` procedure accesses different nodes of the precomputed Falcon tree depending on the intermediate sampling results, which in turn depend on the secret basis. On processors with shared caches, an attacker co-located on the same hardware can observe cache-line access patterns and infer which tree nodes were visited—a classic cache-timing attack. Mitigating this requires loading the entire tree into cache before each signing operation or using constant-time memory access primitives, both of which add complexity and cost.

NIST’s standardisation of Falcon (as FN-DSA) includes explicit guidance that implementers must ensure constant-time behaviour of all floating-point operations—a requirement that few development teams can confidently meet without specialised expertise and extensive side-channel testing.

Table 8.3 ML-DSA vs Falcon: comparative overview. Sizes for NIST Level I (ML-DSA-44 vs Falcon-512). Benchmark speeds on Intel Core i7 (AVX2).

Criterion	ML-DSA-44	Falcon-512
Public key size	1 312 B	897 B
Signature size	2 420 B	666 B
Secret key size	2 560 B	~1 281 B (tree: ~40 KB)
Sign (ops/s)	~3 700	~5 000
Verify (ops/s)	~9 100	~28 000
PK + Sig combined	3 732 B	1 563 B
Hard problem	Module-LWE + SIS	NTRU
Paradigm	Fiat-Shamir w/ aborts	GPV hash-and-sign
Arithmetic	Integer ($\mathbb{Z}_q$)	Floating-point (IEEE 754)
Constant-time	Straightforward	Extremely difficult
Embedded suitability	Good (no FPU needed)	Poor (requires FPU)
Side-channel resistance	Easier to protect	Requires expert care
Implementation complexity	Moderate	High

8.7 ML-DSA vs Falcon: A Comparative Analysis

The two NIST lattice-based signature standards embody fundamentally different answers to the same question: how should a signer prove knowledge of a short lattice vector without revealing it? ML-DSA says: compute a response, check whether it leaks information, and retry if it does. Falcon says: sample from a distribution that is inherently independent of the secret, using a trapdoor that only the key holder possesses. These different philosophies propagate into every aspect of the resulting schemes—size, speed, implementation difficulty, and deployment suitability.

Table 8.3 collects the quantitative comparison at NIST Level I, but the numbers alone do not tell the full story.

8.7.1 Size vs. Simplicity

The most immediately visible difference is size. Falcon-512 signatures are roughly 3.6× smaller than ML-DSA-44 signatures, and the combined public-key-plus-signature footprint is less than half. In protocols where signatures are transmitted frequently—TLS handshakes, certificate chains, blockchain transactions—this difference compounds. A certificate chain with three signatures saves roughly 5 KB per connection by switching from ML-DSA to Falcon, which can matter at scale or over constrained links.

But size is not free. Falcon achieves its compact signatures through the GPV framework’s Gaussian sampling, which requires the floating-point arithmetic and tree-based data structures discussed in Sect. 8.6.6. ML-DSA, by contrast, uses only modular integer arithmetic—addition, multiplication, and comparison in $\mathbb{Z}_q$ —which is straightforward to implement in constant time on any platform. The engineering cost of getting Falcon

right is not just higher; it is qualitatively different, requiring expertise in floating-point microarchitecture that most cryptographic engineering teams do not possess.

8.7.2 Performance Characteristics

Raw throughput numbers can mislead. Falcon signs faster than ML-DSA (roughly 5 000 vs. 3 700 operations per second on a modern desktop) and verifies roughly $3\times$ faster. But ML-DSA's signing speed is highly predictable—each attempt takes the same time, and the rejection loop adds only a constant expected factor. Falcon's signing time, while fast on average, depends on the floating-point behaviour of the specific secret key and message, creating a variable timing profile that an attacker can exploit unless constant-time countermeasures are in place.

For verification, Falcon's advantage is clear and comes with no side-channel risk: the verifier performs one polynomial multiplication and a norm check, with no secret data involved. In applications where verification dominates—public ledgers, broadcast authentication—this advantage is significant.

8.7.3 Deployment Considerations

For the vast majority of applications, ML-DSA is the right choice. Its implementation simplicity, natural constant-time behaviour, and broad platform support—including embedded processors without floating-point hardware—make it the safer engineering decision. NIST designates ML-DSA as the primary standard for digital signatures, and this designation reflects not a judgement about mathematical elegance but a pragmatic assessment of where implementations are most likely to be correct.

Falcon earns its place in a narrower but important niche: applications where signature size dominates all other concerns. Blockchain ledgers that store millions of signatures, certificate transparency logs, bandwidth-constrained satellite links—in these settings, the $3.6\times$ size reduction over ML-DSA may justify the additional engineering investment. Any deployment of Falcon should be backed by expert cryptographic engineering and rigorous side-channel testing; this is not a scheme where a competent but non-specialist team can safely “roll their own” implementation.

The diversity argument. Beyond performance and size, standardising two schemes based on different lattice problems—Module-LWE/SIS for ML-DSA, NTRU for Falcon—provides a cryptographic hedge. If a breakthrough algorithm were discovered against one problem family, the other scheme could serve as a fallback. Cryptographic diversity is not redundancy; it is insurance. This consideration was a key factor in NIST's decision to standardise both.

8.7.4 The Broader Landscape

The lattice-based schemes do not exhaust the post-quantum signature design space. NIST has also standardised SLH-DSA (SPHINCS+), a hash-based signature scheme that requires no lattice assumptions at all (Chap. ??). SLH-DSA’s security rests solely on the collision resistance of its underlying hash function—an assumption that is both older and more conservative than any lattice problem. The price is size: SLH-DSA signatures range from roughly 7 to 49 KB, an order of magnitude larger than either lattice-based scheme.

Together, the three standards span the design space along a clear axis. ML-DSA occupies the practical centre: moderate sizes, straightforward implementation, broad applicability. Falcon trades implementation complexity for compact signatures. SLH-DSA trades signature size for the strongest possible security assumptions. The choice among them is not a matter of which is “best” in the abstract, but which tradeoff profile matches the constraints of the application at hand.

Problems

8.1 Explain why a naïve lattice signature scheme that outputs $\mathbf{z} = \mathbf{s} \cdot \mathbf{c} + \mathbf{y}$ (where $\mathbf{s}$ is the secret, $\mathbf{c}$ the challenge, and $\mathbf{y}$ a random mask) leaks information about $\mathbf{s}$ after multiple signatures. How does rejection sampling solve this?

8.2 In ML-DSA, the signer rejects and restarts if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$. Explain the role of this check in security. What happens to the signing time as the bound γ_1 decreases?

8.3 Consider a three-move identification protocol with commitment space of size N .

- (a) Explain why the Fiat-Shamir transform requires N to be super-polynomial for security. What attack succeeds if N is small?
- (b) In the QROM, an adversary can evaluate H on superpositions. Explain intuitively why the classical “forking lemma” proof technique fails in this setting.

8.4 In the Fiat-Shamir-with-aborts framework, the expected number of restarts is approximately $M = \exp(12\beta/\sigma + 1/(2\sigma^2))$.

- (a) For ML-DSA-44, the parameters give $\beta = 78$ (the product $\tau \cdot \eta$) and $\gamma_1 = 2^{17}$. Estimate the rejection probability per attempt and the expected number of signing attempts.
- (b) Suppose an implementer “optimises” the scheme by removing the rejection step entirely. Describe a concrete attack that recovers the secret key from $\mathcal{O}(n)$ signatures.

8.5 Let $n = 4$, $q = 17$, and consider the ring $R = \mathbb{Z}[X]/(X^4 + 1)$. Let $f = 1 + X$ and $g = 1 - X^2$.

- (a) Verify that f is invertible modulo q by finding $f^{-1} \in R_q$.
- (b) Compute the public key $h = g \cdot f^{-1} \bmod q$.

- (c) Write out the 8×8 basis matrix for the NTRU lattice $\mathcal{L}_h$ (Eq. (8.5)).
- (d) Verify that the vector $(f, g) \in \mathbb{Z}^{2n}$ (interpreted coefficient-wise) belongs to $\mathcal{L}_h$.

8.6 Research-level. In the GPV framework, the Gaussian parameter σ must satisfy $\sigma \geq \|\tilde{\mathbf{B}}\| \cdot \omega(\sqrt{\log n})$. Explain why using σ below this threshold breaks security (not just correctness). Specifically, show that if $\sigma < \|\tilde{\mathbf{b}}_i\|$ for some i , the signature distribution leaks information about the i -th Gram-Schmidt vector.

Chapter 9

Code-Based Cryptography

Abstract Code-based cryptography, rooted in the theory of error-correcting codes, offers the longest track record of any post-quantum approach: the McEliece cryptosystem has resisted cryptanalysis since 1978. This chapter develops the algebraic coding theory needed to understand Classic McEliece (a NIST finalist) and the structured-code schemes HQC and BIKE, connecting throughout to quantum error correction and information theory familiar to physicists.

9.1 Error-Correcting Codes

Error-correcting codes were invented to protect data from noise in communication channels. The cryptographic insight is that *decoding a random linear code is NP-complete*—the same mathematical structure that enables reliable communication also enables public-key encryption, provided the decoder’s structural advantage is kept secret [42].

9.1.1 Linear Codes

The simplest family of codes with useful algebraic structure is the class of linear codes, where codewords form a vector subspace. Linearity is not merely a convenience: it guarantees that the sum of two codewords is again a codeword, that encoding is a matrix multiplication, and that the entire code can be described compactly by a basis rather than listed exhaustively.

Definition 9.1 (Linear Code) An $[n, k]$ linear code C over $\mathbb{F}_2$ is a k -dimensional subspace of $\mathbb{F}_2^n$. The parameter n is the *block length*, k is the *dimension*, and $R = k/n$ is the *rate*.

A linear code admits two equivalent matrix descriptions, each capturing a different perspective on the same subspace. To understand why both are needed, consider the

two fundamental tasks of coding theory: encoding a message, and detecting whether a received word belongs to the code.

Definition 9.2 (Generator and Parity-Check Matrices) A generator matrix $G \in \mathbb{F}_2^{k \times n}$ has the code as its row space: $C = \{\mathbf{m}G : \mathbf{m} \in \mathbb{F}_2^k\}$. It is the encoder's tool—given a k -bit message $\mathbf{m}$, the product $\mathbf{m}G$ produces the corresponding codeword.

A parity-check matrix $H \in \mathbb{F}_2^{(n-k) \times n}$ satisfies $H\mathbf{c}^\top = \mathbf{0}$ for every $\mathbf{c} \in C$. It is the verifier's tool—given a received word $\mathbf{y}$, the product $H\mathbf{y}^\top$ is zero if and only if $\mathbf{y}$ is a valid codeword, and non-zero otherwise.

These are related by $HG^\top = \mathbf{0}$, and one can always choose $G = [I_k \mid P]$ in *systematic form*, yielding $H = [-P^\top \mid I_{n-k}]$ (over $\mathbb{F}_2$, negation is the identity).

9.1.2 Minimum Distance and Error Correction

Encoding alone is useless without a way to measure how much corruption a code can tolerate. The key quantity is the minimum distance: the smallest number of bit positions in which any two distinct codewords differ. A larger minimum distance means more errors can be detected and corrected, but achieving it requires longer codewords—a trade-off that pervades all of coding theory.

Definition 9.3 (Hamming Distance and Weight) The *Hamming weight* of a vector $\mathbf{x} \in \mathbb{F}_2^n$ is $\text{wt}(\mathbf{x}) = |\{i : x_i \neq 0\}|$. The *Hamming distance* is $d(\mathbf{x}, \mathbf{y}) = \text{wt}(\mathbf{x} - \mathbf{y})$.

Definition 9.4 (Minimum Distance) The *minimum distance* of a code C is

$$d(C) = \min_{\mathbf{c}_1 \neq \mathbf{c}_2 \in C} d(\mathbf{c}_1, \mathbf{c}_2) = \min_{\mathbf{0} \neq \mathbf{c} \in C} \text{wt}(\mathbf{c}), \quad (9.1)$$

where the second equality holds because C is linear. A code with parameters $[n, k, d]$ can detect up to $d - 1$ errors and correct up to $t = \lfloor (d - 1)/2 \rfloor$ errors.

The Singleton bound $d \leq n - k + 1$ limits how large the minimum distance can be for given n and k . For binary codes, the more relevant constraint is the *Gilbert–Varshamov bound*: there exist $[n, k, d]$ codes provided $\sum_{i=0}^{d-2} \binom{n-1}{i} < 2^{n-k}$.

9.1.3 Syndrome Decoding

With the generator and parity-check matrices in hand, we can now ask the central question of coding theory: given a received word that may contain errors, how do we recover the original codeword? The parity-check matrix provides an elegant answer through the concept of a syndrome.

Given a received word $\mathbf{y} = \mathbf{c} + \mathbf{e}$ (codeword plus error), the *syndrome* $\mathbf{s} = H\mathbf{y}^\top = H\mathbf{e}^\top$ depends only on the error pattern. The codeword contribution vanishes because $H\mathbf{c}^\top = \mathbf{0}$ by definition. The decoding problem therefore reduces to finding a low-weight vector

$\mathbf{e}$ with a given syndrome—a problem that is easy to state but, for general codes, extraordinarily hard to solve.

Definition 9.5 (Bounded Distance Decoding) Given a parity-check matrix H and a syndrome $\mathbf{s} = H\mathbf{e}^\top$ where $\text{wt}(\mathbf{e}) \leq t$, find $\mathbf{e}$. When $t \leq \lfloor (d-1)/2 \rfloor$, the solution is unique.

For a general linear code, bounded distance decoding is NP-hard. The entire field of coding theory is devoted to finding *structured* code families—Hamming, Reed–Solomon, BCH, Goppa, LDPC, polar—for which efficient decoding algorithms exist. This gap between the hardness of decoding random codes and the tractability of decoding structured codes is the foundation of code-based cryptography.

9.1.4 The Physics Connection

The algebraic framework of linear codes has deep roots in statistical physics and information theory—connections that go well beyond analogy.

Codes, energy, and entropy. The minimum distance of a code corresponds to the energy gap between the ground state and the first excited state in a spin system. Decoding is equivalent to finding the minimum-energy configuration—a connection formalised by Sourlas (1989), who showed that optimal decoding of a random code maps exactly onto the ground-state problem of a random spin glass.

Shannon’s noisy-channel coding theorem provides another bridge. The capacity $C = 1 - H_2(p)$ of the binary symmetric channel, where H_2 is the binary entropy function, marks a phase transition between decodable and undecodable regimes—a statement about the free energy of a random code ensemble at a given noise level.

Perhaps the most direct connection comes from quantum error correction. CSS codes, the workhorses of quantum error correction, are built directly from pairs of classical linear codes $C_1 \supset C_2^\perp$. Syndrome measurements in quantum codes are the direct analogue of classical syndrome decoding, and the stabiliser formalism is fundamentally an extension of linear algebra over $\mathbb{F}_2$.

9.2 Goppa Codes

The McEliece cryptosystem requires a code family that is (i) efficiently decodable by the key holder, and (ii) indistinguishable from a random code by an adversary. *Binary Goppa codes*, introduced by V. D. Goppa in 1970, satisfy both requirements and remain the standard choice after more than fifty years.

Algorithm 13 Patterson's Decoding Algorithm for Binary Goppa Codes

Require: Goppa code $\Gamma(L, g)$, received word $\mathbf{y} = \mathbf{c} + \mathbf{e}$ with $\text{wt}(\mathbf{e}) \leq t$

Ensure: Error vector $\mathbf{e}$

- 1: Compute the syndrome $S(x) = \sum_{i: y_i=1} \frac{1}{x-\alpha_i} \pmod{g(x)}$
 - 2: Compute $T(x) = S(x)^{-1} + x \pmod{g(x)}$
 - 3: Compute $\tau(x) = \sqrt{T(x)} \pmod{g(x)}$ ▷ Square root in $\mathbb{F}_{2^m}[x]/(g(x))$
 - 4: Use the extended Euclidean algorithm to find $a(x), b(x)$ with $a(x) \equiv b(x) \tau(x) \pmod{g(x)}$ and $\deg a \leq \lceil t/2 \rceil, \deg b \leq \lfloor t/2 \rfloor$
 - 5: Form the error-locator polynomial $\sigma(x) = a(x)^2 + x b(x)^2$
 - 6: Find the roots of $\sigma(x)$ in L : $e_i = 1$ iff $\sigma(\alpha_i) = 0$
 - 7: **return** $\mathbf{e}$
-

9.2.1 Definition and Parameters

Goppa codes are defined not by a combinatorial construction but by an algebraic one: membership in the code is determined by a divisibility condition in a polynomial ring. This algebraic structure is what makes efficient decoding possible, while the large number of possible defining polynomials is what makes the codes hard to distinguish from random.

Definition 9.6 (Binary Goppa Code) Let $g(x) \in \mathbb{F}_{2^m}[x]$ be an irreducible polynomial of degree t (the *Goppa polynomial*), and let $L = \{\alpha_1, \dots, \alpha_n\} \subseteq \mathbb{F}_{2^m}$ with $g(\alpha_i) \neq 0$ for all i . The *binary Goppa code* $\Gamma(L, g)$ is

$$\Gamma(L, g) = \left\{ \mathbf{c} \in \mathbb{F}_2^n : \sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)} \right\}. \quad (9.2)$$

The defining condition imposes t independent linear constraints over $\mathbb{F}_{2^m}$, each of which translates to m constraints over $\mathbb{F}_2$. This yields the following parameter bounds.

Theorem 9.1 (Goppa Code Parameters) *The code $\Gamma(L, g)$ is an $[n, k, d]$ binary linear code with block length $n = |L| \leq 2^m$, dimension $k \geq n - mt$, and minimum distance $d \geq 2t + 1$. Thus $\Gamma(L, g)$ can correct at least t errors. The factor of 2 improvement over the design distance (t rather than $\lfloor t/2 \rfloor$) is specific to binary Goppa codes and follows from the algebraic structure of the syndrome equations over $\mathbb{F}_2$.*

9.2.2 Patterson's Decoding Algorithm

The algebraic structure of Goppa codes does more than define the code—it provides a fast decoding algorithm. Patterson (1975) showed that the syndrome equations can be reduced to finding the roots of a single *error-locator polynomial*, a problem solvable in polynomial time.

Patterson's algorithm runs in $O(n^2)$ operations over $\mathbb{F}_{2^m}$ (or $O(n^{3/2} \log n)$ using fast arithmetic), decoding up to t errors—the full designed error-correction capability. This

efficiency is essential: the legitimate receiver can decode in polynomial time, while an adversary facing a random-looking code has no such shortcut.

9.2.3 Why Goppa Codes Are Suitable for Cryptography

Not every efficiently decodable code family is suitable for cryptography. The additional requirement—indistinguishability from a random code—turns out to be extremely demanding, and most natural code families fail it. Binary Goppa codes are one of very few families that satisfy both conditions simultaneously.

Patterson’s algorithm decodes the full t errors in polynomial time, giving the legitimate receiver a decisive advantage. The parity-check matrix of a Goppa code, when put in systematic form, is computationally indistinguishable from a uniformly random binary matrix; no efficient distinguisher is known despite decades of research. The number of irreducible polynomials of degree t over $\mathbb{F}_{2^m}$ is approximately $2^{mt}/t$, providing an exponentially large key space. Finally, as a subclass of alternant codes (and hence of BCH codes), Goppa codes have a solid algebraic theory enabling rigorous security analysis.

Why not other code families? Attempts to replace Goppa codes with other families—Reed–Solomon, Reed–Muller, Gabidulin, generalised Reed–Solomon—have all led to broken schemes. The core issue is that these codes have too much algebraic structure, enabling structural attacks that recover the secret key. Binary Goppa codes sit in a “Goldilocks zone”: enough structure for efficient decoding, not enough for efficient distinguishing.

9.3 The Syndrome Decoding Problem

The previous sections established that structured codes can be decoded efficiently while random codes cannot. We now make this hardness claim precise by defining the syndrome decoding problem—the computational assumption on which all code-based cryptosystems ultimately rest.

9.3.1 Formal Definition and Hardness

The problem can be stated cleanly: given a random system of linear equations over $\mathbb{F}_2$ and a target weight, find a solution of that weight. The randomness of the system is what makes the problem hard—there is no hidden algebraic structure for the solver to exploit.

Algorithm 14 Prange’s Information Set Decoding**Require:** Parity-check matrix $H \in \mathbb{F}_2^{r \times n}$, syndrome $\mathbf{s}$, target weight w **Ensure:** Error vector $\mathbf{e}$ with $H\mathbf{e}^\top = \mathbf{s}$ and $\text{wt}(\mathbf{e}) = w$

```

1: repeat
2:   Choose a random set  $I \subseteq [n]$  with  $|I| = k$  ▷ Information set
3:   Gaussian-eliminate  $H$  on columns  $I$  to obtain  $H' = [I_r \mid \cdot]$ 
4:   Set  $\mathbf{e}_I = \mathbf{0}$  and  $\mathbf{e}_{\bar{I}}$  = the corresponding syndrome solution
5:   if  $\text{wt}(\mathbf{e}) = w$  then
6:     return  $\mathbf{e}$ 
7:   end if
8: until success

```

Definition 9.7 (Syndrome Decoding Problem (SDP)) Given a random binary matrix $H \in \mathbb{F}_2^{r \times n}$ (parity-check matrix), a syndrome $\mathbf{s} \in \mathbb{F}_2^r$, and a target weight w , find an error vector $\mathbf{e} \in \mathbb{F}_2^n$ with $\text{wt}(\mathbf{e}) = w$ such that $H\mathbf{e}^\top = \mathbf{s}$.

Theorem 9.2 (Berlekamp–McEliece–van Tilborg, 1978) *The decision version of the syndrome decoding problem (“does there exist $\mathbf{e}$ with $\text{wt}(\mathbf{e}) \leq w$ and $H\mathbf{e}^\top = \mathbf{s}$?”) is NP-complete.*

The NP-completeness of SDP is the theoretical foundation of all code-based cryptosystems. However, NP-completeness is a worst-case statement; cryptographic security requires average-case hardness. Extensive study has produced no evidence of easy instances among random codes, and the best known algorithms have improved only incrementally over four decades.

For physicists, the syndrome decoding problem has a direct statistical-mechanics interpretation. Finding a weight- w vector $\mathbf{e}$ satisfying $H\mathbf{e}^\top = \mathbf{s}$ is equivalent to finding a spin configuration of magnetisation $(n - 2w)/n$ in a random constraint satisfaction problem. The phase transition between solvable and unsolvable regimes—controlled by the code rate $R = k/n$ and the relative error weight w/n —mirrors the replica symmetry breaking transitions studied in the physics of random systems.

9.3.2 Information Set Decoding

Knowing that syndrome decoding is NP-complete tells us that no polynomial-time algorithm exists (assuming $P \neq NP$), but it says nothing about the actual cost of the best exponential-time algorithms. For setting cryptographic parameters, we need concrete complexity estimates—and the best known algorithms for solving the syndrome decoding problem all belong to the *Information Set Decoding* (ISD) family, first introduced by Prange (1962).

The core idea is simple: select a subset of k coordinates (an “information set”), hope that all errors fall outside this set, and solve the resulting linear system. If the guess is wrong, try again.

The success probability per iteration is $\binom{n-w}{k} / \binom{n}{k}$ —the probability that all w errors fall outside the chosen information set. The expected cost is therefore

Table 9.1 Information Set Decoding variants: asymptotic exponent c in $T = 2^{(c+o(1))n}$ for code rate $R = 1/2$ and relative weight w/n at the Gilbert–Varshamov bound

Algorithm	Exponent c	Key idea
Prange (1962)	0.1207	Brute-force information sets
Lee–Brickell (1988)	0.1164	Allow p errors in $\mathcal{I}$
Stern (1989)	0.1059	Birthday collision on half-syndromes
MMT (2011)	0.0494	Representations technique
BJMM (2012)	0.0480	Near-neighbour search
Both–May (2018)	0.0473	Refined representations

$$T_{\text{Prange}} = \frac{\binom{n}{k}}{\binom{n-w}{k}} \cdot O(n^3) \approx \binom{n}{w} / \binom{r}{w} \cdot O(n^3). \quad (9.3)$$

9.3.3 Improved ISD Variants

Prange’s algorithm is conceptually clean but wasteful: it demands that *all* errors lie outside the information set. Subsequent work relaxed this constraint, allowing a controlled number of errors in the information set and using birthday-type collisions to find them efficiently. Table 9.1 summarises the main variants and the steady but slow improvement in the asymptotic exponent.

The key insight behind Stern’s algorithm is to split the error vector into two halves and search for collisions: vectors whose partial syndromes match. The MMT and BJMM algorithms refine this by exploiting the fact that a weight- w error vector can be *represented* in exponentially many ways as a sum of two vectors, and using near-neighbour search techniques to find matching pairs more efficiently.

9.3.4 Quantum Speedup for ISD

The question that determines whether code-based cryptography survives the quantum era is straightforward: how much faster can a quantum computer solve the syndrome decoding problem? The answer—only polynomially faster—is the reason this entire chapter matters.

Theorem 9.3 (Quantum ISD Complexity) *For a random $[n, k]$ code with w errors, the best classical algorithm (BJMM) runs in $T_{\text{class}} = 2^{(0.0480+o(1))n}$ for rate $R = 1/2$. Grover acceleration yields a naïve quantum speedup of $T_{\text{quant}} \approx \sqrt{T_{\text{class}}}$, but memory constraints limit the practical gain. The best dedicated quantum ISD algorithm, due to Kachigar and Tillich (2017), achieves $T_{\text{quant}} = 2^{(0.0301+o(1))n}$ for rate $R = 1/2$. No exponential quantum advantage is known for syndrome decoding.*

The absence of an exponential quantum speedup is the fundamental reason code-based cryptography is considered post-quantum secure. Unlike integer factorisation

(where Shor's algorithm provides an exponential advantage) or discrete logarithms (same), the syndrome decoding problem appears to admit only a polynomial quantum speedup. This parallels the situation for lattice problems (Chap. 6).

9.4 The McEliece Cryptosystem

We now have all the ingredients for the oldest unbroken public-key encryption scheme. The idea, due to Robert McEliece (1978) [42], is deceptively simple: take a code that we can decode efficiently, disguise it so that it looks like a random code, and publish the disguised version as a public key. Anyone can encode a message using the public key (adding intentional errors), but only the key holder—who knows the secret structure—can decode it. The disguise is the entire security argument. If the public key is indistinguishable from a random generator matrix, then decrypting without the secret key is equivalent to decoding a random code, which we have just seen is NP-hard.

9.4.1 Construction

The construction realises this idea by composing three transformations: a scrambling of the message space (via an invertible matrix S), the structured encoding (via the Goppa generator matrix G), and a permutation of the codeword coordinates (via P). The composition $G' = SGP$ is the public key, and it reveals nothing about the individual factors.

Definition 9.8 (McEliece PKE) The McEliece public-key encryption scheme based on an $[n, k, d]$ binary Goppa code with error-correction capability $t = \lfloor (d - 1)/2 \rfloor$:

KeyGen(1^λ):

1. Choose a random irreducible polynomial $g(x) \in \mathbb{F}_{2^m}[x]$ of degree t and a support set $L = \{\alpha_1, \dots, \alpha_n\} \subseteq \mathbb{F}_{2^m}$.
2. Compute the generator matrix $G \in \mathbb{F}_2^{k \times n}$ of the Goppa code $\Gamma(L, g)$.
3. Choose a random invertible matrix $S \in \mathbb{F}_2^{k \times k}$ and a random permutation matrix $P \in \mathbb{F}_2^{n \times n}$.
4. Set $\text{pk} = (G' = SGP, t)$ and $\text{sk} = (S, G, P, g, L)$.

Enc($\text{pk}, \mathbf{m}$): For message $\mathbf{m} \in \mathbb{F}_2^k$:

1. Sample a random error $\mathbf{e} \xleftarrow{\$} \mathbb{F}_2^n$ with $\text{wt}(\mathbf{e}) = t$.
2. Output $\mathbf{c} = \mathbf{m}G' + \mathbf{e}$.

Dec($\text{sk}, \mathbf{c}$):

1. Compute $\mathbf{c}' = \mathbf{c}P^{-1} = \mathbf{m}SG + \mathbf{e}P^{-1}$.
2. Apply Patterson's algorithm (Algorithm 13) to decode $\mathbf{c}'$, recovering $\mathbf{m}S$.

3. Compute $\mathbf{m} = (\mathbf{m}S)S^{-1}$.

Correctness follows from the fact that P^{-1} is a permutation (preserving Hamming weight), so $\mathbf{e}P^{-1}$ has weight t , which lies within the decoding capability of the Goppa code.

9.4.2 Security Analysis

The security of McEliece rests on two computational assumptions, and it is worth understanding each one independently because they can fail for different reasons.

The first assumption is *generic decoding hardness*: given the public key G' (which looks like a random generator matrix) and a ciphertext $\mathbf{c} = \mathbf{m}G' + \mathbf{e}$, recovering $\mathbf{m}$ requires solving a syndrome decoding instance. The best known attack is ISD (Sect. 9.3.2), which is exponential in the code parameters.

The second assumption is *Goppa code indistinguishability*: the disguised public key $G' = SGP$ is computationally indistinguishable from a uniformly random $k \times n$ binary matrix. Despite extensive research, no efficient distinguisher has been found for binary Goppa codes with appropriate parameters. If either assumption fails—if someone finds a faster generic decoder, or a way to recognise disguised Goppa codes—the scheme breaks. Neither has happened in over 45 years.

45+ years and counting. The McEliece cryptosystem has survived cryptanalysis since 1978—longer than RSA (1977), longer than elliptic curve cryptography (1985), longer than any other public-key scheme in use. The best known attacks have improved only incrementally: from Prange’s 2^{64} work factor for the original parameters to approximately 2^{60} with BJMM, necessitating a parameter increase but not a structural break. No attack has ever exploited the Goppa structure rather than treating the public key as a generic random code.

9.4.3 Classic McEliece: The NIST Candidate

The longevity of McEliece’s original design made it a natural candidate for post-quantum standardisation. Classic McEliece, submitted to the NIST process by Bernstein et al., is a conservative instantiation of the original scheme with modern parameter choices. It advanced to the fourth round of NIST’s process and remains under evaluation.

The public key sizes in Table 9.2 reveal the fundamental tension of classic code-based cryptography: the public key is the systematic form of a $k \times n$ generator matrix, requiring $k(n - k)/8$ bytes of storage. At NIST Level I, this is ~ 255 KB—roughly 1000× larger than ML-KEM-768’s 1,184-byte public key.

Table 9.2 Classic McEliece parameter sets

Parameter set	n	t	PK (bytes)	CT (bytes)	Level
mciece348864	3488	64	261,120	128	I
mciece460896	4608	96	524,160	188	III
mciece6688128	6688	128	1,044,992	240	V
mciece6960119	6960	119	1,047,319	226	V
mciece8192128	8192	128	1,357,824	240	V

Example 9.1 (Key Size Calculation) For `mciece348864`: $n = 3488$, $k = n - mt = 3488 - 12 \cdot 64 = 2720$, $r = n - k = 768$. The public key in systematic form is a $k \times r$ binary matrix:

$$\text{PK size} = \frac{k \cdot r}{8} = \frac{2720 \times 768}{8} = 261,120 \text{ bytes} \approx 255 \text{ KB}. \quad (9.4)$$

The ciphertext is a length- n binary vector: $\lceil 3488/8 \rceil = 436$ bits, but in KEM mode (using a hash) only 128 bytes.

9.4.4 The Key Size Problem

The large public keys of Classic McEliece are not a fixable engineering issue but a fundamental consequence of the security model. The public key must be a dense random-looking matrix to resist distinguishing attacks, and any attempt to introduce algebraic structure for compression typically enables structural attacks that recover the secret key. Every attempt to use structured codes—quasi-cyclic, quasi-dyadic, and others—in a McEliece-like framework has been broken, with the exception of the quasi-cyclic schemes discussed in Sects. 9.5–9.6, which use a fundamentally different construction. The key size scales as $\Theta(n^2)$ in the code length, compared to $\Theta(n)$ for lattice-based schemes—a quadratic penalty that no known trick can eliminate within the classical McEliece paradigm.

Despite this limitation, Classic McEliece has compelling advantages for specific deployment scenarios: long-lived keys (cached once, used many times), high-bandwidth links, and applications where security confidence outweighs bandwidth cost. The next two sections explore schemes that escape the quadratic key-size barrier by abandoning the “disguise a structured code” paradigm entirely.

9.5 HQC: Hamming Quasi-Cyclic

Classic McEliece achieves strong security but pays for it with enormous public keys. HQC (Hamming Quasi-Cyclic), designed by Melchor, Aguilar-Melchor, and collaborators, takes a fundamentally different approach to code-based cryptography: instead

of disguising a structured code as a random one, it builds encryption directly from the hardness of decoding random quasi-cyclic codes, using a concatenated coding scheme for error correction. In August 2024, NIST selected HQC for standardisation as a KEM, providing a code-based alternative to the lattice-based ML-KEM.

9.5.1 Quasi-Cyclic Codes

The key to HQC’s compact key sizes is the quasi-cyclic structure—a symmetry that allows an entire matrix to be represented by a single row. Before defining the HQC construction, we need to understand this structure and why it compresses keys so dramatically.

Definition 9.9 (Quasi-Cyclic Code) An $[n, k]$ code C is *quasi-cyclic* of index ℓ if every cyclic shift of a codeword by n/ℓ positions is also a codeword. Equivalently, the generator matrix has a block-circulant structure: each block is an $r \times r$ circulant matrix, with $r = n/\ell$.

The circulant structure admits a compact representation: an $r \times r$ circulant matrix is determined by its first row, so the entire generator matrix can be stored in $\ell \cdot r$ bits rather than $k \cdot n$. This is the key to achieving small key sizes.

Algebraically, a circulant matrix corresponds to multiplication by a polynomial in the quotient ring $\mathbb{F}_2[x]/(x^r - 1)$. When r is chosen to be prime with $\gcd(r, 2) = 1$, this ring has useful splitting properties that enhance the security analysis.

9.5.2 HQC Construction

The design intuition behind HQC mirrors the LWE-based schemes of Chapter 7, but with codes replacing lattices. The public key is a noisy product in a polynomial ring: $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y}$, where $\mathbf{h}$ is a public random element and $(\mathbf{x}, \mathbf{y})$ are secret sparse vectors. Distinguishing $\mathbf{s}$ from a uniformly random element requires solving a syndrome decoding problem over quasi-cyclic codes—and the quasi-cyclic structure means the public key is just a pair of polynomials rather than a full matrix. The result is key sizes three orders of magnitude smaller than Classic McEliece.

HQC is a KEM built from a public-key encryption scheme via the Fujisaki–Okamoto transform (Chap. 7). The underlying PKE works as follows.

Definition 9.10 (HQC PKE) Let n be prime, and let C be a $[n_1, k, d]$ code with efficient decoding (a tensor product of Reed–Muller and Reed–Solomon codes).

KeyGen(1^λ):

1. Sample $\mathbf{h} \xleftarrow{\$} \mathbb{F}_2[x]/(x^n - 1)$ uniformly at random.
2. Sample $\mathbf{x}, \mathbf{y} \xleftarrow{\$} \mathbb{F}_2[x]/(x^n - 1)$ of fixed Hamming weight w .

Table 9.3 HQC parameter sets (NIST submission, 2024)

Level	n	w	PK (bytes)	CT (bytes)	SS (bytes)
128	17,669	66	2,249	4,497	64
192	35,851	100	4,522	9,042	64
256	57,637	131	7,245	14,485	64

3. Compute $\mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y} \in \mathbb{F}_2[x]/(x^n - 1)$.

4. Set $\text{pk} = (\mathbf{h}, \mathbf{s})$ and $\text{sk} = (\mathbf{x}, \mathbf{y})$.

The public key consists of just two ring elements: the random $\mathbf{h}$ and the noisy product $\mathbf{s}$. The secret key is the pair of sparse polynomials $(\mathbf{x}, \mathbf{y})$ whose low weight is what makes decoding feasible for the key holder.

$\text{Enc}(\text{pk}, \mathbf{m})$: For a message $\mathbf{m} \in \mathbb{F}_2^k$:

1. Encode: $\hat{\mathbf{m}} = \text{Encode}(\mathbf{m}) \in \mathbb{F}_2^{n_1}$.
2. Sample $\mathbf{e}, \mathbf{r}_1, \mathbf{r}_2 \xleftarrow{\$} \mathbb{F}_2[x]/(x^n - 1)$ of fixed weight w_r .
3. Compute $\mathbf{u} = \mathbf{r}_1 + \mathbf{h}\mathbf{r}_2$ and $\mathbf{v} = \hat{\mathbf{m}} + \mathbf{s}\mathbf{r}_2 + \mathbf{e}$.
4. Output $\mathbf{c} = (\mathbf{u}, \mathbf{v})$.

Encryption first protects the message with an inner code C , then buries the encoded message under noise. The first ciphertext component $\mathbf{u}$ is structurally identical to the public key—a noisy product that hides the randomness $\mathbf{r}_2$. The second component $\mathbf{v}$ carries the actual message, masked by the public key’s relationship with the encryptor’s randomness.

$\text{Dec}(\text{sk}, \mathbf{c})$:

1. Compute $\hat{\mathbf{m}}' = \mathbf{v} - \mathbf{u}\mathbf{y} = \hat{\mathbf{m}} + \mathbf{e} + \mathbf{r}_2\mathbf{x} - \mathbf{r}_1\mathbf{y}$.
2. Decode $\hat{\mathbf{m}}'$ using the efficient decoder of C to recover $\mathbf{m}$.

Decryption exploits the key holder’s knowledge of $\mathbf{y}$ to cancel most of the noise. The residual term $\mathbf{e} + \mathbf{r}_2\mathbf{x} - \mathbf{r}_1\mathbf{y}$ is a sparse vector (since all factors are sparse), and the inner code C is chosen with sufficient error-correction capability to remove it.

The term $\mathbf{e} + \mathbf{r}_2\mathbf{x} - \mathbf{r}_1\mathbf{y}$ is a noise vector whose weight is bounded by $w_r \cdot w + w_r$ —guaranteed to lie within the decoding capability of C for appropriate parameter choices. The security relies on the hardness of distinguishing $(\mathbf{h}, \mathbf{s} = \mathbf{x} + \mathbf{h}\mathbf{y})$ from uniform—the *decisional quasi-cyclic syndrome decoding problem*, which is the code-based analogue of the decisional LWE problem.

9.5.3 Parameters and NIST Standardisation

The practical payoff of the quasi-cyclic structure is evident in the parameter sizes.

Compared to Classic McEliece, HQC achieves dramatically smaller public keys (~2 KB vs. ~255 KB at Level I) at the cost of larger ciphertexts (~4.5 KB vs. 128

bytes). The key size reduction comes from the quasi-cyclic structure, which compresses the public key from a full matrix to a pair of polynomials.

NIST selected HQC for standardisation in 2024, citing the value of cryptographic diversity: having a code-based standard alongside the lattice-based ML-KEM ensures that a hypothetical breakthrough against lattice problems would not compromise all post-quantum deployments.

9.6 BIKE: Bit Flipping Key Encapsulation

HQC uses a concatenated code to handle the noise introduced during encryption. BIKE (Bit Flipping Key Encapsulation) takes a different path: it uses a single code family—quasi-cyclic moderate-density parity-check (QC-MDPC) codes—and relies on an iterative bit-flipping decoder rather than an algebraic one. This yields the smallest keys and ciphertexts among the three code-based schemes, but at the cost of a subtle and difficult-to-quantify decoding failure probability. BIKE was a NIST fourth-round candidate alongside Classic McEliece and HQC.

9.6.1 QC-MDPC Codes

The design of BIKE rests on a code family that sits between two extremes. Classical LDPC codes have very sparse parity-check matrices (constant row weight), which makes them efficient to decode but also easy to distinguish from random codes. Fully random codes are indistinguishable but intractable to decode. QC-MDPC codes occupy the middle ground: their parity-check matrices are denser than LDPC but far sparser than random, striking a balance between decodability and pseudorandomness.

Definition 9.11 (QC-MDPC Code) A *quasi-cyclic moderate-density parity-check* code is a binary linear code whose parity-check matrix $H = [H_0 \mid H_1]$ consists of two $r \times r$ circulant blocks, each with row weight $w/2$ (so the total row weight is w). The term “moderate density” distinguishes these from LDPC codes: the row weight $w = O(\sqrt{n})$ is larger than the $O(1)$ weight of LDPC codes but much smaller than $n/2$.

The resulting code has parameters $[n = 2r, k = r, d \geq w + 1]$ (rate $1/2$) and can be decoded using iterative *bit-flipping* algorithms. The quasi-cyclic structure means the parity-check matrix is determined by two polynomials $h_0, h_1 \in \mathbb{F}_2[x]/(x^r - 1)$, giving a compact representation.

9.6.2 BIKE Construction

The BIKE construction encapsulates a shared secret by encoding it with the public quasi-cyclic code and adding intentional errors. Only the holder of the secret sparse parity-check matrix can decode efficiently.

Definition 9.12 (BIKE KEM)

KeyGen(1^λ):

1. Sample $h_0, h_1 \in \mathbb{F}_2[x]/(x^r - 1)$ of weight $w/2$ each.
2. Compute $h = h_1 \cdot h_0^{-1} \bmod (x^r - 1)$.
3. Set $\text{pk} = h$ and $\text{sk} = (h_0, h_1)$.

Key generation produces a public polynomial h that is indistinguishable from random, while the secret key retains the sparse factors h_0 and h_1 needed for efficient decoding. The public key is a single polynomial—just r bits—making BIKE’s keys the most compact of any code-based scheme.

Encaps(pk):

1. Sample a random message $\mathbf{m}$ and derive $(\mathbf{e}_0, \mathbf{e}_1)$ of weight t via a hash function.
2. Compute $\mathbf{c} = (\mathbf{c}_0, \mathbf{c}_1) = (\mathbf{e}_0 + \mathbf{e}_1 \cdot h, \mathbf{m} \oplus \text{Hash}(\mathbf{e}_0, \mathbf{e}_1))$.
3. Shared secret: $K = \text{Hash}(\mathbf{m}, \mathbf{c})$.

Encapsulation deterministically derives an error pattern from a random seed, then uses the public key to produce a syndrome-like ciphertext. The message itself is protected by a one-time pad derived from the error vector, so recovering the shared secret requires recovering the error—which is the syndrome decoding problem.

Decaps($\text{sk}, \mathbf{c}$):

1. Use the secret parity-check matrix $H = [H_0 \mid H_1]$ to compute the syndrome of $(\mathbf{c}_0, \mathbf{c}_1 \cdot h_0) = (\mathbf{e}_0 + \mathbf{e}_1 \cdot h) \cdot h_0$, and apply a bit-flipping decoder to recover $(\mathbf{e}_0, \mathbf{e}_1)$.
2. Recover $\mathbf{m}$ and re-derive K .

Decapsulation succeeds because the secret key’s sparse structure enables the bit-flipping decoder to find the error pattern. The decoder iteratively flips bits that participate in the most unsatisfied parity checks—a heuristic process that works with high probability but, unlike Patterson’s algebraic decoder, can occasionally fail.

9.6.3 The Decoding Failure Challenge

The iterative nature of BIKE’s decoder introduces a problem that Classic McEliece and HQC do not face to the same degree: the *decoding failure rate* (DFR) is not just an inconvenience but a security vulnerability.

Table 9.4 BIKE parameter sets

Level	r	w	PK (bytes)	CT (bytes)	Target DFR
128	12,323	142	1,541	1,573	$< 2^{-128}$
192	24,659	206	3,083	3,115	$< 2^{-192}$
256	40,973	274	5,122	5,154	$< 2^{-256}$

Table 9.5 Comparison of code-based KEMs at NIST Security Level I (128-bit post-quantum security). ML-KEM-768 is included as a lattice-based reference point.

Scheme	PK (B)	CT (B)	PK+CT (B)	Security basis
Classic McEliece	261,120	128	261,248	Binary Goppa
HQC	2,249	4,497	6,746	QC codes
BIKE	1,541	1,573	3,114	QC-MDPC
ML-KEM-768	1,184	1,088	2,272	Module-LWE

Why decoding failures matter for security. A non-negligible decoding failure rate is not merely an engineering nuisance—it is a *security vulnerability*. Guo, Johansson, and Stankovski (2016) showed that an adversary who can trigger decoding failures can recover the secret key through a *reaction attack*: by observing which ciphertexts cause decryption failures, the attacker learns information about the secret key’s structure. This forces BIKE to target an extremely low DFR (below 2^{-128}), which is difficult to prove for iterative decoders.

The BIKE team addresses this through conservative parameter choices that ensure high decoder success probability, combined with the Black–Gray–Flip decoder—an improved bit-flipping algorithm with provably lower failure rates than the basic decoder. Extensive statistical analysis and extrapolation from empirical data are used to certify $\text{DFR} < 2^{-128}$, though a fully rigorous analytical bound remains elusive.

BIKE’s public keys and ciphertexts are the smallest among the three code-based schemes discussed here, but the lack of a rigorous DFR bound (as opposed to an empirical/heuristic one) was a factor in NIST’s decision not to select BIKE for immediate standardisation.

9.7 Comparative Analysis

The three code-based KEMs examined in this chapter occupy distinct points in the design space, each resolving the tension between security confidence, key size, ciphertext size, and implementation maturity in a different way. Table 9.5 makes the trade-offs concrete.

In terms of *security confidence*, Classic McEliece holds the strongest position: 45+ years of cryptanalysis, conservative assumptions, and a reduction to the well-studied

syndrome decoding problem for unstructured codes. HQC and BIKE rely on the quasi-cyclic variant of syndrome decoding, which has a shorter track record but no known weaknesses. ML-KEM relies on Module-LWE, well-studied but post-dating McEliece by decades.

The *size trade-offs* are stark. Classic McEliece has tiny ciphertexts (128 bytes) but enormous public keys (255 KB), making it ideal for scenarios where the public key can be cached. HQC and BIKE achieve practical key sizes through quasi-cyclic structure, with BIKE offering the smallest combined PK+CT among the code-based schemes—roughly half of HQC’s total bandwidth.

Decoding assurance separates the three schemes sharply. Classic McEliece has zero decoding failures thanks to its algebraic decoder. HQC has a negligible but analytically bounded failure probability. BIKE’s failure probability is the hardest to bound rigorously, relying on heuristic analysis of iterative decoders—and as the reaction attack of Guo et al. demonstrates, decoding failures in a KEM are not merely inconvenient but actively dangerous.

From an *implementation* standpoint, Classic McEliece is the simplest to implement correctly, owing to the well-understood algebraic structure of Goppa codes. BIKE requires careful constant-time implementation of the bit-flipping decoder to avoid timing side channels. As of 2024, ML-KEM is the primary NIST standard, HQC was selected for standardisation as a code-based backup, and Classic McEliece and BIKE remain under evaluation.

When to use code-based cryptography. Classic McEliece is the right choice when public keys can be cached (e.g., in long-lived server certificates) and security confidence is paramount—it is the “gold standard” for conservative post-quantum security. HQC provides a balanced code-based alternative suitable for general deployment, and its selection by NIST ensures it will be available as a hedge against lattice breaks. BIKE offers the most compact combined sizes but requires the strongest trust in heuristic decoder analysis.

Problems

9.1 Construct the $[7, 4, 3]$ Hamming code.

- Write its generator matrix G and parity-check matrix H in systematic form.
- Encode the message $\mathbf{m} = (1, 0, 1, 1)$ to obtain a codeword $\mathbf{c}$.
- Introduce a single-bit error in position 3 to form the received word $\mathbf{y} = \mathbf{c} + \mathbf{e}_3$. Compute the syndrome $\mathbf{s} = H\mathbf{y}^\top$ and use it to identify and correct the error.
- What is the maximum number of errors the Hamming code can correct? How does this relate to the minimum distance?

9.2 In the McEliece cryptosystem with a binary Goppa code over $\mathbb{F}_{2^{12}}$ and Goppa polynomial of degree $t = 64$:

- (a) Compute the code parameters $[n, k, d]$ assuming $n = 2^{12}$ support points and using $k \geq n - mt$.
- (b) Compute the public key size in bytes (systematic form).
- (c) Compare this with the public key sizes of ML-KEM-768 (1,184 bytes) and RSA-2048 (256 bytes).
- (d) A 100 Mbps link establishes 1,000 TLS sessions per second. Estimate the bandwidth overhead of replacing ECDH key exchange with McEliece.

9.3 Prange's ISD analysis.

- (a) For a random $[n, k]$ code with w errors, show that the probability that a random information set $\mathcal{I}$ of size k is error-free is $p = \binom{n-w}{k} / \binom{n}{k}$.
- (b) For Classic McEliece parameters $n = 3488$, $k = 2720$, $w = 64$, compute $\log_2(1/p)$ and estimate the work factor of Prange's algorithm.
- (c) Compare your estimate with the claimed 128-bit security level. What does this tell you about the gap between Prange's algorithm and BJMM?

9.4 Quasi-cyclic structure.

- (a) Let $r = 7$ and consider the ring $\mathbb{F}_2[x]/(x^7 - 1)$. Write the 7×7 circulant matrix corresponding to the polynomial $h(x) = 1 + x + x^3$.
- (b) Verify that the product of two circulant matrices is circulant by computing $h(x) \cdot g(x) \bmod (x^7 - 1)$ for $g(x) = 1 + x^2 + x^4$ and checking against the matrix product.
- (c) Explain why storing a circulant matrix requires only r bits rather than r^2 bits, and why this is crucial for the practicality of HQC and BIKE.

9.5 Decoding failure and security.

- (a) Explain why a decoding failure rate of 2^{-64} is insufficient for a KEM claiming 128-bit security. (Hint: consider how an IND-CCA adversary can exploit decryption failures.)
- (b) In the Guo–Johansson–Stankovski reaction attack on QC-MDPC codes, the attacker submits specially crafted ciphertexts and observes whether decryption succeeds or fails. Explain qualitatively how the pattern of failures reveals information about the secret key.
- (c) Why does this attack not apply to Classic McEliece?

9.6 Codes and statistical mechanics.

- (a) Consider the binary symmetric channel with crossover probability p . State Shannon's noisy-channel coding theorem and identify the capacity $C(p) = 1 - H_2(p)$, where H_2 is the binary entropy function.
- (b) The syndrome decoding problem asks for a weight- w vector $\mathbf{e}$ satisfying $r = n - k$ linear constraints. By analogy with a random constraint satisfaction problem, argue that a phase transition occurs at the Gilbert–Varshamov bound: for w/n below a critical threshold (depending on $R = k/n$), solutions exist with high probability; above it, they do not.
- (c) Explain why this phase transition is relevant to setting parameters for code-based cryptosystems.

Chapter 10

Hash-Based Signatures

Abstract Hash-based signatures achieve quantum resistance using only the assumption that hash functions are one-way—the most minimal and conservative foundation in all of post-quantum cryptography. This chapter traces the evolution from Lamport’s one-time signatures through Merkle trees to the stateless SPHINCS+ scheme standardized by NIST as SLH-DSA, developing each construction with full algorithmic detail.

10.1 One-Time Signatures

Every signature scheme we have encountered so far allows a signer to authenticate an unlimited number of messages under a single key pair. What happens if we relax that guarantee to its absolute minimum—a key pair that can sign *exactly one* message? The resulting primitive, called a *one-time signature* (OTS), seems cripplingly limited, but it turns out to be the seed from which all hash-based signature schemes grow. The reason is simple: if we can build a secure OTS from nothing more than a one-way function, then we need no algebraic structure, no lattices, no codes—just hashing. The challenge, which the rest of this chapter addresses, is scaling that one-shot capability into a practical many-time signature scheme.

10.1.1 The Lamport Construction

Lamport [36] observed in 1979 that any one-way function suffices to build a one-time signature. The idea is elegant in its simplicity: pre-commit to two random secrets for every bit of the message, then reveal one secret per bit depending on the message content. Let us make this precise.

Fix a one-way function $H: \{0, 1\}^n \rightarrow \{0, 1\}^n$ and suppose that messages are n -bit strings. The signer prepares two random values for each bit position—one labelled 0, one labelled 1—and publishes their hash images as the public key. To sign a message $m = m_1 m_2 \cdots m_n$, the signer reveals, for each position i , whichever secret corresponds

to the bit m_i . The verifier simply hashes each revealed secret and checks it against the appropriate entry in the public key. Since H is one-way, no adversary can recover the unrevealed secrets, and those are precisely the ones needed to sign a different message.

Definition 10.1 (Lamport One-Time Signature) For $i = 1, \dots, n$, sample $x_i^0, x_i^1 \xleftarrow{\$} \{0, 1\}^n$ uniformly at random and compute $y_i^b = H(x_i^b)$ for $b \in \{0, 1\}$. The signing key is $\text{sk} = (x_i^b)_{i,b}$ and the verification key is $\text{pk} = (y_i^b)_{i,b}$.

To sign a message $m = m_1 m_2 \dots m_n \in \{0, 1\}^n$, the signer outputs $\sigma = (x_1^{m_1}, x_2^{m_2}, \dots, x_n^{m_n})$.

To verify a purported signature $\sigma = (s_1, \dots, s_n)$ on message m , the verifier accepts if $H(s_i) = y_i^{m_i}$ for all i .

Correctness is immediate: the signer reveals exactly those preimages that correspond to the bits of m , and the verifier recomputes the hashes and checks them against the public key.

Theorem 10.1 (Security of Lamport OTS) *If H is a one-way function, then the Lamport scheme is existentially unforgeable under a one-time chosen-message attack (EU-CMA_1).*

Proof sketch. An adversary who observes a signature on m learns the preimages $x_i^{m_i}$ for each bit position i . To forge a signature on a different message $m' \neq m$, the adversary must produce a valid preimage $x_j^{1-m_j}$ for at least one position j where $m'_j \neq m_j$. Since $x_j^{1-m_j}$ was never revealed and H is one-way, producing such a preimage succeeds with at most negligible probability. A formal reduction embeds a one-wayness challenge at a randomly chosen position, succeeding with probability at least ε/n , where ε is the adversary's forgery advantage. $\square$

Why “one-time”? If the same key signs two distinct messages $m \neq m'$, then for every bit position j where $m_j \neq m'_j$, the adversary learns *both* preimages x_j^0 and x_j^1 . After seeing just two signatures, the adversary can forge a signature on any message that agrees with m or m' at every bit position—potentially exponentially many messages.

The Lamport scheme, however, pays a steep price in size. Each secret is an n -bit string, and the signer maintains two secrets per bit position, so for $n = 256$ the signing key occupies $2n \cdot n = 2 \cdot 256 \cdot 256 = 16\,384$ bytes. The verification key is equally large. The signature itself contains one secret per bit position: $n \cdot n = 256 \cdot 256 = 8\,192$ bytes. These sizes are tolerable for a proof of concept, but entirely impractical for real-world deployment. We need a way to compress signatures without abandoning the one-way-function-only paradigm—and that is exactly what the Winternitz construction achieves.

10.1.2 Winternitz OTS (WOTS+)

The fundamental inefficiency of Lamport signatures lies in processing the message one bit at a time: each bit demands its own pair of secrets, yielding signatures with n components. The Winternitz one-time signature asks a natural question: what if we process several bits at once?

The core idea is to replace the single hash evaluation per bit position with a *hash chain*—a sequence of iterated applications of a hash function. Instead of revealing a preimage to authenticate a single bit, the signer reveals a value at a particular *depth* in the chain, and that depth encodes a multi-bit digit of the message. Concretely, if we group bits into base- w digits (each representing $\log_2 w$ bits), then a digit $d \in \{0, 1, \dots, w-1\}$ is authenticated by revealing the value $F^{(d)}(x)$ —the result of applying the hash function d times starting from a secret x . The verifier can check this by applying F an additional $w-1-d$ times and comparing the result against the public key, which is $F^{(w-1)}(x)$.

This is the *Winternitz tradeoff*: larger w means each chain encodes more bits, so fewer chains are needed and signatures shrink. But longer chains require more hash evaluations during signing and verification, so speed decreases. Doubling w roughly halves the number of chains but doubles the average chain length. The parameter w gives the scheme designer a knob to turn between compactness and performance.

Definition 10.2 (Hash Chain) For a function $F: \{0, 1\}^n \rightarrow \{0, 1\}^n$, define the iterated application

$$F^{(j)}(x) = \underbrace{F(F(\dots F(x) \dots))}_j, \quad (10.1)$$

with $F^{(0)}(x) = x$.

With hash chains in hand, we can describe the full construction. In WOTS+, a message digest of n bits is split into $\ell_1 = \lceil n/\log_2 w \rceil$ base- w digits $d_1, \dots, d_{\ell_1}$, each in $\{0, 1, \dots, w-1\}$. A checksum $C = \sum_{i=1}^{\ell_1} (w-1-d_i)$ is appended, encoded in $\ell_2 = \lceil \log_2(\ell_1(w-1))/\log_2 w \rceil + 1$ additional base- w digits $d_{\ell_1+1}, \dots, d_{\ell}$, where $\ell = \ell_1 + \ell_2$.

The checksum is essential. Without it, an adversary who observes a signature could simply apply additional hash iterations to forge signatures on messages with smaller digit values. The checksum ensures that any modification that *decreases* a message digit (which the adversary could handle by hashing further along the chain) must *increase* at least one checksum digit—requiring the adversary to invert a hash chain, which is hard.

Definition 10.3 (WOTS+ Scheme) Let $w \geq 2$, n the security parameter, and $\ell = \ell_1 + \ell_2$ as above.

The signer generates ℓ secret values $x_1, \dots, x_{\ell} \xleftarrow{\$} \{0, 1\}^n$ and computes the public key components $\text{pk}_i = F^{(w-1)}(x_i)$ for $i = 1, \dots, \ell$ —each public key element sits at the far end of its chain.

To sign, the signer computes the base- w representation $(d_1, \dots, d_{\ell})$ of the message digest concatenated with its checksum, then outputs $\sigma = (F^{(d_1)}(x_1), \dots, F^{(d_{\ell})}(x_{\ell}))$. Each signature component is the chain value at depth d_i —shallow for small digits, deep for large ones.

To verify, the verifier computes $F^{(w-1-d_i)}(\sigma_i)$ for each i and checks equality with pk_i . Since the chain is $w - 1$ steps long and the signer revealed the value at step d_i , the verifier needs exactly $w - 1 - d_i$ more steps to reach the public key.

Example 10.1 (WOTS+ Sizes with $w = 16$, $n = 256$) With $w = 16$, each digit encodes $\log_2 16 = 4$ bits. The number of message digits is $\ell_1 = \lceil 256/4 \rceil = 64$. The checksum bound is $\ell_1(w-1) = 64 \cdot 15 = 960$, requiring $\ell_2 = \lfloor \log_{16}(960) \rfloor + 1 = 3$ additional digits. The total number of chains is $\ell = 67$. The signature size drops to $67 \times 32 = 2144$ bytes—nearly four times smaller than Lamport’s 8192 bytes at the same security level. The average number of hash evaluations per signature is $67 \times 7.5 = 502.5$, and verification costs the same on average.

Lamport and WOTS+ solve the one-time problem with differing compactness, but neither addresses the elephant in the room: real-world signing keys must authenticate more than one message. The next section shows how Ralph Merkle’s deceptively simple tree construction extends any OTS into a many-time scheme—at the cost of maintaining state.

10.2 Merkle’s Tree-Based Signatures

A one-time signature authenticates a single message. To sign a thousand messages, we need a thousand key pairs—but publishing a thousand independent public keys defeats the purpose of public-key cryptography. Merkle [44] solved this problem in 1979 with a construction so natural it seems inevitable in hindsight: arrange the OTS public keys as the leaves of a binary hash tree and publish only the root. The root binds all 2^h OTS keys together into a single compact public key, and each signature proves—via a short chain of hashes called an authentication path—that the leaf it used is part of the committed tree.

10.2.1 The Merkle Tree Construction

Fix a tree height h and an OTS scheme (e.g., WOTS+). The signer generates 2^h independent OTS key pairs at setup and organises them into a complete binary tree. Each leaf stores (the hash of) an OTS public key. Each internal node stores the hash of its two children concatenated. The root, which compresses the entire tree into a single hash value, serves as the global public key.

When the signer wishes to authenticate a message, it picks the next unused OTS key pair—say the one at leaf i —signs the message with it, then provides enough information for the verifier to recompute the root starting from that leaf. That “enough information” is the authentication path: the h sibling nodes encountered on the walk from leaf i up to the root.

Definition 10.4 (Merkle Signature Scheme) Generate 2^h independent OTS key pairs $(\text{sk}_i, \text{pk}_i)$ for $i = 0, \dots, 2^h - 1$. Assign each pk_i to a leaf of a complete binary tree of

height h . For each internal node, compute its value as $H(\text{node}_{\text{left}} \parallel \text{node}_{\text{right}})$. The root of the tree is the public key PK.

To sign message m using index i , the signer produces the OTS signature σ_{ots} under sk_i , then computes the authentication path $\text{Auth} = (a_0, a_1, \dots, a_{h-1})$, where a_j is the sibling node at level j on the path from leaf i to the root. The full signature is $\sigma = (i, \sigma_{\text{ots}}, \text{Auth})$.

To verify, the verifier first checks σ_{ots} on m using pk_i . Starting from the leaf hash of pk_i , the verifier iteratively hashes together each pair of sibling nodes along the authentication path. If the reconstructed root equals PK, the signature is accepted.

Theorem 10.2 (Security of the Merkle Scheme) *If the underlying OTS is EU-CMA₁-secure and H is second-preimage resistant, then the Merkle scheme is EU-CMA-secure (many-time), provided no OTS key is reused.*

Proof sketch. Suppose an adversary forges a signature on a new message. The forged signature includes an index i and an OTS signature. If this index was previously used to sign a different message, the adversary has broken the OTS scheme. If the index was used but the adversary provides a different authentication path that still hashes to the root, the adversary has found a second preimage for one of the internal hash computations. Either case contradicts the assumptions. $\square$

10.2.2 Authentication Paths

The authentication path deserves a closer look, because it is the mechanism that makes the entire construction work—and it reappears in every scheme for the rest of this chapter.

Consider a concrete example. Suppose we have a tree of height $h = 3$ with eight leaves $\ell_0, \dots, \ell_7$, and we want to verify a signature that used leaf ℓ_5 . The path from ℓ_5 to the root passes through three internal nodes: the parent of ℓ_5 , then its grandparent, and finally the root. At each level, the verifier needs the *sibling* node—the one it cannot compute from the information it already has—to continue the chain of hashes upward.

Example 10.2 (Authentication Path for Leaf 5 in a Height-3 Tree) In a tree with leaves $\ell_0, \dots, \ell_7$, the path from ℓ_5 to the root passes through nodes (ℓ_5, N_{23}, N_1, R) , where N_{23} is the parent of ℓ_4 and ℓ_5 , and N_1 is the left child of the root. The authentication path consists of the three siblings at each level: $\text{Auth} = (\ell_4, N_{22}, N_0)$.

The verifier begins with the hash of ℓ_5 (obtained from the OTS public key) and proceeds level by level. At the bottom level, it hashes ℓ_4 together with ℓ_5 to reconstruct the parent N_{23} . One level up, it hashes N_{22} (the sibling it received) together with the just-computed N_{23} to reconstruct N_1 . At the top, it hashes N_0 (the other child of the root) together with N_1 to reconstruct the root:

$$\begin{aligned} v_0 &= H(\ell_4 \parallel \ell_5), \\ v_1 &= H(N_{22} \parallel v_0), \\ v_2 &= H(N_0 \parallel v_1), \end{aligned}$$

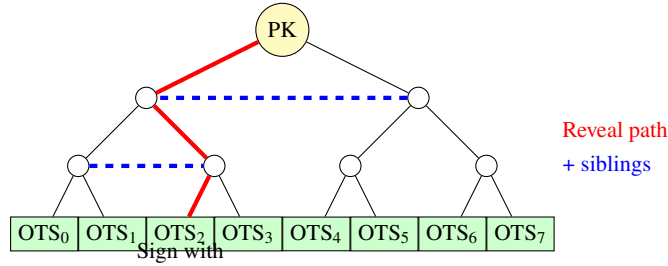


Fig. 10.1 Merkle tree: a single public key authenticates many one-time signatures. The authentication path (red) and required sibling nodes (blue, dashed) enable verification from any leaf to the root.

and accepts if $v_2 = \text{PK}$. The verifier needed exactly $h = 3$ sibling hashes—one per level—to walk from a single leaf all the way to the root.

Figure 10.1 illustrates the Merkle tree structure for a height-3 tree. The root (yellow) serves as the public key, the leaves (green) hold OTS key pairs, and the authentication path for a given signature is highlighted: the red path traces the hash chain from the signing leaf to the root, while the blue dashed edges indicate the sibling nodes that the verifier needs to reconstruct each level.

The authentication path has exactly h nodes—one sibling at each level of the tree. For a tree of height $h = 20$ (supporting $2^{20} \approx 10^6$ signatures) with a 256-bit hash function, the authentication path is $20 \times 32 = 640$ bytes. This is remarkably compact: a few hundred bytes suffice to prove membership in a tree with a million leaves.

10.2.3 Key Generation Cost

The elegance of the Merkle construction comes with a practical burden at key generation time. Generating 2^h OTS key pairs and computing the full tree requires $O(2^h)$ OTS key generations and $O(2^h)$ hash evaluations. For $h = 20$ with WOTS+ ($w = 16$, $n = 256$), this amounts to roughly $2^{20} \times 67 \times 15 \approx 10^9$ hash evaluations—feasible but expensive. Techniques such as *BDS tree traversal* [10] reduce the memory needed to compute authentication paths from $O(2^h)$ to $O(h^2)$, storing only a logarithmic number of nodes.

The Merkle scheme transforms one-time signing into many-time signing, but at a cost: the signer must track which leaf indices have been used, and must never reuse one. This state management requirement, as we shall see, is the central tension driving the next two decades of hash-based signature design.

10.3 Stateful Schemes: XMSS and LMS

The Merkle construction proved that OTS keys could be aggregated into a many-time scheme, but it left the signer with a fragile obligation: maintain a counter, increment it

faithfully, and *never* reuse an index. The *eXtended Merkle Signature Scheme* (XMSS) and the *Leighton–Micali Signature* scheme (LMS)—codified in RFC 8391 [28] and RFC 8554 [43], respectively, with NIST guidance in SP 800-208—are two standardised schemes that refine and harden the Merkle construction for real-world deployment.

10.3.1 XMSS

XMSS improves on the basic Merkle scheme in three important ways. First, it uses WOTS+ as the OTS layer at every leaf, gaining the size benefits of Winternitz chains over Lamport signatures. Second, before hashing two child nodes, XMSS XORs each child with a position-dependent bitmask. This seemingly minor change has a major impact on the security proof: it allows the reduction to exploit a *multi-function multi-target* second-preimage resistance notion, leading to tighter security bounds and smaller parameter choices. Third, the ℓ components of a WOTS+ public key are compressed into a single n -bit value via an unbalanced binary tree called the *L-tree*, which is then placed at the Merkle tree leaf.

Definition 10.5 (XMSS Parameters) An XMSS instance is specified by:

- n : security parameter (hash output length in bytes, typically 32).
- w : Winternitz parameter (typically 16).
- h : tree height; the scheme can sign 2^h messages.

10.3.2 Multi-Tree XMSS (XMSS^{MT})

For large h (e.g., $h = 60$), building a single tree of height 60 is infeasible—it would require 2^{60} leaf computations. XMSS^{MT} addresses this by splitting the total height h into d layers, each of height h/d . The root tree sits at the top and certifies the roots of the trees at the next layer, which in turn certify the layer below, and so on down to the leaves. This layered structure means key generation only requires building the root tree ($2^{h/d}$ leaves, not 2^h), with lower-level trees generated on demand as signatures are produced. The price is that a signature now includes d XMSS signatures and d authentication paths—one per layer—so total signature size scales linearly with d .

10.3.3 LMS

LMS [43] takes a simpler approach than XMSS with broadly similar functionality. It uses *LM-OTS* (Leighton–Micali OTS) as the one-time primitive, which employs direct hash iteration without the bitmask XOR of WOTS+. The security proof relies on standard hash function properties without multi-target refinements, resulting in a design that is slightly simpler to implement—an attractive property for constrained environments

such as embedded systems and hardware security modules. LMS supports a hierarchical multi-tree variant (HSS) analogous to XMSS^{MT}.

Both XMSS and LMS are approved for use by NIST (SP 800-208) in applications where state management can be guaranteed.

10.3.4 The State Management Challenge

Stateful schemes impose a critical operational requirement: the signer must *never* reuse an OTS index. Violating this requirement destroys security—not gradually, but catastrophically.

The state management nightmare. Consider the implications. Restoring a signer from a backup that has index $i = 1000$ after the live system has advanced to $i = 1500$ causes indices 1001–1500 to be reused. Cloning a virtual machine duplicates the state counter, causing immediate reuse. If the state counter is lost to hardware failure, no further signatures can be safely produced. Distributed signing—multiple signers sharing a key—requires coordinating state, introducing a single point of failure.

For these reasons, stateful schemes are recommended only for controlled environments: firmware signing, certificate authority roots, and air-gapped systems.

The fragility of state management is not merely an engineering inconvenience; it is a fundamental barrier to widespread deployment. Eliminating this barrier requires a radically different approach to index selection—one that does not rely on a counter at all. The next two sections develop the ingredients needed for that approach: a signature primitive that tolerates bounded reuse (FORS), and a tree-of-trees architecture (the hypertree) that makes statelessness possible.

10.4 FORS: Few-Time Signatures from Random Subsets

We have seen that one-time signatures break catastrophically when a key is reused. But what if we could build a signature primitive that degrades *gracefully*—remaining secure even after a handful of uses? Such a scheme, called a *few-time signature*, would be exactly what a stateless design needs. If keys are assigned pseudorandomly rather than sequentially, most keys will be used zero or one times, a few will be used twice or three times, and the scheme must tolerate this without collapsing.

The *Forest of Random Subsets* (FORS) scheme was designed specifically to fill this role as the bottom layer of SPHINCS+. Its key idea is to replace the hash-chain mechanism of WOTS+ with a selection-from-subsets approach: instead of revealing

chain values, the signer reveals elements chosen from disjoint forests of secret values, with the selection determined by the message hash.

10.4.1 Construction

FORS organises its secrets into k groups, each containing $t = 2^a$ values arranged as the leaves of a binary hash tree. Signing a message means interpreting its hash as k indices—one per group—and revealing the selected secret along with its authentication path in the corresponding tree. Security comes from the combinatorial difficulty of finding a message whose k indices all land on previously revealed secrets.

Definition 10.6 (FORS Scheme) Fix parameters k (number of trees) and a (tree height, so each tree has $t = 2^a$ leaves).

The signer generates $k \cdot t$ random secret values $\text{sk}_{j,i}$ for $j = 1, \dots, k$ and $i = 0, \dots, t-1$, then builds k binary hash trees, one over each group of t secrets. The public key is the collection of k tree roots, compressed into a single hash.

To sign a message, the signer hashes it to obtain k indices $(d_1, \dots, d_k)$, each in $\{0, \dots, t-1\}$. For each tree j , the signer reveals the secret sk_{j,d_j} together with the authentication path from leaf d_j to the root of tree j . The signature is $\sigma = ((\text{sk}_{j,d_j}, \text{Auth}_j))_{j=1}^k$.

Verification proceeds by hashing each revealed secret and using the authentication path to recompute the root of each tree. The verifier accepts if all k reconstructed roots match the public key.

10.4.2 Few-Time Security

The reason FORS tolerates bounded reuse is quantitative rather than structural. Each signature reveals k out of $k \cdot t$ secret values. After q signatures, an adversary knows at most qk values. To forge a signature on a new message, the adversary needs all k indices of that message to fall among the already-revealed values. The probability that a random message can be forged after q signatures is at most $(q/t)^k$ —a quantity that shrinks exponentially in k as long as q remains much smaller than t .

Example 10.3 (FORS Forgery Probability) With $k = 35$ and $a = 9$ ($t = 512$), after $q = 2^{16}$ signatures the forgery probability is at most $(2^{16}/512)^{35} = (2^7)^{35} = 2^{245}$... but this is per target message. The actual analysis in SPHINCS+ accounts for multi-target attacks and sets parameters so that the overall forgery probability remains below $2^{-\lambda}$ for the target security level λ .

With a few-time signature in hand, we now have every piece needed for the main construction. The remaining question is architectural: how do we assemble FORS, WOTS+, and Merkle trees into a single coherent scheme that needs no state at all?

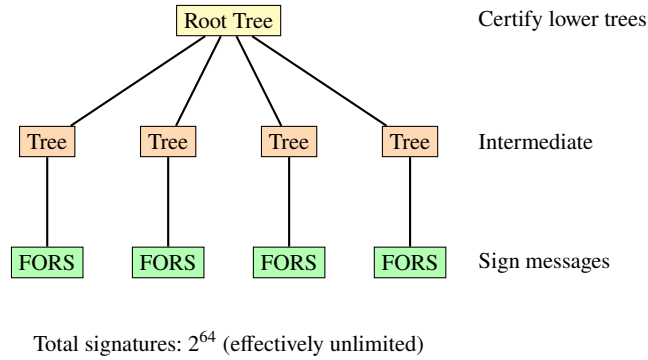


Fig. 10.2 SPHINCS+ hypertree: a tree of Merkle trees that enables stateless signing through massive virtual capacity and few-time signatures at the leaves.

10.5 SLH-DSA (SPHINCS+)

The stateful schemes of the previous section work well in controlled environments, but their fragile state management makes them unsuitable for general-purpose use—web servers, mobile devices, automated TLS endpoints. SPHINCS+ [4, 5], selected by NIST as the sole hash-based signature standard and published as *SLH-DSA* in FIPS 205 [51], eliminates state management entirely. Its central insight is to combine two ideas we have already developed: the hypertree (a tree of Merkle trees, descended from XMSS^{MT}) and randomised index selection that replaces the sequential counter.

10.5.1 The Hypertree Construction

SLH-DSA organises its signatures in a *hypertree*: a tree of XMSS trees arranged in d layers, with total tree height h divided as $h = h' \cdot d$, where h' is the height of each XMSS subtree.

At the top sits a single XMSS tree of height h' whose root is the public key—the only value the verifier needs to store. Each leaf of this top tree certifies (via a WOTS+ signature) the root of an XMSS tree at the next layer down. Those second-layer trees in turn certify the roots of third-layer trees, and so on, until the bottom layer is reached. At the bottom sit FORS instances—the few-time signatures that actually authenticate messages. Figure 10.2 depicts this layered architecture.

A signature therefore consists of the FORS signature on the message, followed by a chain of d WOTS+ signatures and authentication paths, one per hypertree layer:

$$\sigma_{\text{SLH}} = (\sigma_{\text{FORS}}, \sigma_{\text{WOTS}_1^+}, \text{Auth}_1, \dots, \sigma_{\text{WOTS}_d^+}, \text{Auth}_d). \quad (10.2)$$

Each WOTS+ signature in this chain serves as a certificate: it binds the root of a lower-layer tree to a leaf in the layer above, all the way up to the public key.

10.5.2 Removing State: Randomised Hashing

The key insight that eliminates state is deceptively simple. Instead of maintaining a counter to select FORS keys sequentially ($i = 0, 1, 2, \dots$), SLH-DSA derives the FORS key index from the message itself:

$$R \xleftarrow{\$} \{0, 1\}^n, \quad \text{idx} = H_{\text{msg}}(R, \text{pk}, m) \bmod 2^h. \quad (10.3)$$

The signer samples a fresh random value R , hashes it together with the public key and message, and interprets part of the output as an index into the hypertree. Since the hypertree contains 2^h FORS instances and h is large (typically $h \geq 64$), the probability of any single FORS instance being reused more than a few times over the scheme’s entire lifetime is negligible. This is precisely where the few-time security of FORS (Sect. 10.4) earns its keep: even if a FORS key is used two or three times by chance, security degrades gracefully rather than failing completely.

Statefulness to statelessness—a paradigm shift. Stateful schemes (XMSS, LMS) assign indices sequentially: $i = 0, 1, 2, \dots$. SLH-DSA assigns indices pseudorandomly. The price is that the hypertree must be enormous (2^h FORS instances), most of which are never used. But since trees are computed on demand, this virtual size incurs no storage cost—only the signing and verification time to traverse d layers.

10.5.3 Signing and Verification

The signing procedure makes the layered architecture concrete. The signer begins at the bottom of the hypertree by producing a FORS signature on the message, then walks upward through each layer, producing a WOTS+ signature and authentication path that certify the tree root at each level.

Verification mirrors this bottom-up traversal. The verifier recomputes idx from (R, pk, m) , reconstructs the FORS public key from σ_{FORS} , then walks up each hypertree layer, verifying each WOTS+ signature and authentication path until it reaches the root. If the reconstructed root matches the public key, the signature is valid.

10.5.4 Parameter Sets

FIPS 205 defines six parameter sets at three security levels, each available in a “small” (s) and “fast” (f) variant. The distinction between these variants reveals one final tradeoff.

Algorithm 15 SLH-DSA Signing**Require:** Secret key SK, message m **Ensure:** Signature σ

```

1:  $R \xleftarrow{\$} \{0, 1\}^n$  ▷ Randomness for stateless index selection
2:  $\text{digest} \leftarrow H_{\text{msg}}(R, \text{SK.seed}, \text{SK.root}, m)$ 
3: Parse digest as  $(\text{md}, \text{idx})$  ▷ FORS message digest and tree index
4:  $\sigma_{\text{FORS}} \leftarrow \text{FORS.Sig}(\text{md}, \text{SK.seed}, \text{idx})$ 
5:  $\text{pk}_{\text{FORS}} \leftarrow \text{FORS.pkFromSig}(\sigma_{\text{FORS}}, \text{md}, \text{idx})$ 
6: for  $j = 1$  to  $d$  do ▷ Traverse hypertree layers
7:    $\sigma_{\text{WOTS},j} \leftarrow \text{WOTS.Sig}(\text{pk}_{\text{prev}}, \text{SK.seed}, \text{addr}_j)$ 
8:    $\text{Auth}_j \leftarrow$  authentication path in XMSS tree at layer  $j$ 
9:    $\text{pk}_{\text{prev}} \leftarrow$  root of XMSS tree at layer  $j$ 
10: end for
11: return  $\sigma = (R, \sigma_{\text{FORS}}, \sigma_{\text{WOTS},1}, \text{Auth}_1, \dots, \sigma_{\text{WOTS},d}, \text{Auth}_d)$ 

```

Table 10.1 SLH-DSA parameter sets (FIPS 205)

Parameter set	n	h	PK (B)	Sig (B)	Level
SLH-DSA-128s	16	63	32	7 856	I
SLH-DSA-128f	16	66	32	17 088	I
SLH-DSA-192s	24	63	48	16 224	III
SLH-DSA-192f	24	66	48	35 664	III
SLH-DSA-256s	32	64	64	29 792	V
SLH-DSA-256f	32	68	64	49 856	V

The “s” (small) vs. “f” (fast) tradeoff is governed by the hypertree structure. The “small” variants use more XMSS layers (d larger), each with smaller height h' . This reduces the number of WOTS+ chains evaluated per layer but increases the number of layers traversed, yielding smaller signatures at the cost of slower signing and verification. The “fast” variants use fewer layers (d smaller) with taller XMSS trees, producing larger signatures but faster operation—fewer layers means fewer WOTS+ signatures in the overall output.

Example 10.4 (SLH-DSA vs. ML-DSA) At NIST Level I, ML-DSA-44 (lattice-based) produces 2 420-byte signatures at thousands of signatures per second. SLH-DSA-128s produces 7 856-byte signatures at roughly 15 signatures per second—over $3\times$ larger and $100\times$ slower. The justification for deploying SLH-DSA is not performance but *assumption minimality*: its security relies only on hash function properties, providing a hedge against unforeseen lattice cryptanalysis.

This performance gap raises a natural question: if SLH-DSA is so much slower and produces such larger signatures, why standardise it at all? The answer lies in the security analysis that follows.

10.6 Security Analysis

Hash-based signatures occupy a unique position in the post-quantum landscape. Every other signature scheme we study in this book relies on the presumed hardness of a *structured* mathematical problem—finding short vectors in lattices, decoding random linear codes, solving systems of multivariate equations. Hash-based signatures need none of this structure. Their security rests on the weakest assumption known to imply digital signatures at all: the existence of one-way functions.

10.6.1 Minimal Assumptions

The security of SLH-DSA reduces to three properties of the underlying hash function family $\mathcal{H}$. First, **one-wayness**: given $y = H(x)$ for random x , it is hard to find any x' with $H(x') = y$. This property secures the WOTS+ chains and the Lamport preimages at the foundation of every construction in this chapter. Second, **second-preimage resistance**: given x , it is hard to find $x' \neq x$ with $H(x') = H(x)$. This secures the Merkle tree authentication paths—if an adversary could find a second preimage for an internal node, it could forge authentication paths for leaves not in the original tree. Third, **(enhanced) target collision resistance**, a multi-function variant of collision resistance tailored to the bitmask construction used in XMSS trees.

Comparison with lattice assumptions. Lattice-based signatures (ML-DSA) rely on the hardness of Module-LWE and Module-SIS—structured algebraic problems studied for roughly 25 years. Hash-based signatures rely on properties of hash functions studied for over 40 years. The existence of one-way functions is the *weakest* assumption known to imply digital signatures (Rompel, 1990); hash-based schemes directly realise this theoretical minimum.

10.6.2 Grover’s Impact

Grover’s algorithm provides a quadratic speedup for unstructured search, and hash function inversion is precisely such a search problem. For an n -bit hash function, a quantum adversary reduces the cost of preimage search from 2^n to $2^{n/2}$, and the BHT quantum collision algorithm reduces the cost of collision search from $2^{n/2}$ to roughly $2^{n/3}$.

Theorem 10.3 (Quantum Security of Hash Functions) *Against a quantum adversary with access to Grover’s algorithm:*

- A hash function with n -bit output provides $n/2$ bits of preimage resistance.

- *Second-preimage resistance: $n/2$ bits (same as preimage, since second preimage is no easier).*
- *Collision resistance: approximately $n/3$ bits (BHT algorithm).*

Since SLH-DSA relies primarily on one-wayness and second-preimage resistance (not collision resistance), the quantum security level is approximately $n/2$ bits, where n is the hash output length in bits. For SLH-DSA-128s with $n = 16$ bytes = 128 bits, this yields $128/2 = 64$ bits of quantum preimage security—but the scheme compensates by using multiple independent hash evaluations, and the NIST security level analysis accounts for multi-target considerations and the concrete cost of quantum circuits, achieving the claimed 128-bit post-quantum security at Level I.

10.6.3 Multi-Target Attacks

A subtlety arises from the structure of WOTS+ public keys. Each key contains ℓ hash chain endpoints, and an adversary targeting *any* of these ℓ values benefits from a multi-target advantage: the cost of inverting one out of ℓ hash values is reduced by a factor of ℓ . SLH-DSA addresses this through two mechanisms. First, it uses *tweakable hash functions* parameterised by position-dependent addresses, so that each hash evaluation is domain-separated and multi-target gains cannot be pooled across unrelated computations. Second, the parameter n is set large enough that even with the multi-target advantage, the effective security exceeds the target level.

10.6.4 The Quantum Random Oracle Model

Classical security proofs model hash functions as random oracles that adversaries query classically. Against quantum adversaries, the hash function must be modelled as a *quantum random oracle* (QROM) that can be queried in superposition. This is a non-trivial strengthening: some classical random oracle proofs break entirely in the QROM—certain Fiat–Shamir transforms, for instance, lose their security guarantees when the adversary can query the oracle in superposition.

The hash-chain and Merkle tree constructions, however, are robust in this setting. Their security reduces to preimage and second-preimage resistance, which remain hard in the QROM with the appropriate parameter doubling. Security proofs for SLH-DSA have been established in the QROM [51], confirming that the scheme resists adversaries who can make quantum queries to the hash function. This resilience, combined with the minimal assumptions discussed above, is what makes hash-based signatures the most conservative choice in the post-quantum toolkit—the option to reach for when trust in newer algebraic assumptions wavers.

Problems

10.1 Lamport signature security.

- (a) Implement a Lamport one-time signature scheme using SHA-256. Sign a 256-bit message and verify the signature.
- (b) Sign a second, different message with the same key. Identify explicitly which secret values are now known to an eavesdropper who observed both signatures.
- (c) Construct a forged signature on a third message that was never signed. Explain why this is possible and state precisely which bit positions enable the forgery.

10.2 Construct a Merkle tree of height $h = 3$ (supporting 8 one-time signatures) using a hash function of your choice.

- (a) Draw the complete tree, labelling each node with its hash value (abbreviated).
- (b) Show the authentication path for the 5th leaf ($i = 4$ in zero-indexed notation).
- (c) Write out the verification procedure step by step, showing each hash computation.

10.3 WOTS+ parameter analysis. For security parameter $n = 256$ bits:

- (a) Compute the number of chains $\ell = \ell_1 + \ell_2$, the signature size in bytes, and the average number of hash evaluations for signing and verification, for $w \in \{4, 16, 256\}$.
- (b) Plot or tabulate the signature-size vs. signing-cost tradeoff as a function of w .
- (c) Compare the $w = 16$ case with a Lamport signature at the same security level. By what factor is the signature smaller?

10.4 FORS forgery analysis. Consider a FORS instance with $k = 30$ and $a = 8$ ($t = 256$).

- (a) After $q = 1$ signature, what is the probability that a random message can be forged?
- (b) After $q = 100$ signatures, compute the same probability. Is the scheme still secure?
- (c) Find the maximum q such that the forgery probability remains below 2^{-128} .

10.5 Hypertree signature size. Consider SLH-DSA with total height $h = 66$, $d = 22$ layers (so $h' = 3$ per layer), $n = 16$ bytes, $w = 16$, and FORS parameters $k = 33$, $a = 8$.

- (a) Compute the size of a single WOTS+ signature ($\ell \cdot n$ bytes).
- (b) Compute the size of a single authentication path ($h' \cdot n$ bytes).
- (c) Compute the total FORS signature size (including authentication paths for all k trees).
- (d) Compute the total SLH-DSA signature size as the sum of the FORS component plus d WOTS+ signatures and authentication paths, plus the randomness R . Compare with the value in Table 10.1.

10.6 Comparing cryptographic assumptions.

- (a) State the precise assumption needed for ML-DSA security and the precise assumption needed for SLH-DSA security. Which is weaker, and why?
- (b) Explain why the existence of one-way functions is both necessary and sufficient for digital signatures (cite Rompel's result and the trivial direction).

- (c) Suppose a polynomial-time algorithm is discovered that breaks SHA-256 preimage resistance. Which NIST post-quantum standards would be affected, and which would survive? Justify your answer.

Chapter 11

Multivariate, Isogeny-Based, and Other Approaches

Abstract Not every post-quantum proposal has survived scrutiny. This chapter examines the multivariate and isogeny-based families—including the spectacular collapse of SIKE in 2022—not merely as historical curiosities, but as case studies in what makes a cryptographic assumption trustworthy. Understanding how schemes break is as important as understanding how they work.

11.1 Multivariate Polynomial Cryptography

The idea behind multivariate cryptography is seductively simple: solving systems of multivariate quadratic equations over finite fields is NP-complete. Why not build cryptography on this? The answer, as three decades of cryptanalysis have demonstrated, is that NP-completeness of the general problem provides no guarantee whatsoever for the structured instances required by a cryptosystem.

11.1.1 The MQ Problem

Every multivariate scheme rests on a single bet: that systems of quadratic equations in many variables are hard to solve. To make this precise, we need to state the problem carefully—because the gap between the general problem and the instances that actually appear in cryptosystems is where every attack has lived.

Definition 11.1 (Multivariate Quadratic (MQ) Problem) Given m quadratic polynomials $p_1, \dots, p_m$ in n variables over a finite field $\mathbb{F}_q$,

$$p_k(x_1, \dots, x_n) = \sum_{1 \leq i \leq j \leq n} \alpha_{ijk} x_i x_j + \sum_{i=1}^n \beta_{ik} x_i + \gamma_k = 0, \quad k = 1, \dots, m, \quad (11.1)$$

find a solution $(x_1, \dots, x_n) \in \mathbb{F}_q^n$ satisfying all m equations simultaneously.

Theorem 11.1 (NP-Completeness) *The MQ problem over $\mathbb{F}_2$ is NP-complete, even when $m = n$.*

This result fuelled optimism that multivariate systems could serve as one-way functions. The catch is that a cryptosystem cannot use *random* quadratic systems—a trapdoor is needed to decrypt or sign. The trapdoor introduces hidden algebraic structure, and it is this structure that cryptanalysis has repeatedly exploited.

11.1.2 The Oil-Vinegar Construction

If general quadratic systems are hard but random systems have no trapdoor, how does one build a usable scheme? The most enduring answer comes from a culinary metaphor. The Oil-Vinegar scheme, introduced by Patarin in 1997, is the ancestor of most surviving multivariate signature schemes. Its central trick is to partition the variables into two classes that interact in a controlled way—just enough structure to invert, but (one hopes) not enough for an adversary to detect.

Definition 11.2 (Oil-Vinegar Map) Partition the n variables into two sets: *oil variables* $o_1, \dots, o_v$ and *vinegar variables* $v_1, \dots, v_o$, with $n = v + o$. Define a system of o quadratic equations in which no monomial $o_i o_j$ (product of two oil variables) appears:

$$p_k(\mathbf{o}, \mathbf{v}) = \sum_{i,j} \alpha_{ijk} v_i o_j + \sum_{i \leq j} \beta_{ijk} v_i v_j + \text{linear terms} = t_k, \quad k = 1, \dots, o. \quad (11.2)$$

Once the vinegar variables are fixed to random values, the system becomes *linear* in the oil variables and can be solved by Gaussian elimination.

The private key is an invertible affine change of variables S that mixes oil and vinegar, hiding the partition. The public key is the composed system $\mathcal{P} = p \circ S$, which looks like a random quadratic map.

The multivariate design tension. Every multivariate scheme faces the same dilemma: the trapdoor must introduce enough structure for efficient inversion, but this structure must remain invisible in the public key. Thirty-five years of evidence suggest that hiding algebraic structure from Gröbner basis algorithms and differential attacks is extraordinarily difficult.

11.1.3 A Pattern of Breaks

The history of multivariate cryptography is a repeating cycle: proposal, parameter tuning, break.

Table 11.1 Multivariate schemes: a timeline of proposals and breaks

Scheme	Proposed	Broken	Attack method
C*	1988	1995	Linearisation (Patarin)
HFE	1996	2003	Algebraic (Faugère–Joux)
SFLASH	2003	2007	Differential (Dubois et al.)
Rainbow	2005	2022	Rectangular MinRank (Beullens)
GeMSS	2017	2020	Algebraic (key recovery)

C* and HFE.

Matsumoto and Imai’s C* scheme (1988) used a monomial map $x \mapsto x^{1+q^i}$ over an extension field $\mathbb{F}_{q^n}$, composed with affine changes of coordinates to produce a quadratic map over the base field. Patarin (1995) observed that C* satisfies bilinear relations between plaintext and ciphertext, reducing decryption to linear algebra. His repair, the Hidden Field Equations (HFE) scheme, replaced the monomial with a low-degree polynomial over the extension field. Faugère and Joux (2003) showed that the HFE public key generates an ideal in a polynomial ring whose Gröbner basis can be computed efficiently when the hidden polynomial has low degree—precisely the regime needed for practical parameters.

SFLASH.

A signature variant of C* was submitted to NESSIE and selected for the portfolio in 2003. It fell four years later to a differential attack that recovered the secret affine maps by analysing the differential $Dp(\mathbf{x}, \mathbf{y}) = p(\mathbf{x} + \mathbf{y}) - p(\mathbf{x}) - p(\mathbf{y}) + p(\mathbf{0})$ of the public map.

Rainbow and Beullens’ attack.

Rainbow, proposed by Ding and Schmidt (2005), generalised Oil-Vinegar to a multi-layer structure with a “rainbow” of variable partitions. It was a NIST PQC Round 3 finalist, offering 66-byte signatures—by far the most compact among all candidates.

In February 2022, Ward Beullens posted a devastating attack [7]. The core insight: Rainbow’s multi-layer structure implies that certain matrices derived from the public key have unexpectedly low rank. Specifically, the *MinRank problem*—given matrices $M_0, M_1, \dots, M_m$, find a linear combination of rank at most r —can be solved efficiently when the matrix is “rectangular” (more columns than rows). Rainbow’s layered structure creates exactly this rectangular MinRank instance.

Example 11.1 (Breaking Rainbow) Beullens’ attack on the Rainbow-Ia parameter set (NIST Security Level I):

1. Extract a system of bilinear equations from the public key by computing the differential.
2. Formulate the resulting system as a rectangular MinRank instance with target rank $r = v_1$ (the number of vinegar variables in the first layer).
3. Solve the MinRank instance using Kipnis–Shamir modelling combined with Gröbner basis computation.
4. The recovered rank-deficient matrix reveals the layer structure, enabling full key recovery.

The attack runs in 2^{53} operations for Rainbow-Ia—well below the claimed 128-bit security. All three parameter sets (Ia, IIc, Vc) fell.

11.1.4 UOV: The Survivor

Unbalanced Oil-Vinegar (UOV), where the number of vinegar variables greatly exceeds the number of oil variables ($v \gg o$), remains unbroken. The key observation is that the single-layer structure of UOV does not create the rectangular MinRank instances that Beullens exploited. UOV was submitted to NIST’s additional signature call and is under active evaluation.

The cost is large public keys (tens to hundreds of kilobytes), which has motivated compressed variants such as MAYO and VOX. Whether these compressed variants introduce new exploitable structure remains an open question.

The multivariate story, then, is one of persistent structural vulnerability—trapdoors that turn out to be visible from the right angle. But multivariate schemes are not the only post-quantum family to suffer a dramatic collapse. The isogeny-based approach, built on entirely different mathematics, provides an even more striking cautionary tale.

11.2 Isogeny-Based Cryptography

Isogeny-based cryptography was the newest and most mathematically sophisticated approach to post-quantum key exchange. Its collapse in 2022 is the most instructive failure in modern cryptographic history.

11.2.1 Elliptic Curves and Isogenies

We assume familiarity with elliptic curves at the level of an introductory number theory course. The essential objects are the curves themselves and the maps between them. Where classical elliptic-curve cryptography exploits the difficulty of discrete logarithms *within* a single curve, isogeny-based cryptography exploits the difficulty of finding maps *between* curves. To make this precise, we need two definitions.

Definition 11.3 (Elliptic Curve) An *elliptic curve* over a field K is a smooth projective curve of genus 1 with a distinguished rational point. In short Weierstrass form over a field of characteristic $\neq 2, 3$:

$$E : y^2 = x^3 + ax + b, \quad a, b \in K, \quad 4a^3 + 27b^2 \neq 0. \quad (11.3)$$

The set $E(K)$ of K -rational points forms an abelian group under the chord-and-tangent law.

The group structure is what makes elliptic curves useful for classical cryptography, but for isogeny-based schemes the important objects are the *homomorphisms* between curves.

Definition 11.4 (Isogeny) An *isogeny* $\phi: E_1 \rightarrow E_2$ is a non-constant morphism of elliptic curves that maps the identity to the identity. Equivalently, it is a surjective group homomorphism with finite kernel. Its *degree* is $\deg \phi = |\ker \phi|$ (for separable isogenies).

Every finite subgroup $G \leq E$ determines an isogeny $\phi_G: E \rightarrow E/G$ by Vélú's formulae (1971), which give explicit rational functions for the map. The degree of ϕ_G equals $|G|$. Conversely, every separable isogeny from E arises from a finite subgroup of E .

Example 11.2 (Isogenies in Practice) Let $E: y^2 = x^3 + x$ over $\mathbb{F}_p$ and let $P \in E(\mathbb{F}_p)$ have order ℓ . Then $G = \langle P \rangle$ defines an ℓ -isogeny $\phi_G: E \rightarrow E' = E/G$, computed via Vélú's formulae. The curve E' is determined (up to isomorphism) by G alone; different generators of the same subgroup yield the same target curve.

11.2.2 Supersingular Isogeny Graphs

Not all elliptic curves are equally useful for cryptography. Among curves over finite fields, a special class—the supersingular curves—has properties that make it uniquely suited to building hard problems from isogenies. Supersingular curves are rare (roughly $p/12$ of them over $\mathbb{F}_p$), and the network of isogenies connecting them has remarkable combinatorial properties.

Definition 11.5 (Supersingular Curve) An elliptic curve E over $\overline{\mathbb{F}}_p$ is *supersingular* if $E[p] = \{O\}$ (trivial p -torsion over the algebraic closure). Equivalently, $\#E(\mathbb{F}_p) \equiv 1 \pmod{p}$. Over $\overline{\mathbb{F}}_p$, there are approximately $p/12$ supersingular j -invariants.

The supersingular curves and the isogenies between them form a graph, and the structure of this graph is what makes the cryptography possible.

Definition 11.6 (Supersingular Isogeny Graph) Fix a prime $\ell \neq p$. The ℓ -isogeny graph $\mathcal{G}_\ell(p)$ is the graph whose vertices are supersingular j -invariants over $\overline{\mathbb{F}}_p$ and whose edges are ℓ -isogenies (up to isomorphism). Each vertex has degree $\ell + 1$.

These graphs are *Ramanujan graphs*: they are optimal expanders, meaning that random walks mix rapidly and the graph has no exploitable bottlenecks. A walk of length e in $\mathcal{G}_\ell(p)$ corresponds to a composition of e isogenies of degree ℓ , yielding a single isogeny of degree ℓ^e .

The cryptographic hardness assumption is: given two supersingular curves E_0 and E_1 connected by an isogeny of smooth degree ℓ^e , find the isogeny. The best classical algorithm for this “path-finding” problem has exponential complexity $\tilde{O}(\sqrt[p]{p})$, and the best quantum algorithm (using Tani’s claw-finding) achieves $\tilde{O}(\sqrt[p]{p})$.

11.2.3 SIDH/SIKE: Construction and Collapse

Supersingular Isogeny Diffie–Hellman (SIDH), proposed by Jao and De Feo in 2011 [31], translates the path-finding problem into a key exchange protocol. The construction is elegant, but it demands a critical compromise that classical Diffie–Hellman never required.

Definition 11.7 (SIDH Key Exchange) Fix a supersingular curve E_0 over $\mathbb{F}_{p^2}$ where $p = 2^a \cdot 3^b - 1$. Fix generators $\{P_A, Q_A\}$ of $E_0[2^a]$ and $\{P_B, Q_B\}$ of $E_0[3^b]$.

1. **Alice:** Chooses secret integers (α_1, α_2) with $\gcd(\alpha_1, 2^a) = 1$. Computes kernel $\langle P_A + \alpha_2 Q_A \rangle$, yielding a 2^a -isogeny $\phi_A: E_0 \rightarrow E_A$. Sends $(E_A, \phi_A(P_B), \phi_A(Q_B))$ to Bob.
2. **Bob:** Chooses secret integers (β_1, β_2) similarly. Computes $\phi_B: E_0 \rightarrow E_B$ with kernel $\langle P_B + \beta_2 Q_B \rangle$. Sends $(E_B, \phi_B(P_A), \phi_B(Q_A))$ to Alice.
3. **Shared secret:** Alice computes $\phi'_A: E_B \rightarrow E_{AB}$ using kernel $\langle \phi_B(P_A) + \alpha_2 \phi_B(Q_A) \rangle$. Bob computes $\phi'_B: E_A \rightarrow E_{BA}$ analogously. By commutativity, $j(E_{AB}) = j(E_{BA})$.

The auxiliary torsion points $\phi_A(P_B), \phi_A(Q_B)$ (and their counterparts from Bob) are essential: without them, Bob cannot compute his isogeny from E_A , because he does not know Alice’s secret and therefore cannot determine the correct kernel on E_A .

SIKE (Supersingular Isogeny Key Encapsulation) wrapped SIDH in a KEM transform and was submitted to the NIST PQC competition. It offered the smallest key sizes among all candidates—just 335 bytes for a shared secret at the 128-bit security level—and reached Round 4 as an alternate candidate.

The Castryck–Decru attack.

On 30 July 2022, Wouter Castryck and Thomas Decru posted a preprint [12] that broke SIDH in polynomial time. Within days, Maino and Martindale gave an independent attack, and Robert provided a further simplification.

The attack exploits precisely the auxiliary torsion point information that the protocol requires. The key idea:

1. The images $\phi_A(P_B), \phi_A(Q_B)$ reveal how Alice’s secret isogeny acts on the 3^b -torsion of E_0 .

2. By the theory of abelian surfaces, one can “lift” the problem from elliptic curves to the Jacobian of a genus-2 curve, where additional endomorphism structure becomes available.
3. On this abelian surface, the Richelot isogeny (the genus-2 analogue of a 2-isogeny) can be iterated, and each step is determined by the torsion point data.
4. After a steps, Alice’s secret curve E_A is recovered.

Why auxiliary points are fatal. In standard Diffie–Hellman over a group G , Alice publishes g^a but reveals *nothing else* about her secret a . In SIDH, Alice publishes E_A and the images $\phi_A(P_B), \phi_A(Q_B)$ —effectively revealing how her secret isogeny acts on a large subgroup of E_0 . This additional information is not merely helpful; it is sufficient to reconstruct the secret isogeny entirely. The attack uses no quantum resources. It runs in minutes on a standard laptop.

Example 11.3 (Timeline of SIKE’s Collapse)

- 2011: SIDH proposed by Jao and De Feo [31].
- 2017: SIKE submitted to NIST PQC competition.
- 2020: SIKE advances to Round 3 as alternate candidate.
- 2022 (July 5): SIKE advances to Round 4.
- 2022 (July 30): Castryck–Decru attack posted.
- 2022 (August 5): All SIKE parameter sets broken; attack verified independently.
- 2022 (August): Microsoft removes SIKE from cryptographic libraries.
- Result: 11 years of research invalidated by a classical attack using 1990s mathematics.

The SIKE collapse raises an immediate question: is the entire isogeny-based approach doomed, or was the vulnerability specific to SIDH’s design? The answer turns on a single architectural difference.

11.3 Group Actions and CSIDH

The wreckage of SIDH left one isogeny-based construction standing. Commutative Supersingular Isogeny Diffie–Hellman (CSIDH, pronounced “seaside”), proposed by Castryck, Lange, Martindale, Panny, and Renes in 2018 [13], was designed around a fundamentally different mathematical object—a commutative group action rather than a path-finding problem in an isogeny graph. This difference is not cosmetic. It eliminates the auxiliary torsion point information that destroyed SIKE, but it introduces its own challenges.

11.3.1 Class Group Actions

The core idea of CSIDH is to replace the non-commutative isogeny walk of SIDH with the action of an abelian group on a set of elliptic curves. In classical Diffie–Hellman, the protocol works because the group of integers modulo a prime is commutative: $g^{ab} = g^{ba}$. CSIDH achieves the same commutativity, but in a setting where discrete logarithms are hard even for quantum computers—at least, as far as we currently know. The group that plays this role is the ideal class group of an imaginary quadratic order.

Definition 11.8 (Group Action on Elliptic Curves) Let p be a large prime with $p \equiv 3 \pmod{4}$, and let $\mathcal{E}_p$ denote the set of supersingular elliptic curves over $\mathbb{F}_p$ (defined over $\mathbb{F}_p$ itself, not $\mathbb{F}_{p^2}$). The ideal class group $\text{cl}(\mathbb{Z}[\sqrt{-p}])$ acts on $\mathcal{E}_p$: for an ideal class $[\mathfrak{a}]$ and a curve $E \in \mathcal{E}_p$,

$$[\mathfrak{a}] \star E = E/E[\mathfrak{a}], \quad (11.4)$$

where $E[\mathfrak{a}] = \{P \in E(\overline{\mathbb{F}}_p) : \alpha(P) = O \text{ for all } \alpha \in \mathfrak{a}\}$. This action is free, transitive, and—crucially—*commutative*, since $\text{cl}(\mathbb{Z}[\sqrt{-p}])$ is abelian.

The commutativity means that a Diffie–Hellman-style protocol works directly, and—this is the crucial point—without any auxiliary information.

Definition 11.9 (CSIDH Key Exchange) Fix a public supersingular curve E_0 over $\mathbb{F}_p$.

1. **Alice:** Chooses a secret class group element $[\mathfrak{a}]$. Computes $E_A = [\mathfrak{a}] \star E_0$. Sends E_A .
2. **Bob:** Chooses a secret $[\mathfrak{b}]$. Computes $E_B = [\mathfrak{b}] \star E_0$. Sends E_B .
3. **Shared secret:** Alice computes $[\mathfrak{a}] \star E_B = [\mathfrak{a}] \star [\mathfrak{b}] \star E_0$. Bob computes $[\mathfrak{b}] \star E_A = [\mathfrak{b}] \star [\mathfrak{a}] \star E_0$. By commutativity, both obtain the same curve.

11.3.2 Why CSIDH Survives the SIKE Attack

The Castryck–Decru attack on SIDH exploits auxiliary torsion point information. CSIDH transmits *only the target curve* E_A —no images of torsion points, no auxiliary data. There is nothing for the attack to exploit.

More precisely:

- In SIDH, Alice sends $(E_A, \phi_A(P_B), \phi_A(Q_B))$ —the curve plus two points revealing the action of ϕ_A on the 3^b -torsion.
- In CSIDH, Alice sends only E_A . Bob can compute $[\mathfrak{b}] \star E_A$ because the group action is defined intrinsically on curves over $\mathbb{F}_p$, without needing to know Alice’s secret or its action on any torsion subgroup.

The underlying hard problem for CSIDH is the *vectorisation problem*: given E_0 and $E_A = [\mathfrak{a}] \star E_0$, find $[\mathfrak{a}]$. This is a discrete-logarithm-like problem in the class group, and the best known quantum attack uses Kuperberg’s algorithm for the hidden shift problem, achieving subexponential complexity $2^{\tilde{O}(\sqrt{p})}$.

11.3.3 Challenges

Despite surviving the SIKE catastrophe, CSIDH faces significant practical hurdles that have so far prevented it from reaching standardisation.

The first is computational: the class group $\text{cl}(\mathbb{Z}[\sqrt{-p}])$ must be large enough for security, but its structure is difficult to compute for large p , complicating parameter selection. The second is implementational. The group action is computed by walking through small-degree isogenies, and the number of steps depends on the secret key, creating timing side channels. Constant-time implementations (CTIDH) exist but are significantly slower.

The third challenge is the most fundamental: quantum security estimates remain unsettled. Kuperberg’s algorithm places CSIDH in an uncomfortable middle ground—not clearly broken by quantum computers, but not clearly safe either. Achieving NIST Level I security requires primes of 4096 bits or more, yielding public keys of 64 bytes but slow computation (~ 100 ms). Finally, the algebraic structure of a commutative group action, while elegant, is less flexible than lattices for building advanced primitives. The action enables key exchange and, with extensions, signatures via CSI-FiSh, but constructing fully homomorphic encryption or attribute-based schemes from group actions remains an open problem.

CSIDH remains an active research area. Its theoretical elegance—a true group-action-based analogue of classical Diffie–Hellman—ensures continued interest, even if practical deployment is uncertain. Whether isogeny-based cryptography will eventually produce a standardised scheme, or whether the family will remain a theoretical laboratory, depends on resolving these challenges. In the meantime, the contrasting fates of SIKE and CSIDH offer sharp lessons about what makes cryptographic assumptions trustworthy—the subject of our final section.

11.4 Lessons Learned

The failures of multivariate and isogeny-based schemes, combined with the survival of lattice-based and code-based approaches, reveal patterns that should guide the evaluation of any future cryptographic proposal.

11.4.1 What Makes an Assumption Trustworthy?

Not all NP-hard problems make good cryptographic foundations. The collapses catalogued in this chapter suggest four criteria that distinguish assumptions which have survived from those which have not.

The first, and arguably the most important, is the existence of *worst-case to average-case reductions*. Lattice problems enjoy provable connections between worst-case and average-case hardness (Chap. 6): breaking a random instance is at least as hard as

Table 11.2 Structure and its consequences for cryptographic security

Scheme family	Hidden structure	Exploited by	Outcome
C*/HFE	Extension field map	Linearisation, Gröbner	Broken
Rainbow	Layered partitions	Rectangular MinRank	Broken
SIDH/SIKE	Torsion point images	Richelot isogenies	Broken
Lattice (LWE)	Ring/module structure	No known exploit	Survives
Code-based	Goppa code structure	No known exploit	Survives

solving the worst case. Neither the MQ problem nor the supersingular isogeny problem has such a reduction. The presence of a worst-case/average-case connection does not guarantee security—it is a necessary condition, not a sufficient one—but its absence should be treated as a warning.

The second criterion is time under sustained cryptanalytic pressure. McEliece’s cryptosystem has withstood 45 years of concerted attack. Lattice-based schemes have survived over 25 years of scrutiny from a large and active community. SIKE lasted 11 years before a classical attack, using techniques available since the 1990s, dismantled it entirely. There is no shortcut to cryptographic confidence; years of failed attacks are themselves evidence, because each failure constrains the space of possible vulnerabilities.

Third, the *size and diversity of the research community* matters more than is commonly acknowledged. More researchers examining a scheme means more diverse attack strategies, more independent verification of security claims, and a higher probability that structural weaknesses are detected early. The isogeny community, while technically brilliant, was small—perhaps a few dozen active researchers worldwide. The lattice community is an order of magnitude larger, encompassing algebraists, complexity theorists, and practitioners. The probability that a fundamental vulnerability escapes notice is simply lower when more eyes are looking.

The fourth criterion is structural simplicity of the underlying hard problem. The problems at the heart of lattice cryptography (find short vectors in a lattice) and code-based cryptography (decode a random linear code) are simple to state and have been studied as mathematical objects for decades, long before their cryptographic relevance was recognised. The problems underlying multivariate and isogeny schemes involve layers of algebraic structure—hidden field maps, torsion point actions, class group computations—that provide more surfaces for attack. Complexity is not inherently bad, but each additional structural component is a potential entry point for cryptanalysis.

11.4.2 Structured vs. Unstructured Problems

A recurring theme: the structure that enables efficient cryptosystems is the same structure that enables efficient attacks.

The surviving schemes share a property: their structure (ring structure in Module-LWE, Goppa structure in Classic McEliece) has been intensely studied, and the best

known attacks do not exploit it more efficiently than generic attacks on the unstructured variants. This does not mean they are immune, but it provides a degree of confidence that the other schemes lacked.

11.4.3 The Danger of Auxiliary Information

The SIKE attack is a case study in a principle that should be tattooed on every protocol designer's forearm: *every piece of auxiliary information published by a protocol is a potential attack surface*.

In classical Diffie–Hellman, Alice publishes g^a and nothing else. In SIDH, Alice publishes E_A and two points $\phi_A(P_B), \phi_A(Q_B)$ that reveal the action of her secret isogeny on a large torsion subgroup. The protocol requires this information—without it, Bob cannot compute the shared secret—but the information is also sufficient for an attacker to recover the secret.

This is not hindsight bias. Galbraith, Petit, Shani, and Ti raised concerns about torsion point information in 2016. Petit showed in 2017 that certain oversharing variants of SIDH could be attacked. The community was aware of the risk; the surprise was that the “minimal” information required by the protocol was already too much.

Design principle. A key exchange protocol should reveal the minimum information necessary for the counterparty to compute the shared secret. If the protocol inherently requires publishing information about the secret's action on a structured subobject, treat this as a fundamental vulnerability, not a parameter to tune.

11.4.4 Diversification and Crypto-Agility

NIST's decision to standardise schemes from multiple mathematical families—lattices (ML-KEM, ML-DSA), hash-based signatures (SLH-DSA), and codes (under evaluation)—reflects a hard-won lesson: no single family should be trusted unconditionally.

The concept of crypto-agility captures this insight at the systems level. A system that can swap out its cryptographic algorithms without a full redesign is resilient to precisely the kind of sudden collapse that SIKE suffered. Without this flexibility, a broken algorithm means a broken system.

Definition 11.10 (Crypto-Agility) A system exhibits *crypto-agility* if its cryptographic algorithms can be replaced without redesigning the overall architecture. This requires:

- Algorithm negotiation in protocols (as in TLS cipher suites).
- Abstraction layers separating cryptographic primitives from application logic.
- Key and certificate formats that accommodate multiple algorithm families.
- Operational procedures for rapid algorithm rotation.

The SIKE collapse vindicated this approach. Had SIKE been the sole post-quantum standard, the 2022 attack would have been a civilisation-scale infrastructure crisis. Because NIST had diversified across families, SIKE’s failure was a research setback, not an operational catastrophe.

The practical implication for system designers: architect systems so that the cryptographic layer is a replaceable module, not a load-bearing wall. The history of cryptography guarantees that algorithms will break; the only question is when. The migration strategies and protocol-level mechanisms for achieving this agility in practice are the subject of Chapter 14.

Problems

11.1 Consider a system of 2 quadratic equations in 3 variables over $\mathbb{F}_2$:

$$\begin{aligned}x_1x_2 + x_2x_3 + x_1 &= 0, \\x_1x_3 + x_2 + x_3 &= 0.\end{aligned}$$

- (a) Enumerate all solutions by brute force (there are only $2^3 = 8$ candidates).
- (b) How many candidates would there be for a system in n variables over $\mathbb{F}_2$? At what value of n does brute-force search exceed 2^{128} operations?
- (c) Explain why the NP-completeness of the MQ problem does not directly imply that any specific multivariate cryptosystem is secure.

11.2 In the Oil-Vinegar construction with o oil variables and v vinegar variables:

- (a) Explain why fixing the vinegar variables reduces the system to linear equations in the oil variables.
- (b) Kipnis and Shamir (1999) showed that the original balanced Oil-Vinegar scheme ($v = o$) can be broken. Why does making the scheme “unbalanced” ($v \gg o$) defeat their attack? (Hint: consider the rank of the matrix that the attack needs to find.)
- (c) Rainbow uses multiple layers of Oil-Vinegar. Explain intuitively why the layered structure creates low-rank matrices that Beullens’ MinRank attack can exploit.

11.3(a) In the SIDH key exchange, identify the auxiliary torsion point information that Alice sends to Bob. Why is this information necessary for the protocol to work?

- (b) Explain, at a high level, how the Castryck–Decru attack uses this auxiliary information to recover Alice’s secret isogeny.
- (c) CSIDH does not transmit any auxiliary torsion point information. Explain how Bob can still compute the shared secret without this information.
- (d) Does CSIDH’s immunity to the Castryck–Decru attack guarantee its long-term security? What other threats exist?

11.4 Propose a set of at least four criteria for evaluating the trustworthiness of a post-quantum cryptographic assumption. Apply your criteria to: (a) Module-LWE, (b) the syndrome decoding problem, (c) the MQ problem, (d) the supersingular isogeny problem (post-SIKE). Rank them from most to least trustworthy and justify your ranking.

11.5 Design exercise.

You are architecting a TLS-based communication system that must remain secure for 30 years.

- (a) Explain why choosing a single post-quantum algorithm family is risky, citing specific historical examples from this chapter.
- (b) Design a layered architecture that achieves crypto-agility. Specify what is abstracted, what negotiation is needed, and how algorithm rotation would work operationally.
- (c) Discuss the performance and complexity costs of your design. Is crypto-agility free?

Part III
Practice and Future

Chapter 12

Implementation and Side-Channel Resistance

Abstract Mathematical security is necessary but not sufficient: more cryptographic deployments are compromised through implementation flaws than through mathematical breakthroughs. This chapter addresses the engineering challenges of post-quantum cryptography, from constant-time programming and side-channel resistance to random number generation and the bandwidth implications of larger keys and ciphertexts.

12.1 The Constant-Time Imperative

A cryptographic algorithm may be provably secure in the mathematical model, yet a careless implementation can leak the entire secret key through execution time alone. Timing side channels arise whenever the duration of a computation depends on secret data: a branch taken or not taken, a cache line hit or miss, a variable-latency instruction—any of these can reveal bits of the key to an attacker who measures wall-clock time over many executions [50].

12.1.1 Why Timing Variations Leak Secrets

Consider an attacker who can query a decapsulation oracle and measure response time with nanosecond precision. If the implementation contains a single conditional branch whose outcome depends on a secret coefficient, the attacker observes two clusters of timings. By correlating timing with chosen ciphertexts, the attacker partitions the secret space and recovers coefficients one by one. Kocher’s seminal work showed that even *average* timing differences of a few microseconds suffice when the attacker collects millions of measurements.

Example 12.1 (A Timing Disaster: Barrett Reduction) The Barrett reduction computes $a \bmod q$ using multiplication by a precomputed factor instead of division. A naive implementation finishes with a conditional correction:

```

1 // INSECURE: timing depends on secret value
2 int16_t barrett_reduce(int32_t a) {
3     int32_t t = (a * BARRETT_FACTOR) >> BARRETT_SHIFT;
4     t = a - t * KYBER_Q;
5     if (t >= KYBER_Q) { // SECRET-DEPENDENT BRANCH
6         t -= KYBER_Q;
7     }
8     return (int16_t)t;
9 }

```

The branch on line 5 executes conditionally depending on t , which correlates with the secret polynomial coefficient being reduced. Over millions of NTT operations, the timing difference between the taken and not-taken paths leaks enough information to reconstruct the secret key.

The constant-time fix replaces the branch with an arithmetic mask:

```

1 // SECURE: constant-time Barrett reduction
2 int16_t barrett_reduce_ct(int32_t a) {
3     int32_t t = (a * BARRETT_FACTOR) >> BARRETT_SHIFT;
4     t = a - t * KYBER_Q;
5     t -= KYBER_Q & ~(t - KYBER_Q) >> 31); // branchless
6     return (int16_t)t;
7 }

```

The expression $(t - \text{KYBER_Q}) \gg 31$ produces an all-ones mask when $t < q$ and an all-zeros mask when $t \geq q$. The subtraction is performed unconditionally; the mask selects whether the result is kept.

12.1.2 Constant-Time Programming Patterns

The Barrett example illustrates a single instance of a pervasive problem. Every arithmetic routine, every comparison, every loop that touches secret data must be scrutinised for variable-time behaviour. The discipline reduces to four rules:

The four rules of constant-time programming. Every implementation that processes secret data must obey:

1. **No secret-dependent branches.** All conditional logic on secret values must be replaced by arithmetic or bitwise select operations.
2. **No secret-dependent memory access patterns.** Array indices derived from secret data cause cache-timing leaks; use constant-index sweeps with conditional moves.
3. **No variable-latency instructions on secret data.** On many microarchitectures, integer division and floating-point operations have data-dependent latency.

Table 12.1 Constant-time verification tools

Tool	Approach	Scope
ctgrind	Valgrind-based taint tracking	Detects branches and memory accesses on tainted (secret) data
timecop	Valgrind memcheck extension	Flags uninitialised-value-style errors for secret-dependent control flow
dudect	Statistical black-box testing	Measures timing distributions; no source access needed
ct-verif	LLVM-based formal verification	Proves constant-time at the IR level

4. **No early termination.** Comparison routines must always examine every byte; loops must execute a fixed number of iterations.

These rules apply to all three NIST standards. ML-KEM [50] uses the Fujisaki–Okamoto transform, which requires a constant-time comparison of re-encrypted ciphertext. ML-DSA [49] rejection-samples signatures in a loop that must always run a fixed number of iterations. SLH-DSA [51] uses hash-based tree traversal with secret-dependent leaf selection that must be protected.

12.1.3 Verification Tools

Manual inspection is insufficient for verifying constant-time behaviour. The following tools automate the analysis:

For production code, a combination of static analysis (`ct-verif`) and dynamic testing (`dudect`) provides the strongest assurance. Constant-time discipline eliminates timing as a side channel, but timing is not the only channel an attacker can exploit. Power consumption, electromagnetic emanation, and cache behaviour all leak information—and lattice-based schemes introduce attack surfaces that have no classical analogue.

12.2 Side Channels in Lattice Cryptography

The NTT, polynomial sampling, and the Fujisaki–Okamoto decapsulation check each present distinct attack opportunities that deserve individual attention.

12.2.1 Timing Attacks on the NTT

The NTT is the performance-critical kernel of ML-KEM and ML-DSA: it reduces polynomial multiplication from $O(n^2)$ to $O(n \log n)$. Each butterfly operation involves a modular multiplication followed by modular reduction. If the reduction uses a conditional subtraction (as in example 12.1), the attacker can correlate timing with individual coefficients of the secret polynomial.

A constant-time butterfly avoids all branches:

```

1 // Constant-time NTT butterfly with Montgomery reduction
2 void ct_butterfly(int16_t *a, int16_t *b, int16_t zeta) {
3     int32_t t = (int32_t)montgomery_reduce((int32_t)*b * zeta);
4     *b = *a - t;
5     *a = *a + t;
6     // Branchless conditional reduction
7     *a -= KYBER_Q & -((*a >= KYBER_Q));
8     *b += KYBER_Q & -((*b < 0));
9 }

```

12.2.2 Power and Electromagnetic Analysis

Timing attacks require only network access, but an attacker with physical proximity can extract far more. Simple Power Analysis (SPA) and Differential Power Analysis (DPA) exploit the fact that different instructions and data values draw different amounts of current. A single power trace of an NTT execution can reveal the structure of the butterfly network, and by extension the secret polynomial.

SPA on lattice operations. The NTT butterfly has a characteristic power signature: the multiply-and-reduce step draws measurably more current than addition. When the reduction step is conditional, the power trace directly reveals whether the subtraction occurred. For ML-DSA, the rejection sampling loop produces a variable number of NTT calls per signature; the number of iterations visible in the power trace leaks information about the secret key.

DPA on polynomial coefficients. By collecting thousands of power traces during decapsulation and correlating with hypothesised coefficient values, an attacker mounts a DPA attack that recovers the secret polynomial coefficient by coefficient. The attack succeeds because each butterfly operation processes one coefficient against one twiddle factor, creating a localised power-secret dependency.

Electromagnetic emanation attacks. EM probes placed near the processor die can achieve spatial resolution sufficient to isolate individual pipeline stages. This is particularly threatening for embedded implementations (smart cards, IoT devices) where the attacker may have physical access.

12.2.3 Cache-Timing Attacks

Power and EM attacks require physical proximity, but cache-timing attacks can be mounted by any process sharing the same hardware—including a co-tenant in a cloud virtual machine. Table-based implementations of modular arithmetic or hash functions are vulnerable to cache-timing attacks. When a lookup table exceeds one cache line (typically 64 bytes), the access pattern reveals which entries were used. In lattice cryptography, this affects:

- **NTT twiddle factor tables.** If the table of roots of unity is large enough to span multiple cache lines, the access pattern during the NTT reveals structural information about the transform.
- **Discrete Gaussian sampling via CDT.** The cumulative distribution table (CDT) method performs a binary search; the sequence of cache lines accessed reveals the sampled value.
- **SHAKE/SHA-3 round functions.** The Keccak permutation is naturally constant-time, but software implementations that use lookup tables for the χ step may leak.

The countermeasure is to ensure all table accesses sweep the entire table in constant time, selecting the desired entry via conditional moves.

12.2.4 Masking Countermeasures for the NTT

Constant-time code defeats timing and cache attacks, but it cannot hide power consumption or electromagnetic emanation—the physics of computation guarantees that different data values draw different currents. Masking is the standard countermeasure against power and EM analysis. The idea is to split every secret value x into $d + 1$ shares $x_0, \dots, x_d$ such that $x = x_0 + x_1 + \dots + x_d \bmod q$, where each share is individually uniformly random. An attacker who observes fewer than $d + 1$ shares learns nothing about x .

Definition 12.1 (Boolean and Arithmetic Masking) Let $x \in \mathbb{Z}_q$ be a secret value.

- **Boolean masking:** choose $r \xleftarrow{\$} \{0, 1\}^k$ and store $(x \oplus r, r)$.
- **Arithmetic masking:** choose $r \xleftarrow{\$} \mathbb{Z}_q$ and store $(x - r \bmod q, r)$.

A d -th order masking scheme uses $d + 1$ shares and is secure against attackers who can probe at most d intermediate values simultaneously.

Applying masking to the NTT is non-trivial because the butterfly mixes shares through both addition and multiplication. The state of the art proceeds as follows:

1. Mask all input coefficients with arithmetic shares before entering the NTT.
2. Perform the NTT independently on each share (the NTT is linear over $\mathbb{Z}_q$, so $\text{NTT}(x_0 + x_1) = \text{NTT}(x_0) + \text{NTT}(x_1)$).

3. For pointwise multiplication (which is non-linear), use a secure multiplication gadget that refreshes shares after every product.
4. Recombine shares only at the final output, after all secret-dependent operations are complete.

The overhead of first-order masking ($d = 1$) is roughly 2–3× in execution time and 2× in memory. Higher-order masking scales as $O(d^2)$ due to the secure multiplication gadget.

Remark 12.1 (Masking vs. Hiding) Masking (splitting secrets into shares) and hiding (adding random delays or shuffling operation order) are complementary. Production implementations targeting smart-card certification (Common Criteria EAL4+ or higher) typically combine both.

All the attacks discussed so far—timing, power, EM, cache—are *passive*: the attacker observes but does not interfere. A stronger adversary can actively corrupt a computation and exploit the faulty output.

12.3 Fault Injection Attacks

Fault injection encompasses a family of techniques—voltage glitches, clock manipulation, laser pulses, electromagnetic pulses—that induce physical perturbations and exploit the resulting errors to recover secrets. The attacker’s toolkit ranges from inexpensive glitching hardware to precision laser stations costing hundreds of thousands of euros.

12.3.1 Fault Models

The standard fault models, ordered by attacker strength, are:

1. **Instruction skip.** A single instruction is replaced by a NOP. This is achievable with voltage or clock glitches and requires only coarse temporal control.
2. **Random byte/word fault.** A storage element (register or SRAM cell) is set to a random value. Laser fault injection achieves this with spatial precision of a few micrometres.
3. **Bit flip.** A single bit in a register or memory cell is toggled. Focused ion beam or precise laser attacks can target individual bits.
4. **Stuck-at fault.** A bit is forced to 0 or 1 permanently. This models persistent hardware damage.

12.3.2 Attacks on ML-DSA (Dilithium)

ML-DSA [49] uses rejection sampling: after computing a candidate signature $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, the signer checks $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$. If the check fails, the signer restarts with fresh randomness.

Skipping the rejection check. If an attacker can induce an instruction skip that bypasses the norm check, the faulty signature is output without rejection. This signature leaks information about $\mathbf{s}_1$: without the bound on $\|\mathbf{z}\|_\infty$, the distribution of $\mathbf{z}$ is no longer independent of the secret. Collecting a few hundred faulty signatures enables full key recovery via lattice reduction.

Faulting the hash computation. If the challenge hash $c = H(\mu\|\mathbf{w}_1)$ is corrupted, the resulting signature $(c', \mathbf{z})$ satisfies $\mathbf{z} = \mathbf{y} + c'\mathbf{s}_1$ for a known c' . Two signatures with the same $\mathbf{y}$ but different challenges yield $\mathbf{z}_1 - \mathbf{z}_2 = (c'_1 - c'_2)\mathbf{s}_1$, recovering $\mathbf{s}_1$ directly.

12.3.3 Attacks on ML-KEM (Kyber)

ML-KEM [50] uses the Fujisaki–Okamoto (FO) transform, which re-encrypts the decrypted message and compares the result to the received ciphertext. If they match, the shared secret is derived from the message; otherwise, an implicit rejection value is returned.

Faulting the FO comparison. If the attacker skips the re-encryption check, the oracle always returns the “real” shared secret, converting the CCA-secure scheme into a CPA-only scheme. The attacker then mounts a chosen-ciphertext attack against the underlying CPA encryption, recovering the secret key.

Faulting decryption coefficients. A random fault during inverse-NTT corrupts a single coefficient of the decrypted polynomial. By submitting the same ciphertext multiple times and comparing correct and faulted outputs, the attacker isolates individual coefficients of the secret key.

12.3.4 Countermeasures

The most effective strategy combines algorithmic redundancy (recompute and compare) with randomised fault detection (infection). An attacker who cannot distinguish a detected fault from a natural rejection gains no information.

Remark 12.2 (The Physics of Fault Injection) Laser fault injection uses focused infrared pulses ($\lambda \approx 1064 \text{ nm}$) to generate photo-currents in SRAM cells, flipping bits with spatial resolution of $\sim 1 \mu\text{m}$. Electromagnetic fault injection (EMFI) uses a coil probe to induce transient currents, offering lower precision but no requirement for decapsulation of the chip package. Both techniques are available as commercial test equipment for security evaluation laboratories.

Table 12.2 Fault injection countermeasures and their overhead

Countermeasure	Mechanism	Overhead
Redundant computation	Execute critical operations twice and compare results	$\sim 2\times$ time
Algorithm-level check-sums	Verify $\mathbf{As} + \mathbf{e} = \mathbf{t}$ after key generation	Low ($< 5\%$)
Instruction flow monitoring	Hardware watchdog or software control flow integrity	10–20%
Infection countermeasure	Randomise output on detected fault (no error signal)	Negligible

Side channels and fault attacks exploit the physical computation; a different class of vulnerability targets the inputs to that computation. If the random numbers feeding key generation or signing are weak, no amount of constant-time discipline or masking can save the scheme.

12.4 Random Number Generation

Lattice-based cryptography is extraordinarily hungry for randomness. ML-KEM key generation requires sampling an entire matrix $\mathbf{A} \in \mathbb{Z}_q^{k \times k}$ of polynomials plus secret and error vectors—thousands of random coefficients per operation. Poor randomness does not merely weaken security; it can destroy it completely.

12.4.1 DRBG Requirements

NIST SP 800-90A specifies three approved Deterministic Random Bit Generators (DRBGs): CTR_DRBG (based on AES), Hash_DRBG (based on SHA-2), and HMAC_DRBG. All three NIST post-quantum standards use SHAKE-128 or SHAKE-256 (instances of SHA-3) to expand a seed into the required random bytes, effectively using the seed-expansion paradigm internally.

The critical requirement is the quality of the initial seed. A DRBG seeded with 256 bits of true entropy provides 2^{256} security regardless of how many bytes are subsequently generated. A DRBG seeded with 15 bits of entropy (as in the 2008 Debian OpenSSL disaster) reduces the entire key space to 2^{15} , and the lattice structure makes key recovery even easier than brute force: the error distribution collapses, and simple linear algebra suffices.

12.4.2 Hardware and Quantum Random Number Generation

Hardware RNG (HRNG). Modern processors include hardware entropy sources: Intel’s RDRAND/RDSEED (thermal noise in a metastable circuit), ARM’s TRNG (ring oscillator jitter). These provide the seed entropy that DRBGs expand. The Linux kernel’s `getrandom()` system call blocks until sufficient entropy is available, making it the recommended interface.

Quantum RNG (QRNG). Quantum random number generators exploit the fundamental indeterminacy of quantum measurements—typically photon arrival times, vacuum fluctuations, or beam splitter outcomes. Unlike classical entropy sources, whose unpredictability rests on complexity assumptions, QRNG unpredictability is guaranteed by the laws of physics. For post-quantum cryptography, there is a pleasing conceptual closure: the same quantum mechanics that threatens classical cryptography provides provably random seeds for its replacement.

12.4.3 Deterministic Key Generation

All three NIST standards support deterministic key generation from a single 32-byte seed:

```

1 // ML-KEM: deterministic key generation (FIPS 203, Algorithm 16)
2 void ml_kem_keygen(uint8_t *pk, uint8_t *sk, const uint8_t seed
   [32]) {
3     uint8_t expanded[64];
4     shake256(expanded, 64, seed, 32);
5     // expanded[0..31] -> seed for matrix A
6     // expanded[32..63] -> seed for secret/error
7     expand_A(A, expanded);
8     sample_secret(s, e, expanded + 32);
9     pk = encode(A, A*s + e);
10    sk = encode(s, pk, H(pk), z); // z = implicit rejection seed
11 }

```

Advantages: reproducible key generation for testing, reduced RNG exposure surface (one call instead of many), and the ability to recover keys from a backed-up seed.

Risks: seed compromise is total compromise, and the seed expansion itself must be protected against side channels.

Remark 12.3 (Hedged Randomness) A prudent implementation *hedges* by mixing the output of the system RNG with a deterministic component: $\text{seed} = H(\text{DRBG output} \parallel \text{nonce} \parallel \text{context})$. If either source is compromised, the other provides a safety net.

With constant-time code, side-channel countermeasures, fault resistance, and sound randomness in place, the implementation is secure—but it is also larger. The next section quantifies the cost in bytes.

Table 12.3 Key, signature, and ciphertext sizes for NIST PQC standards (bytes)

Algorithm	Public key	Secret key	Sig./CT	Security
<i>Classical baselines</i>				
ECDH-P256	64	32	64 ^{CT}	128-bit
Ed25519	32	32	64 ^{Sig}	128-bit
RSA-3072	384	384	384 ^{Sig}	128-bit
<i>ML-KEM (FIPS 203)—Key Encapsulation</i>				
ML-KEM-512	800	1,632	768 ^{CT}	128-bit
ML-KEM-768	1,184	2,400	1,088 ^{CT}	192-bit
ML-KEM-1024	1,568	3,168	1,568 ^{CT}	256-bit
<i>ML-DSA (FIPS 204)—Digital Signature</i>				
ML-DSA-44	1,312	2,560	2,420 ^{Sig}	128-bit
ML-DSA-65	1,952	4,032	3,309 ^{Sig}	192-bit
ML-DSA-87	2,592	4,896	4,627 ^{Sig}	256-bit
<i>SLH-DSA (FIPS 205)—Hash-Based Signature</i>				
SLH-DSA-128s	32	64	7,856 ^{Sig}	128-bit
SLH-DSA-128f	32	64	17,088 ^{Sig}	128-bit
SLH-DSA-192s	48	96	16,224 ^{Sig}	192-bit
SLH-DSA-256s	64	128	29,792 ^{Sig}	256-bit

12.5 Key and Ciphertext Sizes

The most immediate practical consequence of post-quantum cryptography is larger keys and ciphertexts. Table 12.3 presents a comprehensive comparison of all NIST post-quantum standards against classical baselines.

12.5.1 Impact on TLS

A TLS 1.3 handshake with X25519 + Ed25519 transmits approximately 1.5 KB of cryptographic material (key shares, certificates, signatures). Replacing this with ML-KEM-768 + ML-DSA-65 increases the cryptographic payload to roughly 9 KB—a 6× increase. With a typical certificate chain of three certificates, the total signature contribution alone is $3 \times 3,309 = 9,927$ bytes, requiring TCP fragmentation and potentially an additional round trip.

Example 12.2 (Handshake Overhead Calculation) Consider replacing X25519 (32 B public key) + Ed25519 (32 B public key, 64 B signature) with ML-KEM-768 + ML-DSA-65 in a TLS 1.3 handshake:

- Key exchange overhead: $(1,184 + 1,088) - (32 + 32) = +2,208$ B.
- Authentication overhead (3-cert chain): $3 \times (1,952 + 3,309) - 3 \times (32 + 64) = +15,495$ B.
- Total additional bytes: $\approx 17,700$ B.

With a 1,500-byte MTU, the classical handshake fits in ~ 2 TCP segments; the post-quantum handshake requires ~ 14 segments and may need an additional round trip.

12.5.2 Storage and Compression

Certificate databases. A certificate authority storing 10^6 end-entity certificates sees public-key storage grow from 32 MB (Ed25519) to 1.95 GB (ML-DSA-65)—a $61\times$ increase that has infrastructure implications for database indexing, replication, and backup.

Blockchain. A Bitcoin transaction is approximately 250 bytes; replacing ECDSA with ML-DSA-65 would increase this to $\sim 3,500$ bytes, accelerating blockchain growth by $14\times$.

Compression techniques. Lattice public keys exhibit algebraic structure that enables moderate compression:

- **Seed expansion.** Store only the 32-byte seed for the matrix $\mathbf{A}$ and transmit $\mathbf{t} = \mathbf{As} + \mathbf{e}$; the receiver re-expands $\mathbf{A}$ from the seed. This is already standard in ML-KEM and ML-DSA.
- **Coefficient compression.** ML-KEM compresses ciphertext coefficients from 12 bits to 10 or 4 bits (depending on the component), reducing ciphertext size by $\sim 25\%$ at the cost of a small decryption failure probability.
- **Hybrid schemes.** During the transition period, using a compact classical algorithm alongside PQC (e.g., X25519 + ML-KEM-768) adds only ~ 32 bytes over the PQC-only cost while providing backward security.

Larger artefacts demand not only bandwidth but also confidence: a single incorrect coefficient in a public key renders the scheme insecure. The final section addresses how implementations are tested, validated, and formally verified.

12.6 Testing and Validation

Deploying a new cryptographic algorithm requires not only correct implementation but also formal evidence of correctness. The testing and validation ecosystem for PQC is still maturing, but the foundational elements are in place.

12.6.1 Known-Answer Tests

A Known-Answer Test (KAT) consists of a fixed input (seed, message, key) and the expected output (ciphertext, signature, shared secret). NIST published KAT vectors for all parameter sets of ML-KEM, ML-DSA, and SLH-DSA as part of the FIPS

standardisation process. Any conforming implementation must reproduce these vectors bit-for-bit.

KATs verify functional correctness but not security properties: an implementation that passes all KATs may still have timing leaks, fault vulnerabilities, or incorrect error handling on malformed inputs. KATs are necessary but not sufficient.

12.6.2 FIPS 140-3 and the CMVP

The Cryptographic Module Validation Program (CMVP), jointly operated by NIST and the Canadian Centre for Cyber Security, certifies cryptographic modules under FIPS 140-3. The standard defines four security levels:

1. **Level 1.** Basic requirements: approved algorithms, KATs, no physical security.
2. **Level 2.** Tamper-evidence (seals, coatings), role-based authentication.
3. **Level 3.** Tamper-response (active zeroisation), identity-based authentication.
4. **Level 4.** Environmental failure protection, complete physical penetration resistance.

With the publication of FIPS 203, 204, and 205 in 2024, the CMVP began accepting validation submissions for modules implementing the post-quantum standards. The validation process tests:

- Algorithm correctness via KATs.
- Self-tests (power-up and conditional) as required by FIPS 140-3 §4.9.
- Entropy source validation (SP 800-90B) for the seed generation path.
- Key zeroisation: secret keys must be erased from memory when no longer needed.

12.6.3 Formal Verification

Formal verification provides mathematical proof that an implementation correctly realises its specification. Two approaches are gaining traction for PQC:

EasyCrypt and Jasmin. The Jasmin language compiles to verified constant-time assembly, and EasyCrypt provides machine-checked proofs of cryptographic security. The FORMOSA project used this toolchain to produce a formally verified implementation of ML-KEM that is provably functionally correct and constant-time at the assembly level.

HACL* and Vale. The HACL* library (written in F*) provides verified C implementations of cryptographic primitives. The Vale framework targets verified assembly for performance-critical routines. Extensions to cover ML-KEM and ML-DSA are under active development.

Remark 12.4 (The Verification Gap) As of 2026, formally verified implementations of the NIST PQC standards exist only for ML-KEM. ML-DSA and SLH-DSA lack complete verified implementations, and no verified implementation includes side-channel

countermeasures beyond constant-time execution (i.e., no verified masking). Closing this gap is one of the most important open problems in applied PQC.

A secure, validated implementation is a necessary foundation, but deploying it into production infrastructure—replacing classical algorithms in TLS, PKI, code signing, and beyond—raises an entirely different set of challenges. The next chapter turns from implementation to migration strategy.

12.1 The following C function performs modular reduction. Identify the timing side channel and rewrite it in constant time.

```

1 int16_t reduce(int32_t a, int16_t q) {
2     int16_t r = a % q;
3     if (r < 0) r += q;
4     return r;
5 }
```

Hint: Replace the branch with an arithmetic mask derived from the sign bit of `r`.

12.2 A TLS 1.3 handshake currently fits in a single TCP round trip. Compute the additional bytes introduced by replacing X25519 + Ed25519 with ML-KEM-768 + ML-DSA-65. Will the handshake still fit in one round trip assuming a typical MTU of 1,500 bytes? What if SLH-DSA-128s is used for authentication instead of ML-DSA-65?

12.3 Consider an NTT butterfly that computes $a' = a + \omega b$ and $b' = a - \omega b$ over $\mathbb{Z}_q$. Suppose the inputs are arithmetically masked: $a = a_0 + a_1 \bmod q$ and $b = b_0 + b_1 \bmod q$. Show that performing the butterfly independently on each share pair (a_0, b_0) and (a_1, b_1) produces valid shares of the output, i.e., $a'_0 + a'_1 = a' \bmod q$. Why does this argument fail for pointwise multiplication $c = a \cdot b$?

12.4 An attacker can inject a single instruction-skip fault during ML-DSA signing. Describe which instruction the attacker should target to bypass rejection sampling. If the attacker collects N faulty signatures, outline how to mount a key-recovery attack. Propose a software countermeasure and argue that it defeats the single-fault model.

12.5 Write a C function `ct_equal(const uint8_t *a, const uint8_t *b, size_t len)` that returns 1 if the two byte arrays are identical and 0 otherwise, in constant time. Verify that your implementation has no early-exit path. Explain why `memcmp()` from the standard library is *not* suitable for comparing secret data.

```

1 // Skeleton: fill in the body
2 int ct_equal(const uint8_t *a, const uint8_t *b, size_t len) {
3     // Your implementation here
4 }
```


Chapter 13

Hardware Security Analysis of Post-Quantum Cryptography

Abstract Deploying post-quantum cryptographic algorithms in hardware introduces attack surfaces that have no analogue in software implementations. Every logic gate transition leaks information through power consumption and electromagnetic emanation; every register holds secrets that persist until actively destroyed. This chapter develops hardware security analysis from first principles—the physics of side-channel leakage, the mathematics of masking countermeasures, and the engineering of fault-resilient architectures—then applies the full framework to ML-KEM, ML-DSA, and SLH-DSA. We map the attack surfaces of each algorithm to specific countermeasure techniques with concrete cost estimates, analyse the cross-component interactions that make system-level security harder than the sum of its parts, and survey the certification landscape that governs whether a hardened implementation is accepted for deployment.

13.1 The Entropy-to-Cryptography Pipeline

The previous chapter treated PQC algorithms as software: sequences of arithmetic instructions executed on a processor, where security means constant-time code and absence of secret-dependent branches. Hardware changes the game entirely. A dedicated PQC accelerator processes secrets not as abstract variables but as physical voltages on wires, currents through transistors, and charges stored in capacitors. Every one of these physical quantities is, in principle, observable by an attacker with the right equipment.

The central object of this chapter is the *entropy-to-cryptography pipeline*: the complete data path from the entropy source that generates randomness, through the deterministic random bit generator (DRBG) that conditions and expands it, to the PQC algorithms that consume it for key generation, encapsulation, and signing. In a typical hardware PQC implementation, this pipeline has three stages:

1. **Entropy source.** A physical random number generator (often a quantum random number generator, QRNG) that produces raw random bits. These bits have high but imperfect min-entropy and must pass health tests before being trusted.

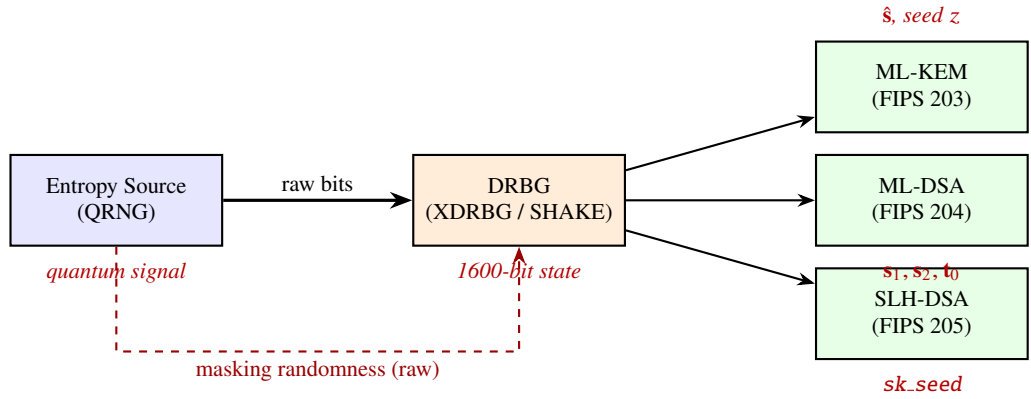


Fig. 13.1 The entropy-to-cryptography pipeline. Solid arrows carry conditioned randomness; the dashed arrow carries raw entropy directly to the DRBG’s masking logic, breaking the circular dependency discussed in Sect. 13.7. Red labels indicate the primary secrets at each stage.

2. **DRBG.** A cryptographic construction—typically based on SHAKE-256 or a similar extendable-output function—that absorbs entropy and produces the uniform random streams needed by PQC algorithms. Its internal state (1600 bits for a Keccak-based DRBG) is the most valuable secret in the system: compromising it yields every key, nonce, and mask generated until the next reseed.
3. **PQC algorithms.** ML-KEM, ML-DSA, and SLH-DSA, each consuming random bits from the DRBG and processing long-term secrets (private keys) and ephemeral secrets (nonces, masks).

The DRBG is the single chokepoint through which all entropy passes. An attacker who recovers the DRBG state does not need to attack any individual PQC algorithm—all outputs become predictable. Conversely, perfectly securing the PQC algorithms is worthless if the DRBG leaks its state through a power trace.

Figure 13.1 illustrates this data flow. Each box processes at least one secret, and each arrow represents an interface that must be protected for both integrity (corruption leads to security failure) and confidentiality (observation leads to entropy leakage).

The weakest-link principle. In a hardware cryptographic module, the system security is determined by the least protected component in the entropy-to-cryptography pipeline. Hardening ML-KEM to withstand 10^6 attack traces is futile if the DRBG that generates ML-KEM’s nonces can be broken with 10^3 traces of unmasked Keccak. Security analysis must be holistic: every component, every interface, every shared resource.

Table 13.1 summarises the attack surface of each pipeline component. The DRBG stands out: its internal state is a long-lived secret processed repeatedly by Keccak, and without masking, CPA recovers the state in 10^2 – 10^4 traces [2]. Multiple invocations

Table 13.1 Pipeline components, their secrets, and primary attack surfaces

Component	Primary Secret	Key Operations	Primary Attack Surface
Entropy source	Raw quantum signal	Detection, ADC, health tests	Physical manipulation, EM emanation
DRBG	Internal state (1600 bits)	Keccak- f [1600]	CPA on Keccak, DFA on last rounds
ML-KEM	$\hat{s}$, seed z	NTT, compress	CPA on NTT, FI on comparison
ML-DSA	s_1, s_2, t_0	NTT, rejection sampling	Timing channel, CPA on Keccak
SLH-DSA	sk_seed	Keccak ($\sim 35,000\times$)	CPA on WOTS+ chains

between reseeds give the attacker correlated traces of the same secret, making the DRBG a more attractive target than any individual PQC algorithm.

This chapter analyses each stage of the pipeline, but the emphasis falls on the aspects unique to PQC. Classical hardware security techniques (power analysis of AES, fault attacks on RSA-CRT) are well-documented [41]; here we focus on the new attack surfaces that arise from lattice-based and hash-based constructions—the NTT, rejection sampling, polynomial compression, WOTS+ chains—and on the system-level interactions that make PQC hardware security qualitatively different from securing a single AES core.

The remainder of the chapter is organised as follows. Section 13.2 develops side-channel analysis from the physics of CMOS circuits through to the statistics of Correlation Power Analysis. Section 13.3 introduces the countermeasure toolkit: masking (Boolean and arithmetic), shuffling, fault injection defences, and zeroisation. Sections 13.4–13.6 apply this toolkit to each of the three NIST-standardised PQC algorithms. Section 13.7 examines the cross-component interactions that only manifest at the system level. Section 13.8 surveys the certification frameworks that determine whether a hardened implementation is accepted for regulated deployment.

13.2 Side-Channel Analysis from First Principles

A cryptographic algorithm, viewed as mathematics, processes secrets without any physical trace. A cryptographic *implementation*, viewed as a circuit, processes secrets by moving electrons through transistors—and every electron movement is, in principle, observable. The gap between the mathematical abstraction and the physical reality is the side channel.

13.2.1 The Physics of Leakage

Consider a single CMOS inverter: a p-type transistor connected to the power supply (V_{DD}) and an n-type transistor connected to ground. When the input transitions from 0

to 1, the n-transistor turns on and discharges the output capacitance to ground; when the input transitions from 1 to 0, the p-transistor turns on and charges the output capacitance from V_{DD} . Each transition draws a transient current from the power supply. A transition from 0 to 1 at the input produces a measurable current pulse; a 1 to 1 transition produces none.

Scale this to a register holding n bits. When the register is clocked to update from a value v_{old} to a value v_{new} , the number of bit positions that change is the *Hamming distance* $HD(v_{old}, v_{new}) = HW(v_{old} \oplus v_{new})$, where HW denotes the Hamming weight (number of 1-bits). Each changing bit charges or discharges a parasitic capacitance C , drawing an energy of approximately $\frac{1}{2}CV_{DD}^2$ per transition. The total power consumed in that clock cycle is therefore approximately proportional to $HD(v_{old}, v_{new})$, plus a data-independent component (clock tree, leakage current, other logic).

This is the *Hamming distance power model* [41]:

$$P(t) \approx \alpha \cdot HD(v_{old}, v_{new}) + \beta + N(t), \quad (13.1)$$

where α is a gain factor depending on the capacitance and supply voltage, β captures the data-independent power, and $N(t)$ is measurement noise. The same physics produces electromagnetic emanation: each current pulse generates a transient magnetic field that can be measured with a near-field EM probe, sometimes with better spatial resolution than power measurements.

The fundamental problem. If v_{old} is known (or constant, such as an initialisation value) and v_{new} depends on a secret k , then measuring $P(t)$ leaks information about k . No amount of algorithmic cleverness can eliminate this leakage—it is a consequence of the laws of physics applied to digital circuits. The only options are to make the leakage statistically undetectable (masking) or to reduce the number of observations the attacker can collect (limiting trace availability).

13.2.2 What a Power Trace Looks Like

An attacker places a small resistor (1–10 Ω) in series with the target device's power supply and measures the voltage drop across it with a digital oscilloscope sampling at 1–5 GS/s. The result is a *power trace*: a time series of voltage samples, one per nanosecond or better, spanning the duration of the cryptographic operation. For a Keccak permutation running at 100 MHz, a single invocation takes $24 \text{ rounds} \times 1 \text{ cycle/round} = 240 \text{ ns}$, producing roughly 1000 samples at 4 GS/s.

Each clock cycle appears as a spike in the trace, with amplitude modulated by the data being processed. The trace of an ML-KEM decapsulation at 100 MHz might span $50 \mu\text{s}$ and contain 200,000 samples. Hidden in those samples is enough information to recover the secret key—if the attacker knows where to look and how to extract the signal from the noise.

13.2.3 Simple Power Analysis

The most direct attack is *Simple Power Analysis* (SPA): visually inspecting a single (or very few) power trace(s) to identify operations or data values. In RSA, the square-and-multiply pattern reveals the exponent bits directly from one trace. In PQC, SPA targets are less dramatic but still real: the number of iterations in ML-DSA's rejection sampling loop is visible as repeated blocks in the trace, and the accept/reject decision may produce a distinguishable power signature.

SPA requires no statistics—just a good oscilloscope and knowledge of the algorithm's structure. It is the reason that every conditional branch on a secret must be implemented as a constant-time, data-independent operation: not merely because of timing differences, but because the power profile of taken versus not-taken branches differs measurably.

In PQC, SPA is less immediately devastating than in RSA (where the entire private key can be read from one trace), but it still matters. The variable-time rejection loop in ML-DSA signing is an SPA-class vulnerability: the number of loop iterations is visible in the trace as a repeated pattern of NTT, multiplication, and norm-check blocks. Even if the per-iteration power profile does not directly reveal secret coefficients, the iteration count leaks information about the secret key over many signatures (Sect. 13.5.1).

13.2.4 Differential and Correlation Power Analysis

When a single trace does not reveal the secret directly, the attacker turns to statistics. *Differential Power Analysis* (DPA), introduced by Kocher, Jaffe, and Jun [34], and its refined successor *Correlation Power Analysis* (CPA) [9], exploit the statistical relationship between the secret and the measured power across many executions of the same operation with different inputs.

The CPA procedure, explained step by step, proceeds as follows.

Definition 13.1 (Correlation Power Analysis) Let k^* be the secret (e.g., one coefficient of the ML-KEM secret polynomial $\hat{s}$), and let x_i be the known input for the i -th trace. The attacker:

1. Collects N power traces $\{P_i(t)\}_{i=1}^N$, each corresponding to the processing of input x_i under the same secret k^* .
2. For each key hypothesis $k \in \mathcal{K}$ (e.g., $k \in \{0, 1, \dots, q-1\}$ for a single coefficient mod q):
 - (a) Computes the hypothetical intermediate value $v_i(k) = f(x_i, k)$ for each trace, where f is a known function (e.g., a modular multiplication in the NTT butterfly).
 - (b) Predicts the power consumption $h_i(k) = \text{HW}(v_i(k))$ or $\text{HD}(v_i^{\text{prev}}, v_i(k))$ using the Hamming weight or Hamming distance model.
 - (c) Computes the Pearson correlation coefficient between the vector $(h_1(k), \dots, h_N(k))$ and the measured power at each time sample:

$$\rho_k(t) = \frac{\sum_{i=1}^N (h_i(k) - \bar{h}_k)(P_i(t) - \bar{P}(t))}{\sqrt{\sum_{i=1}^N (h_i(k) - \bar{h}_k)^2 \cdot \sum_{i=1}^N (P_i(t) - \bar{P}(t))^2}}. \quad (13.2)$$

3. Selects the hypothesis k that achieves the highest absolute correlation: $\hat{k} = \arg \max_k \max_t |\rho_k(t)|$.

If N is large enough, $\hat{k} = k^*$ with overwhelming probability.

The intuition behind CPA is worth pausing over, because it is the foundation of all statistical side-channel attacks. For the correct key hypothesis $k = k^*$, the predicted power $h_i(k^*)$ is *genuinely correlated* with the measured power at the relevant time sample: both reflect the same underlying data value. For any incorrect hypothesis $k \neq k^*$, the predicted power is a function of a *wrong* intermediate value and therefore uncorrelated with the measurement. As N grows, the correlation for the correct hypothesis converges to a non-zero value (determined by the model accuracy and the SNR), while the correlations for incorrect hypotheses converge to zero by the law of large numbers. The attack succeeds when the gap between the correct peak and the tallest incorrect peak becomes statistically significant.

The critical question is: how large must N be? For an unprotected hardware implementation of AES, the answer is 50–500 traces. For an unprotected ML-KEM NTT, published attacks demonstrate full key recovery in 10^3 – 10^4 traces [55, 52]. The ML-KEM trace count is higher than AES for two reasons: the modulus $q = 3329$ means each coefficient has 3329 possible values (versus 256 for an AES byte), spreading the correlation signal across more hypotheses; and the NTT butterfly involves multiple operations per coefficient, blurring the temporal localisation. The exact number depends on the signal-to-noise ratio (SNR): a dedicated ASIC with low switching noise requires fewer traces than an FPGA sharing its power supply with other logic.

A useful rule of thumb relates the SNR to the trace count. For a first-order attack (no masking), the minimum number of traces scales as:

$$N \approx \frac{\alpha}{\text{SNR}^2}, \quad (13.3)$$

where α is a constant depending on the model accuracy and the number of hypotheses (typically $\alpha \approx 3$ – 10). For a first-order masked implementation, the attacker must perform a *second-order* analysis—correlating products or combinations of leakage from two different time samples—and the trace count scales as $N \approx \alpha/\text{SNR}^4$. Each additional masking order squares the exponent, explaining the exponential security gain with masking order.

Example 13.1 (CPA on an NTT Butterfly) Consider a single NTT butterfly computing $a' = a + \omega \cdot b \bmod q$ where ω is a known twiddle factor and b is a secret coefficient of $\hat{s}$. The attacker knows a (it is derived from the public ciphertext) and ω (it is a fixed constant). For each hypothesis $b = k$, the attacker predicts $v(k) = a + \omega \cdot k \bmod q$ and models the power as $\text{HW}(v(k))$. After collecting traces from 5000 decapsulations (with different ciphertexts but the same secret key), the correlation peak for the correct

$k = b$ rises above the noise, while all incorrect hypotheses remain at the noise floor. Repeating for each of the 256 coefficients recovers the full secret polynomial.

To make this concrete with numbers: for $q = 3329$, $\omega = 17$, and $a = 1000$, the intermediate value for $k = 42$ is $v(42) = 1000 + 17 \times 42 \bmod 3329 = 1714$, with $\text{HW}(1714) = \text{HW}(0x6B2) = 7$. For $k = 43$, $v(43) = 1731$, with $\text{HW}(1731) = 8$. Different key hypotheses produce different Hamming weights, and the correlation distinguishes the correct one—provided enough traces are averaged to suppress the noise.

13.2.5 Electromagnetic Emanation

Power analysis measures the *aggregate* current drawn by the entire chip. Electromagnetic (EM) analysis measures the *local* magnetic field above a specific region of the die, using a near-field probe with a loop diameter of 100–500 μm . The physics is identical—current flow creates magnetic fields—but EM probes offer two advantages: spatial selectivity (targeting the specific register that processes the secret, ignoring noise from unrelated logic) and the ability to bypass on-chip power filtering that attenuates the power-supply signal.

For FPGA implementations, where the placement of logic is partially under the designer’s control, EM probes can isolate the Keccak core, the NTT multiplier, or the comparison logic. The cost of a professional EM measurement setup ranges from \$50K to \$500K, compared to \$5K–\$20K for a power analysis setup. However, the information gained per trace is often higher, reducing the total number of traces needed.

The practical implication for PQC hardware designers is that power-rail filtering and decoupling capacitors—standard practice for signal integrity—provide some attenuation of the power-supply signal but offer no protection against EM probes. A device that “passes” a power-analysis test may still fail an EM test. Certification laboratories (Sect. 13.8) typically test both channels.

13.2.6 Microarchitectural Side Channels

When PQC algorithms run on general-purpose processors rather than dedicated hardware, a different class of side channels emerges. *Cache-timing attacks* exploit the fact that memory access time depends on whether the data is in the cache (fast) or main memory (slow). If the secret determines which memory addresses are accessed—as in a lookup table indexed by a secret value—the attacker can infer the secret by measuring access times or by priming and probing cache lines.

These attacks are devastating against table-based implementations of Keccak or polynomial sampling, but they are primarily a software concern. In dedicated hardware, there is no cache. We mention them here for completeness, because many “hardware PQC” deployments actually use a soft-core processor running PQC software on an FPGA, inheriting all the software side channels plus the hardware ones.

Table 13.2 Side-channel attack classes, equipment cost, and typical trace requirements for unprotected PQC implementations

Attack Class	Equipment Cost	Traces Needed	Primary Target
SPA	\$5K–\$20K	1–10	Control flow, iteration count
DPA/CPA (power)	\$5K–\$20K	10^3 – 10^4	NTT coefficients, Keccak state
CPA (EM)	\$50K–\$500K	10^2 – 10^3	Targeted registers
Cache timing	\$0 (software)	10^2 – 10^6	Table lookups, branches
Fault injection	\$10K–\$100K	1–10	Comparison logic, loop counters

The distinction between “hardware PQC” and “software PQC on hardware” is crucial for security analysis. A soft-core RISC-V processor on an FPGA running the reference C implementation of ML-KEM is vulnerable to cache-timing attacks, branch-prediction attacks, and power analysis of the processor’s ALU—simultaneously. A dedicated RTL implementation of ML-KEM on the same FPGA has none of the microarchitectural side channels but is fully exposed to power and EM analysis of its custom datapath. The threat models are fundamentally different, and countermeasures that work for one may be irrelevant for the other. This chapter focuses on dedicated hardware implementations; Chapter 12 addressed the software case.

13.2.7 Threat Model Summary

Table 13.2 summarises the landscape. The attacker’s capability is not exotic: a \$10K oscilloscope and physical access to the device suffice for power-based CPA. Certification laboratories (Sect. 13.8) routinely perform these attacks as part of the evaluation process. The remainder of this chapter is devoted to the countermeasures that make these attacks fail—or at least raise their cost beyond practical feasibility.

13.3 Countermeasure Foundations

The previous section established that side-channel leakage is a physical inevitability. This section develops the mathematical and engineering tools for rendering that leakage useless to an attacker. The central technique is *masking*: splitting every secret into random shares such that no single share reveals anything about the original. But masking alone is not enough—we also need defences against fault injection, techniques for secure memory management, and a clear understanding of where each countermeasure applies within the PQC pipeline.

13.3.1 Boolean Masking

The simplest form of masking splits a secret x into two shares x_0 and x_1 such that $x = x_0 \oplus x_1$, where x_0 is chosen uniformly at random. The share $x_1 = x \oplus x_0$ is then also uniformly random (since XOR with a uniform value produces a uniform result). An attacker who observes only x_0 or only x_1 —through a side channel that captures one register at a time—learns nothing about x . Recovering x requires combining information from *both* shares simultaneously, which requires a qualitatively different (and much harder) attack.

Example 13.2 (Boolean Masking of a Single Byte) Let $x = 0xA7$ (binary 10100111). We choose a uniform random mask $x_0 = 0x3C$ (binary 00111100). The second share is $x_1 = x \oplus x_0 = 0x9B$ (binary 10011011). The Hamming weight of x is 5, but the Hamming weights of x_0 and x_1 are 4 and 5 respectively—both unrelated to the secret. An attacker measuring the power consumed when processing x_0 or x_1 sees a value uniformly distributed over all 256 possible bytes, regardless of x . To recover x , the attacker would need to observe x_0 and x_1 in the same clock cycle (or in two cycles and combine them statistically), which requires a second-order attack.

More generally, d -th order Boolean masking splits x into $d + 1$ shares:

$$x = x_0 \oplus x_1 \oplus \cdots \oplus x_d, \quad (13.4)$$

where $x_0, \dots, x_{d-1}$ are independent uniform random values and $x_d = x \oplus x_0 \oplus \cdots \oplus x_{d-1}$. A d -th order attack—one that combines leakage from $d + 1$ time samples simultaneously—is needed to extract information about x . The number of traces required for a d -th order attack scales roughly as SNR^{-2d} , where SNR is the signal-to-noise ratio [41]. First-order masking ($d = 1$) is the practical minimum for certification; it squares the trace requirement compared to an unmasked implementation.

The beauty of Boolean masking is that *linear operations are free*. If we want to compute $z = x \oplus y$ where both x and y are masked, we simply XOR corresponding shares: $z_0 = x_0 \oplus y_0$, $z_1 = x_1 \oplus y_1$. No fresh randomness is needed, and each share computation touches only one share of each input.

The difficulty arises with *non-linear operations*. Computing $z = x \wedge y$ (bitwise AND) from shares requires evaluating cross-terms: $(x_0 \oplus x_1) \wedge (y_0 \oplus y_1) = (x_0 \wedge y_0) \oplus (x_0 \wedge y_1) \oplus (x_1 \wedge y_0) \oplus (x_1 \wedge y_1)$. The cross-terms $x_0 \wedge y_1$ and $x_1 \wedge y_0$ each depend on shares of different secrets, and computing them in the same combinational circuit creates a transient moment where both shares influence the same wire—exactly the leakage that masking was supposed to prevent.

Definition 13.2 (ISW Scheme) The Ishai–Sahai–Wagner (ISW) scheme [30] computes a masked AND by introducing *fresh randomness* to break the correlation between cross-terms. For first-order masking ($d = 1$), the masked AND of (x_0, x_1) and (y_0, y_1) proceeds as:

1. Draw a fresh random bit r .
2. Compute $z_0 = (x_0 \wedge y_0) \oplus r \oplus (x_0 \wedge y_1) \oplus (x_1 \wedge y_0)$.

3. Compute $z_1 = (x_1 \wedge y_1) \oplus r$.

Each intermediate computation depends on at most one share of each input, plus fresh randomness. The cost: one fresh random bit per AND gate.

In practice, the ISW scheme alone is insufficient for hardware implementations. The reason is *glitches*: in combinational logic, signals do not arrive at a gate simultaneously. If $x_0 \wedge y_1$ is computed slightly before r is XORed in, there is a brief moment where the wire carries a value that depends on both x_0 and y_1 —and that glitch is observable in the power trace.

13.3.2 Domain-Oriented Masking

Domain-Oriented Masking (DOM) [24] solves the glitch problem by inserting a *pipeline register* between the computation of cross-domain terms and their recombination. The register ensures that each share is stable for a full clock cycle before it interacts with shares from another domain, preventing transient glitch-based leakage.

Why do glitches matter so much? In a hardware implementation, combinational logic does not compute instantaneously. When inputs to a gate arrive at different times (due to different propagation delays through preceding logic), the gate output may transition through intermediate values before settling. If a masked AND gate receives x_0 before r (the fresh random bit), there is a transient nanosecond-scale window where the output carries $x_0 \wedge y_1$ —a cross-domain term that depends on shares from both the x and y secrets. This transient draws current from the power supply, and the current is observable in the power trace. The ISW scheme is provably secure in the *probing model* (where the attacker reads individual wires), but not in the *glitch-extended probing model* that captures this physical reality. DOM closes this gap.

For first-order DOM (DOM-1), the overhead per AND gate is one fresh random bit plus one pipeline cycle of latency. For a complex function like the Keccak permutation, the overhead concentrates in the single non-linear step: the χ function. All other steps of Keccak (θ , ρ , π , ι) are linear (XOR-based) and apply independently to each share at zero cost. The χ step computes $a[i] = a[i] \oplus ((\neg a[i+1]) \wedge a[i+2])$ —one AND per bit of the 1600-bit state. DOM-1 on χ requires 1600 fresh random bits per round $\times$ 24 rounds = 38,400 fresh random bits per Keccak- $f[1600]$ invocation.

At a Keccak throughput of 100 invocations per second, the masking randomness demand is approximately 3.84 Mbit/s—a non-trivial but achievable requirement for a hardware random number generator.

13.3.3 Arithmetic Masking

Boolean masking works naturally with XOR-based operations (Keccak, bitwise comparisons). But PQC algorithms spend much of their time in modular arithmetic: the NTT operates in $\mathbb{Z}_q$, polynomial multiplication is coefficient-wise multiplication mod q , and

key generation involves additions mod q . For these operations, *arithmetic masking* is the natural choice:

$$x = x_0 + x_1 \bmod q. \quad (13.5)$$

Under arithmetic masking, all linear modular operations—addition, subtraction, multiplication by a *public* constant—apply independently to each share at no extra cost. If c is a public constant and $x = x_0 + x_1 \bmod q$, then $c \cdot x = (c \cdot x_0) + (c \cdot x_1) \bmod q$. No fresh randomness, no cross-terms.

This observation has a profound consequence for PQC.

The NTT is free under arithmetic masking. The NTT butterfly computes $a' = a + \omega b \bmod q$ and $b' = a - \omega b \bmod q$, where ω is a public twiddle factor. Since ω is public, the multiplication $\omega \cdot b$ is a public-constant-times-secret operation: linear under arithmetic masking. The additions and subtractions are also linear. Therefore, the *entire* NTT and pointwise polynomial multiplication (where the public matrix $\mathbf{A}$ multiplies the secret vector $\hat{\mathbf{s}}$) can be masked by simply running each share through the NTT independently. Cost: $2\times$ the multiplier area, zero fresh randomness.

Where, then, does the masking cost concentrate? At the *boundaries* between arithmetic and Boolean domains.

13.3.4 The A2B/B2A Conversion Problem

PQC algorithms inevitably switch between arithmetic operations (NTT, modular multiply) and Boolean operations (Keccak for hashing, bitwise comparison for decapsulation checks, compression via rounding). At each switch, the masked representation must be converted:

- **A2B (Arithmetic to Boolean):** Given shares x_0, x_1 with $x = x_0 + x_1 \bmod q$, produce shares y_0, y_1 with $x = y_0 \oplus y_1$. This requires a *masked carry chain*: the addition involves carries that propagate bit by bit, and each carry computation is a non-linear (AND) operation that must be masked. Cost: $O(n)$ masked ANDs for n -bit values.
- **B2A (Boolean to Arithmetic):** The reverse conversion, typically implemented via Goubin’s algorithm [23] or optimised variants.

The concrete cost depends on the bit-width of q . For ML-KEM with $q = 3329$ (12 bits): approximately 12 cycles and 12 bits of fresh randomness per conversion. For ML-DSA with $q = 8380417$ (23 bits): approximately 23 cycles and 23 bits of fresh randomness. The cost scaling with bit-width means ML-DSA masking is roughly twice as expensive per coefficient as ML-KEM masking—a ratio that pervades the entire hardening analysis.

The A2B/B2A conversion is not merely an overhead—it is a *verified security boundary*. Each conversion must be proven to maintain the masking invariant: after the

conversion, each output share must be statistically independent of the secret. Errors in the conversion logic (incorrect handling of carry propagation, off-by-one in the modular reduction) are among the most subtle and dangerous implementation bugs in masked PQC hardware, because they silently break the security guarantee without producing any functional error.

13.3.5 Shuffling

Shuffling processes polynomial coefficients in a random permutation that changes with every execution. If the attacker expects coefficient i to appear at time step i , but the implementation processes coefficient $\pi(i)$ instead (for a random permutation π), the attacker's power model predictions are misaligned with the actual computation. The number of traces required increases by a factor proportional to the number of possible positions—up to $256\times$ for a polynomial of 256 coefficients.

Shuffling is implemented via the Fisher–Yates algorithm in hardware (approximately 256 clock cycles of setup, consuming roughly 2 Kbits of randomness) or via a cheaper partial permutation (random start plus coprime stride, which offers fewer permutations but is simpler to implement).

A critical point: shuffling is a *statistical* countermeasure, not a cryptographic one. It increases the number of traces needed but does not provide information-theoretic protection. It must always *complement* masking, never replace it.

Example 13.3 (Shuffling the NTT) Consider an NTT on a polynomial of 256 coefficients. Without shuffling, the attacker knows that coefficient i is processed at butterfly stage j at a specific clock cycle. With shuffling, the processing order is a random permutation. The attacker's CPA hypothesis for coefficient i must be tested against all 256 possible time positions, and the correlation signal is diluted by a factor of $\sqrt{256} = 16$. Combined with first-order masking, the effective trace requirement increases from roughly 10^6 (masking alone) to roughly $10^6 \times 256 \approx 2.6 \times 10^8$ (masking plus shuffling). This is well beyond practical feasibility for most attack scenarios.

13.3.6 Fault Injection Countermeasures

Side-channel analysis is a *passive* attack: the attacker observes but does not interfere. *Fault injection* (FI) is an *active* attack: the attacker deliberately disrupts the computation—via voltage glitching, clock manipulation, laser pulses, or electromagnetic pulses—to induce errors that reveal secrets or bypass security checks.

Three classes of countermeasures address fault injection:

Temporal redundancy. Compute the operation twice and compare results before outputting. If a transient fault corrupts one computation, the comparison fails and the device aborts. Cost: $2\times$ time or, with parallel redundancy, $2\times$ area. Temporal redundancy is effective against *transient* faults (voltage glitches, single laser pulses) but not against

permanent faults (circuit modifications by a well-resourced attacker). For the highest threat levels, two independent implementations on different die regions provide stronger guarantees.

Information redundancy. Maintain parity, error-correcting codes, or CRCs on intermediate values, verified at each pipeline stage. A fault that modifies a secret value also corrupts the check bits with high probability, triggering detection. The error-detection code must be chosen so that the fault model of interest (single-bit flips for voltage glitches, multi-bit burst errors for laser pulses) falls within the code’s detection capability.

FSM protection. The control logic (finite state machine) that sequences cryptographic operations is itself a target. One-hot encoding with illegal-state detection, or Hamming-coded states with a trap state that triggers immediate zeroisation, prevents an attacker from skipping critical steps (such as the re-encryption check in ML-KEM decapsulation or the norm check in ML-DSA signing). The trap state is critical: when an illegal state is detected, the module must not simply reset (which might leave secrets in memory) but must actively zeroise all secret-holding locations before any further operation.

For PQC specifically, high-value FI targets include the comparison logic in ML-KEM’s implicit rejection, the rejection loop counter in ML-DSA, and the address register in SLH-DSA’s Merkle tree traversal. We will examine each in the per-algorithm sections that follow.

13.3.7 Zeroisation

After a cryptographic operation completes, every register and memory location that held secret data must be actively overwritten with a known pattern (typically all zeros, then all ones, then all zeros again). Simply deasserting a signal or relying on power-down to clear memory is insufficient: FPGA block RAMs (BRAMs) do not self-clear on reset, and register contents may persist through soft resets.

The zeroisation path must be *independent* of the normal operational path—otherwise, a single fault injection that disrupts the normal completion sequence also bypasses cleanup. FIPS 140-3 mandates zeroisation under SP 800-140D [48]; Common Criteria covers it under FCS_CKM.4 [14]. Verification of zeroisation completeness—confirming that *every* secret-holding location is covered—is one of the most tedious but most important steps in hardware security certification.

For PQC implementations, the zeroisation scope is larger than for classical algorithms. An AES core has a 128-bit key register and a 128-bit state register—two locations to clear. An ML-KEM decapsulation core has the secret polynomial $\hat{s}$ (multiple kilobits across NTT butterfly buffers), the seed z , intermediate Keccak states, masked shares, and the comparison result—dozens of locations distributed across the datapath. A systematic zeroisation analysis begins by enumerating every register and BRAM location that touches secret data at any point during the computation, then verifying that each location is overwritten on: (a) normal completion, (b) fault/tamper detection, and (c) power-down or reset.

FPGA BRAMs do not self-clear. Unlike flip-flops, which typically power up in a defined state, FPGA block RAMs may retain their contents across configuration reloads and soft resets. A “cold boot” attack on an FPGA that stores ML-KEM secret keys in BRAM is realistic: removing power briefly and re-reading the BRAM may recover partial key material. Active zeroisation on shutdown—triggered by a voltage-drop detector with enough energy storage to complete the overwrite sequence—is the countermeasure.

With the countermeasure toolkit in hand, we now apply it to each of the three NIST-standardised PQC algorithms, identifying the specific attack surfaces and the cost of hardening each one.

13.4 ML-KEM: Attack Surfaces and Hardening

ML-KEM (FIPS 203 [50]) operates over $\mathbb{Z}_q$ with $q = 3329$ (12 bits). Among the three PQC standards, it is the most amenable to hardware masking, because most of its arithmetic is linear under arithmetic masking and the modulus is small. The primary concern is the *decapsulation* operation, which processes the long-term secret key $\hat{s}$ once per key exchange—potentially millions of times over the key’s lifetime.

13.4.1 Decapsulation: A Guided Tour of the Attack Surface

ML-KEM decapsulation proceeds through six stages, each with distinct security implications. We walk through them in order, marking the secret data processed at each step and the applicable attack class.

Step 1: NTT and pointwise multiplication. The ciphertext vector $\hat{u}$ (public) is multiplied by the secret key $\hat{s}$ in the NTT domain. As established in Sect. 13.3.3, this is fully linear under arithmetic masking: each share of $\hat{s}$ is multiplied by the public $\hat{u}$ independently. Without masking, CPA recovers the secret coefficients in 10^3 – 10^4 traces [55]. With first-order arithmetic masking, the attack requires second-order statistical analysis, raising the trace requirement to roughly 10^6 – 10^8 depending on SNR.

Step 2: Inverse NTT and decompression. The inverse NTT is also linear (public twiddle factors operating on masked shares). Decompression involves scaling by powers of 2, which are public constants—still linear.

Step 3: CBD sampling. When re-encapsulating to verify the ciphertext (the Fujisaki–Okamoto transform), the secret polynomial is regenerated from a seed using the Centred Binomial Distribution. This involves: (a) Keccak hashing of the secret seed, requiring DOM-1 Keccak; (b) a popcount operation on secret data, which is non-linear and requires Boolean masking. The B2A conversion (from the Boolean domain of Keccak

Table 13.3 ML-KEM decapsulation: attack surfaces and countermeasures

Operation	Risk	Attack Class	Countermeasure
NTT / pointwise multiply	Critical	CPA (10^3 – 10^4 traces)	Arithmetic masking (free)
SHAKE for secret sampling	Critical	CPA on Keccak state	DOM-1 Keccak
CBD popcount	High	SCA on non-linear op	Masked popcount, B2A
Compression rounding	Medium	SCA at unmasking point	A2B, late unmasking
Decaps comparison	High	Timing/power oracle	Constant-power balanced MUX
FI on comparison	High	CCA oracle	Temporal redundancy
Secret key in BRAM	Medium	Probing, cold boot	Masked storage, ECC

output to the arithmetic domain of polynomial operations) costs approximately 12 cycles per coefficient for $q = 3329$.

Step 4: Compression. The polynomial is compressed by rounding: a non-linear operation that requires A2B conversion (from arithmetic masking to Boolean representation for the rounding comparison). This is a localised but critical unmasking point.

Step 5: Comparison (implicit rejection). The re-encrypted ciphertext c' is compared bitwise against the received ciphertext c . This comparison must be *constant-time* and *constant-power*: the implementation must not reveal whether $c' = c$ or $c' \neq c$. A distinguishable comparison gives the attacker a chosen-ciphertext oracle, breaking CCA security. The standard approach is a “balanced MUX” that always computes both the real and rejection shared secrets and selects the output using a secret bit derived from the comparison result.

Step 6: Shared secret derivation. Keccak processes the secret seed z (used for implicit rejection) and the derived shared secret. The seed z must be masked throughout.

13.4.2 Masking Cost Summary

The cost structure of masking ML-KEM is lopsided in a fortunate way. The most computationally intensive operations—the NTT and pointwise multiplication—are free under arithmetic masking (aside from the $2\times$ multiplier area for two shares). The real cost concentrates at the domain boundaries:

- **CBD sampling:** B2A conversion, $\sim 256 \times 12 = 3,072$ cycles per polynomial.
- **Compression:** A2B conversion, $\sim 256 \times 12 = 3,072$ cycles per polynomial.
- **Keccak:** DOM-1 adds one pipeline cycle per round (24 cycles per invocation) and demands 38,400 bits of fresh randomness per invocation.
- **Comparison + MUX:** A2B for bitwise compare, plus balanced MUX logic.

For a hardware implementation running at 100 MHz, the masking overhead adds roughly 20–30% to the decapsulation latency—dominated by the Keccak DOM-1 cost and the A2B/B2A conversions, not by the NTT.

13.4.3 The Implicit Rejection Mechanism

ML-KEM’s CCA security relies on the Fujisaki–Okamoto transform [27]: after decrypting, the module re-encrypts the plaintext and compares the result with the received ciphertext. If they match, the real shared secret is returned; if not, a pseudorandom “rejection” shared secret (derived from the secret seed z and the ciphertext) is returned instead. The attacker must not be able to distinguish which case occurred.

In hardware, this means: (1) both the real and rejection shared secrets must *always* be computed, regardless of the comparison result; (2) the multiplexer that selects between them must draw the same power in both cases (a *balanced MUX*); (3) the comparison must evaluate all bytes without early termination; and (4) the seed z must remain masked throughout, since it is a long-term secret that persists across all decapsulations.

A fault injection that forces the comparison to always return “equal” converts ML-KEM into a CCA-vulnerable scheme: the attacker can submit malformed ciphertexts and always receive the real shared secret, enabling an adaptive chosen-ciphertext attack. The countermeasure is temporal redundancy on the re-encryption and comparison, with independent comparison logic that would require two simultaneous faults to bypass.

With ML-KEM’s compact modulus and linear NTT, first-order masking is both effective and affordable. ML-DSA, as we shall see, is a substantially harder problem.

13.5 ML-DSA: Attack Surfaces and Hardening

ML-DSA (FIPS 204 [49]) operates over $\mathbb{Z}_q$ with $q = 8380417$ (23 bits)—nearly double the bit-width of ML-KEM. But the larger modulus is not the primary challenge. ML-DSA introduces three complications absent from ML-KEM: a *structural timing side channel* from rejection sampling, *non-linear rounding operations* that resist masking, and a *deterministic mode* that gives the attacker unlimited identical traces.

13.5.1 The Rejection Sampling Timing Channel

ML-DSA signing uses rejection sampling: the signer generates a random masking vector $\mathbf{y}$, computes the signature candidate $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ (where c is the challenge and $\mathbf{s}_1$ is the secret key), and checks whether $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$. If the check fails, the signer restarts with fresh $\mathbf{y}$. The number of iterations required depends on the secret $\mathbf{s}_1$: keys with larger coefficients cause more rejections.

For ML-DSA-65, the expected number of iterations is 4–7, with significant variance. The attacker does not need to determine the exact count. The total signing time is proportional to the iteration count and is trivially measurable—even remotely, via network timing. With 10^4 – 10^5 timed signatures, statistical analysis can distinguish keys.

This is not a bug in the implementation; it is a structural property of the algorithm. The only countermeasure is *constant-time padding*: always execute $N_{\max}$ iterations, discarding the extra results. The value $N_{\max}$ must be calibrated so that $\Pr[\text{iterations} > N_{\max}] < 2^{-128}$. All iterations—including the discarded ones—must write to the output buffer unconditionally. Only the valid result is selected at the end via a masked multiplexer.

The constant-time padding has a severe performance cost. For ML-DSA-65 with $p \approx 0.2$ success probability per iteration, the geometric distribution gives $\Pr[\text{iterations} > N] = (1-p)^N = 0.8^N$. To achieve $0.8^{N_{\max}} < 2^{-128}$, we need $N_{\max} > 128/\log_2(1/0.8) \approx 398$ iterations. Since the expected number of iterations is only $1/p = 5$, the worst-case latency exceeds the average by a factor of roughly 80. This is the price of constant-time execution for a rejection-sampling-based scheme, and it is one of the principal engineering arguments for preferring ML-KEM (which has no rejection sampling) when latency-sensitive hardware is a priority.

13.5.2 Deterministic vs. Hedged Mode

In deterministic mode, the randomness parameter `rnd` is set to zero. The same message and secret key produce identical intermediate values across executions. For DPA, this is the worst-case scenario: the attacker can average unlimited traces of the same computation, driving the noise to zero.

In *hedged mode*, `rnd` is drawn fresh from the DRBG for each signature. Different executions with the same message now produce different intermediate values, preventing trace averaging. For hardware implementations with access to a high-quality random number generator, hedged mode is strictly superior from a side-channel perspective.

Recommendation. Hardware implementations with a DRBG backed by a physical entropy source should implement hedged mode only. Omitting deterministic mode reduces the certification analysis scope significantly and eliminates the DPA worst case. If deterministic mode must be offered (e.g., for interoperability testing), it requires higher-order masking or a formal bound on the number of signatures permitted with the same message.

13.5.3 Non-Linear Rounding and Norm Checks

ML-DSA's `HighBits` and `LowBits` functions decompose a polynomial coefficient into high-order and low-order parts via integer division and rounding. These operations are non-linear and difficult to mask, especially for the non-power-of-2 modulus $q = 8380417$. The transition from the masked to unmasked domain immediately before rounding is a concentrated leakage point. The practical approach is *late unmasking*:

maintain the arithmetic masking as deep into the computation as possible, converting to Boolean representation only at the final moment before the non-linear rounding, and minimising the temporal window during which the unmasked value exists.

The infinity-norm check ($\|\mathbf{z}\|_\infty < \gamma_1 - \beta$) must evaluate all 256 coefficients without early exit. An early exit—stopping as soon as one coefficient exceeds the threshold—reveals which coefficient failed, leaking information about the secret. In the masked domain, each norm comparison requires A2B conversion (23 bits per coefficient), making this one of the most expensive masked operations in ML-DSA.

13.5.4 Fault Injection on SampleInBall

The function `SampleInBall` generates the challenge polynomial c with exactly τ non-zero coefficients (each ± 1). A fault that forces $c = 0$ reduces the signature equation to $\mathbf{z} = \mathbf{y}$: the output is the nonce itself. Two such faults with different (real) challenges allow the attacker to compute $c_1 \mathbf{s}_1 - c_2 \mathbf{s}_1 = \mathbf{z}_1 - \mathbf{z}_2$, recovering the secret key algebraically.

The countermeasure is an integrity check on c : after sampling, verify that exactly τ coefficients are non-zero and that their values are in $\{-1, +1\}$. This check is cheap (a single pass over 256 coefficients) and catches the most dangerous fault scenario. Redundant encoding of the rejection loop counter adds a second layer of defence.

13.5.5 The 23-Bit Cost Scaling Problem

Every masked operation in ML-DSA inherits a cost penalty from the 23-bit modulus. Compared to ML-KEM's 12-bit $q = 3329$:

- Multiplier area: $\sim 4\times$ (area scales roughly as the square of bit-width for schoolbook multiplication).
- A2B/B2A conversion: $\sim 2\times$ (cost is linear in bit-width: 23 vs. 12 masked carry-chain steps).
- BRAM for masked polynomials: $\sim 2\times$ (23-bit shares vs. 12-bit shares).

This cost scaling limits hardware reuse between ML-KEM and ML-DSA. A unified PQC accelerator must either provision the wider datapath (wasting area when running ML-KEM) or maintain separate arithmetic units (wasting area when only one algorithm is active).

13.5.6 Hint Generation and Secret Leakage

The hint vector $\mathbf{h}$ in ML-DSA is a public output—it is included in the signature. However, its *generation* involves operations on the secret vectors $\mathbf{s}_2$ and $\mathbf{t}_0$. The function `MakeHint` computes whether adding a correction changes the high-order bits of a value, which

requires evaluating a rounding condition on secret-derived data. The hint itself does not leak the secret (it is designed not to), but the *process* of computing it—the power consumption, the timing of conditional evaluations—may leak through a side channel.

The practical impact is that hint computation must be treated as a secret operation even though its output is public. The masked domain must extend through the hint generation, with unmasking deferred until the hint is written to the output buffer.

13.5.7 Countermeasure Summary for ML-DSA

The complete hardening of ML-DSA requires:

- Constant-time rejection loop with $N_{\max}$ padding and unconditional buffer writes.
- Hedged mode as the default (or only) signing mode.
- DOM-1 Keccak for all SHAKE invocations on secret data.
- Arithmetic masking (23-bit) on all operations involving s_1 , s_2 , and nonce y .
- Late unmasking for HighBits/LowBits, with minimal temporal exposure.
- Full-vector norm evaluation without early exit.
- Integrity verification of the challenge polynomial c after SampleInBall.
- Redundant encoding of the rejection loop counter.
- Shuffling of NTT coefficient processing order.

The combined overhead is substantially higher than for ML-KEM: roughly 40–60% additional latency and 3–4× additional area for the masked arithmetic, dominated by the 23-bit multipliers and the constant-time rejection loop padding.

13.6 SLH-DSA: Attack Surfaces and Hardening

SLH-DSA (FIPS 205 [51]) is architecturally the opposite of ML-KEM and ML-DSA. There is no lattice algebra, no NTT, no polynomial multiplication. All cryptographic operations reduce to invocations of Keccak/SHAKE. This makes the security surface *homogeneous*—the Keccak core is the only target—but the attack volume is enormous.

13.6.1 The Sheer Volume Problem

A single SLH-DSA-128f signature requires approximately 35,000 Keccak invocations. Many of these process secret data: the WOTS+ chain computations iterate $\text{value}_{i+1} = H(\text{value}_i)$, where each intermediate value is a secret derived from the master seed sk_seed . The key derivation step $\text{PRF}(\text{sk_seed}, \text{address}_i)$ is the ideal CPA scenario: a fixed secret (sk_seed), a known varying input (the public address), and many traces.

For a key used to sign 1000 messages, the attacker accumulates roughly 35×10^6 Keccak traces over related secrets. If the per-trace SNR is moderate (as on an FPGA),

first-order masking may be insufficient: second-order statistical attacks over millions of traces become feasible.

13.6.2 WOTS+ Chains as CPA Targets

Each WOTS+ chain computes a sequence of hash values: starting from a secret leaf value derived from `sk_seed`, each step applies Keccak to produce the next value in the chain. Successive values are functionally related—each is the hash of the previous one—giving the attacker a “related-input” advantage beyond standard CPA. The chain values that are revealed in the signature provide known plaintexts for verification of partial attacks.

This is qualitatively different from attacking ML-KEM’s NTT, where each decapsulation processes an independent ciphertext. In SLH-DSA, the secret material is structurally correlated across invocations, and the attacker benefits from this correlation.

13.6.3 Masking Order: DOM-1 vs. DOM-2

The decision between first-order masking (DOM-1) and second-order masking (DOM-2) for SLH-DSA’s Keccak should be driven by a quantitative evaluation:

$$\text{traces per key} \times \text{SNR}^{2d} \quad \text{vs.} \quad \text{attack complexity at order } d. \quad (13.6)$$

If the product targets high-volume signing (e.g., a server signing thousands of TLS certificates per hour), the trace budget $N = 35,000 \times \text{signatures}$ may exceed the first-order attack threshold. In such cases, DOM-2 for SLH-DSA’s Keccak is justified even if DOM-1 suffices for ML-KEM and ML-DSA. DOM-2 roughly cubes the Keccak area and requires $2 \times 1600 = 3,200$ fresh random bits per round (76,800 per invocation)—a significant but not prohibitive cost for a well-resourced hardware implementation.

13.6.4 Fault Injection on Address and Leaf Index

Forcing the address register to repeat a value causes the same WOTS+ leaf to be used for two different messages. Each signature reveals a subset of the chain values; two signatures from the same leaf reveal a larger subset, potentially enabling forgery. The countermeasure is redundant encoding of the address register (dual rail or ECC) with illegal-value detection and abort.

Complete signature buffering before emission is also required: SLH-DSA signatures are large (7–50 KB depending on the parameter set), and streaming partial signatures before the full computation completes would expose partial authentication paths without integrity guarantees.

13.6.5 Countermeasure Summary

SLH-DSA hardening is conceptually simple—protect Keccak, protect the address logic, buffer outputs—but the scale is formidable. The masking randomness demand at DOM-1 is $35,000 \times 38,400 \approx 1.3 \times 10^9$ bits per signature, or roughly 170 MB. At DOM-2, this doubles. The entropy source must be dimensioned accordingly, and the randomness delivery path must itself be protected against fault injection.

An additional countermeasure specific to SLH-DSA is *shuffling of independent hash chain computations* within a WOTS+ signature. A WOTS+ signature consists of ℓ independent chains (where ℓ depends on the parameter set, typically 35–67). The order in which these chains are computed can be randomised without affecting the output. This prevents the attacker from aligning traces to specific chains, multiplying the required number of traces by ℓ .

Key generation deserves special attention: it computes the full hypertree, requiring millions of Keccak invocations over `sk_seed`. This is a massive SCA trace volume concentrated in a single operation. The recommendation is to perform key generation in a controlled environment (e.g., during manufacturing, with physical shielding) or with an elevated masking order.

SLH-DSA is the first PQC algorithm certifiable with SCA resistance once a DOM-protected Keccak core is available, precisely because its security surface is homogeneous. There are no lattice-specific complications, no domain conversions, no rejection sampling—just Keccak, at scale.

13.7 Cross-Component Interactions

The previous three sections analysed each PQC algorithm in isolation. But a real hardware cryptographic module does not contain a single algorithm in a vacuum. It contains an entropy source, a DRBG, multiple PQC cores, shared bus interconnects, and a finite-state machine that orchestrates them. The interfaces between these components introduce systemic risks that only manifest at the integration level. Testing components individually and declaring each “secure” provides no guarantee about the integrated system—the interactions themselves are the attack surface.

13.7.1 The DRBG as Systemic Vulnerability

The DRBG’s internal state (1600 bits for a Keccak-based XDRBG [32]) is more valuable than any individual PQC key: compromising the state yields all keys, nonces, and masks generated until the next reseed. The state is a long-lived secret processed repeatedly by Keccak, and each invocation provides the attacker with a correlated trace. Between reseeds, if the DRBG services K requests, the attacker has K traces of the same

(evolving) state. Without masking, CPA recovers the state; with DOM-1, the attacker needs second-order analysis across the K traces.

Differential Fault Analysis (DFA) on the Keccak permutation is equally dangerous: a single-byte fault in rounds 22–24 of Keccak- f [1600] enables full state recovery with as few as 2–4 faults [2]. This is a more direct attack than CPA and requires fewer interactions, making fault-injection countermeasures on the DRBG’s Keccak core at least as important as masking.

The DRBG’s FSM also presents a target. A fault that causes a reseed to be interpreted as a generate operation means the state is not refreshed—potentially yielding correlated outputs. The reverse fault (generate interpreted as reseed) may overwrite state with unprocessed data. Redundant FSM encoding with trap-state detection mitigates both scenarios.

13.7.2 The Reseed Integrity Problem

The entropy path from the physical random number generator to the DRBG crosses a physical bus. If a fault injection corrupts the entropy bits in transit, the DRBG absorbs corrupted data as if it were genuine entropy. This is *worse* than no reseed: if the corruption is attacker-controlled, the attacker can steer the DRBG state to a known value, compromising all subsequent outputs.

The countermeasure is an integrity check on the reseed path: a CRC or MAC computed over the entropy block, verified by the DRBG before absorption, with abort and zeroisation on mismatch. The integrity check itself must be protected against fault injection—a redundant computation with independent comparison logic.

13.7.3 The Circular Dependency

Consider the following dependency chain:

1. The DRBG (e.g., XDRBG based on SHAKE-256) produces randomness for PQC algorithms.
2. The DRBG’s Keccak core must be masked (DOM-1 or DOM-2) to prevent state recovery via CPA.
3. Masking Keccak requires fresh random bits—38,400 per invocation at DOM-1.
4. Those fresh random bits must come from... the DRBG?

This is circular: the DRBG cannot produce the randomness needed to protect itself. The solution is to break the cycle by sourcing the masking randomness for the DRBG’s Keccak *directly from the physical entropy source*—raw bits that have passed the health test but have not yet been conditioned by the DRBG. These bits are not cryptographically conditioned, but they do not need to be: masking randomness requires statistical uniformity, not computational unpredictability. The entropy source’s raw output, post-health-test, is sufficient.

This creates a clear hierarchy: the entropy source feeds raw randomness to the DRBG's masking logic (bypassing the DRBG), while the DRBG feeds conditioned randomness to the PQC algorithms' masking logic and operational randomness needs.

The architecture has three layers, each with a different randomness quality:

1. **Raw entropy** (post-health-test, pre-conditioning): feeds DRBG masking logic. Needs statistical uniformity but not cryptographic unpredictability.
2. **Conditioned entropy** (DRBG output): feeds PQC algorithm masking logic and shuffling. Needs computational unpredictability.
3. **Algorithm-level randomness** (also DRBG output, domain-separated): feeds PQC key generation, nonces, encapsulation coins. Needs full cryptographic quality with per-consumer domain separation.

The domain separation at layer 3 is essential for composability: the security proof of each PQC algorithm assumes independent randomness. If two algorithms share a DRBG without domain separation (i.e., without distinct personalisation strings), a correlation between their random inputs may exist, and the composition is not provably secure [50, 49].

13.7.4 Shared Keccak Core Contention

In area-constrained designs, a single Keccak core may be shared among the DRBG, ML-KEM, ML-DSA, and SLH-DSA. Sharing creates two problems. First, *contention*: when two modules need Keccak simultaneously, one must wait, and the wait time depends on the other module's secret-dependent computation—a timing side channel. Second, *state residue*: if the Keccak core's registers are not zeroised between invocations from different security domains, secret state from one domain may leak into the power trace of another domain's computation.

The countermeasures are either separate Keccak instances per security domain (expensive in area) or a constant-time arbiter that services requests in a fixed schedule regardless of demand, combined with mandatory zeroisation between context switches.

The choice between these approaches is an engineering trade-off with certification implications. Separate instances provide stronger isolation (each domain has no way to influence another's timing) but require more silicon area and more masking randomness. A shared instance with a constant-time arbiter is more area-efficient but requires a careful security argument documenting why the arbiter's scheduling policy does not create cross-domain timing channels. The security argument must address not just the Keccak invocations themselves but also the queuing delays when multiple requests arrive simultaneously.

Table 13.4 Comparative hardening cost for first-order masking across NIST PQC standards

Aspect	ML-KEM	ML-DSA	SLH-DSA
Modulus bit-width	12 bits	23 bits	N/A (hash-only)
NTT masking cost	Free (linear)	Free (linear)	N/A
A2B/B2A cost	~12 cycles	~23 cycles	N/A
Keccak invocations	~6 per decaps	~10 per sign	~35,000 per sign
DOM-1 randomness	~230 Kbit	~384 Kbit	~1.3 Gbit
Timing channel	None (structural)	Rejection sampling	None (structural)
Latency overhead	~20–30%	~40–60%	~20–40%
Area overhead	~2× multiplier	~4× multiplier	~2× Keccak

13.7.5 Frequent Reseeding as a Side-Channel Countermeasure

An underappreciated advantage of high-throughput entropy sources is their impact on side-channel resistance. If the DRBG reseeds every K invocations, the attacker has at most K traces per DRBG state. With DOM-1 Keccak requiring approximately 10^6 traces for a successful second-order attack, setting $K \ll 10^6$ ensures the state changes before the attacker accumulates enough traces.

This is a quantifiable, multiplicative security gain. A DRBG reseeded every 1000 invocations from a high-throughput entropy source has an effective SCA resistance that compounds with the masking order: the attacker needs traces *per state*, but each state lasts only K invocations. This turns the reseed frequency into a formal security parameter, with a documented bound: “maximum K invocations between reseeds, where K is chosen such that $K \ll$ traces needed for order- d attack.”

Example 13.4 (Reseed Frequency Dimensioning) Suppose a DOM-1 masked Keccak requires 10^6 traces for a successful second-order CPA attack (a reasonable estimate for moderate SNR on an FPGA). If the DRBG reseeds every $K = 1000$ invocations, the attacker has 1000 traces per state—three orders of magnitude below the attack threshold. Even if the attacker collects traces across 10^3 different states, each batch is independent and cannot be combined. The reseed frequency effectively reduces the attacker’s useful trace budget by a factor of 1000×. A physical entropy source capable of delivering a new 256-bit seed every 10 ms (25.6 Kbit/s—trivial for any hardware RNG) supports this reseed frequency at the cost of one additional Keccak invocation per reseed.

13.7.6 Comparative Hardening Cost

Table 13.4 summarises the hardening cost across the three algorithms. The differences are stark: ML-KEM is the cheapest to protect, SLH-DSA is the simplest (only Keccak), and ML-DSA is the most complex (wider datapath, rejection sampling, rounding).

The cross-component interactions analysed in this section—reseed integrity, circular dependencies, shared resource contention, and reseed frequency—must all be

documented in the security target for any hardware certification. We now turn to the certification frameworks themselves.

13.8 The Certification Landscape

A hardened implementation is only as valuable as the certification it can achieve. Regulated industries—government, defence, finance, critical infrastructure—require cryptographic modules to be independently evaluated and certified against recognised standards. This section surveys the three major certification frameworks and maps the security requirements from the previous sections to specific framework clauses.

13.8.1 FIPS 140-3

The *Federal Information Processing Standard 140-3* [48] defines security requirements for cryptographic modules at four levels of increasing stringency. It is mandatory for all cryptographic modules used by US federal agencies and widely adopted by private industry as a baseline.

Definition 13.3 (FIPS 140-3 Security Levels)

- **Level 1:** Functional correctness. Algorithm validation via Known Answer Tests (KATs) and the Automated Cryptographic Validation Protocol (ACVP). No physical security requirements.
- **Level 2:** Role-based authentication plus evidence of tamper (tamper-evident coatings or seals). Minimal physical security.
- **Level 3:** Tamper resistance (active response to physical penetration) and *non-invasive side-channel testing* per ISO 17825 [29]. This is the level at which SCA countermeasures become mandatory.
- **Level 4:** Complete envelope of physical protection. Environmental failure protection and formal verification methods. Few commercial products target Level 4.

For PQC hardware, the critical transition is from Level 2 to Level 3: the moment SCA testing becomes mandatory. A Level 3 evaluation requires the module to resist non-invasive attacks (power analysis, EM analysis) as tested by an accredited laboratory. The masking, shuffling, and fault countermeasures developed in this chapter are precisely what Level 3 demands.

13.8.2 Common Criteria

The *Common Criteria* (CC) [14] is an international framework (ISO/IEC 15408) that evaluates products against a *Protection Profile* (PP) specifying the required security

functionality. Evaluation Assurance Levels (EAL) range from EAL1 (functionally tested) to EAL7 (formally verified design and tested). For cryptographic hardware, the critical metric is the *vulnerability assessment* level:

- **AVA_VAN.3:** Resistance to attackers with “enhanced-basic” attack potential. Basic SCA testing.
- **AVA_VAN.4:** Resistance to attackers with “moderate” attack potential. Laboratory-grade SCA testing with power and EM analysis is mandatory.
- **AVA_VAN.5:** Resistance to attackers with “high” attack potential. Includes fault injection testing.

Products targeting the European market (particularly government and payment systems) typically require EAL4+ with AVA_VAN.4 or AVA_VAN.5. The CC evaluation is more flexible than FIPS 140-3 in that the Protection Profile can be tailored, but also more demanding at the highest levels: AVA_VAN.5 explicitly requires resistance to fault injection attacks, which FIPS 140-3 Level 3 addresses only indirectly through physical tamper resistance.

13.8.3 BSI Guidelines

The German Federal Office for Information Security (BSI) publishes Technical Reports that often set the de facto standard for European evaluations. For hardware security:

- **TR-03158:** Side-channel analysis resistance evaluation methodology [11]. Specifies the measurement setup, statistical tests, and pass/fail criteria.
- **AIS 20/31:** Evaluation of random number generators. Defines Physical True Random Number Generator classes (PTG.1, PTG.2, PTG.3) with increasing requirements on the stochastic model, health testing, and conditioning.

BSI requirements are roughly aligned with CC but often more prescriptive in methodology. For a PQC hardware module targeting both FIPS 140-3 Level 3 and Common Criteria EAL4+/AVA_VAN.4, the BSI TR-03158 methodology provides the most detailed blueprint for the SCA evaluation.

13.8.4 Certification Mapping

13.8.5 The Practical Path

For a vendor developing a PQC hardware module, the certification process involves several stages. First, algorithm validation: each PQC algorithm must pass NIST’s ACVP testing (for FIPS) or equivalent functional testing (for CC). Second, documentation: the security target (CC) or security policy (FIPS) must describe all cryptographic boundaries, key management, and security mechanisms. Third, laboratory evaluation:

Table 13.5 Mapping security requirements to certification framework clauses

Requirement	FIPS 140-3	Common Criteria	BSI
Algorithm correctness	KATs + ACVP (SP 800-140D)	ADV_FSP + ATE_COV	AIS 20/31 (RNG)
SCA resistance	ISO 17825 (Level 3+)	AVA_VAN.4/.5	TR-03158
FI resistance	Physical security (Level 3+)	AVA_VAN.5	TR-03158
Zeroisation	SP 800-140D	FCS_CKM.4	CC-aligned
RNG quality	SP 800-90A/B/C	FCS_RBG	AIS 20/31
Formal verification	Expected at Level 4	ADV_SPM (EAL5+)	EAL5+

an accredited lab performs SCA testing (power, EM, sometimes laser fault injection) against the claimed security level. Fourth, report and certification body review.

Timeline and cost. A FIPS 140-3 Level 3 certification typically takes 12–24 months from submission to certificate, depending on laboratory queue length and the number of findings that require remediation. Costs range from \$200K to \$500K for laboratory fees, plus internal engineering effort. Common Criteria EAL4+/AVA_VAN.4 is comparable in duration but often more expensive due to the depth of documentation required (the Security Target alone can exceed 200 pages). Delta certifications—updating an existing certificate for a new algorithm (e.g., adding ML-KEM to an already-certified module)—are faster and cheaper, typically 6–12 months.

The PQC-specific complication is that certification laboratories are still developing their testing methodologies for lattice-based and hash-based algorithms. The attack libraries used for SCA testing of AES and RSA do not directly apply to NTT-based schemes. Early movers in PQC hardware certification face the challenge—and the advantage—of helping to define the evaluation methodology.

13.8.6 What Certification Does Not Cover

It is worth being explicit about the limits of certification. FIPS 140-3 and Common Criteria certify a *specific version* of a *specific product* against a *specific set of requirements*. They do not guarantee security in perpetuity. New attack techniques—discovered after certification—may invalidate the security claims. The certification is a snapshot, not a proof.

For PQC specifically, several open questions remain in the certification landscape as of 2026:

- **NTT-specific SCA test vectors:** Certification labs have decades of experience testing AES and RSA/ECC implementations. Standardised test procedures for NTT-based leakage detection are still being developed.

- **Rejection sampling evaluation:** How to evaluate the timing channel from ML-DSA’s variable-time signing in a systematic, reproducible way.
- **Hash-based signature volume:** Whether the SCA evaluation for SLH-DSA should account for the cumulative trace volume over the key’s expected lifetime, not just a single operation.
- **DRBG-PQC composition:** Whether the DRBG and PQC modules can be certified separately or must be evaluated as a composed system.

These gaps mean that a vendor pursuing early PQC hardware certification must engage proactively with the evaluation laboratory, proposing test methodologies rather than waiting for published standards. The technical analysis in this chapter provides the foundation for those proposals.

The interplay between hardware security and certification creates a virtuous cycle: the threat analysis drives the countermeasure design, the countermeasure design structures the certification evidence, and the certification process validates (or challenges) the threat analysis. A well-designed PQC hardware module treats security analysis and certification planning as parallel activities, not sequential ones.

The next chapter addresses a different but equally practical challenge: how organisations migrate their existing cryptographic infrastructure to post-quantum algorithms. Where this chapter asked “how do we build a secure PQC implementation?”, the next asks “how do we deploy it into a world that still runs on RSA and elliptic curves?” The hardware security analysis developed here—particularly the certification requirements and the cross-component interaction model—will reappear when we discuss the constraints that certified hardware modules impose on migration timelines and hybrid protocol design.

Problems

13.1 CPA walkthrough.

An attacker targets a single coefficient b of the ML-KEM secret polynomial $\hat{s}$, where $q = 3329$. The targeted NTT butterfly computes $a' = a + \omega b \bmod q$ with public twiddle factor $\omega = 17$ and known input a .

- For each key hypothesis $k \in \{0, 1, \dots, 3328\}$, write the expression for the predicted intermediate value $v(k) = a + 17k \bmod 3329$.
- Using the Hamming weight model, compute $\text{HW}(v(k))$ for $k = 0, 1, 2, 3$ and $a = 1000$. Show that different hypotheses produce different predicted power values.
- Suppose the attacker collects $N = 5000$ traces with different values of a (drawn from decapsulations with different ciphertexts). For the correct hypothesis $k = b$, the predicted power values are correlated with the measured power at the targeted clock cycle. Explain why incorrect hypotheses $k \neq b$ produce near-zero correlation as N grows.
- If the SNR is 0.1 (typical for an FPGA), estimate the minimum number of traces needed for a successful first-order CPA using the rule of thumb $N \approx \alpha/\text{SNR}^2$

with $\alpha \approx 5$. How does this change if the implementation uses first-order arithmetic masking?

13.2 Masking cost estimation.

A hardware ML-KEM-768 decapsulation on FPGA runs at 100 MHz and takes 50 μ s unmasked.

- The NTT involves $256 \times \log_2 256/2 = 1024$ butterfly operations. Under first-order arithmetic masking, the NTT must be computed on two shares. If the multiplier is not duplicated (i.e., shares are processed sequentially), what is the NTT's new latency? If the multiplier is duplicated, what is the area cost?
- The decapsulation requires 6 Keccak invocations on secret data. DOM-1 adds one pipeline cycle per Keccak round. At 100 MHz, compute the additional latency for the 6 invocations.
- There are approximately 1536 A2B conversions (256 coefficients $\times$ 6 polynomials) during decapsulation. At 12 cycles per conversion, compute the total conversion overhead.
- Sum the overheads from (a)–(c) and express the masked decapsulation latency as a percentage increase over the unmasked baseline. Identify the dominant cost contributor.

13.3 SLH-DSA masking randomness budget.

SLH-DSA-128f requires approximately 35,000 Keccak invocations per signature.

- At DOM-1, each Keccak invocation consumes 38,400 bits of fresh randomness. Compute the total randomness per signature in bits and in megabytes.
- If the device signs at 10 signatures per second, what is the required randomness throughput in Mbit/s?
- A typical FPGA-based QRNG produces 100 Mbit/s of raw (post-health-test) output. Is this sufficient for DOM-1? For DOM-2 (which doubles the per-invocation randomness)?
- If the randomness source cannot keep up, the signing operation must stall. Explain why this stall is itself a potential side channel and propose a mitigation.

13.4 ML-DSA rejection sampling timing channel.

Consider ML-DSA-65, where each signing attempt succeeds with probability $p \approx 0.2$.

- The number of attempts follows a geometric distribution. Compute the expected number of attempts and the standard deviation.
- If each attempt takes 100 μ s, and the attacker can measure total signing time with 1 μ s precision, how many distinct timing bins can the attacker observe?
- Explain why constant-time padding to $N_{\max}$ iterations eliminates this channel. Compute $N_{\max}$ such that $\Pr[\text{iterations} > N_{\max}] < 2^{-128}$.
- What is the worst-case signing latency with this padding, and how does it compare to the expected latency without padding?

13.5 Certification mapping exercise.

A vendor is developing a PQC hardware module that implements ML-KEM-768 and ML-DSA-65. The module targets FIPS 140-3 Level 3 certification and Common Criteria EAL4+/AVA_VAN.4.

- (a) List the specific countermeasures (from Sects. 13.3–13.5) that are mandatory to satisfy the SCA resistance requirements at these certification levels.
- (b) The vendor’s DRBG uses SHAKE-256 (XDRBG construction). Which certification framework clause governs the DRBG’s algorithm correctness? Which governs its SCA resistance?
- (c) The module shares a single Keccak core between the DRBG and both PQC algorithms. Identify the cross-component risks (Sect. 13.7) that must be documented in the Security Target and tested by the evaluation laboratory.
- (d) If the vendor later adds SLH-DSA-128f support, what additional security analysis is required? Can this be handled as a delta certification?

13.6 The circular dependency.

A PQC hardware module uses a DRBG with DOM-1 masked Keccak. The masking requires 38,400 bits of fresh randomness per Keccak invocation.

- (a) Explain why the DRBG cannot use its own output as masking randomness for its own Keccak core.
- (b) The module has access to a QRNG that produces 200 Mbit/s of raw output. The DRBG processes 100 Keccak invocations per second. Compute the randomness consumed by the DRBG’s masking and the fraction of the QRNG’s output this represents.
- (c) The remaining QRNG output is conditioned by the DRBG for use as masking randomness by the PQC modules. If ML-KEM decapsulation requires 6 masked Keccak invocations and runs at 1000 decapsulations per second, compute the total masking randomness demand for ML-KEM’s Keccak alone. Is the conditioned output from the DRBG sufficient?
- (d) Propose an architecture that resolves the circular dependency while ensuring that both the DRBG and PQC modules receive adequate masking randomness.

Chapter 14

Migration Strategies

Abstract The post-quantum transition is not merely an algorithm swap—it requires rethinking protocols, infrastructure, and organizational processes. This chapter provides a practical framework for migrating to post-quantum cryptography, covering hybrid deployment modes, protocol updates for TLS and code signing, blockchain-specific challenges, organizational roadmaps aligned with CNSA 2.0 and European guidance, and the timeline pressures captured by Mosca’s inequality.

14.1 Hybrid Modes

The safest path to post-quantum security runs through *hybrid cryptography*: combining a classical and a post-quantum algorithm so that the overall construction remains secure as long as *either* component is secure. This is not overcaution—it is learning from history. The Dual EC DRBG episode (2006–2013) demonstrated that even standardized algorithms can conceal fatal weaknesses. Hybrid modes protect against classical breaks of the new PQC primitive, implementation flaws in immature code, quantum timeline uncertainty, and future changes to standards.

14.1.1 Composite KEMs

A *composite KEM* runs two key encapsulation mechanisms in parallel and combines their shared secrets through a key derivation function.

Definition 14.1 (Composite KEM) Let $\text{KEM}_1 = (\text{KeyGen}_1, \text{Encaps}_1, \text{Decaps}_1)$ and $\text{KEM}_2 = (\text{KeyGen}_2, \text{Encaps}_2, \text{Decaps}_2)$ be two KEMs. The *composite KEM* KEM_H operates as follows:

1. KeyGen_H : Run $(pk_1, sk_1) \leftarrow \text{KeyGen}_1$ and $(pk_2, sk_2) \leftarrow \text{KeyGen}_2$. Return $pk_H = (pk_1, pk_2)$, $sk_H = (sk_1, sk_2)$.

2. $\text{Encaps}_H(pk_H)$: Compute $(c_1, K_1) \leftarrow \text{Encaps}_1(pk_1)$ and $(c_2, K_2) \leftarrow \text{Encaps}_2(pk_2)$. Return $c_H = (c_1, c_2)$ and $K_H = \text{KDF}(K_1 \| K_2)$.
3. $\text{Decaps}_H(sk_H, c_H)$: Recover $K_1 \leftarrow \text{Decaps}_1(sk_1, c_1)$ and $K_2 \leftarrow \text{Decaps}_2(sk_2, c_2)$. Return $K_H = \text{KDF}(K_1 \| K_2)$.

The critical observation is that the KDF (e.g., HKDF-SHA256) is modelled as a random oracle: even if one shared secret is completely known to the adversary, the output remains pseudorandom as long as the other secret has sufficient entropy.

Theorem 14.1 (IND-CCA2 Security of Composite KEMs) *Let KEM_1 be (t_1, ϵ_1) -IND-CCA2 secure and KEM_2 be (t_2, ϵ_2) -IND-CCA2 secure. In the random-oracle model for KDF, the composite KEM_H is (t, ϵ) -IND-CCA2 secure with $t = \min(t_1, t_2)$ and $\epsilon \leq \epsilon_1 + \epsilon_2$. In particular, KEM_H is IND-CCA2 secure if either component is.*

Proof sketch. Assume an adversary $\mathcal{A}$ breaks KEM_H with advantage ϵ . We construct a hybrid argument over two games: Game 0 replaces K_1 with a uniformly random key (reduction to KEM_1), and Game 1 then replaces K_2 (reduction to KEM_2). In the random-oracle model, $\text{KDF}(K_1 \| K_2)$ is indistinguishable from random whenever at least one input is random, so $\epsilon \leq \epsilon_1 + \epsilon_2$. See Bindel et al. [8] for the full proof.

14.1.2 Dual Signatures

For digital signatures, the hybrid approach produces two independent signatures and requires both to verify:

Definition 14.2 (Composite Signature) Given signature schemes $\Sigma_1 = (\text{KeyGen}_1, \text{Sign}_1, \text{Verify}_1)$ and $\Sigma_2 = (\text{KeyGen}_2, \text{Sign}_2, \text{Verify}_2)$, the composite scheme signs as $\sigma_H = (\text{Sign}_1(sk_1, m), \text{Sign}_2(sk_2, m))$ and accepts if and only if *both* components verify.

This “AND” composition guarantees existential unforgeability (EUF-CMA) if either component scheme is EUF-CMA secure. The price is paid in signature and public-key size: for an RSA-2048 + ML-DSA-65 combination, the composite signature is $256 + 3,309 = 3,565$ bytes, with a composite public key of $294 + 1,952 = 2,246$ bytes.

Figure 14.1 illustrates the nested-layer structure common to both composite KEMs and composite signatures: each cryptographic layer independently protects the data, so an adversary must break both to compromise confidentiality or integrity.

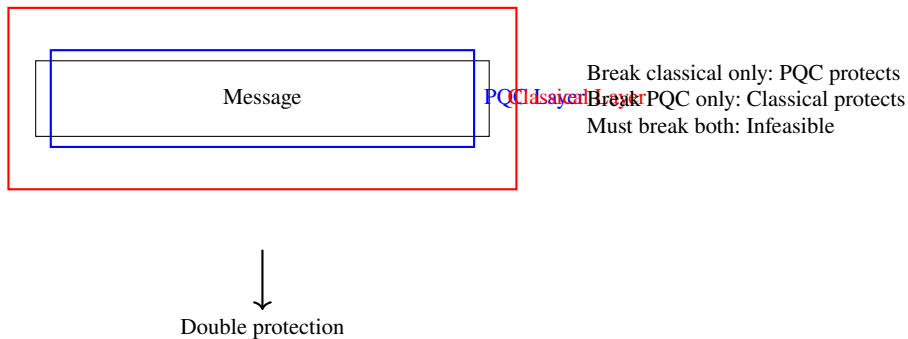


Fig. 14.1 Nested hybrid layers. The message is protected by both a post-quantum and a classical cryptographic layer; security is maintained as long as at least one layer remains unbroken.

14.1.3 X.509 Hybrid Certificates

Deploying hybrid signatures within the X.509 public-key infrastructure requires encoding two public keys and two signatures into a single certificate. Three strategies have been proposed:

1. **Dual-key certificate:** A single certificate carries both public keys and is signed with both CA keys. The IETF draft `draft-ounsworth-pq-composite-sigs` defines composite OIDs for this approach.
2. **Catalyst certificate:** The classical certificate includes the PQC public key and signature in a non-critical X.509v3 extension. Legacy clients ignore the extension; PQC-aware clients verify both.
3. **Paired certificates:** Two separate certificates—one classical, one PQC—are bound by a shared serial number. The server sends both; the client verifies whichever it supports.

The catalyst approach offers the best backward compatibility: existing TLS stacks parse the certificate without modification, while updated clients gain quantum resistance. The cost is roughly 5–6 KB of additional data per certificate (dominated by the ML-DSA public key and CA signature).

14.1.4 Performance Impact

The computational overhead of hybrid KEMs is modest ($< 2\times$), because ML-KEM operations are fast. The dominant cost is *bandwidth*: the combined ciphertext and public-key material adds roughly 2 KB per handshake, which matters on constrained links but is negligible on broadband.

Table 14.1 Hybrid KEM overhead: X25519 + ML-KEM-768

Operation	X25519	ML-KEM-768	Hybrid	Factor
Key generation	45 μ s	35 μ s	80 μ s	1.8×
Encapsulation	45 μ s	44 μ s	89 μ s	2.0×
Decapsulation	45 μ s	40 μ s	85 μ s	1.9×
Ciphertext size	32 B	1,088 B	1,120 B	35×

14.2 TLS Migration

TLS 1.3 protects billions of HTTPS connections daily and is the highest-priority migration target. Its clean key-exchange design—a single round-trip with ephemeral Diffie–Hellman—makes post-quantum integration relatively straightforward compared with older protocols.

14.2.1 TLS 1.3 with Post-Quantum Key Exchange

TLS 1.3 negotiates key exchange via *named groups* advertised in the `supported_groups` extension. Adding a post-quantum or hybrid KEM requires:

1. Assigning a new named-group code point (e.g., `0x6399` for X25519Kyber768).
2. Extending buffer sizes: the `ClientHello` key-share grows from 32 bytes (X25519) to $\approx 1,216$ bytes; the `ServerHello` encapsulation adds $\approx 1,088$ bytes.
3. Combining the shared secrets via the TLS 1.3 key schedule: $SS_{\text{hybrid}} = \text{HKDF-Extract}(K_{\text{X25519}} \| K_{\text{ML-KEM}})$.

Crucially, the handshake *message count* does not change—the post-quantum material is embedded in the existing `key_share` extension—so the protocol retains its 1-RTT structure.

14.2.2 Chrome and Cloudflare Experiments

Example 14.1 (CECPQ2 and Beyond) Google and Cloudflare have conducted the largest post-quantum TLS experiments to date:

- **2019:** CECPQ2 deployed hybrid ECDH + NTRU-HRSS in Chrome, covering $\sim 30\%$ of TLS connections to Google servers. Median latency increase: 1 ms. User complaints: zero.
- **2022:** Cloudflare switched to hybrid X25519 + Kyber-768, making PQC available across 20+ million websites.
- **2023–2024:** Chrome enabled X25519Kyber768 by default. By late 2024, over 10% of all global HTTPS traffic used a post-quantum key exchange.

Table 14.2 TLS handshake size and latency overhead

Component	Classical	PQC/Hybrid	Increase
Key exchange	96 B	2,336 B	24×
Certificate chain (3 certs)	3,150 B	17,235 B	5.5×
Total handshake	~3.5 KB	~20 KB	5.7×
Median latency	baseline	+1.3 ms	—
95th-pct latency	baseline	+21 ms	—

These experiments demonstrated that Internet-scale hybrid deployment is feasible with negligible user-facing impact.

14.2.3 X25519Kyber768

X25519Kyber768 (also known as X25519MLKEM768 after the FIPS 203 [50] renaming) is the first hybrid KEM deployed at scale. It concatenates the 32-byte X25519 shared secret with the 32-byte ML-KEM-768 shared secret and feeds the 64-byte result into the TLS 1.3 HKDF key schedule. The combined ciphertext is $32 + 1,088 = 1,120$ bytes—well within a single TCP segment, avoiding handshake fragmentation in most network conditions.

14.2.4 Certificate Chain Sizes and Latency

While key exchange adds ~2 KB, the larger concern is *certificate chains*. A three-certificate chain with ML-DSA-65 signatures grows from ~3 KB (RSA-2048) to ~17 KB, a 5.5× increase.

On desktop broadband, the extra latency is invisible. On mobile networks with high round-trip times, the 95th-percentile overhead can reach 20–90 ms, primarily due to TCP-level fragmentation and retransmission when the handshake exceeds the initial congestion window. Session resumption via TLS 1.3 PSK mode avoids this cost entirely and becomes *more important* in a post-quantum world.

The middlebox problem. Corporate firewalls and deep-packet-inspection appliances often impose hard limits on TLS record sizes (commonly 8 KB or 16 KB). PQC certificate chains that exceed these limits are silently dropped, causing mysterious connection failures. Early deployers report 2–6 weeks of debugging before identifying the root cause. The fix is to fragment certificates across multiple TLS records or to use certificate compression (RFC 8879).

14.3 Code Signing and Software Supply Chain

Code signing presents challenges distinct from TLS: signatures are verified *years* after creation, backward compatibility with legacy verifiers is critical, and signature size directly affects software distribution costs.

14.3.1 Firmware Updates

Embedded devices—IoT sensors, medical implants, industrial controllers—verify firmware signatures using public keys burned into hardware at manufacture. These keys may need to remain valid for 10–20 years, making them high-priority targets for “harvest now, decrypt later” attacks on confidentiality and, more critically, for future signature-forgery attacks on integrity. A quantum adversary who can forge a firmware signature can push malicious updates to every device in the fleet.

Stateful hash-based signature schemes (LMS, XMSS), standardized in NIST SP 800-208, are attractive for firmware signing because their security rests purely on hash-function properties and they produce compact signatures (~2.5 KB for LMS with $H = 20$). The main operational burden is state management: each private key must track which one-time keys have been used, and reuse is catastrophic. For CA root keys that sign infrequently, this burden is manageable.

14.3.2 Package Managers and Repository Signing

Software supply chains (npm, PyPI, apt, Homebrew) sign repository metadata and individual packages. A single compromised signing key can inject malicious code into millions of systems. Migration considerations include:

- **Dual signatures:** Repositories can distribute both classical and PQC signatures in parallel. Clients verify whichever their toolchain supports.
- **Transparency logs:** Systems like Sigstore already decouple signatures from distribution, easing algorithm transitions.
- **Threshold signing:** Multi-party signing ceremonies (common for root keys) must be updated to use PQC-compatible threshold protocols.

14.3.3 The Signature Size Challenge

For a typical software update containing 1,000–10,000 signed files, switching from RSA-2048 to ML-DSA-65 increases total signature overhead from ~250 KB to ~3.3 MB per update. SLH-DSA offers the advantage of minimal public-key size and conservative, hash-only security assumptions, but its signatures are 2–3× larger than ML-DSA. The

Table 14.3 Signature sizes for code-signing candidates

Scheme	Signature	Public key	Security basis
RSA-2048	256 B	294 B	Factoring
ECDSA-P256	64 B	64 B	ECDLP
ML-DSA-65	3,309 B	1,952 B	Module-LWE
SLH-DSA-128s	7,856 B	32 B	Hash functions
LMS ($H=20$)	2,508 B	56 B	Hash functions

practical choice depends on whether the deployment is bandwidth-constrained (favour ML-DSA) or demands maximal cryptographic conservatism (favour SLH-DSA or LMS for root keys).

The signature-size tension is annoying for software distribution; for blockchain systems, where every byte is replicated across thousands of nodes and stored forever, it becomes an existential design constraint.

14.4 Blockchain and Cryptocurrencies

Blockchain systems face a “perfect storm” of post-quantum migration challenges: immutable transaction histories, size-sensitive economics, decentralized governance, and catastrophic failure modes.

14.4.1 ECDSA Vulnerability

Bitcoin, Ethereum, and most cryptocurrencies derive address security from ECDSA over the `secp256k1` curve. Shor’s algorithm recovers private keys from public keys in polynomial time. In Bitcoin, the public key is exposed whenever coins are *spent* from a pay-to-public-key-hash (P2PKH) address. An adversary with a cryptographically relevant quantum computer (CRQC) could:

1. Monitor the mempool for pending transactions that reveal a public key.
2. Extract the private key before the transaction is confirmed.
3. Submit a competing transaction redirecting the funds.

More dangerously, coins held in addresses where the public key was previously exposed (estimated at over 4 million BTC, roughly 20% of all supply) are vulnerable even without monitoring the mempool.

14.4.2 Migration Challenges

The ECDSA vulnerability is clear enough; what makes blockchain migration uniquely difficult is that every proposed remedy collides with a core design property.

Consider immutability first. The blockchain's defining feature—that no past record can be altered—becomes a liability when past records contain quantum-vulnerable signatures. Historical ECDSA signatures cannot be replaced, only supplemented with new PQC protections going forward. This stands in sharp contrast to a TLS deployment, where rotating to new certificates immediately protects all future sessions and the old certificates simply expire.

Transaction size compounds the problem. Replacing ECDSA (~72 B per signature) with ML-DSA-44 (~2,420 B) inflates a simple Bitcoin transaction from ~250 B to ~3,500 B—a 14× increase. Block capacity drops from ~4,000 to ~285 transactions, fees rise proportionally, and full-node storage grows from ~50 GB/year to ~700 GB/year. In a system where every full node stores the complete history, this is not merely a bandwidth cost; it reshapes the economics of participation.

Governance adds a further constraint. Any signature-scheme change requires a hard fork or, at minimum, a soft fork with broad community agreement. In decentralized networks, achieving such agreement can take years—the Bitcoin block-size debate (2015–2017) is a cautionary precedent.

Finally, address migration demands active participation from every user. Funds must be moved to quantum-safe addresses through on-chain transactions, which means every wallet holder must act. Funds in dormant wallets—including those belonging to lost keys, estimated at 3–4 million BTC—remain permanently vulnerable, creating a systemic risk that no protocol upgrade can eliminate unilaterally.

14.4.3 Frozen Funds Risk

Coins whose owners cannot or will not migrate before a CRQC arrives face outright theft. The community has proposed several mitigation strategies, none of them painless.

The most conservative approach is *commit-reveal*: users commit a hash of a PQC public key to the blockchain now, then reveal the actual key only when spending. Because the quantum-vulnerable classical public key never enters the mempool, a CRQC operator cannot intercept and forge a competing transaction. The limitation is clear—commit-reveal protects future transactions but does nothing for the millions of addresses whose public keys were exposed in past spends.

A second line of defence is the *quantum-safe sidechain*: a parallel chain secured by PQC signatures that accepts bridged assets from the main chain. Users lock funds on the classical chain and receive equivalent tokens on the sidechain, migrating at their own pace without requiring a hard fork. The approach is technically sound but adds operational complexity and splits network effects between two chains.

The most drastic proposal is *deadline-based freezing*. After a community-agreed date, the network would reject transactions from addresses that have not migrated to

PQC keys. This protects the system from quantum theft but permanently locks affected funds—including coins belonging to deceased holders or lost wallets. The measure is politically explosive; it amounts to a collective decision to sacrifice some holders' assets for the security of the whole, a trade-off with no precedent in Bitcoin's history.

Example 14.2 (Ethereum's Account Abstraction Path) Ethereum's account-abstraction roadmap (EIP-4337 and successors) decouples signature verification from the protocol layer, allowing smart-contract wallets to adopt PQC signature schemes without a hard fork. The planned sequence is: enable smart-contract wallets (2024–2025), support hybrid signatures (2025–2027), incentivize migration via gas discounts (2027–2029), and disable raw ECDSA for new transactions (2030+). This is the most credible blockchain migration plan to date, but it still requires years of coordinated effort.

Blockchain migration illustrates, in extreme form, the tension between technical necessity and organizational inertia. The next section generalizes this tension into a structured roadmap applicable to any organization, blockchain or otherwise.

14.5 Organizational Roadmap

The preceding sections examined specific migration targets—TLS, code signing, blockchain—each with its own technical constraints. But even perfect technical solutions fail without organizational execution. Organizations must run a structured migration program that touches procurement, engineering, compliance, and operations in coordinated sequence. The six-phase roadmap below distills guidance from NIST SP 1800-38, the NSA's CNSA 2.0 advisory, and European agency recommendations into a practical framework. It is ordered by dependency: each phase produces the inputs the next phase requires.

14.5.1 Phase 1: Cryptographic Inventory

No migration can succeed without knowing what must be moved. The first phase is a comprehensive discovery of every use of public-key cryptography across the organization: TLS certificates, code-signing keys, VPN configurations, SSH keys, API tokens, hardware security modules, embedded-device firmware, and third-party dependencies. The analogy to physical logistics is apt—before relocating a warehouse, one inventories every pallet; skipping this step guarantees that critical items are left behind.

Automated scanning tools (Venafi, Keyfactor, or open-source cbom generators) provide a starting point, but they rarely find everything. Manual review is essential, particularly for shadow IT and legacy systems maintained by teams outside central security. A typical medium enterprise discovers that 40–50% of its cryptographic assets were unknown to the security team before the inventory—a sobering figure that explains why so many migrations stall in later phases.

Table 14.4 Risk classification matrix

Data category	Lifetime	Risk level	Action
Government / military	25–50 years	Critical	Immediate
Healthcare records	50–100 years	Critical	Immediate
Financial records	7–10 years	High	By 2027
Trade secrets / IP	10–20 years	High	By 2027
Personal data (GDPR)	Lifetime	Medium	By 2029
Ephemeral session keys	Hours	Low	By 2031

With a complete inventory in hand, the organization can move to risk assessment: not all assets face the same quantum exposure, and treating them uniformly wastes resources.

14.5.2 Phase 2: Risk Assessment

The inventory tells us *what* the organization has; risk assessment determines *what is at stake*. Each asset is classified along two dimensions: *data sensitivity lifetime*—how long must the data remain confidential or its signatures remain trusted?—and *migration difficulty*—how hard is it to change the algorithm? Assets that score high on both dimensions (long-lived secrets protected by deeply embedded cryptography) demand immediate attention; ephemeral session keys on modern, patchable servers can wait.

14.5.3 Phase 3: Prioritize

Risk assessment produces a ranked list; prioritization turns that list into a work plan. The principle is straightforward—start with the highest-risk, longest-lived data—but the practical ordering requires judgment about dependencies and organizational capacity.

VPN tunnels and TLS sessions protecting classified or regulated data typically top the list, because they are actively exposed to harvest-now-decrypt-later attacks and Mosca’s inequality (Sect. 14.6.1) is already violated for multi-decade secrets. Root CA and code-signing keys follow closely: they are long-lived, their compromise has cascading impact, and their migration is largely independent of application-layer changes. Data-at-rest encryption keys for archives with multi-decade retention come next, followed by customer-facing web services, which benefit from well-understood migration paths (Sect. 14.2) and high public visibility.

The prioritized list feeds directly into the testing phase, where each migration path is validated before it touches production traffic.

14.5.4 Phase 4: Test

Before any PQC algorithm touches production traffic, it must survive contact with reality. Hybrid modes are deployed first in staging and pre-production environments, where failures are cheap and reversible.

Testing spans several dimensions. Functional correctness is the baseline: do the hybrid handshakes complete, and do the resulting session keys match on both sides? Latency and throughput under load reveal whether the larger key material degrades performance beyond acceptable thresholds—the middlebox problem described in Sect. 14.2.4 is routinely discovered at this stage. Interoperability testing with legacy clients and middleboxes is critical, because a migration that breaks 2% of connections will be rolled back before it protects anyone. Failure-mode behavior must be explicitly validated: does the system fall back gracefully to classical-only when the PQC component is unavailable, or does it fail hard? Finally, key and certificate lifecycle operations—renewal, revocation, rotation—must work end-to-end, because a migration that cannot maintain itself is a migration that will decay.

Testing failures are not setbacks; they are the point. Every incompatibility caught here is one that does not become a 2 AM incident in production.

14.5.5 Phase 5: Deploy

Deployment follows the same graduated-rollout discipline used for any high-risk infrastructure change—the difference is that the stakes include long-term cryptographic exposure, not just transient outages.

The rollout begins with a canary deployment to roughly 1% of traffic, chosen to include a representative mix of client populations. At this scale, gross incompatibilities surface—broken middleboxes, misconfigured load balancers, clients that choke on larger handshakes—without affecting the majority of users. Once the canary is stable, traffic shifts to 10% (early adopters), where performance under realistic load is validated against the baselines established in testing. The move to 50% exercises the system at scale: tail latencies, connection-retry rates, and certificate-chain failures that appear only under heavy load become visible. Full deployment at 100% completes the migration, but the classical algorithm is not removed—it remains available as a fallback until the organization is confident that every client ecosystem has adapted.

Throughout each stage, monitoring dashboards track handshake failure rates, latency distributions, and certificate-validation errors. Any regression beyond predefined thresholds triggers an automatic rollback to the previous traffic split.

Table 14.5 CNSA 2.0 transition deadlines (NSA, 2022)

Use case	Algorithm	Deadline
Software / firmware signing	CNSA 2.0 exclusive	2025
Web servers / cloud services	CNSA 2.0 preferred	2025
Traditional networking (VPN, routers)	CNSA 2.0 preferred	2026
Operating systems	CNSA 2.0 support	2027
Custom / embedded applications	CNSA 2.0 exclusive	2030
Legacy infrastructure	CNSA 2.0 exclusive	2033

14.5.6 Phase 6: Verify and Maintain

A migration that declares victory at 100% deployment and moves on has failed. Cryptographic infrastructure is a living system: new services are provisioned, dependencies are updated, developers under deadline pressure reach for the defaults their frameworks provide—and those defaults may still be classical.

Continuous compliance therefore requires several ongoing disciplines. Monitoring detects *algorithm drift*, where newly deployed services silently revert to classical-only configurations. Automated alerting fires when certificates or keys use deprecated algorithms, catching regressions before they reach production. Periodic reassessment accounts for advances in cryptanalysis: a parameter set considered safe in 2026 may need revision by 2030 as lattice-reduction techniques improve.

The deepest lesson of this phase is the importance of *crypto-agility*—the architectural ability to swap algorithms without redesigning protocols or redeploying hardware. Crypto-agility is not a feature to add later; it is a design principle that must be embedded from Phase 1 onward. Organizations that achieve it will navigate not only the post-quantum transition but every subsequent cryptographic migration with far less friction.

The CNSA 2.0 timeline and European guidance that follow provide the external deadlines against which these six phases must be scheduled.

14.5.7 CNSA 2.0 Timeline

The six phases above describe *how* to migrate; the CNSA 2.0 timeline specifies *when*. The NSA's Commercial National Security Algorithm Suite 2.0 sets mandatory deadlines for U.S. national-security systems, and these deadlines increasingly serve as de facto benchmarks for the private sector as well:

14.5.8 European Guidance

CNSA 2.0 is a U.S.-centric framework; European agencies set their own timelines, informed by different threat models and regulatory structures, but converging on similar urgency.

ANSSI (France) recommends hybrid modes as mandatory during the transition period and requires PQC for new systems handling classified data from 2025 onward. Notably, ANSSI explicitly endorses FrodoKEM alongside ML-KEM as a conservative alternative for high-security applications—reflecting a preference for schemes with stronger theoretical security reductions, even at a performance cost. BSI (Germany) recommends beginning migration planning immediately, with hybrid deployments by 2025 and full PQC by 2030; BSI requires a “defence in depth” approach combining lattice-based and hash-based schemes, ensuring that no single cryptanalytic breakthrough compromises the entire infrastructure. At the EU level, ENISA published a post-quantum roadmap in 2022 urging member states to conduct cryptographic inventories by 2025 and complete critical-infrastructure migration by 2030.

The convergence across these agencies—American, French, German, European—is striking. Despite differing institutional cultures and threat priorities, all place the start of serious migration activity no later than 2025 and demand completion for critical systems by 2030. Whether these deadlines are achievable depends on the timeline pressures we examine next.

14.6 Timeline and Urgency

The roadmap and regulatory deadlines of the previous section raise an obvious question: how late is too late? The answer depends not on policy but on arithmetic—a simple inequality that formalizes the harvest-now-decrypt-later threat into a planning tool.

14.6.1 Mosca’s Inequality

Michele Mosca formalized the urgency of post-quantum migration with a simple but powerful inequality:

Definition 14.3 (Mosca’s Inequality) Let:

- x = the number of years the data must remain secure,
- y = the number of years needed to migrate the system to PQC,
- z = the number of years until a CRQC exists.

If $x + y > z$, then the data is *already at risk today*, because an adversary can harvest ciphertext now and decrypt it once a CRQC is available before the confidentiality requirement expires.

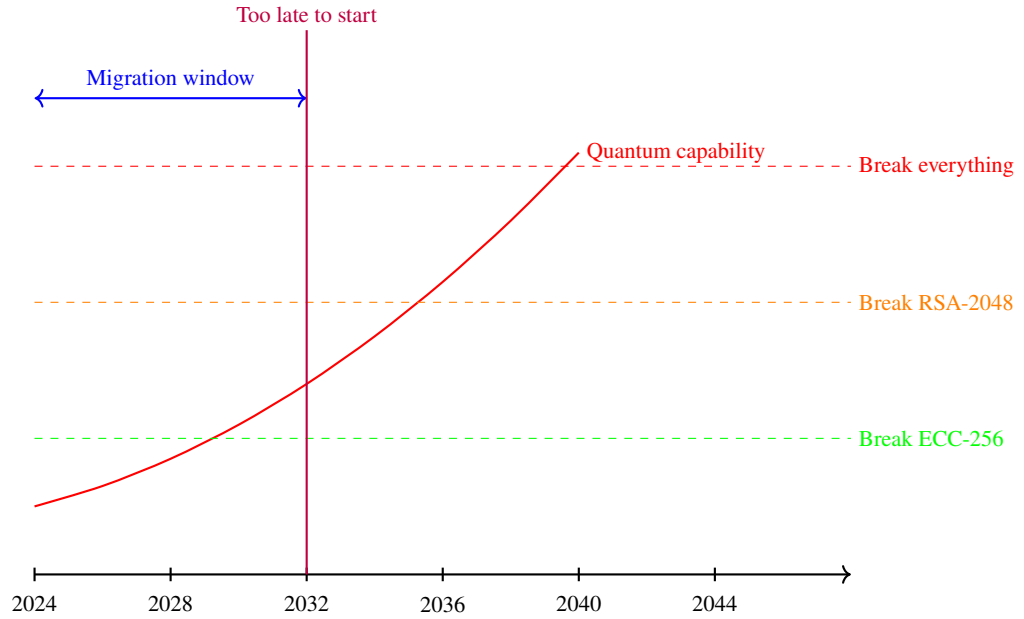


Fig. 14.2 Projected quantum capability versus classical cryptographic thresholds. The migration window closes once quantum hardware crosses the first dashed threshold; organisations that have not begun transitioning by that point face irreversible exposure.

Example 14.3 (Applying Mosca’s Inequality) A hospital system stores patient records that must remain confidential for 50 years ($x = 50$). Their IT department estimates that a full cryptographic migration will take 5 years ($y = 5$). Assuming a CRQC arrives by 2035 ($z = 2035 - 2026 = 9$), we have $x + y = 55 \gg 9 = z$. The inequality has been violated for decades: any patient record encrypted before 2026 with a classical algorithm is exposed to a harvest-now-decrypt-later attack.

The uncomfortable implication: for any data with a multi-decade confidentiality requirement, migration should have started years ago. Figure 14.2 plots projected quantum capability against the cryptographic thresholds for current algorithms, making the shrinking migration window visually concrete.

14.6.2 Sector-Specific Deadlines

Mosca’s inequality is most useful when applied to concrete sectors, because different industries face radically different values of x , y , and z :

Table 14.6 Sector-specific Mosca analysis (assuming CRQC by 2035)

Sector	x (yr)	y (yr)	$x + y$	Must start by
Intelligence	50	8	58	Already overdue
Healthcare	50	5	55	Already overdue
Finance	10	4	14	2026
Telecoms	5	3	8	2027
Retail / web	2	2	4	2031

14.6.3 Cost of Delay

Delaying migration incurs compounding costs:

1. **Harvest-now exposure grows daily.** Every additional day of classical-only encryption adds one more day of ciphertext to the adversary's stockpile.
2. **Technical debt accumulates.** New systems deployed today without PQC support become tomorrow's "legacy infrastructure"—the most expensive category to migrate.
3. **Talent scarcity.** PQC expertise is limited. Organizations that wait will compete for the same engineers during a compressed timeline, driving up costs.
4. **Regulatory risk.** Once mandates like CNSA 2.0 take effect, non-compliant systems face operational restrictions or loss of certification.

Lessons from previous cryptographic migrations. The AES transition (2001–2010) succeeded because it offered a drop-in replacement with better performance. The IPv6 transition (1998–present, still incomplete) stalled because it required infrastructure changes with no immediate benefit. PQC migration resembles IPv6 more than AES: the threat is not yet tangible, the changes are invasive, and the performance trade-offs are unfavorable. Without deliberate urgency, a 20-year transition is a realistic worst case.

Problems

14.1 Apply Mosca's inequality to the following scenario: a hospital system stores patient records that must remain confidential for 50 years. Their IT department estimates that a full cryptographic migration will take 5 years. Assuming a cryptographically relevant quantum computer arrives in 2035:

- (a) By what year must the migration begin?
- (b) Repeat the analysis for a retail company whose transaction data must remain confidential for only 2 years, with a migration time of 18 months.
- (c) In which case is the urgency greater, and why?

14.2 Design a hybrid KEM that combines X25519 and ML-KEM-768. Specify how the shared secrets are combined (concatenation followed by a KDF), and argue—using the structure of Theorem 14.1—that your construction is IND-CCA2 secure if either component KEM is secure.

14.3 A TLS server presents a certificate chain of depth 3 (end-entity, intermediate CA, root CA). Compute the total chain size in bytes for each of the following configurations:

- (a) All certificates use RSA-2048 (public key: 294 B, signature: 256 B, overhead: 200 B per cert).
- (b) All certificates use ML-DSA-65 (public key: 1,952 B, signature: 3,309 B, overhead: 200 B per cert).
- (c) A hybrid catalyst certificate (RSA public key + ML-DSA public key and signature in an extension).

Discuss which configuration exceeds a typical TCP initial congestion window of 14,600 bytes and what mitigations are available.

14.4 Bitcoin transactions currently average 250 bytes with an ECDSA signature of 72 bytes. Suppose ECDSA is replaced by ML-DSA-44 (signature: 2,420 B; public key: 1,312 B).

- (a) Compute the new average transaction size.
- (b) If the block size limit remains 4 MB, estimate the new maximum number of transactions per block.
- (c) Discuss two alternative approaches that could mitigate the throughput reduction.

14.5 An enterprise operates 3,000 TLS endpoints, 200 code-signing certificates, and 50 VPN gateways. Using the six-phase roadmap of Sect. 14.5:

- (a) Identify which assets should be migrated first and justify your prioritization using Mosca’s inequality.
- (b) Propose a 4-year timeline with milestones for each phase.
- (c) Explain why crypto-agility (Phase 6) is essential even after migration is “complete.”

Chapter 15

Advanced Cryptographic Primitives

Abstract Lattice-based cryptography enables capabilities far beyond encryption and signatures. This chapter introduces fully homomorphic encryption, zero-knowledge proofs, and secure multiparty computation—primitives that were theoretical curiosities until lattice theory made them practical. The noise dynamics of LWE-based schemes play a central role throughout, and the techniques for managing noise reappear in different guises across all three primitives. The chapter closes with a survey of lattice-based advanced constructions that point toward a richer cryptographic future.

15.1 Fully Homomorphic Encryption

Imagine encrypting your medical records before uploading them to the cloud, having the cloud run a diagnostic algorithm on that encrypted data, and receiving encrypted results that only you can decrypt. The cloud processes your data faithfully yet learns nothing about it. This seemingly impossible capability—*fully homomorphic encryption* (FHE)—has moved from theoretical breakthrough to practical deployment in under two decades, powered by the same lattice problems that underpin post-quantum security.

15.1.1 The Concept: Computing on Encrypted Data

Definition 15.1 (Homomorphic Encryption) An encryption scheme $(\text{Gen}, \text{Enc}, \text{Dec}, \text{Eval})$ is *fully homomorphic* if it supports an efficient evaluation algorithm Eval such that for any circuit f and any plaintexts $m_1, \dots, m_k$:

$$\text{Dec}(sk, \text{Eval}(f, \text{Enc}(pk, m_1), \dots, \text{Enc}(pk, m_k))) = f(m_1, \dots, m_k), \quad (15.1)$$

where Eval works without access to the secret key sk .

It suffices that Eval supports addition and multiplication of ciphertexts, since these two operations are *universal*—any Boolean or arithmetic circuit can be built from

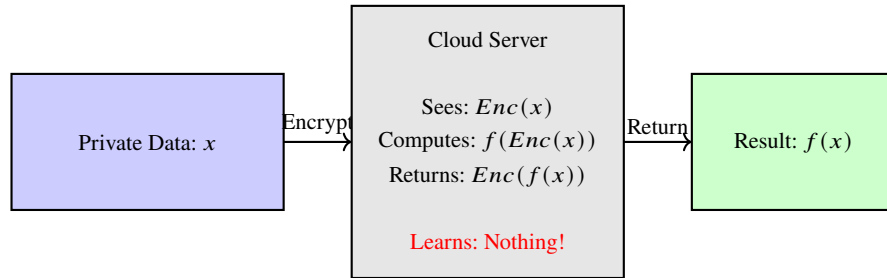


Fig. 15.1 Fully homomorphic encryption: computing on encrypted data without decryption. The cloud processes ciphertexts and returns an encrypted result, learning nothing about the underlying data.

them. Schemes supporting only one operation (e.g., unpadded RSA is multiplicatively homomorphic) are called *partially homomorphic*; schemes supporting both but only to a bounded depth are called *somewhat homomorphic* (SHE).

Why FHE matters. Public-key encryption protects data *at rest* and *in transit*. FHE protects data *in use*. This is the missing piece for privacy-preserving cloud computing: a hospital can outsource genomic analysis, a bank can outsource fraud detection, and a government can outsource census statistics—all without exposing raw data to the computing party.

Figure 15.1 summarises the FHE workflow: a client encrypts private data, a cloud server computes on the ciphertext without ever seeing the plaintext, and the client decrypts the result to obtain the output of the computation.

15.1.2 Gentry’s Blueprint: Somewhat HE Plus Bootstrapping

The existence of FHE was an open problem for over 30 years, from Rivest, Adleman, and Dertouzos’s 1978 conjecture to Gentry’s 2009 resolution [20]. Gentry’s construction follows a two-step blueprint that remains the template for all modern FHE schemes:

1. **Build a somewhat homomorphic scheme.** Construct an encryption scheme that supports both homomorphic addition and multiplication, but only for circuits up to some bounded depth L .
2. **Bootstrap to full homomorphism.** Use the SHE scheme to homomorphically evaluate its *own decryption circuit*, thereby “refreshing” a noisy ciphertext into a fresh one. If the decryption circuit has depth less than L , one application of *Eval* resets the noise without exceeding the scheme’s capacity, and the process can be repeated indefinitely.

The bootstrapping step is the conceptual leap. The evaluator encrypts the secret key under the public key (a so-called *bootstrapping key*), then homomorphically computes

Table 15.1 Modern FHE scheme families and their characteristics

Scheme	Plaintext space	Best for	Noise management
BGV	Integers mod p	Exact integer arithmetic	Modulus switching
BFV	Integers mod p	Exact integer arithmetic	Scale-invariant
CKKS	Approximate reals	Machine learning, statistics	Rescaling
TFHE	Single bits	Boolean circuits, fast bootstrap	Gate bootstrapping

decryption “inside the encryption.” The output is a fresh ciphertext of the same plaintext, with noise reset to the level introduced by one decryption-circuit evaluation.

At first glance, bootstrapping seems to violate a basic principle: computation on noisy data should only increase the noise, never reduce it. The resolution is that noise is not destroyed—it is *paid for*. The bootstrapping procedure decrypts and re-encrypts homomorphically, and the computational cost of doing so (time, memory, ciphertext expansion) is the price of resetting the noise floor. The bookkeeping is exact: any circuit of depth D that would otherwise require a modulus q exponential in D can instead use a polynomial modulus at the cost of D/L bootstrapping operations, each with latency proportional to the decryption-circuit depth. Nothing is free; the noise budget is traded for a time budget.

15.1.3 LWE-Based FHE: BGV, BFV, CKKS, and TFHE

All modern FHE schemes derive their security from the Learning With Errors problem (Chap. 7) or its ring/module variants [56]. They share a common algebraic skeleton—a ciphertext is an LWE (or Ring-LWE) sample $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e + \Delta m)$ where Δ is an encoding factor and e is the noise term—but they differ in two fundamental design choices: how plaintexts are encoded and how noise is kept under control.

BGV and BFV both operate over integers modulo a prime p , making them natural choices for exact integer arithmetic. They diverge in noise management: BGV uses *modulus switching*, progressively shrinking the ciphertext modulus after each multiplication to reduce the relative noise, while BFV adopts a *scale-invariant* approach that keeps the modulus fixed and instead manages the ratio between signal and noise through careful encoding. CKKS takes a fundamentally different path by encoding approximate real numbers—accepting small rounding errors in the plaintext in exchange for native support of floating-point workloads such as machine learning and statistics. TFHE occupies the opposite extreme, encrypting single bits and evaluating Boolean circuits gate by gate, but compensating with extremely fast bootstrapping (sub-millisecond on modern hardware) that resets noise after every gate. Table 15.1 summarises these families.

Despite these differences, the four families share the same fundamental bottleneck. Homomorphic addition adds two LWE samples and the noise grows additively; homomorphic multiplication involves a tensor product that causes the noise to grow

multiplicatively. This multiplicative noise growth is what limits the depth of circuits that can be evaluated without bootstrapping, and managing it is the central challenge of every FHE implementation.

15.1.4 Noise Growth and the Depth Barrier

The central constraint on any LWE-based FHE scheme is noise growth. Every homomorphic operation increases the noise in the ciphertext, and once the noise exceeds a threshold, decryption fails. Understanding the precise rate of growth is essential for choosing parameters and deciding when bootstrapping is needed.

Definition 15.2 (Noise Budget) A fresh ciphertext in an LWE-based scheme carries a noise term e with magnitude $\|e\| \approx \sigma$. After homomorphic operations, the noise evolves as:

$$\text{Addition: } \|e_{\text{add}}\| \approx \|e_1\| + \|e_2\|, \quad (15.2)$$

$$\text{Multiplication: } \|e_{\text{mult}}\| \approx \|e_1\| \cdot \|e_2\| \cdot \text{poly}(n). \quad (15.3)$$

Decryption fails when $\|e\| \geq q/2$, where q is the ciphertext modulus. The *noise budget* is $\log_2(q/2 \|e\|)$ bits.

Equation (15.3) is the crux: after L levels of multiplication, noise grows as $\sigma^{2^L} \cdot \text{poly}(n)^L$. Without intervention, even modest circuits exhaust the noise budget.

The analogy to signal processing is precise, not merely suggestive. In a chain of analog amplifiers, each stage adds thermal noise and amplifies the noise from all previous stages; after L stages the signal-to-noise ratio degrades exponentially. The standard engineering solution is a *regenerative repeater*: decode the analog signal to digital, then re-encode it, resetting the noise floor at the cost of latency. FHE bootstrapping does exactly this—it homomorphically decrypts (decodes) and re-encrypts (re-encodes), resetting noise at the cost of computational latency. The parallel is structural, not metaphorical: both systems face exponential noise growth from iterated nonlinear operations, and both solve it by interleaving decode-reencode steps. The difference is that in FHE the “decoding” itself must happen on encrypted data, which is why bootstrapping is so expensive.

15.1.5 Bootstrapping: The Noise Reset

Bootstrapping is the operation that transforms somewhat homomorphic encryption into *fully* homomorphic encryption. Given a ciphertext c with high noise, bootstrapping produces a ciphertext c' encrypting the same plaintext with low noise:

1. Publish an encrypted secret key: $\tilde{sk} = \text{Enc}(pk, sk)$.
2. Homomorphically evaluate the decryption function: $c' \leftarrow \text{Eval}(\text{Dec}(\cdot, \tilde{sk}), c)$.

3. The output c' is a fresh encryption of the original plaintext m , with noise determined only by the depth of the decryption circuit.

The circular-security assumption—that it is safe to encrypt the secret key under its own public key—is required but widely believed to hold for LWE-based schemes.

Example 15.1 (Bootstrapping Cost Over Time) The practical viability of FHE is measured by bootstrapping speed. Gentry’s original scheme required approximately 30 minutes per bootstrap in 2009. TFHE reduced this to 13 ms by 2016, and current GPU-accelerated implementations achieve sub-millisecond bootstraps. This six-orders-of-magnitude improvement in 15 years has brought FHE from theoretical curiosity to deployable technology.

15.1.6 Practical FHE Today

Several open-source libraries have brought FHE from research papers to production code. Microsoft SEAL implements BFV and CKKS and has become the de facto platform for privacy-preserving machine learning, with bindings in C++, .NET, and Python. TFHE-rs, developed by Zama, implements TFHE with fast gate bootstrapping and targets Boolean circuits and integer operations where per-gate refresh is essential. OpenFHE provides a unified API spanning BGV, BFV, CKKS, and TFHE, making it possible to prototype across scheme families within a single codebase and select the best fit for a given workload.

Example 15.2 (Private Medical Diagnosis) A hospital encrypts a patient’s genomic data (approximately 3 GB) and uploads it to a cloud server. The cloud homomorphically evaluates a cancer risk model—comparing one million single-nucleotide polymorphisms—and returns encrypted risk scores. The hospital decrypts the result; the cloud learns nothing about the patient. Current systems perform this computation in a few hours on commodity hardware, at a fraction of the cost of building an in-house secure computation facility.

15.2 Zero-Knowledge Proofs

A zero-knowledge proof allows a *prover* to convince a *verifier* that a statement is true, without revealing *any* information beyond the truth of the statement itself. This paradoxical-sounding capability is one of the most powerful ideas in modern cryptography.

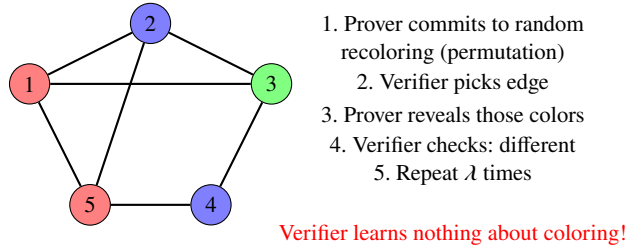


Fig. 15.2 Zero-knowledge proof of graph 3-colorability: the prover demonstrates knowledge of a valid coloring without revealing it, by committing to random permutations and revealing only queried edges.

15.2.1 The Concept: Proving Without Revealing

Definition 15.3 (Zero-Knowledge Proof) An interactive protocol between a prover P and a verifier V for a language $\mathcal{L}$ satisfies three properties:

- **Completeness:** If $x \in \mathcal{L}$, the honest prover convinces the verifier with probability 1 (or $1 - \text{negl}(\lambda)$).
- **Soundness:** If $x \notin \mathcal{L}$, no cheating prover convinces the verifier except with probability $\text{negl}(\lambda)$.
- **Zero-knowledge:** The verifier's view can be *simulated* without interaction with the prover. Formally, there exists a polynomial-time simulator S such that $S(x) \approx \text{View}_V(P \leftrightarrow V)(x)$.

The zero-knowledge property is the remarkable one: the verifier learns that “ $x \in \mathcal{L}$ ” but gains no additional knowledge—not even a single bit about *why* the statement is true.

The zero-knowledge property has a flavour that readers with a physics background may recognise. Measuring a qubit in the computational basis yields one bit and irreversibly collapses the state, destroying access to all other information the qubit encoded. A zero-knowledge proof achieves something analogous by design rather than physics: the verifier receives exactly one bit of information—“the statement is true”—and the simulator argument guarantees, mathematically, that no additional information can be extracted. The mechanisms are entirely different (cryptographic simulation versus quantum collapse), but the operational consequence is the same: a hard, provable bound on what the receiving party can learn.

A classical illustration of this concept is the graph 3-colorability protocol, shown in Figure 15.2. The prover knows a valid 3-coloring of a graph and convinces the verifier of this fact by repeatedly committing to a random recoloring (a permutation of the colors), letting the verifier pick an edge, and revealing only the two endpoint colors. After sufficiently many rounds the verifier is convinced the coloring is valid, yet has learned nothing about the actual coloring itself.

15.2.2 Interactive Proofs and the Fiat–Shamir Heuristic

The classical structure of an interactive zero-knowledge proof follows a three-move *sigma protocol*:

1. **Commitment.** The prover sends a commitment a (e.g., a randomised encryption).
2. **Challenge.** The verifier sends a random challenge $c \xleftarrow{\$} \{0, 1\}^\lambda$.
3. **Response.** The prover sends a response z that the verifier checks against (a, c) .

To eliminate interaction, Fiat and Shamir [17] proposed replacing the verifier’s random challenge with the output of a cryptographic hash function applied to the commitment and the statement:

$$c = H(a \parallel x), \quad (15.4)$$

where H is modelled as a random oracle. The resulting *non-interactive* proof $\pi = (a, z)$ can be verified by anyone, without live interaction.

Definition 15.4 (Post-Quantum Fiat–Shamir Security) In the quantum random oracle model (QROM), the Fiat–Shamir transform remains sound but incurs a security loss: an adversary making q quantum queries to H achieves soundness error $O(\sqrt{q} \cdot \varepsilon)$ rather than ε , where ε is the soundness error of the interactive protocol. This $\sqrt{q}$ factor must be accounted for when setting security parameters.

The Fiat–Shamir transform is the mechanism behind the CRYSTALS-Dilithium (ML-DSA) signature scheme (Chap. 8).

15.2.3 Lattice-Based Zero-Knowledge: Stern-Type Protocols

Lattices enable efficient zero-knowledge proofs for algebraically structured statements. The prototypical example is *proving knowledge of a short vector*:

Example 15.3 (ZK Proof of a Short Preimage) **Statement:** “I know $\mathbf{s} \in \mathbb{Z}^m$ with $\|\mathbf{s}\| \leq \beta$ such that $\mathbf{A}\mathbf{s} = \mathbf{t} \pmod{q}$.”

A Stern-type protocol proceeds as follows:

1. **Commit:** Sample a mask $\mathbf{y} \xleftarrow{\$} D_{\mathbb{Z}^m, \sigma}$ (discrete Gaussian) and send $\mathbf{w} = \mathbf{A}\mathbf{y} \pmod{q}$.
2. **Challenge:** Receive a random challenge $c \in \{0, 1, 2\}$.
3. **Respond:**
 - If $c = 0$: reveal $\mathbf{y}$ and $\mathbf{s}$ in permuted form.
 - If $c = 1$: reveal $\mathbf{z} = \mathbf{y} + \mathbf{s}$ and a commitment opening.
 - If $c = 2$: reveal $\mathbf{y}$ and a commitment to $\mathbf{s}$.
4. **Verify:** Check the appropriate linear relation and norm bound.

Table 15.2 Comparison of proof systems for post-quantum zero-knowledge

System	Proof size	Verification	Assumption
Classical SNARK	~128 bytes	~10 ms	Pairings (quantum-broken)
STARK	10–200 KB	~50 ms	Hash functions
Lattice SNARK	10–50 KB	~100 ms	LWE / SIS
Stern-type	~100 KB	~200 ms	LWE / SIS

Each round has soundness error $2/3$. After λ rounds, the soundness error drops to $(2/3)^\lambda$, which is negligible for $\lambda \approx 200$. The mask $\mathbf{y}$ information-theoretically hides the secret $\mathbf{s}$, provided σ is large enough relative to $\|\mathbf{s}\|$.

Stern-type proofs are the basis of several post-quantum anonymous credential schemes and group signature proposals.

15.2.4 Post-Quantum SNARKs and STARKs

Succinct Non-interactive Arguments of Knowledge (SNARKs) compress proofs to a few hundred bytes and enable verification in milliseconds, regardless of the complexity of the proven statement. Most deployed SNARKs rely on elliptic-curve pairings and are therefore vulnerable to quantum attack.

Two post-quantum alternatives have emerged, pursuing different trade-offs. **STARKs** (Scalable Transparent Arguments of Knowledge) replace pairings entirely with hash-based polynomial commitments. The resulting proofs are larger—tens of kilobytes rather than hundreds of bytes—but they rely only on collision-resistant hash functions, which are already quantum-safe, and they require no trusted setup. This transparency is a significant practical advantage: there is no toxic waste, no multi-party ceremony, and no trust assumption beyond the hash function itself.

Lattice-based SNARKs take the opposite approach, replacing pairing-based polynomial commitments with lattice-based ones while preserving the algebraic structure that makes SNARKs succinct. Active research has produced proof sizes in the range of 10–50 KB with verification times around 100 ms—competitive with STARKs in size, but with the added benefit of richer algebraic operations. The main challenge is prover time, which can be orders of magnitude slower than pairing-based provers due to the overhead of lattice arithmetic.

15.2.5 Applications of Zero-Knowledge

Zero-knowledge proofs underpin a surprisingly wide range of privacy-preserving applications. In *anonymous authentication*, a user proves membership in a group—“I hold a valid credential issued by this authority”—without revealing which credential, enabling systems where identity verification does not require identity disclosure. *Range*

proofs allow a party to demonstrate that a private value lies in an interval (“my account balance exceeds the required minimum”) without revealing the value itself; these are essential in financial compliance, where regulations demand verification but privacy law demands confidentiality. *Verifiable computation* inverts the trust model of cloud outsourcing: instead of trusting the server, a client receives a proof that the computation was executed correctly, turning trust into mathematics. Finally, zero-knowledge proofs are the engine behind privacy-preserving blockchains, where transaction validity must be publicly verifiable even though sender, receiver, and amount remain hidden.

Each of these applications currently relies on pre-quantum proof systems. As the transition to post-quantum cryptography proceeds, they will need quantum-safe instantiations—making the lattice-based and hash-based proof systems discussed above not just theoretically interesting but practically urgent.

15.3 Secure Multiparty Computation

In secure multiparty computation (MPC), n parties holding private inputs $x_1, \dots, x_n$ jointly compute a function $f(x_1, \dots, x_n)$ such that each party learns the output but nothing else about the other parties’ inputs—even if some parties are corrupted.

15.3.1 The Millionaires’ Problem

Yao’s *millionaires’ problem* (1982) is the canonical MPC example: Alice and Bob each know their net worth and wish to determine who is richer without revealing the actual amounts. The problem generalises immediately: any function computable by a circuit can be evaluated securely by multiple parties.

Definition 15.5 (Secure Computation) A protocol Π among n parties securely computes f if, for any adversary corrupting up to t parties, there exists a simulator S such that the adversary’s view in Π is computationally indistinguishable from S ’s output given only the corrupted parties’ inputs and the function output.

15.3.2 Shamir Secret Sharing

The foundation of most MPC protocols is *secret sharing*: distributing a secret among n parties so that any $t + 1$ can reconstruct it, but any t learn nothing.

Definition 15.6 (Shamir’s $(t + 1, n)$ -Secret Sharing) To share a secret $s \in \mathbb{Z}_q$:

1. Choose random coefficients $a_1, \dots, a_t \xleftarrow{\$} \mathbb{Z}_q$.
2. Define the polynomial $p(x) = s + a_1x + \dots + a_tx^t \pmod{q}$.
3. Give party i the share $s_i = p(i)$ for $i = 1, \dots, n$.

Any $t + 1$ shares determine p uniquely by Lagrange interpolation; any t shares are uniformly distributed and reveal nothing about s .

Shamir secret sharing is information-theoretically secure—its security does not rely on any computational assumption and therefore survives quantum computers without modification.

Secret sharing as a hologram. In holography, every fragment of a holographic plate contains information about the entire image, but the image can only be reconstructed from enough fragments. Shamir shares behave analogously: each share individually is uniformly random (contains no information about the secret), yet any sufficiently large collection of shares perfectly reconstructs the original.

15.3.3 Garbled Circuits

Yao's *garbled circuits* protocol provides a general method for two-party secure computation:

1. The *garbler* (Alice) takes the Boolean circuit for f , assigns random cryptographic labels to each wire's 0 and 1 values, and creates an encrypted truth table for each gate.
2. Alice sends the garbled circuit plus labels for her own input bits.
3. Bob obtains labels for his input bits via *oblivious transfer* (OT)—a sub-protocol in which Alice sends one of two values and learns nothing about Bob's choice, while Bob learns only his chosen value.
4. Bob evaluates the garbled circuit gate by gate and obtains labels for the output wires, which Alice helps decode.

The security of garbled circuits reduces to the security of the symmetric encryption used for the truth tables and the security of oblivious transfer.

15.3.4 MPC from Lattice-Based Oblivious Transfer

Oblivious transfer is the critical building block. Classical OT protocols rely on the Diffie–Hellman assumption or RSA, both quantum-vulnerable. Post-quantum OT can be constructed from the LWE assumption:

1. The receiver generates an LWE public key that encodes their choice bit $b \in \{0, 1\}$.
2. The sender encrypts both messages m_0, m_1 under keys derived from the receiver's public key, such that only m_b is decryptable.
3. The receiver decrypts m_b ; the LWE assumption guarantees that m_{1-b} is computationally hidden.

Table 15.3 Post-quantum MPC overhead compared to classical MPC

Operation	Classical	Post-quantum	Overhead
AES evaluation	13 ms	18 ms	1.4×
10^6 AND gates	1.2 s	2.1 s	1.75×
ML inference	5 s	8 s	1.6×

This LWE-based OT, combined with garbled circuits, yields a complete post-quantum two-party computation protocol. Extension to n parties uses standard techniques (e.g., the BMR protocol or SPDZ with lattice-based preprocessing).

The overhead of post-quantum MPC (Table 15.3) is modest: typically 1.4–1.8× compared to classical protocols. The bottleneck is the larger ciphertext and key sizes of lattice-based OT, not any fundamental efficiency barrier.

15.4 Lattice-Based Advanced Constructions

The three primitives above—FHE, zero-knowledge proofs, and MPC—are general-purpose tools. But the algebraic richness of lattices, particularly the trapdoor and preimage-sampling techniques developed by Gentry, Peikert, and Vaikuntanathan, enables a family of *structured* primitives that control access to encrypted data in ways no other post-quantum assumption can match. These constructions share a common mechanism: embedding access-control logic into the lattice structure itself, so that the ability to decrypt depends on algebraic relationships between keys, identities, attributes, or functions. We survey four such constructions, in order of increasing generality.

15.4.1 Identity-Based Encryption

Public-key encryption as deployed today requires a cumbersome scaffolding: every user generates a key pair, a certificate authority vouches that a given public key belongs to a given person, and relying parties must validate these certificates before encrypting. *Identity-based encryption* (IBE) collapses this machinery. A user’s public key *is* their identity—an email address, a phone number, any unique string—and a trusted authority issues corresponding secret keys on demand. The concept dates to Shamir (1984), but efficient constructions had to wait for lattice trapdoors.

The lattice realisation works as follows. A master authority generates a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ together with a short basis (trapdoor) for the lattice $\mathcal{L}^\perp(\mathbf{A})$. When a user with identity id requests a key, the authority applies the trapdoor to sample a short vector $\mathbf{s}$ satisfying $H(id) \cdot \mathbf{s} = \mathbf{0} \pmod{q}$, where H is a public function mapping identities to matrices. Anyone who wishes to encrypt a message to id simply uses $H(id)$ as the public key; the LWE assumption guarantees that only the holder of $\mathbf{s}$ can decrypt. Lattice IBE achieves *adaptive* security under standard LWE assumptions—a stronger

guarantee than the pairing-based schemes it replaces, and one that survives quantum adversaries.

15.4.2 Attribute-Based Encryption

IBE binds decryption to a single identity string. In practice, access control is rarely that simple: a classified document should be readable by anyone in the Physics department with Secret clearance, regardless of individual identity. *Attribute-based encryption* (ABE) generalises IBE to handle exactly this kind of Boolean policy. Ciphertexts carry attributes (“Department = Physics”, “Clearance = Secret”), and decryption keys encode policies (“Department = Physics **AND** Clearance $\geq$ Secret”). Decryption succeeds if and only if the ciphertext’s attributes satisfy the key’s policy—and the encryptor need not even know who will eventually decrypt.

Definition 15.7 (Ciphertext-Policy ABE) A ciphertext-policy ABE scheme consists of:

- $\text{Setup}(\lambda) \rightarrow (mpk, msk)$: Generate master public/secret keys.
- $\text{Enc}(mpk, P, m) \rightarrow ct$: Encrypt message m under access policy P .
- $\text{KeyGen}(msk, S) \rightarrow sk_S$: Issue a secret key for attribute set S .
- $\text{Dec}(sk_S, ct) \rightarrow m$: Decrypt if and only if S satisfies P .

The lattice realisation of ABE rests on the technique of *attribute-encoding* lattice trapdoors, introduced by Gentry, Sahai, and Waters. The idea is to embed the policy evaluation directly into the lattice structure: each attribute contributes a matrix, and the policy determines how these matrices combine. A user whose attribute set satisfies the policy can, through the algebraic structure, reconstruct a short vector that enables decryption; a user whose attributes fall short cannot, because the corresponding lattice problem remains hard. Current lattice-based ABE constructions support policies expressible as Boolean formulas or even arbitrary circuits, though ciphertext and key sizes grow with policy complexity—the main barrier to deployment.

15.4.3 Threshold Signatures and Distributed Key Generation

A single compromised signing key can be catastrophic: a stolen certificate-authority key undermines every certificate it ever issued, and a stolen cryptocurrency wallet key drains the account irreversibly. *Threshold signatures* eliminate this single point of failure by distributing signing authority among n parties so that any $t + 1$ can jointly produce a valid signature, but t or fewer—even if fully corrupted—cannot sign at all.

For classical schemes like ECDSA, threshold protocols are well understood. Lattice-based signatures introduce a complication that has no classical analogue: the signing process involves sampling from a discrete Gaussian distribution, and the noise in the

resulting signature must fall within tight bounds for verification to succeed. When signing is distributed, each party contributes a partial signature with its own noise, and the combined noise must still satisfy the global bound. Achieving this requires *distributed Gaussian sampling* protocols—each party samples locally, the partial samples are combined, and a rejection-sampling step ensures the aggregate has the correct distribution without leaking information about individual shares. Recent constructions build on Shamir-style secret sharing of the lattice signing key, combined with these distributed sampling techniques, to achieve threshold variants of both Dilithium-style (Fiat–Shamir with aborts) and Falcon-style (NTRU trapdoor) signatures. The main remaining obstacle is round complexity: current protocols require more interaction rounds than their classical counterparts.

15.4.4 Functional Encryption

IBE and ABE control *who* can decrypt. *Functional encryption* (FE) goes further: it controls *what* the decryptor learns. A decryption key sk_f , associated with a function f , allows its holder to compute $f(m)$ from a ciphertext encrypting m —and nothing beyond $f(m)$. Where ABE is an access-control mechanism, FE is an information-control mechanism: even authorised parties see only the specific derived quantity they need.

Example 15.4 (Inner-Product Functional Encryption) A hospital encrypts a patient record $\mathbf{m} \in \mathbb{Z}_q^n$. A researcher holds a key $sk_{\mathbf{y}}$ for a vector $\mathbf{y}$. Decryption yields the inner product $\langle \mathbf{m}, \mathbf{y} \rangle \pmod{q}$ —e.g., a weighted health score—without revealing any other component of $\mathbf{m}$. Lattice-based inner-product FE is efficient and secure under standard LWE assumptions.

The gap between theory and practice in functional encryption is wide. For restricted function classes—inner products and, more recently, degree-two polynomials—lattice-based constructions are efficient enough for deployment in privacy-preserving data analytics. For arbitrary functions, FE constructions exist but rely on indistinguishability obfuscation, a primitive that is itself barely practical. The frontier of FE research is therefore about pushing the boundary of “which function classes can we support efficiently?”—a question where lattice techniques have made the most progress and are likely to continue leading.

These four constructions span a wide range of maturity, from deployed systems to active research frontiers. Table 15.4 summarises the current state. FHE is the clear leader: the theory is settled and multiple production libraries exist, with performance as the remaining bottleneck. IBE and threshold signatures have mature theoretical foundations and working prototypes, but deployment is limited by key-management complexity and round costs respectively. ABE has strong theoretical results but faces a practical barrier in ciphertext and key sizes that grow with policy complexity. Functional encryption for general functions remains the most open problem—a frontier where even the theoretical foundations are incomplete.

Table 15.4 Maturity of lattice-based advanced constructions

Primitive	Theory	Practice	Main challenge
FHE	Mature	Deployed	Performance
IBE	Mature	Prototypes	Key management
ABE	Mature	Research	Ciphertext/key size
Threshold signatures	Active	Prototypes	Distributed sampling
Functional encryption	Partial	Limited	General functions

Problems

15.1 FHE noise budget.

In an LWE-based FHE scheme, a fresh ciphertext has noise magnitude σ . After one homomorphic multiplication, the noise grows to approximately $\sigma^2 \cdot n$ where n is the lattice dimension. If the modulus is q and decryption fails when noise exceeds $q/2$:

- Show that after L levels of multiplication the noise is approximately $\sigma^{2^L} \cdot n^{2^L-1}$.
- Derive the maximum multiplicative depth $L_{\max}$ as a function of σ , n , and q by solving $\sigma^{2^L} \cdot n^{2^L-1} = q/2$.
- For $\sigma = 3.2$, $n = 1024$, $q = 2^{27}$, compute $L_{\max}$. How many multiplications can be performed before bootstrapping is required?

15.2 Stern-type soundness.

A Stern-type zero-knowledge protocol has soundness error $2/3$ per round.

- How many rounds are needed to achieve soundness error below 2^{-128} ?
- If each round requires sending three commitments of 256 bits each, estimate the total proof size.
- The Fiat–Shamir transform converts this interactive proof into a non-interactive one. In the quantum random oracle model, the soundness loss is a multiplicative factor of $\sqrt{q_H}$ where q_H is the number of hash queries. If we allow $q_H = 2^{64}$ quantum queries, how many additional rounds must be added to maintain 2^{-128} soundness?

15.3 Secret sharing arithmetic.

Consider Shamir $(3, 5)$ -secret sharing over $\mathbb{Z}_{17}$. The dealer wishes to share $s = 11$.

- Choose random coefficients $a_1, a_2 \in \mathbb{Z}_{17}$ and compute the five shares.
- Verify that any three shares reconstruct s via Lagrange interpolation.
- Verify that any two shares are consistent with *every* possible secret $s' \in \mathbb{Z}_{17}$.
- Suppose two parties hold shares of secrets s and s' respectively (shared with the same evaluation points). Explain how the parties can compute shares of $s + s'$ without interaction, and shares of $s \cdot s'$ with one round of communication. Why does multiplication increase the threshold?

15.4 Research-level.

Functional encryption for inner products over $\mathbb{Z}_q$ allows a key holder to learn $\langle \mathbf{m}, \mathbf{y} \rangle$ from an encryption of $\mathbf{m}$. Explain why this does *not* trivially allow recovery of $\mathbf{m}$ itself

(even if the adversary can request keys for many different vectors $\mathbf{y}$), provided $\dim(\mathbf{m}) >$ the number of key queries. What happens when the number of queries equals $\dim(\mathbf{m})$? Discuss the implications for bounding the number of functional decryption keys in a practical deployment.

Chapter 16

Open Problems and Research Directions

Abstract Post-quantum cryptography is a field in rapid motion. This final chapter surveys the principal open problems and research directions that will shape the discipline over the coming decades. We examine quantum cryptanalysis of lattice problems, physical side-channel research on PQC implementations, hardware optimization for constrained devices, the engineering challenge of cryptographic agility, contingency planning for new mathematical breaks, and long-term visions for the quantum internet era. The chapter concludes with a numbered catalogue of concrete open problems suitable for doctoral research.

16.1 Quantum Cryptanalysis

Every cryptographic system lives under a sword: the possibility that someone will find a faster way to break it. For RSA and elliptic-curve cryptography, that sword fell decades ago in theory—Shor’s algorithm [59] factors integers and computes discrete logarithms in polynomial time on a quantum computer. The only reprieve is engineering: no quantum machine yet has the scale to execute the attack. Lattice-based cryptography was chosen as the successor precisely because no analogous quantum algorithm is known. But “not known” is not “impossible,” and the question of whether quantum computers can efficiently solve lattice problems is arguably the most important open question in all of cryptography.

What makes this question so difficult is the nature of the gap. Shor’s algorithm succeeds because factoring and discrete logarithms are instances of the abelian hidden subgroup problem, and quantum Fourier transforms over abelian groups are efficient. Lattice problems, by contrast, are related to *non-abelian* hidden subgroup problems, where quantum Fourier transforms over the relevant groups are not known to help. The challenge is not merely to find a faster algorithm, but to discover an entirely new structural insight—a way to make quantum interference reveal short lattice vectors the way it reveals hidden periods.

16.1.1 The Quantum Algorithm Toolkit

Before asking what quantum computers might achieve against lattices, we should take stock of what quantum algorithms can do at all. The known families relevant to cryptanalysis are surprisingly few:

1. **Hidden subgroup algorithms** (Shor): exponential speedup for abelian hidden subgroup problems. Lattice problems are related to *non-abelian* hidden subgroup problems, for which no efficient quantum algorithm is known [56].
2. **Grover search**: a generic $\sqrt{N}$ speedup for unstructured search. Applied to lattice sieving, Grover's algorithm reduces the exponent from $2^{0.292n}$ to approximately $2^{0.265n}$ [35].
3. **Quantum walks**: polynomial speedups for graph-based search, applicable to certain enumeration subroutines.
4. **Quantum sampling**: algorithms such as HHL that solve linear systems exponentially fast under favourable conditions.

The striking feature of this toolkit is its narrowness. Decades of quantum algorithm research have produced, for cryptanalytic purposes, exactly one transformative technique (hidden subgroup methods) and several that offer modest polynomial speedups. This is not for lack of trying—it reflects something deep about the structure of quantum computation.

The gap. There is a conspicuous absence in this list: no quantum algorithm exploits the algebraic structure of lattice problems in the way Shor exploits the group structure of $\mathbb{Z}_N^*$. Closing—or proving the impossibility of closing—this gap is the central challenge of quantum cryptanalysis.

A breakthrough could come from an unexpected direction. Perhaps the geometry of lattices admits a quantum encoding that we have not yet imagined. Perhaps a new model of quantum computation—topological, measurement-based, or hybrid classical-quantum—will make visible a structure that the circuit model obscures. Or perhaps lattice problems are genuinely hard for quantum computers, and the absence of progress reflects a real barrier rather than a failure of imagination. Distinguishing between these possibilities is the work of the coming decades.

16.1.2 Quantum Speedups for Sieving and Enumeration

If no transformative quantum algorithm exists for lattice problems, the best we can hope for is to accelerate the known classical approaches. Classical lattice sieving algorithms find short vectors by generating exponentially many lattice vectors and combining those that are close together. The best classical sieving algorithms run in time $2^{0.292n+o(n)}$. Applying Grover search to the nearest-neighbour subroutine yields a quantum sieving

complexity of approximately $2^{0.265n+o(n)}$, at the cost of exponential quantum memory [35].

Definition 16.1 (Quantum Sieving Complexity) Let $T_{\text{class}}(n) = 2^{c_1 n+o(n)}$ be the classical sieving time and $T_{\text{quant}}(n) = 2^{c_2 n+o(n)}$ the quantum sieving time. Current best known values are $c_1 \approx 0.292$ and $c_2 \approx 0.265$. The *quantum sieving advantage* is the ratio $c_1/c_2 \approx 1.10$.

A 10% reduction in the exponent is significant but far from the exponential collapse that Shor inflicts on RSA. To put it concretely: for the lattice dimensions relevant to ML-KEM-768, the quantum sieving advantage shaves roughly 20 bits off the security level—enough to matter for parameter selection, but not enough to threaten the scheme’s viability. For enumeration-based algorithms, quantum walks on the enumeration tree can provide a modest polynomial speedup, but the overall complexity remains $2^{\Theta(n)}$.

The deeper question is whether this 10% is the end of the story or merely the beginning. Could a cleverer use of quantum resources—better nearest-neighbour data structures, more efficient quantum memory access, or entirely new sieving strategies that are natively quantum—push the exponent significantly lower? No concrete proposal has achieved this, but neither has anyone proved it impossible.

16.1.3 Quantum Walks on Lattice Graphs

Quantum walk algorithms offer an alternative approach that sidesteps the Grover framework entirely. The idea is to define a graph whose vertices correspond to lattice vectors and whose edges connect “close” pairs, then deploy a quantum walk to search for short vectors. The difficulty is fundamental: the graph has $2^{\Theta(n)}$ vertices, and quantum walks provide at best a quadratic speedup over classical random walks on generic graphs.

Recent work has explored whether *structured* quantum walks—exploiting the periodicity of the lattice—can do better. An analogy from condensed-matter physics is suggestive: Bloch’s theorem shows that the periodicity of a crystal lattice constrains the electron wavefunctions to Bloch waves. If a similar structural constraint could be imposed on a quantum walk over the lattice graph, it might yield a super-quadratic speedup. No concrete algorithm of this type is known.

The analogy, however, is more tantalising than it may first appear. In a physical crystal, periodicity creates band structure, and band structure creates forbidden and allowed energy regions that profoundly shape transport properties. A lattice in the cryptographic sense has its own periodicity—the lattice vectors tile space—and a quantum walk that respects this periodicity might exhibit analogous transport phenomena. Whether these phenomena can be harnessed for finding short vectors remains entirely open. The tools exist in mathematical physics; what is missing is the bridge to algorithmic design.

16.1.4 The SVP Holy Grail

All of the preceding discussion converges on a single question that looms over the entire field.

The SVP question. Is there a polynomial-time quantum algorithm for SVP? A positive answer would break all lattice-based cryptography. A negative answer—a proof that SVP is hard even for quantum computers—would place lattice-based cryptography on the firmest possible theoretical footing. Neither result appears within reach.

Several intermediate results constrain what is possible. The best known quantum lower bound for exact SVP is trivial ($\Omega(n)$, the cost of reading the input). On the upper-bound side, there is no known quantum algorithm that beats $2^{0.265n}$ asymptotically. Gidney and Ekerå [22] have provided sharp quantum resource estimates for Shor’s algorithm against RSA and elliptic curves, but comparable resource analyses for quantum lattice attacks remain incomplete—a significant gap in the literature.

We are, in a sense, building an entire cryptographic infrastructure on a conjecture: that lattice problems are hard for quantum computers. The evidence for this conjecture is substantial—decades of effort by talented researchers have failed to refute it—but evidence is not proof. The field proceeds, as it must, on the best available knowledge, while keeping one eye on the horizon for any sign that the conjecture might be wrong.

16.2 Physical Side-Channel Research

The previous sections asked whether the *mathematics* of PQC can be broken. This section asks a different and equally important question: can the *physics* of PQC implementations be exploited? Cryptographic implementations leak information through physical channels—power consumption, electromagnetic (EM) radiation, timing, and cache behaviour—and post-quantum algorithms introduce new side-channel surfaces that are not yet fully understood.

The transition from RSA and elliptic curves to lattice-based schemes is not merely a change of mathematical assumptions; it is a change of computational workload. Where classical schemes perform modular exponentiations over large integers, PQC schemes perform polynomial arithmetic over moderate-sized rings. The leakage characteristics of these operations are fundamentally different, and the countermeasures developed over two decades for classical schemes do not transfer automatically. Understanding this new leakage landscape is urgent: the algorithms are being standardised now, and implementations are being deployed into hardware that will be in the field for a decade or more.

16.2.1 Electromagnetic Analysis of PQC

The NTT butterfly operations at the heart of MODULE-LWE-based schemes such as ML-KEM produce characteristic EM signatures. Each butterfly involves a modular multiplication whose operand values influence switching activity on the data bus. Near-field EM probes can resolve individual butterfly stages, potentially recovering polynomial coefficients that encode the secret key.

Example 16.1 (EM Leakage in NTT) Consider an NTT of length $n = 256$ over $\mathbb{Z}_q$ with $q = 3329$ (the ML-KEM parameter set). The NTT consists of $\log_2 n = 8$ stages, each performing $n/2 = 128$ butterfly operations. An EM probe positioned over the arithmetic unit of an unprotected microcontroller can distinguish twiddle-factor multiplications involving small coefficients (low Hamming weight) from those involving large coefficients, leaking partial information about the secret polynomial after as few as 1 000 traces.

The implications extend beyond a single attack. The NTT is not an optional component—it is the computational backbone of ML-KEM and ML-DSA, executed during every key generation, encapsulation, and signing operation. Any EM vulnerability in the NTT is therefore a vulnerability in the entire scheme. Protecting it requires understanding not just which coefficients leak, but how the leakage propagates through the NTT's butterfly structure and how it compounds across multiple operations with the same key.

16.2.2 Deep Learning for Side-Channel Attacks

Classical side-channel attacks require the analyst to identify exploitable leakage points manually—a process that demands both domain expertise and considerable effort. Deep learning models—convolutional and recurrent neural networks—upend this paradigm by learning leakage models directly from raw power or EM traces, bypassing the need for expert feature engineering. For PQC implementations, this shift is particularly consequential because:

- The leakage model for polynomial arithmetic is more complex than for classical modular exponentiation.
- Masking countermeasures that are effective against classical template attacks may be insufficient against neural-network-based profiling.
- The large key and ciphertext sizes of PQC schemes produce longer traces with more potential leakage points.

The concern is not speculative. Recent experiments have demonstrated that deep learning models can extract secret keys from masked PQC implementations using fewer traces than classical statistical attacks. The arms race between attack and defence in this domain is accelerating, and the defensive side is not clearly winning.

16.2.3 Physics-Informed Countermeasures

Effective countermeasures require understanding the physical leakage mechanism, not merely the algorithmic one. A masking scheme that is provably secure in an idealised leakage model may fail spectacularly when the leakage model does not match the actual physics of the chip. Examples of physics-informed approaches include:

1. **Dual-rail logic:** balancing switching activity at the transistor level so that every operation dissipates the same energy regardless of operand values.
2. **EM shielding:** designing package-level Faraday cages calibrated to the frequencies at which NTT operations emit.
3. **Noise injection:** adding controlled random switching activity whose spectrum masks the signal of interest. The noise budget must be set by modelling the channel capacity of the EM side channel.

Each of these approaches carries costs—in area, in power, in design complexity—and none is a silver bullet. The research challenge is to find the right combination for each deployment context, from billion-unit smart cards where every gate counts to server-class hardware where area is cheap but throughput is paramount.

16.2.4 Quantum Side Channels

As quantum computers begin to execute cryptographic algorithms, they introduce an entirely new class of side channels that has no precedent in classical hardware:

- **Qubit crosstalk:** in superconducting processors, the resonant frequency of a qubit shifts depending on the state of its neighbours, potentially leaking information about the computation.
- **Photon statistics in QKD:** detector timing jitter and after-pulsing probabilities depend on photon energy, creating a side channel in quantum key distribution systems.
- **Cryogenic thermal signatures:** heat dissipation in dilution refrigerators varies with the number and type of gate operations, providing a coarse-grained side channel on quantum algorithm execution.

These quantum side channels are largely unexplored from a cryptanalytic perspective. The physics community understands the mechanisms intimately—crosstalk is a major engineering challenge in scaling quantum processors—but has not yet examined them through the lens of an adversary. This intersection of quantum hardware engineering and cryptographic security represents one of the most fertile and least explored research frontiers in the field.

Table 16.1 NTT implementation tradeoffs for ML-KEM ($n = 256$, $q = 3329$)

Architecture	Area (kGE)	Latency (μ s)	Energy (nJ)
Software (ARM Cortex-M4)	—	~100	~10 000
1 butterfly unit	~5	~10	~50
4 butterfly units	~15	~3	~20
128 butterfly units	~400	~0.1	~10

16.3 Hardware Optimization

Moving from theory to practice, we confront a blunt engineering reality: PQC algorithms are more computationally expensive than their classical predecessors, and the devices that must run them are not getting bigger. Deploying PQC on resource-constrained devices—smart cards, IoT sensors, embedded controllers—requires hardware implementations that meet strict area, power, and latency budgets. The question is not whether PQC can be implemented in hardware; it is whether it can be implemented within the budgets that the application demands.

16.3.1 FPGA and ASIC Implementations of NTT

The Number Theoretic Transform dominates the computational cost of MODULE-LWE-based schemes, just as modular exponentiation dominated the cost of RSA. On a general-purpose CPU, an NTT of length $n = 256$ with $q = 3329$ requires approximately $n \log_2 n = 2048$ butterfly operations, each involving a modular multiplication and addition. A dedicated hardware butterfly unit can execute one butterfly per clock cycle, completing the full NTT in $\log_2 n = 8$ cycles when $n/2 = 128$ butterfly units operate in parallel.

The numbers in Table 16.1 reveal a classic area–latency tradeoff, but they also tell a more nuanced story. Energy decreases with parallelism even as area grows, because the parallel design completes faster and spends less time in energy-consuming idle states. For IoT applications where energy is the binding constraint, the single-butterfly design is often preferred; for high-throughput servers, fully parallel implementations are justified. The design space between these extremes is large and largely unexplored—finding Pareto-optimal points for specific deployment scenarios is an active research area.

16.3.2 Gaussian Sampling Hardware

Schemes based on Gaussian noise (including earlier NTRU variants and certain signature schemes) require sampling from a discrete Gaussian distribution. This is a

deceptively difficult problem in hardware. The sampler must simultaneously satisfy three competing requirements:

- **Statistical quality:** the output distribution must be indistinguishable from ideal to prevent statistical side-channel attacks.
- **Constant-time execution:** rejection sampling introduces data-dependent timing; hardware implementations must mask this variation.
- **Entropy consumption:** each sample requires fresh random bits; the TRNG must keep pace with the sampling rate.

Cumulative Distribution Table (CDT) sampling and the Knuth–Yao algorithm are the two dominant hardware approaches. CDT sampling uses a precomputed table and a single comparison per sample, achieving constant time at the cost of table storage. The Knuth–Yao algorithm is more memory-efficient but requires a variable number of random bits per sample. Neither approach is clearly superior—the choice depends on the available silicon budget and the paranoia level of the threat model.

16.3.3 Energy-Efficient PQC for IoT

For battery-powered IoT devices, the energy cost of a single key encapsulation may determine whether PQC is feasible at all. ML-KEM-512, the lightest NIST-standardised parameter set, requires approximately 150 000 arithmetic operations for encapsulation. On an ARM Cortex-M0+ at 1.8 V and 16 MHz, this translates to roughly 10 μ J—orders of magnitude more than an ECDH key exchange.

This gap is not merely inconvenient; it is potentially disqualifying. A sensor node running on a coin-cell battery that performs one key exchange per hour can afford 10 μ J. A node that performs one per second cannot. Reducing this energy cost through algorithmic, architectural, and circuit-level optimisation is an active research area with direct practical impact—one that will determine whether billions of deployed IoT devices can participate in a post-quantum world or must be left behind.

16.4 Cryptographic Agility

History teaches that every cryptographic algorithm eventually falls. DES lasted 20 years. MD5 lasted 13. SHA-1 lasted 12 from the first theoretical cracks to practical collision. The ability to replace algorithms quickly and safely—*cryptographic agility*—is therefore not a luxury but a survival requirement. Yet most deployed systems treat their cryptographic algorithms as load-bearing walls rather than replaceable components, and the resulting brittleness has turned every past algorithm deprecation into a multi-year ordeal.

16.4.1 Algorithm-Agnostic Design

What would it mean for a system to be truly algorithm-agnostic? Not merely to support multiple algorithms, but to treat the choice of algorithm as a runtime configuration parameter rather than a compile-time decision.

Definition 16.2 (Cryptographic Agility) A system exhibits *cryptographic agility* if:

1. Algorithms are identified by negotiable object identifiers (OIDs), not hard-coded implementations.
2. Multiple algorithms can coexist in the same deployment.
3. Algorithm transitions require configuration changes, not code changes.
4. Monitoring infrastructure tracks which algorithms are in active use.

Achieving agility requires abstracting the cryptographic layer behind a stable interface. The provider pattern—where algorithm implementations are loaded dynamically based on policy—is the standard architectural approach. However, agility introduces its own risks: a larger algorithm menu expands the attack surface (downgrade attacks) and increases testing burden. The engineering challenge is to gain the flexibility of agility without the fragility that comes from supporting too many options.

16.4.2 Protocol Negotiation

TLS 1.3 demonstrates effective protocol-level agility: the client advertises supported groups and signature algorithms, and the server selects the strongest mutually supported option. Extending this mechanism to include PQC algorithms—as done in hybrid key exchange drafts—is straightforward in principle. The challenges are operational:

- Larger key shares increase the ClientHello size, sometimes exceeding a single TCP segment and triggering fragmentation.
- Certificate chains containing PQC signatures are significantly larger, straining bandwidth-limited links.
- Negotiation must handle the case where a previously trusted algorithm is abruptly deprecated.

These are not theoretical concerns. Early deployments of hybrid TLS have already encountered middleboxes that drop oversized ClientHellos, certificate verification failures due to unexpected signature algorithms, and interoperability problems between implementations at different stages of standards compliance. The path from “the protocol supports it” to “the internet works with it” is long and paved with operational surprises.

16.4.3 Lessons from SHA-1 and MD5

The deprecation of MD5 and SHA-1 provides instructive—and sobering—precedents. MD5 collision attacks were first demonstrated in 2004, yet MD5-signed certificates persisted in the Web PKI until at least 2012. SHA-1 collisions were demonstrated in 2017 (the SHAttered attack); major browsers did not fully remove SHA-1 certificate support until 2019—two years after the break and twelve years after the first theoretical weaknesses.

Lesson. The time between a cryptographic break and the completion of migration is measured in years to decades, not days. Systems that lack agility transform an algorithmic weakness into a systemic crisis. The PQC transition is our opportunity to build agility into infrastructure *before* it is needed.

The PQC transition differs from the SHA-1 and MD5 episodes in one crucial respect: we are migrating *before* the break, not after. This is unprecedented in the history of cryptography, and it gives us a window—narrow but real—to design the migration infrastructure correctly. Whether we use that window well will determine whether the next cryptographic break is a managed transition or a global emergency.

16.5 Preparing for New Breaks

The preceding section argued that agility is essential. This section asks a harder question: what specifically should we be agile *toward*? The NIST PQC standards are based primarily on the hardness of lattice problems (MODULE-LWE, RING-LWE, Module-NTRU). This is a deliberate bet—lattice problems have the strongest theoretical foundations and the best performance characteristics. But it is a bet nonetheless, and responsible engineering demands a contingency plan. What happens if these foundations crack?

16.5.1 What if Module-LWE Breaks?

A polynomial-time algorithm for MODULE-LWE would invalidate ML-KEM and ML-DSA simultaneously, disabling both key encapsulation and digital signatures. The impact would be comparable to a polynomial-time factoring algorithm in the pre-quantum era—but broader, because NIST has concentrated its standards on a single hardness assumption. In the classical world, at least, RSA and Diffie–Hellman rested on different (though related) problems. The lattice monoculture is efficient but fragile.

Fallback options under active standardisation or study include:

- **Code-based cryptography:** Classic McEliece (NIST round 4) rests on the hardness of decoding random linear codes, studied for over 45 years. Key sizes are large (hundreds of kilobytes) but ciphertext sizes are competitive.
- **Hash-based signatures:** SPHINCS⁺ (standardised as SLH-DSA) requires only the security of a hash function—a minimal assumption. Signature sizes are large (~8–50 KB) but the scheme is a robust fallback.
- **Multivariate-quadratic schemes:** These have a chequered history of breaks, but research continues on hardened variants.

None of these alternatives matches the performance of lattice-based schemes. Classic McEliece keys are too large for many applications; SLH-DSA signatures are too slow for high-throughput signing. The research imperative is clear: we need either to improve the efficiency of these fallback schemes or to identify new hardness assumptions that yield practical constructions.

16.5.2 Defence in Depth

The vulnerability of any single assumption motivates combining multiple assumptions so that an adversary must break all of them to compromise the system.

Definition 16.3 (Cryptographic Defence in Depth) A system employs *cryptographic defence in depth* if it combines multiple independent hardness assumptions such that an adversary must break *all* of them to compromise the system. The standard realisation is *hybrid* key exchange: the shared secret is the hash of a classical ECDH key and a PQC KEM key, so the system is secure as long as *either* assumption holds.

Hybrid modes are now supported in TLS (e.g., X25519+ML-KEM-768) and are recommended by multiple national agencies during the transition period. The cost is modest: approximately double the computation and bandwidth of a single key exchange. This is a price worth paying for insurance against a catastrophic break—but it is not free, and systems that adopt hybrid modes must budget for the overhead in their capacity planning.

16.5.3 Monitoring Cryptanalysis

Defence in depth is a structural safeguard. Monitoring is the operational one. Early warning of approaching breaks requires systematic tracking of progress on several fronts:

1. **Lattice dimension records:** track the largest dimension in which SVP or CVP has been solved exactly, and the approximation factors achieved in challenge lattices.
2. **Algorithmic improvements:** a reduction in the sieving exponent c (currently $c \approx 0.292$ classically, $c \approx 0.265$ quantumly) would directly affect security estimates.

3. **Quantum hardware milestones:** logical qubit counts, error rates, and demonstrated algorithm execution times.
4. **New mathematical connections:** unexpected reductions between problems—such as a reduction from `MODULE-LWE` to a problem known to be easy—would be a red flag.

No single indicator will flash red the day before a break. Cryptographic collapses are usually the culmination of a series of incremental advances, each individually unthreatening. The discipline required is to track these increments systematically and to have the institutional courage to act when the trendline points toward danger—even when no individual result is yet alarming.

16.6 The 50-Year Horizon

Cryptographic infrastructure protects data whose sensitivity may span decades: medical records, state secrets, financial archives, intellectual property. A birth certificate issued today may need to be verifiable in 2076. A classified intelligence assessment filed this year must remain confidential for at least 25 years, often 50. Planning on this timescale forces us to reason about technologies and mathematical discoveries that do not yet exist, and to design systems that will survive surprises we cannot anticipate.

This is, in some ways, an absurd task. Fifty years ago, public-key cryptography itself did not exist—Diffie and Hellman’s seminal paper was still a year away. The notion that we can predict the cryptographic landscape of 2076 is hubris. Yet the alternative—building infrastructure that assumes today’s assumptions will hold forever—is worse than hubris; it is negligence. The goal is not to predict the future but to build systems that are robust to a wide range of futures.

16.6.1 The Quantum Internet

The most transformative development on the horizon is the quantum internet: a global network connecting quantum processors via entanglement distribution. A mature quantum internet would enable quantum key distribution (QKD) over arbitrary distances, offering information-theoretic key exchange—security guaranteed by the laws of physics rather than the conjectured hardness of a mathematical problem.

The appeal is obvious. Information-theoretic security does not degrade with advances in computing or mathematics. A key exchanged via QKD today will be just as secure against a quantum computer in 2076 as it is against a classical computer today. For data that truly must remain secret for decades, this guarantee is qualitatively different from anything computational cryptography can offer.

However, QKD alone does not replace public-key cryptography. It requires an authenticated classical channel (which itself needs PQC signatures), does not provide digital signatures, and its throughput is orders of magnitude below that of computa-

tional schemes. A QKD link might exchange a few thousand key bits per second over a metropolitan fibre; a PQC key encapsulation runs in microseconds and requires no dedicated quantum hardware. The realistic future is not one in which QKD replaces PQC, but one in which they coexist in a *layered* security stack—PQC providing the bulk computational cryptography, QKD hardening the highest-sensitivity links where the cost and complexity are justified.

16.6.2 Post-Quantum Plus Quantum: The Full Stack

The long-term security architecture, then, will not be a single technology but an integrated stack of complementary layers:

1. **Physical layer:** QKD and quantum random number generation provide information-theoretic primitives.
2. **Computational layer:** PQC algorithms handle authentication, general-purpose encryption, and signatures.
3. **Protocol layer:** agile protocols negotiate the best available combination of physical and computational security.
4. **Policy layer:** automated monitoring and migration tools manage algorithm lifecycles.

No single layer is sufficient. QKD cannot authenticate; PQC relies on computational assumptions that may eventually fail; protocols without agility become liabilities; policy without automation is too slow. Defence in depth across layers is the only sustainable strategy [54].

The analogy to network architecture is deliberate. The TCP/IP stack succeeded not because any single layer was perfect, but because each layer could evolve independently while maintaining stable interfaces with its neighbours. The cryptographic stack of the future must achieve the same modularity. A breakthrough in quantum repeater technology should strengthen the physical layer without requiring changes to the PQC algorithms above it. A new post-quantum signature scheme should slot into the computational layer without disrupting the QKD infrastructure below. Achieving this clean separation of concerns—while maintaining the tight integration needed for security—is perhaps the grandest engineering challenge of the PQC era.

16.6.3 Information-Theoretic vs. Computational Security

Underlying these architectural questions is a philosophical one that has divided cryptographers since Shannon: should we aim for information-theoretic security (where no amount of computation can break the system) or accept computational security (where breaking the system requires more computation than an adversary can muster)?

The practical answer, as Table 16.2 makes clear, is hybrid: information-theoretic security for the most sensitive, low-volume channels; computational security for the vast majority of communications. The trade-off is not between good and bad, but

Table 16.2 Information-theoretic vs. computational security for long-term planning

Criterion	Information-theoretic	Computational
Key management	Requires pre-shared or QKD keys	Public-key infrastructure
Bandwidth cost	One-time pad: $ K \geq M $	Compact ciphertexts
Assumption	None (physics only)	Hardness of a math problem
Scalability	Poor (point-to-point)	Excellent (public-key)
Longevity	Permanent	Until assumption fails

between two kinds of risk. Information-theoretic security eliminates computational risk but introduces physical risk (the security of QKD depends on correct modelling of the quantum channel and the absence of implementation flaws in the quantum hardware). Computational security eliminates physical risk but introduces mathematical risk (the hardness assumption might fall).

The PQC transition is about securing the computational layer against quantum adversaries, while the parallel development of quantum networks strengthens the physical layer. Fifty years from now, both layers will look very different from what we build today. The measure of our success will not be whether we chose the right algorithms—we almost certainly will not have—but whether we built the right abstractions to let them be replaced gracefully when better ones arrive.

16.7 Concrete Open Problems

The following problems are open as of early 2026. Each combines a precise statement with a brief discussion of why the problem matters, what approaches have been tried, and where a new perspective might succeed.

1. **Quantum complexity of SVP.** Determine the quantum query complexity of exact SVP in n dimensions. Is $2^{\Theta(n)}$ optimal, or does a sub-exponential quantum algorithm exist? The best known quantum upper bound is $2^{0.265n+o(n)}$ from quantum sieving; no super-linear lower bound is known. Even proving a $2^{\omega(\sqrt{n})}$ quantum lower bound would be a major advance [56].

This is the foundational question of the field. A sub-exponential quantum algorithm would require re-parameterising every lattice-based standard; a strong lower bound would justify the entire PQC programme on firm theoretical ground. Either result would reshape the landscape.

2. **Tight reductions for Module-LWE.** The security proof of ML-KEM reduces MODULE-LWE to worst-case lattice problems, but the reduction incurs a polynomial loss in the security parameter [54]. Can this loss be eliminated? Tight reductions would allow smaller parameters and thus more efficient schemes, directly impacting every deployment.

The polynomial loss is not merely an aesthetic blemish—it forces parameter choices to be conservative, adding kilobytes to keys and microseconds to operations across

billions of devices. Closing this gap would have immediate, tangible engineering consequences.

3. **Quantum resource estimates for lattice attacks.** Provide end-to-end quantum resource estimates—logical qubits, T-gate count, circuit depth, wall-clock time under realistic error correction—for the best known quantum attacks on ML-KEM-768 and ML-KEM-1024. Gidney and Ekerå [22] performed this analysis for RSA-2048 and elliptic curves; the lattice case is missing.
Without these estimates, we are setting security parameters based on asymptotic analysis rather than concrete costs. The RSA estimates revealed that Shor’s algorithm requires fewer resources than previously assumed; a similar analysis for lattices could either confirm or challenge current parameter choices.
4. **Side-channel security of NTT implementations.** Characterise the information-theoretic leakage of masked NTT implementations. Specifically: for an NTT masked at order d , what is the minimum number of EM traces required to recover a secret coefficient, as a function of d and the signal-to-noise ratio? Current bounds are empirical, not theoretical.
Theoretical leakage bounds would transform side-channel protection from an empirical art into an engineering discipline, allowing designers to choose masking orders with provable security guarantees rather than relying on attack-and-patch cycles.
5. **Constant-time Gaussian sampling with minimal entropy.** Design a constant-time discrete Gaussian sampler that is provably secure against timing side channels and consumes fewer than $2\lceil\log_2(\sigma)\rceil + 3$ random bits per sample, where σ is the standard deviation. The current best constructions consume significantly more.
Entropy is a scarce resource in constrained hardware. A sampler that uses fewer random bits per sample directly reduces the load on the true random number generator, which is often the throughput bottleneck in embedded PQC implementations.
6. **Sub-linear-area NTT for constrained devices.** Design an NTT hardware architecture for ML-KEM that fits within 3 kGE (gate equivalents) while completing a full NTT in fewer than 5 000 clock cycles. Achieving this would make PQC viable on the smallest smart-card ICs.
Smart cards and similar constrained devices are deployed in the billions, and many must support cryptographic operations for a decade or more. If PQC cannot fit on these devices, the post-quantum transition will leave a vast installed base unprotected.
7. **Formal verification of PQC implementations.** Develop a formally verified (machine-checked) implementation of ML-KEM in a language suitable for embedded deployment (e.g., C or Rust). The implementation must be verified for both functional correctness (agreement with the specification) and constant-time execution (absence of secret-dependent branching and memory access).
The history of cryptographic implementation is littered with bugs that formal methods could have caught—Heartbleed, Lucky 13, countless timing side channels. A verified reference implementation would not only prevent such bugs but would serve as a trusted baseline against which other implementations can be validated.
8. **Hybrid protocol overhead bounds.** Determine the minimum additional latency and bandwidth overhead of hybrid PQC/classical key exchange in TLS 1.3 as a function of network round-trip time and MTU. Current empirical measurements vary widely; a clean analytical model validated against measurement would be valuable.

Hybrid key exchange is the recommended deployment mode during the transition, but the overhead is poorly characterised. Network operators need reliable models to plan capacity, and protocol designers need them to optimise handshake flows.

9. **Worst-case to average-case reductions for code-based problems.** The Ajtai-style worst-case to average-case connection that gives lattice-based cryptography its strong theoretical foundation [45] has no known analogue for code-based problems (syndrome decoding of random linear codes). Establishing such a connection—or proving it impossible—would clarify the comparative theoretical footing of the two families. Code-based cryptography is the leading fallback if lattice assumptions fail. Without a worst-case to average-case reduction, its security rests on decades of failed attacks rather than on a provable connection to a well-studied hard problem. Resolving this question would either elevate code-based cryptography to the same theoretical standing as lattice-based schemes or clarify why such standing is unattainable.
10. **Post-quantum oblivious transfer.** Construct an efficient post-quantum oblivious transfer (OT) protocol with communication complexity sub-linear in the security parameter. OT is a foundational primitive for secure multi-party computation; current PQC-based OT constructions are significantly less efficient than their classical counterparts.

Secure multi-party computation is increasingly used in privacy-preserving machine learning, auctions, and financial computations. If OT remains expensive in the post-quantum setting, these applications will face a performance cliff when classical assumptions are deprecated.

11. **Quantum side-channel taxonomy.** Develop a systematic taxonomy of side channels in superconducting and photonic quantum processors, including qubit crosstalk, readout crosstalk, and thermal signatures. For each channel, determine the information leakage rate (bits per operation) and propose countermeasures. No comprehensive framework exists.

As quantum computers begin executing cryptographic algorithms—both for attack (running Shor’s algorithm) and defence (QKD, quantum random number generation)—the security of these computations against physical observers becomes a practical concern. A taxonomy is the first step toward principled protection.

12. **Automated cryptographic migration.** Design and evaluate a system that can automatically detect the use of deprecated cryptographic algorithms in a large code base ($> 10^6$ lines), propose replacements compatible with the existing API contracts, and verify that the replacement preserves functional behaviour. Current migration efforts are overwhelmingly manual.

The SHA-1 deprecation took over a decade in part because every migration was a bespoke engineering effort. Automated migration tooling would compress the timeline from years to weeks, transforming cryptographic agility from an aspiration to an operational reality.

16.1 Gidney and Ekerå [22] estimate that factoring a 2048-bit RSA modulus requires approximately 20 million noisy qubits over 8 hours. Suppose a quantum sieving algorithm for SVP in dimension n requires $2^{0.265n}$ quantum operations, each implemented as a circuit of depth $D = \text{poly}(n)$. Under the surface-code error correction model with a physical error rate of 10^{-3} , estimate the number of physical qubits and wall-clock

time required to attack ML-KEM-768 (which targets 192-bit security, corresponding to $n \approx 700$ in the lattice dimension). Compare with the Gidney–Ekerå RSA estimate and discuss which system is more vulnerable in the medium term.

16.2 Design a cryptographic agility framework for a web service currently using TLS 1.3 with X25519 and Ed25519. Your framework must allow swapping to any NIST-standardised PQC algorithm within 48 hours of a break announcement. Identify the components that need abstraction (TLS library, certificate infrastructure, key storage, monitoring) and the operational procedures required (key rotation, certificate reissuance, client compatibility testing). Discuss the tradeoff between agility and the risk of downgrade attacks.

16.3 Consider a TLS 1.3 handshake using hybrid key exchange (X25519 + ML-KEM-768). The combined key share is approximately $1,184 + 32 = 1,216$ bytes (ML-KEM-768 public key plus X25519 public key). Assuming a network MTU of 1,500 bytes:

- (a) Determine whether the ClientHello (including the hybrid key share, supported groups, signature algorithms, and typical extensions) fits within a single TCP segment. If not, estimate the additional round-trip latency from TCP fragmentation.
- (b) Repeat the analysis for ML-KEM-1024 (public key $\approx 1,568$ bytes).
- (c) Discuss strategies (e.g., sending the PQC key share in a HelloRetryRequest, or using TCP Fast Open) to mitigate the overhead.

16.4 Choose one of the open problems listed in Sect. 16.7. Write a 2-page research proposal outlining: (a) the problem’s significance, (b) your proposed approach, (c) what tools or techniques from physics you would bring to bear, and (d) what a successful outcome would look like. Include a brief literature review (at least five references) and a realistic 12-month timeline.

Appendix A

Proof Supplements

A.1 Ajtai's Worst-Case to Average-Case Reduction

This section provides a detailed proof sketch of theorem 6.5 (Sect. 6.4.2), which establishes that solving the Short Integer Solutions problem on a random matrix is at least as hard as solving worst-case lattice problems.

Theorem A.1 (Ajtai [1], Micciancio–Regev [46]) *Let n, m, q be integers with $m = O(n \log q)$ and $q = \text{poly}(n)$, and let $\beta < q$. If there exists a probabilistic polynomial-time algorithm $\mathcal{A}$ that, given a uniformly random matrix $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, finds a non-zero $\mathbf{x} \in \mathbb{Z}^m$ with $\|\mathbf{x}\| \leq \beta$ and $\mathbf{A}\mathbf{x} = \mathbf{0} \pmod{q}$ with non-negligible probability, then there exists a probabilistic polynomial-time algorithm that solves γ -SIVP on every n -dimensional lattice, where $\gamma = \beta \cdot \tilde{O}(\sqrt{n})$.*

Proof sketch. The reduction receives a worst-case lattice $\mathcal{L} \subset \mathbb{R}^n$ (given by some basis $\mathbf{B}$) and must find n linearly independent vectors of norm at most $\gamma \cdot \lambda_n(\mathcal{L})$. It proceeds in three stages.

Stage 1: Gaussian sampling on the target lattice. Using the iterative Klein/GPV sampler, the reduction draws lattice vectors from the discrete Gaussian distribution $D_{\mathcal{L},s}$ with parameter s chosen above the *smoothing parameter* $\eta_\varepsilon(\mathcal{L})$ (definition 6.3). Recall that $\eta_\varepsilon(\mathcal{L})$ is the smallest s such that $\rho_{1/s}(\mathcal{L}^* \setminus \{\mathbf{0}\}) \leq \varepsilon$, where $\rho_t(\mathbf{x}) = \exp(-\pi \|\mathbf{x}\|^2 / t^2)$. Above the smoothing parameter, the reduction modulo the fundamental parallelepiped of $\mathcal{L}$ produces a distribution that is statistically close to uniform over the parallelepiped. This property is crucial: it allows the reduction to *embed* the target lattice into a random-looking SIS instance.

Stage 2: Constructing a random SIS instance. The reduction builds the matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ column by column. For each column j :

1. Sample $\mathbf{v}_j \leftarrow D_{\mathcal{L},s}$.
2. Compute $\mathbf{a}_j = \mathbf{B}^{-1}\mathbf{v}_j \pmod{q}$.

Because $s \geq \eta_\varepsilon(\mathcal{L})$, the vectors $\mathbf{a}_j$ are within statistical distance ε of uniform over $\mathbb{Z}_q^n$. Hence $\mathbf{A}$ is indistinguishable from a genuinely random matrix, and the SIS oracle $\mathcal{A}$ applies.

Stage 3: From a SIS solution to short lattice vectors. The oracle returns a non-zero $\mathbf{x} \in \mathbb{Z}^m$ with $\|\mathbf{x}\| \leq \beta$ and $\mathbf{A}\mathbf{x} = \mathbf{0} \pmod{q}$. Unravelling the construction, this means that the vector

$$\mathbf{w} = \sum_{j=1}^m x_j \mathbf{v}_j \quad (\text{A.1})$$

lies in $q \cdot \mathcal{L}$ (it maps to zero modulo q). Since each $\mathbf{v}_j$ is a Gaussian sample of expected norm $s\sqrt{n}$ and $\|\mathbf{x}\| \leq \beta$, a standard tail bound gives $\|\mathbf{w}\| \leq \beta \cdot s \cdot \tilde{O}(\sqrt{n})$. The vector $\mathbf{w}/q$ is therefore a lattice vector of $\mathcal{L}$ with norm at most $\beta \cdot s \cdot \tilde{O}(\sqrt{n})/q$.

Repeating the procedure n times with independent randomness (and verifying linear independence, which holds with overwhelming probability for $m \gg n$) yields n independent short vectors in $\mathcal{L}$. Setting the Gaussian parameter s appropriately relative to $\lambda_n(\mathcal{L})$ ensures the resulting vectors have norm at most $\gamma \cdot \lambda_n(\mathcal{L})$ with $\gamma = \beta \cdot \tilde{O}(\sqrt{n})$, thereby solving γ -SIVP. $\square$

Why this matters. Ajtai's reduction is the reason lattice-based cryptography stands on firmer theoretical ground than RSA or elliptic-curve cryptography. An efficient SIS solver would imply an efficient algorithm for *every* instance of approximate SIVP—a problem that has resisted attack for decades. The reduction is *classical* (no quantum computation required), unlike Regev's LWE reduction below.

A.2 Regev's LWE Hardness Reduction

This section expands on theorem 7.2 (Sect. 7.2.1), presenting the main ideas of Regev's quantum reduction from worst-case lattice problems to the average-case Learning With Errors problem.

Theorem A.2 (Regev [56]) *Let $n \geq 1$, let $q \leq \text{poly}(n)$ be prime, and let $\alpha \in (0, 1)$ with $\alpha q \geq 2\sqrt{n}$. If there exists a probabilistic polynomial-time algorithm that solves decision-LWE($n, q, D_{\mathbb{Z}, \alpha q}$) with non-negligible advantage, then there exists a quantum polynomial-time algorithm that solves $\tilde{O}(n/\alpha)$ -GAP SVP on every n -dimensional lattice.*

Proof sketch. The reduction receives a worst-case lattice $\mathcal{L}$ (given by a basis $\mathbf{B}$) and a threshold d , and must decide whether $\lambda_1(\mathcal{L}) \leq d$ or $\lambda_1(\mathcal{L}) > \gamma d$. It uses an LWE oracle and quantum computation as follows.

Step 1: Quantum state preparation. For each dual lattice vector $\mathbf{w} \in \mathcal{L}^*$, consider the Gaussian function $\rho_r(\mathbf{x} - \mathbf{w}) = \exp(-\pi \|\mathbf{x} - \mathbf{w}\|^2 / r^2)$ centred at $\mathbf{w}$. For a suitable parameter r , the reduction prepares (on a quantum computer) the superposition

$$|\psi\rangle = \sum_{\mathbf{w} \in \mathcal{L}^*} \rho_r(\mathbf{w}) |\mathbf{w}\rangle. \quad (\text{A.2})$$

This state encodes the dual lattice weighted by a Gaussian envelope. Preparing $|\psi\rangle$ is the step that requires quantum computation; it uses Gaussian superposition sampling over the lattice, building on the iterative quantum sampler of Regev.

Step 2: Quantum Fourier transform. Apply the quantum Fourier transform (QFT) to $|\psi\rangle$. By the Poisson summation formula, the QFT of a Gaussian sum over $\mathcal{L}^*$ is a Gaussian sum over the primal lattice $\mathcal{L}$ with reciprocal width:

$$\text{QFT } |\psi\rangle \propto \sum_{\mathbf{v} \in \mathcal{L}} \rho_{1/r}(\mathbf{v}) |\mathbf{v}\rangle. \quad (\text{A.3})$$

Measuring this state yields a lattice point $\mathbf{v} \in \mathcal{L}$ distributed according to $D_{\mathcal{L}, 1/r}$.

Step 3: From lattice samples to LWE samples. The classical part of the reduction converts each discrete Gaussian sample $\mathbf{v} \leftarrow D_{\mathcal{L}, 1/r}$ into an LWE sample as follows.

Choose $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$ and compute

$$b = \langle \mathbf{a}, \mathbf{v} \rangle + e \pmod{q}, \quad (\text{A.4})$$

where e is a rounding term arising from the discretisation of $\mathbf{v}$ to coordinates modulo q . The distribution of e is (close to) the discrete Gaussian $D_{\mathbb{Z}, \alpha q}$ when the lattice sampling parameter r is set appropriately in terms of α . The pair $(\mathbf{a}, b)$ is therefore an LWE sample with secret determined by the lattice point.

Step 4: Using the LWE oracle. Feed the constructed LWE samples to the oracle. If the oracle distinguishes LWE from uniform, the reduction can extract information about the Gaussian width $1/r$ on $\mathcal{L}$, which is directly related to $\lambda_1(\mathcal{L})$. Specifically:

- If $\lambda_1(\mathcal{L})$ is small (the YES case of GAP-SVP), the Gaussian mass concentrates on short vectors, and the resulting LWE samples have a detectable structure.
- If $\lambda_1(\mathcal{L})$ is large (the NO case), the Gaussian over $\mathcal{L}$ is essentially supported on the origin, the samples carry no information, and the oracle sees uniform pairs.

The oracle's ability to distinguish the two cases therefore solves GAP-SVP . $\square$

Why the reduction is quantum. The quantum step is Step 1: preparing a Gaussian superposition over $\mathcal{L}^*$. No efficient classical procedure is known for sampling from this distribution on an arbitrary lattice. The QFT in Step 2 is the natural quantum analogue of Poisson summation and exploits the same position–momentum duality familiar from quantum mechanics (see the grey box in Sect. 7.2.1).

Crucially, the *attacker* need not be quantum. Even a purely classical LWE solver would, via this reduction, yield a *quantum* algorithm for GAP-SVP . The quantum nature is a property of the proof, not of the threat model.

Classical alternatives. Peikert (2009) and Brakerski et al. (2013) later established purely classical reductions from GAP-SVP to LWE , at the cost of requiring either a super-polynomial modulus q or a larger approximation factor γ . For the tightest parameters used in practice (polynomial q , polynomial γ), Regev’s quantum reduction remains the strongest known result.

A.3 Correctness of the Fujisaki–Okamoto Transform

This section proves theorem 7.6 (Sect. 7.6): the Fujisaki–Okamoto transform upgrades IND-CPA security to IND-CCA2 security in the random oracle model, provided the underlying scheme has negligible decryption failure probability.

Theorem A.3 (FO Security [19, 27]) *Let $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be an IND-CPA -secure public-key encryption scheme with δ -correctness (i.e., $\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m; r)) \neq m] \leq \delta$ over the randomness of key generation and r). Let $\Pi' = (\text{KeyGen}', \text{Encaps}, \text{Decaps})$ be the FO-transformed KEM (definition 7.8). Then in the random oracle model, for any IND-CCA2 adversary $\mathcal{A}$ making at most q_D decapsulation queries and q_H hash queries,*

$$\text{Adv}_{\Pi'}^{\text{IND-CCA2}}(\mathcal{A}) \leq 2 \text{Adv}_{\Pi}^{\text{IND-CPA}}(\mathcal{B}) + q_H \cdot \delta + \frac{q_D}{|\mathcal{M}|}, \quad (\text{A.5})$$

where $\mathcal{B}$ is an IND-CPA adversary with comparable running time and $|\mathcal{M}|$ is the message space size.

Proof (game-hopping argument). We proceed through a sequence of games, starting from the real IND-CCA2 experiment. Let $\mathcal{A}$ be an adversary that receives a challenge ciphertext c^* encapsulating either the real key K_0^* or a random key K_1^* .

Game 0 (Real game). This is the standard IND-CCA2 experiment for the KEM Π' . The challenger runs KeyGen' , computes $(K_0^*, c^*) \leftarrow \text{Encaps}(\text{pk})$, samples $K_1^* \xleftarrow{\$} \{0, 1\}^\ell$, flips a bit b , and gives $\mathcal{A}$ the pair (K_b^*, c^*) . The adversary may query Decaps on any ciphertext $c \neq c^*$ and the random oracles H, G . Let W_0 denote the event that $\mathcal{A}$ guesses b correctly.

Game 1 (Decapsulation uses table lookup). Modify the decapsulation oracle as follows: instead of decrypting c with Dec , scan the random oracle table for any query m such that $G(m, \text{pk})$ produces randomness yielding $\text{Enc}(\text{pk}, m; G(m, \text{pk})) = c$. If found, return $K = H(m, \text{pk})$; otherwise return $H(s, c)$ (implicit rejection).

Games 0 and 1 differ only when $\mathcal{A}$ submits a ciphertext c that is valid (i.e., Dec recovers some m' with $\text{Enc}(\text{pk}, m'; G(m', \text{pk})) = c$) but $\mathcal{A}$ never queried G on m' . In the random oracle model, $G(m', \text{pk})$ is uniformly random and independent of $\mathcal{A}$ ’s view, so the probability that $\text{Enc}(\text{pk}, m'; G(m', \text{pk})) = c$ holds by accident is at most $1/|\mathcal{M}|$ per query. Hence:

$$|\Pr[W_1] - \Pr[W_0]| \leq \frac{q_D}{|\mathcal{M}|}. \quad (\text{A.6})$$

Game 2 (Abort on decryption failure of c^*). Modify the game so that if the message m^* encrypted in the challenge ciphertext c^* would cause a decryption failure (i.e., $\text{Dec}(\text{sk}', c^*) \neq m^*$), the game aborts. This event has probability at most δ by the correctness assumption, so:

$$|\Pr[W_2] - \Pr[W_1]| \leq \delta. \quad (\text{A.7})$$

Game 3 (Replace K_0^* with random). Replace the real key $K_0^* = H(m^*, \text{pk})$ with a uniformly random value. Since H is a random oracle and m^* is chosen uniformly at random by Encaps , the value $H(m^*, \text{pk})$ is uniformly random unless $\mathcal{A}$ queries H on m^* . If $\mathcal{A}$ ever queries H on m^* , it has effectively recovered the plaintext, which breaks IND-CPA security. Formally, construct an IND-CPA adversary $\mathcal{B}$ that simulates Game 2 for $\mathcal{A}$ and outputs 1 if $\mathcal{A}$ queries $H(m^*, \text{pk})$. Then:

$$|\Pr[W_3] - \Pr[W_2]| \leq \text{Adv}_{\Pi}^{\text{IND-CPA}}(\mathcal{B}) + (q_H - 1) \cdot \delta, \quad (\text{A.8})$$

where the additional δ terms account for the possibility that $\mathcal{A}$ queries H on a message m' whose encryption coincidentally equals c^* due to a decryption failure event.

In Game 3, both K_0^* and K_1^* are uniformly random and independent of $\mathcal{A}$'s view, so $\Pr[W_3] = 1/2$.

Conclusion. Combining the bounds:

$$\text{Adv}_{\Pi'}^{\text{IND-CCA2}}(\mathcal{A}) = |2 \Pr[W_0] - 1| \quad (\text{A.9})$$

$$\leq 2 \text{Adv}_{\Pi}^{\text{IND-CPA}}(\mathcal{B}) + q_H \cdot \delta + \frac{q_D}{|\mathcal{M}|}. \quad (\text{A.10})$$

For ML-KEM, $\delta < 2^{-140}$ and $|\mathcal{M}| = 2^{256}$, making the loss terms negligible. $\square$

The role of re-encryption. The re-encryption check in Decaps ($c' \stackrel{?}{=} c$) is what makes Game 1 possible: it ensures that any ciphertext accepted by Decaps was honestly generated from a message known to the random oracle. Without re-encryption, an adversary could submit malformed ciphertexts that decrypt to related messages, gradually extracting the secret key. Implicit rejection (returning $H(s, c)$ for invalid ciphertexts) ensures that the adversary cannot even detect whether a ciphertext was rejected, closing timing side channels.

A.4 The Rejection Sampling Lemma

This section proves theorem 8.1 (Sect. 8.3.3), the key technical tool that ensures lattice signatures based on the Fiat–Shamir-with-aborts paradigm do not leak the secret key.

Lemma A.1 (Rejection Sampling [38]) *Let V be a finite set of vectors with $\|\mathbf{v}\|_\infty \leq \beta$ for all $\mathbf{v} \in V$, and let $h : V \rightarrow [0, 1]$ be a probability function. Let $\sigma = \omega(\beta\sqrt{\log m})$ where m is the ambient dimension. Define $M = \exp(12\beta/\sigma + 1/(2\sigma^2))$. Consider the following experiment:*

1. Sample $\mathbf{v} \leftarrow h$ and $\mathbf{y} \leftarrow D_\sigma^m$.
2. Set $\mathbf{z} = \mathbf{y} + \mathbf{v}$.
3. Output $\mathbf{z}$ with probability $\min(\frac{D_\sigma^m(\mathbf{z})}{M \cdot D_{\mathbf{v},\sigma}^m(\mathbf{z})}, 1)$; otherwise restart.

Then:

- (a) The probability of outputting $\mathbf{z}$ (not restarting) in each iteration is at least $1/M$.
- (b) The statistical distance between the distribution of $\mathbf{z}$ (conditioned on output) and D_σ^m is at most $2^{-\omega(\log m)}/M$.

In particular, the output distribution is (essentially) D_σ^m regardless of which $\mathbf{v}$ was used.

Proof sketch. The argument is a direct calculation exploiting the rapid decay of Gaussians.

Masking property. For any fixed $\mathbf{v}$ with $\|\mathbf{v}\|_\infty \leq \beta$, the ratio of the centred Gaussian to the shifted Gaussian at any point $\mathbf{z}$ satisfies

$$\frac{D_\sigma^m(\mathbf{z})}{D_{\mathbf{v},\sigma}^m(\mathbf{z})} = \exp\left(\frac{-2\langle \mathbf{z}, \mathbf{v} \rangle + \|\mathbf{v}\|^2}{2\sigma^2}\right). \quad (\text{A.11})$$

Since $\mathbf{z} = \mathbf{y} + \mathbf{v}$ with $\mathbf{y} \leftarrow D_\sigma^m$, the inner product $\langle \mathbf{z}, \mathbf{v} \rangle$ concentrates around $\|\mathbf{v}\|^2$ (which is at most $m\beta^2$). The condition $\sigma = \omega(\beta\sqrt{\log m})$ ensures that the ratio concentrates in $[1/M, 1]$ with overwhelming probability. Specifically, a Gaussian tail bound gives

$$\Pr\left[\frac{D_\sigma^m(\mathbf{z})}{D_{\mathbf{v},\sigma}^m(\mathbf{z})} < \frac{1}{M}\right] \leq 2^{-\omega(\log m)}. \quad (\text{A.12})$$

Acceptance probability. The acceptance probability equals $\mathbb{E}\left[\min\left(\frac{D_\sigma^m(\mathbf{z})}{M \cdot D_{\mathbf{v},\sigma}^m(\mathbf{z})}, 1\right)\right] = \frac{1}{M}$ (up to the negligible tail), because the expected value of $\min(R/M, 1)$ for a ratio R distributed as above is $1/M$ by construction of M .

Output distribution. Conditioned on acceptance, the density of $\mathbf{z}$ is proportional to $D_{\mathbf{v},\sigma}^m(\mathbf{z}) \cdot \frac{D_\sigma^m(\mathbf{z})}{M \cdot D_{\mathbf{v},\sigma}^m(\mathbf{z})} = \frac{1}{M} D_\sigma^m(\mathbf{z})$, which is exactly D_σ^m up to normalisation. The negligible tail where the ratio falls below $1/M$ contributes at most $2^{-\omega(\log m)}/M$ statistical distance. Since this holds for every $\mathbf{v} \in V$, the output distribution is independent of the secret (up to negligible error). $\square$

A.5 Security of Merkle's Signature Scheme

This section proves theorem 10.2 (Sect. 10.2): the Merkle tree construction yields an existentially unforgeable many-time signature scheme from any secure one-time signature.

Theorem A.4 (Merkle Tree Security) *Let OTS be an EU-CMA₁-secure one-time signature scheme and H a second-preimage-resistant hash function. Then the Merkle signature scheme with tree height h (supporting $N = 2^h$ signatures) is EU-CMA-secure. Concretely, for any forger $\mathcal{F}$ making at most N signing queries:*

$$\text{Adv}_{\text{Merkle}}^{\text{EU-CMA}}(\mathcal{F}) \leq N \cdot \text{Adv}_{\text{OTS}}^{\text{EU-CMA}_1}(\mathcal{B}) + N \cdot \text{Adv}_H^{\text{SPR}}(C), \quad (\text{A.13})$$

where $\mathcal{B}$ and C are efficient reductions.

Proof (hybrid argument over leaves). Suppose forger $\mathcal{F}$ outputs a valid forgery (m^*, σ^*) on a message not previously signed. The forgery contains an authentication path from some leaf i^* to the root. We distinguish two cases.

Case 1: OTS forgery. The authentication path uses a leaf OTS key pair $(\text{sk}_{i^*}, \text{pk}_{i^*})$ that was used in one of the $\leq N$ signing queries. In this case, the forger has produced a second valid signature under a one-time key, which constitutes an OTS forgery.

To reduce to OTS security, construct $\mathcal{B}$ as follows. $\mathcal{B}$ receives an OTS challenge public key pk^* and must produce a forgery. It guesses a random index $i' \xleftarrow{\$} \{0, \dots, N-1\}$, generates all other OTS key pairs honestly, and embeds pk^* at leaf i' . The Merkle tree is built normally. When $\mathcal{F}$ requests a signature using leaf i' , $\mathcal{B}$ forwards the message to its OTS signing oracle. If $\mathcal{F}$'s forgery uses leaf $i^* = i'$ (which happens with probability $\geq 1/N$), $\mathcal{B}$ obtains a valid OTS forgery.

Case 2: Hash collision on the path. The authentication path in the forgery reaches the correct root PK but diverges from the honest tree at some internal node. That is, there exists an internal node where the forger's claimed child values (L^*, R^*) differ from the honest values (L, R) , yet $H(L^* || R^*) = H(L || R)$. This is a second-preimage for H .

Construct C that embeds its second-preimage challenge at a random internal node and simulates the scheme for $\mathcal{F}$. The guess succeeds with probability at least $1/N$ (there are $\leq N$ internal nodes on any root-to-leaf path times the number of leaves).

Combining. Since every forgery falls into one of the two cases, a union bound gives the claimed advantage. $\square$

Tightness. The factor of $N = 2^h$ in the security loss is inherent in the guessing step and represents a real cost: a Merkle tree supporting 2^{20} signatures loses 20 bits of security relative to the underlying OTS. This motivates the use of multi-tree constructions (XMSS^{MT}, HSS) that split the height across layers to reduce the effective loss per layer, as discussed in Sect. 10.3.

Appendix B

Reference Implementations

This appendix provides self-contained Python examples that illustrate the algorithms discussed throughout the book. The educational LWE example uses only standard libraries; all remaining examples require `liboqs-python` (<https://github.com/open-quantum-safe/liboqs-python>). Install it with `pip install liboqs-python`.

Warning. None of these scripts are suitable for production use. They are intended solely as pedagogical companions to the main text.

B.1 LWE Encryption (Educational)

The following pure-Python script implements a toy LWE encryption scheme with parameters $n = 16$, $q = 97$. These parameters provide *no* security; they are chosen so that every intermediate value fits comfortably on screen. The code demonstrates the three core operations—key generation, single-bit encryption, and decryption—and highlights how noise accumulation can cause decryption failures when parameters are too aggressive.

```
1 import random
2
3 # --- Parameters (educational only) ---
4 n = 16      # lattice dimension
5 q = 97      # modulus (prime)
6 m = 32      # number of public-key samples
7 sigma = 2   # noise bound (uniform in [-sigma, sigma])
8
9 def sample_error():
10     """Sample small noise uniformly from [-sigma, sigma]."""
11     return random.randint(-sigma, sigma) % q
12
13 def keygen():
14     """Return (public_key, secret_key)."""
15     s = [random.randrange(q) for _ in range(n)]      # secret
16     vector
```

```

16     A = [[random.randrange(q) for _ in range(n)]
17           for _ in range(m)]                # random
18           matrix
19     e = [sample_error() for _ in range(m)]    # noise
20           vector
21     b = [(sum(A[i][j]*s[j] for j in range(n)) + e[i]) % q
22           for i in range(m)]                # b = As +
23           e
24     return (A, b), s
25
26 def encrypt(pk, bit):
27     """Encrypt a single bit in {0,1}."""
28     A, b = pk
29     # Choose a random binary selector vector r
30     r = [random.randint(0, 1) for _ in range(m)]
31     # Sum selected rows of A and corresponding entries of b
32     u = [sum(r[i]*A[i][j] for i in range(m)) % q
33           for j in range(n)]
34     v = sum(r[i]*b[i] for i in range(m)) % q
35     # Encode the bit: add floor(q/2) if bit == 1
36     v = (v + bit * (q // 2)) % q
37     return (u, v)
38
39 def decrypt(sk, ct):
40     """Decrypt ciphertext to recover the bit."""
41     u, v = ct
42     s = sk
43     # Compute v - <u, s> mod q
44     d = (v - sum(u[j]*s[j] for j in range(n))) % q
45     # Map to the nearest bit: 0 if close to 0, 1 if close to q/2
46     if d > q // 2:
47         d -= q # centered representative
48     return 0 if abs(d) < q // 4 else 1
49
50 # --- Quick self-test ---
51 pk, sk = keygen()
52 for bit in (0, 1):
53     ct = encrypt(pk, bit)
54     assert decrypt(sk, ct) == bit, "Decryption_failure!"
55 print("LWE_encrypt/decrypt_OK_for_bits_0_and_1")

```

Listing B.1 Toy LWE encryption (NOT secure)

B.2 ML-KEM with liboqs

ML-KEM (FIPS 203) is the NIST standard key-encapsulation mechanism derived from CRYSTALS-Kyber. The script below exercises all three parameter sets through the liboqs Python bindings, prints object sizes, and measures wall-clock time for each operation.

```

1 import oqs

```



```

2 import time
3
4 PARAM_SETS = ["ML-KEM-512", "ML-KEM-768", "ML-KEM-1024"]
5 ITERS = 100
6
7 for name in PARAM_SETS:
8     kem = oqs.KeyEncapsulation(name)
9
10    # --- Key generation ---
11    t0 = time.perf_counter()
12    for _ in range(ITERS):
13        pk = kem.generate_keypair()
14    kg_ms = (time.perf_counter() - t0) / ITERS * 1000
15
16    # --- Encapsulation ---
17    t0 = time.perf_counter()
18    for _ in range(ITERS):
19        ct, ss_enc = kem.encap_secret(pk)
20    enc_ms = (time.perf_counter() - t0) / ITERS * 1000
21
22    # --- Decapsulation ---
23    t0 = time.perf_counter()
24    for _ in range(ITERS):
25        ss_dec = kem.decaps_secret(ct)
26    dec_ms = (time.perf_counter() - t0) / ITERS * 1000
27
28    assert ss_enc == ss_dec, "Shared secrets do not match!"
29
30    print(f"\n=== {name} ===")
31    print(f"Public key: {len(pk):>5} bytes")
32    print(f"Ciphertext: {len(ct):>5} bytes")
33    print(f"Shared sec.: {len(ss_enc):>5} bytes")
34    print(f"KeyGen: {kg_ms:>7.3f} ms")
35    print(f"Encaps: {enc_ms:>7.3f} ms")
36    print(f"Decaps: {dec_ms:>7.3f} ms")

```

Listing B.2 ML-KEM key encapsulation with liboqs

B.3 ML-DSA with liboqs

ML-DSA (FIPS 204), derived from CRYSTALS-Dilithium, is the primary NIST post-quantum signature standard. The code below demonstrates key generation, signing, and verification for ML-DSA-65 (the recommended security level) and reports sizes and timings.

```

1 import oqs
2 import time
3
4 sig = oqs.Signature("ML-DSA-65")
5 message = b"Post-quantum cryptography is here."
6 ITERS = 100

```

```

7
8 # --- Key generation ---
9 t0 = time.perf_counter()
10 for _ in range(ITERs):
11     pk = sig.generate_keypair()
12 kg_ms = (time.perf_counter() - t0) / ITERs * 1000
13
14 # --- Signing ---
15 t0 = time.perf_counter()
16 for _ in range(ITERs):
17     signature = sig.sign(message)
18 sign_ms = (time.perf_counter() - t0) / ITERs * 1000
19
20 # --- Verification ---
21 verifier = oqs.Signature("ML-DSA-65")
22 t0 = time.perf_counter()
23 for _ in range(ITERs):
24     valid = verifier.verify(message, signature, pk)
25 ver_ms = (time.perf_counter() - t0) / ITERs * 1000
26
27 assert valid, "Signature verification failed!"
28
29 print("=== ML-DSA-65 ===")
30 print(f"Public key: {len(pk):>5} bytes")
31 print(f"Signature: {len(signature):>5} bytes")
32 print(f"KeyGen: {kg_ms:>7.3f} ms")
33 print(f"Sign: {sign_ms:>7.3f} ms")
34 print(f"Verify: {ver_ms:>7.3f} ms")

```

Listing B.3 ML-DSA digital signatures with liboqs

B.4 SLH-DSA with liboqs

SLH-DSA (FIPS 205), based on SPHINCS⁺, is a stateless hash-based signature scheme. Its main trade-off is between the “small” variants (shorter signatures, slower signing) and the “fast” variants (larger signatures, faster signing). The script below makes this trade-off concrete.

```

1 import oqs
2 import time
3
4 VARIANTS = [
5     ("SLH-DSA-SHA2-128s", "128-bit, small"),
6     ("SLH-DSA-SHA2-128f", "128-bit, fast"),
7 ]
8 message = b"Hash-based signatures need no lattice assumptions."
9 ITERs = 10 # SLH-DSA signing is slow; fewer iterations
10
11 for name, label in VARIANTS:
12     sig = oqs.Signature(name)
13     pk = sig.generate_keypair()

```

```

14
15     # --- Signing ---
16     t0 = time.perf_counter()
17     for _ in range(ITERES):
18         signature = sig.sign(message)
19     sign_ms = (time.perf_counter() - t0) / ITERES * 1000
20
21     # --- Verification ---
22     verifier = oqs.Signature(name)
23     t0 = time.perf_counter()
24     for _ in range(ITERES):
25         valid = verifier.verify(message, signature, pk)
26     ver_ms = (time.perf_counter() - t0) / ITERES * 1000
27
28     assert valid
29     print(f"\n===_{name}_{label}===")
30     print(f"Public key:_{len(pk):>6}_bytes")
31     print(f"Signature:_{len(signature):>6}_bytes")
32     print(f"Sign:_{sign_ms:>9.1f}_ms")
33     print(f"Verify:_{ver_ms:>9.1f}_ms")

```

Listing B.4 SLH-DSA: small vs. fast parameter comparison

B.5 Hybrid TLS with Post-Quantum KEM

Deploying post-quantum algorithms in TLS 1.3 requires an OpenSSL build that supports the OQS provider. The following commands assume OpenSSL 3.x compiled with the oqs-provider plugin (see <https://github.com/open-quantum-safe/oqs-provider> for build instructions).

Step 1. Generate a hybrid certificate. The example below creates an ML-DSA-65 CA and issues a server certificate that uses a hybrid key exchange during the TLS handshake.

```

1 # Generate CA key and self-signed certificate
2 openssl req -x509 -new -newkey mldsa65 -keyout ca.key \
3   -out ca.crt -nodes -subj "/CN=PQC_Test_CA" -days 365
4
5 # Generate server key and CSR
6 openssl req -new -newkey mldsa65 -keyout server.key \
7   -out server.csr -nodes -subj "/CN=localhost"
8
9 # Sign the server certificate with the CA
10 openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key \
11   -CAcreateserial -out server.crt -days 365

```

Listing B.5 Generate an ML-DSA-65 CA and server certificate

Step 2. Test a hybrid TLS connection. Use `s_server` and `s_client` to verify the handshake with a hybrid key exchange group (X25519 + ML-KEM-768).

```

1 # Terminal 1 -- start the server
2 openssl s_server -cert server.crt -key server.key \
3   -groups x25519_mlkem768 -www -accept 4433
4
5 # Terminal 2 -- connect as a client
6 openssl s_client -connect localhost:4433 \
7   -groups x25519_mlkem768 -CAfile ca.crt

```

Listing B.6 Hybrid TLS 1.3 handshake test

A successful connection will display the negotiated group (x25519_mlkem768) in the handshake output, confirming that both classical and post-quantum key exchange took place.

B.6 Performance Benchmarks

The script below benchmarks every NIST-standardised PQC algorithm alongside RSA-2048 and ECDSA-P256, printing the results as a formatted table. It requires `liboqs-python` and the `cryptography` package.

```

1 import oqs, time
2 from cryptography.hazmat.primitives.asymmetric import rsa, ec,
3   padding, utils
4 from cryptography.hazmat.primitives import hashes, serialization
5
6 ITERS = 100
7 MSG = b"Benchmark_payload_for_signature_schemes."
8
9 def bench(fn, n=ITERS):
10     t0 = time.perf_counter()
11     for _ in range(n):
12         result = fn()
13     return (time.perf_counter() - t0) / n * 1000, result
14
15 rows = []
16
17 # --- ML-KEM ---
18 for name in ("ML-KEM-512", "ML-KEM-768", "ML-KEM-1024"):
19     k = oqs.KeyEncapsulation(name)
20     kg, pk = bench(k.generate_keypair)
21     en, (ct, ss) = bench(lambda: k.encap_secret(pk))
22     de, _ = bench(lambda: k.decap_secret(ct))
23     rows.append((name, f"{len(pk)}", f"{len(ct)}",
24                 f"{kg:.3f}", f"{en:.3f}", f"{de:.3f}"))
25
26 # --- ML-DSA ---
27 for name in ("ML-DSA-44", "ML-DSA-65", "ML-DSA-87"):
28     s = oqs.Signature(name)
29     kg, pk = bench(s.generate_keypair)
30     si, sig = bench(lambda: s.sign(MSG))
31     v = oqs.Signature(name)

```

```

31     ve, _ = bench(lambda: v.verify(MSG, sig, pk))
32     rows.append((name, f"{len(pk)}", f"{len(sig)}",
33                  f"{kg:.3f}", f"{si:.3f}", f"{ve:.3f}"))
34
35 # --- SLH-DSA (fewer iterations -- signing is slow) ---
36 for name in ("SLH-DSA-SHA2-128s", "SLH-DSA-SHA2-128f"):
37     s = oqs.Signature(name)
38     kg, pk = bench(lambda: s.generate_keypair(), 10)
39     si, sig = bench(lambda: s.sign(MSG), 10)
40     v = oqs.Signature(name)
41     ve, _ = bench(lambda: v.verify(MSG, sig, pk), 10)
42     rows.append((name, f"{len(pk)}", f"{len(sig)}",
43                  f"{kg:.3f}", f"{si:.3f}", f"{ve:.3f}"))
44
45 # --- RSA-2048 (sign/verify only) ---
46 sk = rsa.generate_private_key(65537, 2048)
47 pk_rsa = sk.public_key()
48 sig_rsa = sk.sign(MSG, padding.PKCS1v15(), hashes.SHA256())
49 kg_r, _ = bench(lambda: rsa.generate_private_key(65537, 2048), 10)
50 si_r, _ = bench(lambda: sk.sign(MSG, padding.PKCS1v15(),
51                                 hashes.SHA256()), 10)
52 ve_r, _ = bench(lambda: pk_rsa.verify(sig_rsa, MSG,
53                                       padding.PKCS1v15(),
54                                       hashes.SHA256()), 10)
55 pk_bytes = pk_rsa.public_bytes(serialization.Encoding.DER,
56                                serialization.PublicFormat.SubjectPublicKeyInfo)
57 rows.append(("RSA-2048", f"{len(pk_bytes)}", f"{len(sig_rsa)}",
58             f"{kg_r:.3f}", f"{si_r:.3f}", f"{ve_r:.3f}"))
59
60 # --- ECDSA P-256 ---
61 sk_ec = ec.generate_private_key(ec.SECP256R1())
62 pk_ec = sk_ec.public_key()
63 sig_ec = sk_ec.sign(MSG, ec.ECDSA(hashes.SHA256()))
64 kg_e, _ = bench(lambda: ec.generate_private_key(ec.SECP256R1()), 10)
65 si_e, _ = bench(lambda: sk_ec.sign(MSG, ec.ECDSA(hashes.SHA256()))), 10)
66 ve_e, _ = bench(lambda: pk_ec.verify(sig_ec, MSG,
67                                       ec.ECDSA(hashes.SHA256()))), 10)
68 pk_ec_b = pk_ec.public_bytes(serialization.Encoding.DER,
69                               serialization.PublicFormat.SubjectPublicKeyInfo)
70 rows.append(("ECDSA-P256", f"{len(pk_ec_b)}", f"{len(sig_ec)}",
71             f"{kg_e:.3f}", f"{si_e:.3f}", f"{ve_e:.3f}"))
72
73 # --- Print table ---
74 hdr = f"'Algorithm':<20}\{'PK(B)':>7}\{'CT/Sig(B)':>9}\ \
75       f'\{'KG(ms)':>8}\{'Op1(ms)':>8}\{'Op2(ms)':>8}"
76 print(hdr)
77 print("-" * len(hdr))
78 for r in rows:
79     print(f"{r[0]:<20}\{r[1]:>7}\{r[2]:>9}\ \
80           f"{r[3]:>8}\{r[4]:>8}\{r[5]:>8}")

```

Listing B.7 Comprehensive PQC benchmark script

In the table above, **Op1** denotes encapsulation (for KEMs) or signing (for signatures), and **Op2** denotes decapsulation or verification, respectively. Readers are encouraged to

run the script on their own hardware and compare results with the figures in the main text.

Solutions

Problems of Chapter 1

1.1 The HNDL inequality requires $x + y \leq z$, where $x = 30$ (shelf life), y is migration time, and z the CRQC arrival year. Migration must be complete by $z - x = 2035 - 30 = 2005$. Since 2005 has passed, the agency is *already* in the danger window—any ciphertext captured from 2005 onward remains sensitive when the CRQC arrives. Migration must begin immediately.

1.2 (a) Grover searches a space of N in $O(\sqrt{N})$ queries, so AES-128 ($N = 2^{128}$) requires 2^{64} quantum evaluations: 64-bit security. (b) AES-256 requires 2^{128} Grover queries—128-bit post-quantum security. Each query demands a full AES quantum circuit ($\sim 10^4$ T-gates), and Grover is inherently sequential, so this is firmly infeasible. (c) Grover halves the exponent ($2^k \rightarrow 2^{k/2}$); the fix is to double the key. Shor gives an *exponential* speedup (sub-exponential $\rightarrow$ polynomial), which no key-length increase can compensate.

Problems of Chapter 2

2.1 In $R_q = \mathbb{Z}_{17}[x]/(x^4 + 1)$, first compute $a \cdot b$ in $\mathbb{Z}[x]$: $(3x^3 + x + 2)(2x^2 + x + 1) = 6x^5 + 3x^4 + 5x^3 + 5x^2 + 3x + 2$. Reduce mod $x^4 + 1$ using $x^4 \equiv -1$, $x^5 \equiv -x$: $c(x) = 5x^3 + 5x^2 + (3 - 6)x + (2 - 3) = 5x^3 + 5x^2 - 3x - 1$. Mod 17: $-3 \equiv 14$, $-1 \equiv 16$. Result: $5x^3 + 5x^2 + 14x + 16$.

2.2 $M = 5 \cdot 7 \cdot 11 = 385$. Partial products: $M_1 = 77$, $M_2 = 55$, $M_3 = 35$. Inverses: $77 \equiv 2 \pmod{5}$, $2^{-1} \equiv 3$; $55 \equiv 6 \pmod{7}$, $6^{-1} \equiv 6$; $35 \equiv 2 \pmod{11}$, $2^{-1} \equiv 6$. $x = 3 \cdot 77 \cdot 3 + 4 \cdot 55 \cdot 6 + 2 \cdot 35 \cdot 6 = 693 + 1320 + 420 = 2433 \equiv 123 \pmod{385}$. Check: $123 \equiv 3 \pmod{5} \checkmark$; $123 \equiv 4 \pmod{7} \checkmark$; $123 \equiv 2 \pmod{11} \checkmark$.

2.4 $\Delta(D_1, D_2) = \frac{1}{2} \sum |D_1(x) - D_2(x)| = \frac{1}{2}(0.1 + 0 + 0 + 0 + 0.1) = 0.1$. By the statistical distance lemma, replacing D_1 by D_2 in one hybrid step changes the adversary's advantage by at most 0.1.

Problems of Chapter 3

3.1 (a) $n = 61 \cdot 53 = 3233$, $\phi(n) = 60 \cdot 52 = 3120$. Choose $e = 17$. Extended Euclidean: $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$, so $d = 2753$. (b) By repeated squaring: $42^{17} \bmod 3233 = 2557$. Decrypt: $2557^{2753} \bmod 3233 = 42$. ✓ (c) With $n = 3233$, trial division factors it instantly. RSA-2048 resists GNFS ($\sim 2^{112}$ ops) but falls to Shor's algorithm in $O(n^3)$.

3.2 (a) $A = 5^6 \bmod 23 = 8$, $B = 5^{15} \bmod 23 = 19$. $K = B^A = 19^8 \bmod 23$: $19^2 \equiv 16$, $19^4 \equiv 3$, $19^6 \equiv 48 \equiv 2$. Check: $A^B = 8^{19} \bmod 23 = 2$. ✓ (b) Eve tries $a \in \{1, \dots, 21\}$ until $5^a \equiv 8$, finding $a = 6$ in ≤ 21 steps. For 2048-bit primes the search space is $\sim 2^{2048}$; even NFS-DL requires $\sim 2^{112}$ operations.

Problems of Chapter 4

4.1 $f(x) = 11^x \bmod 15$: values cycle as 1, 11, 1, 11, ... with period $r = 2$. Check: r is even ✓; $11^{r/2} = 11 \not\equiv -1 \pmod{15}$ ✓. $\gcd(11^1 - 1, 15) = \gcd(10, 15) = 5$, $\gcd(11^1 + 1, 15) = \gcd(12, 15) = 3$. Factors: $15 = 3 \times 5$.

4.4 (a) BBBV proves $\Omega(\sqrt{N})$ queries are necessary for unstructured search. For AES-256, $N = 2^{256}$, so $\geq 2^{128}$ quantum oracle calls are required—128-bit security holds. (b) BBBV applies to *black-box* oracles. Shor's algorithm exploits the algebraic structure of modular exponentiation (periodicity) via the QFT. The oracle is not a black box; its internal structure is essential, so the BBBV bound does not apply.

Problems of Chapter 5

5.2 (a) $K_{\text{final}} = \text{HKDF}(K_C \| K_P)$. If CDH holds but MODULE-LWE is broken, the adversary learns K_P but K_C is random, so K_{final} is pseudorandom via the extractor property of HKDF. The symmetric argument applies if MODULE-LWE holds but CDH is broken. Security requires only *one* assumption.

(b) X25519 alone: $32 + 32 = 64$ B. ML-KEM-768 alone: $1,184 + 1,088 = 2,272$ B. Hybrid: $(32 + 1,184) + (32 + 1,088) = 2,336$ B total; overhead vs. X25519 is 2,272 B.

(c) During a cryptographic transition, confidence is asymmetric: ECC has 40 years of cryptanalysis; lattice assumptions have far less. The hybrid protects against a surprise classical break of MODULE-LWE (X25519 still holds) *and* against an early CRQC (ML-

KEM still holds). The ~ 2 KB overhead is negligible against the risk of betting on a single assumption family.

Problems of Chapter 6

6.1 (a) $\mathbf{B} = \begin{pmatrix} 1 & 1/2 \\ 0 & \sqrt{3}/2 \end{pmatrix}$, $\det(\mathcal{L}) = \sqrt{3}/2$. This is the hexagonal (triangular) lattice—the densest packing in $\mathbb{R}^2$. (b) All six nearest neighbours ($\pm \mathbf{b}_1, \pm \mathbf{b}_2, \pm(\mathbf{b}_1 - \mathbf{b}_2)$) have norm 1, so $\lambda_1 = \lambda_2 = 1$. (c) Alternative basis: $\mathbf{b}'_1 = (1, 0)$, $\mathbf{b}'_2 = \mathbf{b}_2 - \mathbf{b}_1 = (-1/2, \sqrt{3}/2)$; determinant = $\sqrt{3}/2$, unchanged. ✓ (d) $\mathbf{B}^* = (\mathbf{B}^{-1})^T$: dual basis $\mathbf{d}_1 = (1, -1/\sqrt{3})$, $\mathbf{d}_2 = (0, 2/\sqrt{3})$.

6.5 Basis $\mathbf{B} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ (rows), $\det(\mathbf{B}) = 6$.

(a) $\mathbf{B}^{-1} = \frac{1}{6} \begin{pmatrix} 3 & -1 \\ 0 & 2 \end{pmatrix}$. Dual basis (columns of $\mathbf{B}^{-1}$): $\mathbf{d}_1 = (1/2, 0)$, $\mathbf{d}_2 = (-1/6, 1/3)$. Verify: $\langle \mathbf{b}_i, \mathbf{d}_j \rangle = \delta_{ij}$ ✓. (b) $\det(\mathcal{L}) \cdot \det(\mathcal{L}^*) = 6 \cdot \frac{1}{6} = 1$. ✓ (c) $\lambda_1(\mathcal{L}) = \|(2, 1)\| = \sqrt{5}$. $\lambda_1(\mathcal{L}^*) = \|(-1/6, 1/3)\| = \sqrt{5}/6$. Transference: $\sqrt{5} \cdot \sqrt{5}/6 = 5/6 < 2 = n$. ✓

Problems of Chapter 7

7.2 Public key: $(\mathbf{a}_1, b_1) = ((7, 11), 2)$, $(\mathbf{a}_2, b_2) = ((13, 4), 15)$, $(\mathbf{a}_3, b_3) = ((1, 9), 14)$. Parameters: $n = 2$, $q = 17$, $\mathbf{s} = (3, 5)^T$, subset $S = \{1, 3\}$, encrypt $\mu = 1$.

$\mathbf{u} = \mathbf{a}_1 + \mathbf{a}_3 = (8, 20) \equiv (8, 3) \pmod{17}$. $v = b_1 + b_3 + \lfloor q/2 \rfloor \cdot \mu = 2 + 14 + 8 = 24 \equiv 7 \pmod{17}$. Ciphertext: $((8, 3), 7)$.

Decrypt: $v - \langle \mathbf{u}, \mathbf{s} \rangle = 7 - (24 + 15) = 7 - 39 \equiv 7 - 5 = 2 \pmod{17}$. Since $\lfloor q/2 \rfloor = 8$ and $|2| < q/4 \approx 4.25$, the decoder outputs $\mu = 0$. This is a decryption error: the accumulated noise $e_S = -6$ exceeds the margin. Such errors are expected in toy examples with $q = 17$; the exercise demonstrates the encryption/decryption mechanics.

7.5 (a) A CPA-secure scheme may be malleable: given ciphertext c encrypting m , an adversary can produce c' decrypting to a related m' . In a KEM (active adversary), this lets the adversary manipulate the shared secret.

(b) The FO transform re-encrypts after decryption: compute $c' = \text{Enc}(pk, m'; G(m'))$ and check $c' = c$. Failure means the ciphertext was tampered with. This converts CPA to CCA security.

(c) Explicit rejection (returning $\perp$) is a distinguishable event enabling Bleichenbacher-style attacks. Implicit rejection returns $K' = H(\text{randomness} \| c)$, a pseudorandom key indistinguishable from a valid one, closing the side channel.

Problems of Chapter 8

8.1 In the naïve scheme, each signature $\mathbf{z} = \mathbf{s}c + \mathbf{y}$ has its distribution centred at $\mathbf{s}c$. After many signatures, $\mathbb{E}[\mathbf{z} | c] = \mathbf{s}c$ (since $\mathbb{E}[\mathbf{y}] = 0$), so linear regression recovers $\mathbf{s}$.

Rejection sampling fixes this: accept $\mathbf{z}$ with probability $\propto \rho_{\sigma}(\mathbf{z})/\rho_{\sigma}(\mathbf{z}-\mathbf{s}c)$, ensuring the output distribution is D_{σ} regardless of $\mathbf{s}$ and c . No information about $\mathbf{s}$ leaks, regardless of the number of signatures.

8.3 (a) With commitment space size N , a forger enumerates all N commitments, computes the challenge $c = H(\text{com})$ for each, and checks if it can produce a valid response. If N is polynomial this is efficient—the scheme is broken. Security requires N super-polynomial.

(b) The classical forking lemma rewinds the adversary to the same commitment with a different challenge. In the QROM, the adversary queries H in superposition; measuring which input was queried collapses the state, and rewinding is forbidden by no-cloning. Alternative techniques (e.g., lossy-mode frameworks) are needed, typically with a looser security bound.

Problems of Chapter 9

9.1 (a) Systematic $[7, 4, 3]$ Hamming code: $G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$, $H = \begin{pmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}$.
 (b) $\mathbf{c} = (1, 0, 1, 1)G = (1, 0, 1, 1, 1, 0, 0)$. (c) $\mathbf{y} = (1, 0, 0, 1, 1, 0, 0)$. Syndrome $H\mathbf{y}^T = (1, 0, 1)^T$, matching column 3 of H . Flip bit 3: recover $\mathbf{c}$. ✓ (d) Minimum distance $d = 3$; corrects $t = \lfloor (d-1)/2 \rfloor = 1$ error.

9.2 (a) $n = 4096$, $m = 12$, $t = 64$: $k \geq n - mt = 3328$, $d \geq 2t + 1 = 129$. Parameters: $[4096, 3328, 129]$. (b) Public key (systematic form): $(n-k) \times k$ bits = $768 \times 3328/8 = 319,488$ B ≈ 312 KB. (c) ML-KEM-768: 1,184 B; RSA-2048: 256 B. McEliece is $\sim 260\times$ larger than ML-KEM. (d) $1,000$ sessions/s $\times 312$ KB ≈ 312 MB/s = 2.5 Gbps, far exceeding the 100 Mbps link. Impractical without key caching.

Problems of Chapter 10

10.3 $n = 256$ bits, so each hash value is 32 bytes. $\ell_1 = \lceil 256/\log_2 w \rceil$, $\ell_2 = \lfloor \log_2(\ell_1(w-1)) \rfloor / \log_2 w + 1$:

	$w = 4$	$w = 16$	$w = 256$
ℓ_1	128	64	32
ℓ_2	5	3	2
ℓ	133	67	34
Sig. (bytes)	4,256	2,144	1,088
Avg. hashes	~200	~503	~4,335

(b) As w increases, signatures shrink but hash cost grows—the fundamental WOTS tradeoff. $w = 16$ is the standard compromise. (c) Lamport at 256-bit security: $512 \times 32 = 16,384$ B. WOTS+ ($w = 16$): 2,144 B—a $7.6\times$ reduction.

10.4 FORS: $k = 30$ trees, $t = 2^a = 256$ leaves each. A forgery requires all k target indices to match previously revealed leaves.

(a) After $q = 1$: per-tree match probability = $1/256$; forgery probability = $(1/256)^{30} = 2^{-240}$. (b) After $q = 100$: ≤ 100 leaves per tree revealed; $(100/256)^{30} \approx 2^{-41}$. Far above 2^{-128} —security is lost. (c) Need $(q/t)^k < 2^{-128}$: $30 \log_2(q/256) < -128$, giving $q < 256 \cdot 2^{-128/30} \approx 13$. At most ~ 13 signatures maintain 2^{-128} security.

Problems of Chapter 11

11.1 (a) Brute-force over $\mathbb{F}_2^3$:

x_1	x_2	x_3	f_1	f_2	Sol?
0	0	0	0	0	✓
0	0	1	0	1	
0	1	0	0	1	
0	1	1	1	0	
1	0	0	1	0	
1	0	1	1	1	
1	1	0	1	1	
1	1	1	0	0	✓

Solutions: $(0, 0, 0)$ and $(1, 1, 1)$.

(b) 2^n candidates; exceeds 2^{128} when $n > 128$.

(c) NP-completeness is worst-case. Specific multivariate cryptosystems use structured instances (with trapdoors), and structure-specific attacks may apply. MI, SFLASH, and Rainbow were all broken via such attacks despite generic MQ remaining hard.

11.3 (a) Alice sends E_A plus the images $\phi_A(P_B)$, $\phi_A(Q_B)$ of Bob's torsion basis. Without these, Bob cannot determine the kernel of his secret isogeny on E_A .

(b) Castryck–Decru uses the torsion images to lift the problem to a higher-dimensional abelian variety (Kani–Richelot framework, dimension 4 or 8), where the isogeny can be computed in polynomial time.

(c) CSIDH uses commutativity of the ideal class group action on supersingular curves over $\mathbb{F}_p$: $[a][b]E_0 = [b][a]E_0$. No auxiliary torsion points are needed.

(d) No: CSIDH faces Kuperberg's subexponential quantum attack on the hidden shift problem, classical meet-in-the-middle attacks, and performance orders of magnitude slower than lattice alternatives.

Problems of Chapter 12

12.2 Current: X25519 (32 + 32 = 64 B key exchange) + Ed25519 (32 B pk + 64 B sig).

ML-KEM-768 + ML-DSA-65: KE = 1,184 + 1,088 = 2,272 B; auth = 1,952 + 3,309 = 5,261 B. Additional bytes: (2,272 - 64) + (5,261 - 96) = 2,208 + 5,165 = 7,373 B. At 1,500 B MTU (~1,460 payload), this spans ~6 extra TCP segments. The handshake no longer fits in one round trip but remains manageable with pipelining.

With SLH-DSA-128s (sig: 7,856 B; pk: 32 B): a depth-2 cert chain carries ~15.6 KB in signatures alone, exceeding the TCP initial congestion window (~14.6 KB) and forcing an extra RTT. Not viable for latency-sensitive handshakes without certificate compression.

12.3 The butterfly $a' = a + \omega b$, $b' = a - \omega b$ is linear. With shares $a = a_0 + a_1$, $b = b_0 + b_1$: compute $a'_i = a_i + \omega b_i$, $b'_i = a_i - \omega b_i$ independently. Then $a'_0 + a'_1 = (a_0 + a_1) + \omega(b_0 + b_1) = a + \omega b = a'$. ✓

For multiplication $c = ab$: $(a_0 + a_1)(b_0 + b_1) = a_0b_0 + a_0b_1 + a_1b_0 + a_1b_1$. Computing shares independently gives $a_0b_0 + a_1b_1 \neq ab$; the cross-terms $a_0b_1 + a_1b_0$ require share interaction (e.g., ISW multiplication gadget), leaking information without a re-masking protocol.

Problems of Chapter 14

14.1 Mosca: begin migration by year $z - x - y$ (CRQC year minus shelf life minus migration time).

(a) Hospital: $2035 - 50 - 5 = 1980$. Decades past—records encrypted today remain sensitive until 2076, so HNDL exposure is immediate. (b) Retail: $2035 - 2 - 1.5 = 2031.5$. Migration can wait until mid-2031. (c) The hospital's urgency is far greater: the 50-year shelf life means today's data is already vulnerable to harvest-now-decrypt-later. The retail company's 2-year shelf life limits exposure to recent transactions.

14.3 Depth-3 chain (3 certs); each cert = pk + sig + 200 B overhead.

(a) RSA-2048: $(294 + 256 + 200) \times 3 = 2,250$ B. (b) ML-DSA-65: $(1,952 + 3,309 + 200) \times 3 = 16,383$ B. (c) Hybrid catalyst: $(294 + 256 + 1,952 + 3,309 + 200) \times 3 = 18,033$ B.

TCP initial congestion window $\approx 14,600$ B. Config (a) fits. Configs (b) and (c) exceed it by 1.8 KB and 3.4 KB respectively, forcing one extra RTT. Mitigations: TLS certificate compression (RFC 8879), caching intermediates at clients, or chain truncation.

Problems of Chapter 15

15.2 Stern soundness error: $2/3$ per round.

(a) Need $(2/3)^r < 2^{-128}$: $r > 128/\log_2(3/2) = 128/0.585 \approx 219$ rounds. (b) Per round: 3 commitments $\times$ 32 B = 96 B, plus challenge and responses (~ 100 B). Total $\approx 219 \times 196 \approx 43$ KB. (c) QROM loss: $(2/3)^r \cdot 2^{32} < 2^{-128}$ requires $(2/3)^r < 2^{-160}$, so $r > 274$. Need 55 additional rounds.

15.3 (3, 5)-sharing over $\mathbb{Z}_{17}$, $s = 11$. Choose $a_1 = 3$, $a_2 = 7$; polynomial $p(x) = 11 + 3x + 7x^2$.

(a) Shares: $p(1) = 21 \equiv 4$, $p(2) = 45 \equiv 11$, $p(3) = 83 \equiv 15$, $p(4) = 135 \equiv 16$, $p(5) = 201 \equiv 14$.

(b) Reconstruct from (1, 4), (2, 11), (3, 15): Lagrange basis at $x = 0$: $L_1 = \frac{(-2)(-3)}{(-1)(-2)} = 3$, $L_2 = \frac{(-1)(-3)}{(1)(-1)} = -3 \equiv 14$, $L_3 = \frac{(-1)(-2)}{(2)(1)} = 1$. $s = 4 \cdot 3 + 11 \cdot 14 + 15 \cdot 1 = 12 + 154 + 15 = 181 \equiv 11 \pmod{17}$. ✓

(c) Two shares fix two of three coefficients. The free coefficient $a_2 \in \mathbb{Z}_{17}$ yields 17 distinct polynomials, each giving a different $p(0)$. Every secret is equally consistent: perfect secrecy.

(d) Addition: locally compute $p(i) + p'(i)$; the sum polynomial has the same degree, so no interaction is needed. Multiplication: $p(i) \cdot p'(i)$ gives shares of a degree-4 polynomial, doubling the threshold from 3 to 5. Degree reduction requires one round of communication.

Problems of Chapter 16

16.3 (a) Typical ClientHello ≈ 300 B. Hybrid key share: $32 + 1,184 = 1,216$ B. Total $\approx 1,516$ B, exceeding the $\sim 1,460$ B TCP payload by ~ 56 B. Split across two segments, but both transmit in the same flight—negligible added latency (< 1 ms).

(b) ML-KEM-1024: $32 + 1,568 = 1,600$ B key share. ClientHello $\approx 1,900$ B, spanning 2 segments. Qualitatively similar, though some middleboxes reject multi-segment ClientHello messages.

(c) Mitigations: (i) **HelloRetryRequest**—send ClientHello without the PQC share; server requests it in a second round (one extra RTT, guaranteed compatibility). (ii) **QUIC**—uses UDP with its own framing, avoiding TCP segmentation. (iii) **TCP Fast Open**—offsets latency by including data in the SYN.

References

1. Ajtai, M.: Generating hard instances of lattice problems. In: Proceedings of the 28th Annual ACM Symposium on Theory of Computing, pp. 99–108 (1996). DOI 10.1145/237814.237838
2. Bagheri, N., Ghaedi, N., Sanadhya, S.K.: Differential fault analysis of SHA-3. In: International Conference on Cryptology in India — INDOCRYPT 2015, *LNCS*, vol. 9462, pp. 253–269. Springer (2015). DOI 10.1007/978-3-319-26617-6_14
3. Bennett, C.H., Bernstein, E., Brassard, G., Vazirani, U.: Strengths and weaknesses of quantum computing. *SIAM Journal on Computing* **26**(5), 1510–1523 (1997). DOI 10.1137/S0097539796300933
4. Bernstein, D.J., Hopwood, D., Hülsing, A., Lange, T., Niederhagen, R., Papachristodoulou, L., Schneider, M., Schwabe, P., Wilcox-O’Hearn, Z.: SPHINCS: Practical stateless hash-based signatures. In: Advances in Cryptology — EUROCRYPT 2015, *LNCS*, vol. 9056, pp. 368–397. Springer (2015). DOI 10.1007/978-3-662-46800-5_15
5. Bernstein, D.J., Hopwood, D., Hülsing, A., Lange, T., Niederhagen, R., Papachristodoulou, L., Schneider, M., Schwabe, P., Wilcox-O’Hearn, Z.: SPHINCS: Practical stateless hash-based signatures. In: Advances in Cryptology — EUROCRYPT 2015, *LNCS*, vol. 9056, pp. 368–397. Springer (2015). DOI 10.1007/978-3-662-46800-5_15
6. Bernstein, D.J., Lange, T.: Post-quantum cryptography. *Nature* **549**, 188–194 (2017). DOI 10.1038/nature23461
7. Beullens, W.: Breaking Rainbow takes a weekend on a laptop. In: Advances in Cryptology — CRYPTO 2022, *LNCS*, vol. 13508, pp. 464–479. Springer (2022). DOI 10.1007/978-3-031-15979-4_16
8. Bindel, N., Brendel, J., Fischlin, M., Goncalves, B., Stebila, D.: Hybrid key encapsulation mechanisms and authenticated key exchange. In: Post-Quantum Cryptography — PQCrypto 2019, *LNCS*, vol. 11505, pp. 206–226. Springer (2019). DOI 10.1007/978-3-030-25510-7_12
9. Brier, E., Clavier, C., Olivier, F.: Correlation power analysis with a leakage model. In: Cryptographic Hardware and Embedded Systems — CHES 2004, *LNCS*, vol. 3156, pp. 16–29. Springer (2004). DOI 10.1007/978-3-540-28632-5_2
10. Buchmann, J., Dahmen, E., Szydło, M.: Hash-based digital signature schemes. In: D.J. Bernstein, J. Buchmann, E. Dahmen (eds.) Post-Quantum Cryptography, pp. 35–93. Springer (2009). DOI 10.1007/978-3-540-88702-7_3
11. Bundesamt für Sicherheit in der Informationstechnik: TR-03158: Evaluation of side-channel analysis resistance. Tech. rep., BSI (2023)
12. Castryck, W., Decru, T.: An efficient key recovery attack on SIDH. In: Advances in Cryptology — EUROCRYPT 2023, *LNCS*, vol. 14008, pp. 423–447. Springer (2023). DOI 10.1007/978-3-031-30589-4_15. Preliminary version on ePrint 2022/975
13. Castryck, W., Lange, T., Martindale, C., Panny, L., Renes, J.: CSIDH: An efficient post-quantum commutative group action. In: Advances in Cryptology — ASIACRYPT 2018, *LNCS*, vol. 11274, pp. 395–427. Springer (2018)
14. Common Criteria Recognition Arrangement: Common criteria for information technology security evaluation, version 3.1, revision 5. Tech. rep., CCRA (2022). <https://www.commoncriteriaportal.org>
15. Diffie, W., Hellman, M.E.: New directions in cryptography. *IEEE Transactions on Information Theory* **22**(6), 644–654 (1976). DOI 10.1109/TIT.1976.1055638
16. Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS – Dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2018**(1), 238–268 (2018). DOI 10.13154/tches.v2018.i1.238-268
17. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Advances in Cryptology — CRYPTO ’86, *LNCS*, vol. 263, pp. 186–194. Springer (1987)
18. Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: Falcon: Fast-fourier lattice-based compact signatures over NTRU. Tech. rep., NIST PQC Submission (2020). Specification v1.2

19. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: *Advances in Cryptology — CRYPTO '99, LNCS*, vol. 1666, pp. 537–554. Springer (1999)
20. Gentry, C.: A fully homomorphic encryption scheme. Ph.D. thesis, Stanford University (2009)
21. Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. In: *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, pp. 197–206 (2008). DOI 10.1145/1374376.1374407
22. Gidney, C., Ekerå, M.: How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum* **5**, 433 (2021). DOI 10.22331/q-2021-04-15-433
23. Goubin, L.: A sound method for switching between Boolean and arithmetic masking. In: *Cryptographic Hardware and Embedded Systems — CHES 2001, LNCS*, vol. 2162, pp. 3–15. Springer (2001). DOI 10.1007/3-540-44709-1_2
24. Gross, H., Mangard, S., Korak, T.: Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. In: *ACM Workshop on Theory of Implementation Security — TIS 2016*, pp. 3–14. ACM (2016). DOI 10.1145/2996366.2996426
25. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pp. 212–219 (1996). DOI 10.1145/237814.237866
26. Hoffstein, J., Pipher, J., Silverman, J.H.: NTRU: A ring-based public key cryptosystem. In: *Algorithmic Number Theory — ANTS-III, LNCS*, vol. 1423, pp. 267–288. Springer (1998)
27. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transform. In: *Theory of Cryptography Conference — TCC 2017, LNCS*, vol. 10677, pp. 341–371. Springer (2017). DOI 10.1007/978-3-319-70500-2_12
28. Huelsing, A., Butin, D., Gazdag, S.L., Rijneveld, J., Mohaisen, A.: XMSS: eXtended Merkle signature scheme. RFC 8391, Internet Engineering Task Force (2018). DOI 10.17487/RFC8391
29. International Organization for Standardization: Testing methods for the mitigation of non-invasive attack classes against cryptographic modules. International Standard ISO 17825:2024, ISO (2024)
30. Ishai, Y., Sahai, A., Wagner, D.: Private circuits: Securing hardware against probing attacks. In: *Advances in Cryptology — CRYPTO 2003, LNCS*, vol. 2729, pp. 463–481. Springer (2003). DOI 10.1007/978-3-540-45146-4_27
31. Jao, D., De Feo, L.: Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. In: *Post-Quantum Cryptography — PQCrypto 2011, LNCS*, vol. 7071, pp. 19–34. Springer (2011)
32. Kampanakis, P., Dang, Q.: The extended deterministic random bit generator (XDRBG). In: *NIST Lightweight Cryptography Workshop* (2023)
33. Koblitz, N.: Elliptic curve cryptosystems. *Mathematics of Computation* **48**(177), 203–209 (1987)
34. Kocher, P., Jaffe, J., Jun, B.: Differential power analysis. In: *Advances in Cryptology — CRYPTO '99, LNCS*, vol. 1666, pp. 388–397. Springer (1999). DOI 10.1007/3-540-48405-1_25
35. Laarhoven, T.: Search problems in cryptography: From fingerprinting to lattice sieving. Ph.D. thesis, Eindhoven University of Technology (2015). DOI 10.6100/IR795867. ISBN 978-90-386-3991-5
36. Lamport, L.: Constructing digital signatures from a one-way function. Technical Report CSL-98, SRI International (1979)
37. Lenstra, A.K., Lenstra, H.W., Lovász, L.: Factoring polynomials with rational coefficients. *Mathematische Annalen* **261**, 515–534 (1982). DOI 10.1007/BF01457454
38. Lyubashevsky, V.: Lattice signatures without trapdoors. In: *Advances in Cryptology — EUROCRYPT 2012, LNCS*, vol. 7237, pp. 738–755. Springer (2012). DOI 10.1007/978-3-642-29011-4_43
39. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: *Advances in Cryptology — EUROCRYPT 2010, LNCS*, vol. 6110, pp. 1–23. Springer (2010). DOI 10.1007/978-3-642-13190-5_1
40. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. *Journal of the ACM* **60**(6), 43:1–43:35 (2013). DOI 10.1145/2535925
41. Mangard, S., Oswald, E., Popp, T.: *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer (2007). DOI 10.1007/978-0-387-38162-6
42. McEliece, R.J.: A public-key cryptosystem based on algebraic coding theory. *DSN Progress Report* **42–44**, 114–116 (1978)

43. McGrew, D., Curcio, M., Fluhrer, S.: Leighton-Micali signature. RFC 8554, Internet Engineering Task Force (2019). DOI 10.17487/RFC8554
44. Merkle, R.C.: A certified digital signature. In: *Advances in Cryptology — CRYPTO '89, LNCS*, vol. 435, pp. 218–238. Springer (1990)
45. Micciancio, D., Goldwasser, S.: Complexity of Lattice Problems: A Cryptographic Perspective, *The Kluwer International Series in Engineering and Computer Science*, vol. 671. Springer (2002). DOI 10.1007/978-1-4615-0897-7
46. Micciancio, D., Regev, O.: Worst-case to average-case reductions based on Gaussian measures. *SIAM Journal on Computing* **37**(1), 267–302 (2007). DOI 10.1137/S0097539705447360
47. Miller, V.S.: Use of elliptic curves in cryptography. In: *Advances in Cryptology — CRYPTO '85, LNCS*, vol. 218, pp. 417–426. Springer (1986)
48. National Institute of Standards and Technology: Security requirements for cryptographic modules. Federal Information Processing Standard (FIPS) 140-3, NIST (2019). DOI 10.6028/NIST.FIPS.140-3
49. National Institute of Standards and Technology: Module-lattice-based digital signature standard. Federal Information Processing Standards Publication 204, NIST (2024). DOI 10.6028/NIST.FIPS.204
50. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication 203, NIST (2024). DOI 10.6028/NIST.FIPS.203
51. National Institute of Standards and Technology: Stateless hash-based digital signature standard. Federal Information Processing Standards Publication 205, NIST (2024). DOI 10.6028/NIST.FIPS.205
52. Ngo, K., Dubrova, E., Guo, Q., Johansson, T.: A side-channel attack on a masked IND-CCA secure saber KEM implementation. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2021**(4), 676–707 (2021). DOI 10.46586/tches.v2021.i4.676-707
53. Nielsen, M.A., Chuang, I.L.: *Quantum Computation and Quantum Information*, 10th anniversary edn. Cambridge University Press (2010)
54. Peikert, C.: A decade of lattice cryptography. *Foundations and Trends in Theoretical Computer Science* **10**(4), 283–424 (2016). DOI 10.1561/04000000074
55. Ravi, P., Roy, S.S., Chattopadhyay, A., Bhasin, S.: Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, pp. 307–335 (2020). DOI 10.13154/tches.v2020.i3.307-335
56. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM* **56**(6), 34:1–34:40 (2009). DOI 10.1145/1568318.1568324. Preliminary version in STOC 2005
57. Rivest, R.L., Shamir, A., Adleman, L.: A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM* **21**(2), 120–126 (1978). DOI 10.1145/359340.359342
58. Shor, P.W.: Algorithms for quantum computation: Discrete logarithms and factoring. In: *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134 (1994). DOI 10.1109/SFCS.1994.365700
59. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing* **26**(5), 1484–1509 (1997). DOI 10.1137/S0097539795293172

Index

Symbols

50-year horizon 270

A

A2B conversion
 security boundary 207
address register (SLH-DSA) 216
advantage 27
adversary 27
AEAD 35
AES 34
Ajtai's reduction 92
algorithmic diversity 70
amplitude amplification 58
approximation factor 87
attribute-based encryption 254
authentication path 155
average-case hardness 26

B

Barrett reduction 183
basis reduction 88
BBBV lower bound 57
Beullens, Ward 169
BIKE 145
 construction 146
 decoding failure rate 146
BKZ algorithm 90
 block size 90
block cipher 34
bounded distance decoding 135
BPP 25
BQP 25
BRAM
 zeroisation 210

BSI 222

C

cache-timing attack 187
cache-timing attacks 203
Castryck–Decru attack 172
CBD sampling 210
CDH assumption 47
CDH problem 38
CECPQ2 230
centred binomial distribution 101
centred representative 19
certificate authority 42
certificate chain
 PQC overhead 231
Certificate Transparency 43
challenger 27
Chinese Remainder Theorem 20
circular dependency (masking randomness)
 218
Classic McEliece 141
closest vector problem *see* CVP
CMVP 194
CNSA 2.0 238
code distance 59
code signing 232
code-based cryptography
 comparison 147
collision resistance 35
Common Criteria 221
complexity class 25
composite KEM 74
compression (ML-KEM) 211
computational hardness 46
coNP 25
constant-time arbiter 219
constant-time padding 213

continued fractions 54
 core-SVP model 91
 Correlation Power Analysis 201
 CPA 201
 cryptanalysis
 monitoring 269
 cryptanalytic break 9
 crypto-agility 79, 177, 238
 cryptographic agility 266
 definition 267
 cryptographic inventory 235
 cryptographically relevant quantum computer 5
 cryptography
 symmetric 33
 CSIDH 173
 CVP 87
 approximate 87
 cyclotomic polynomial 18
 cyclotomic ring 104
D
 DDH assumption 47
 Debian OpenSSL bug 190
 decoding problem 87
 deep learning
 side-channel attacks 263
 defence in depth 269
 deterministic mode 213
 Differential Fault Analysis 218
 Differential Power Analysis 201
 Diffie–Hellman 3, 37
 key exchange 38
 diffraction analogy 52
 Bragg peaks 54
 DigiNotar 43
 digital signature 39
 quantum threat 7
 discrete Gaussian 101
 discrete Gaussian distribution 23
 tail bound 23
 discrete logarithm 4
 discrete logarithm problem 47
 quantum attack 55
 DOM 206
 Domain-Oriented Masking 206
 DPA 186, 201
 DRBG 190, 198
 chokepoint 198
 dual attack 102
 dual-rail logic 264
E
 ECDHE 43

ECDLP 39, 48
 quantum attack 55
 ECDSA 41
 electromagnetic analysis 203
 elliptic curve 38, 171
 elliptic curve cryptography 3, 38
 entropy source 197
 entropy-to-cryptography pipeline 197
 EUF-CMA 29, 40
 Euler’s theorem 19, 38
 Euler’s totient function 17
 existential unforgeability 36
 extended Euclidean algorithm 19

F

failure-boosting attack 111
 FALCON 9
 Falcon
 tree 125
 fault injection 188, 208
 fault-tolerant quantum computation 59
 Fermat’s little theorem 19
 few-time signature 158
 FFT 105
 FHE *see* fully homomorphic encryption
 Fiat–Shamir heuristic 41
 Fiat–Shamir transform 114
 Fiat–Shamir with aborts 116
 field 18
 finite 18
 finite field 18
 FIPS 140-3 221
 FORS 158
 frozen funds risk 234
 FSM protection 209
 Fujisaki–Okamoto transform 107
 fully homomorphic encryption 243
 functional encryption 255
 fundamental parallelepiped 82

G

GapSVP 88
 garbled circuits 252
 gate complexity 52
 Gaussian heuristic 83
 Gaussian sampling 93
 hardware 265
 GCM 35
 General Number Field Sieve 46
 generator 16
 generator matrix 134
 Gilbert–Varshamov bound 134
 glitches 206
 Goppa code 135

- definition 136
- parameters 136
- Goppa polynomial 136
- GPV framework 123
- Gram–Schmidt orthogonalisation 89
- group 15
 - abelian 16
 - cyclic 16
 - order 16
- group action 174
- Grover’s algorithm 4, 36, 56
 - diffusion operator 56
 - Grover operator 56
 - hash-based signatures 163
 - impact on symmetric ciphers 12
 - impact on symmetric crypto 58
 - oracle 56
 - resonance analogy 57

H

- Hamming distance 134, 200
- Hamming distance model 200
- Hamming weight 134, 200
- hardware
 - PQC optimization 265
- harvest now, decrypt later 5
- hash function 35
- hash-based signatures
 - security assumptions 163
- hedged mode 213
- Hermite factor 85
- hidden subgroup problem 4, 48, 61
- HighBits 213
- HKDF 36, 44
- HMAC 36
- HQC 142
 - construction 143
- hybrid cryptography 227
- hybrid method 57
- hypertree 160

I

- ideal lattice 105
- identification scheme 115
- identity-based encryption 253
- implicit rejection 107
- IND-CCA2 28, 107
 - composite KEM 228
- IND-CPA 28, 103
- information redundancy 209
- information set decoding 138
 - quantum 139
 - variants 139

- integer factorisation 4
- integer factorisation problem 46
- IoT
 - PQC energy 266
- isogeny 171
- isogeny graph 171
- ISW scheme 205

K

- Karp reduction 25
- KAT 193
- Keccak
 - shared core contention 219
- KEM 107
 - composite 227
- key distribution problem 36

L

- Lagrange’s theorem 16
- Lamport signature 151
- lattice 81
 - basis 81
 - basis quality 84
 - computational problems 86
 - definition 81
 - determinant 82
 - dimension hardness 82
 - dual 83
- lattice problems
 - quantum complexity 61
- leftover hash lemma 22
- linear code 133
 - block length 133
 - dimension 133
 - rate 133
- LLL algorithm 89
 - guarantees 89
 - reduced basis 89
- LMS 157, 232
- LowBits 213
- LWE
 - decision 99
 - error distribution 99
 - search 99
 - search-decision equivalence 100

M

- MAC 36
- magic state distillation 59
- masking 187, 204
 - A2B conversion 207
 - arithmetic 187, 207

- B2A conversion 207
- Boolean 187, 205
- mass surveillance 6
- McEliece cryptosystem 140
- MD5
 - deprecation 268
- measurement 52
- Merkle signature scheme 154
- Merkle tree 154
- millionaires' problem 251
- minimum distance 134
- Minkowski's theorem 83
- MinRank problem 169
- ML-DSA 9, 198
 - hardware security 212
- ML-KEM 9, 108, 198
 - compression 110
 - decryption failure 111
 - hardware security 210
 - security estimate 95
- mode of operation 35
- modular exponentiation
 - quantum 54
- modular reduction 19
- Module-LWE 106
 - break scenario 268
- Mosca's inequality 239
- MQ problem 167
- multiparty computation 251

N

- negacyclic convolution 17
- negacyclic matrix 104
- negligible function 22
- NIST PQC standardisation 9
- NP (complexity class) 25
- NP-complete 26
- NP-complete problems
 - quantum complexity 60
- NP-hard 26
- NTRU
 - lattice 122
- NTT *see* Number Theoretic Transform, 104, 105
 - hardware 265
 - masking cost 207
 - side channel 186
 - side-channel leakage 263
- Number Theoretic Transform 21

O

- OAEP 38
- oblivious transfer 252

- Oil-Vinegar 168
- one-time signature 151
- one-way trapdoor function 37
- open problems 272
- order
 - multiplicative 53
- orthogonality defect 85

P

- P (complexity class) 25
- parity-check matrix 134
- Patterson's algorithm 136
- period finding 53
- PKI 42
- Pollard's rho 48
- polynomial ring 17
 - quotient 17
- post-quantum cryptography
 - design criterion 61
 - history 7
- power trace 200
- Prange's algorithm 138
- preimage resistance 35
- preimage sampling 123
- primal attack 102
- public-key cryptography 3, 37

Q

- QC-MDPC code 145
 - definition 145
- QRNG 191
- QROM 30, 249
- quantum algorithms
 - toolkit 260
- quantum circuit model 52
- quantum computing
 - limitations 60
 - model 51
- quantum cryptanalysis 260
- quantum Fourier transform 52
 - circuit depth 52
- quantum gate 51
- quantum hardware
 - current state 60
- quantum internet 270
- quantum random oracle model 30, 115, 164
- quantum resource estimates 4, 59
- quantum walks 261
- quasi-cyclic code 143
 - definition 143
- quasi-cyclic syndrome decoding 144
- qubit 51
- quotient ring 17

R

Ramanujan graph 172
 random oracle 249
 random oracle model 30, 115
 reduction
 Karp 25
 Turing 25
 Regev's encryption 103
 Regev's reduction 101
 rejection sampling
 timing channel 212
 rejection sampling lemma 117
 ring 16
 commutative 17
 Ring-LWE 104
 definition 105
 hardness 105
 security 106
 ROM *see* random oracle model
 root Hermite factor 91
 root of unity 20
 RSA 3, 37
 RSA problem 46
 RSA-PSS 40

S

Schnorr signature 40
 secret sharing 251
 security
 EUF-CMA 29
 full stack 271
 IND-CCA2 28
 IND-CPA 28
 security game 27
 security loss 29
 security reduction 29
 loss 29
 tightness 30
 SHA-1
 deprecation 268
 SHA-2 35
 SHA-3 35
 Shor's algorithm 4, 26, 48, 52
 circuit 54
 ECDSA 233
 example 55
 factoring reduction 53
 original paper 7
 qubit count 54
 shortest vector problem *see* SVP
 shuffling 208
 WOTS+ chains 217
 side channel 199

side-channel analysis
 PQC 262
 quantum 264
 SIDH 172
 sieving
 quantum 261
 sigma protocol 249
 signature
 composite 228
 Simple Power Analysis 201
 SIS 92
 hardness 92
 module variant 118
 SIVP 88
 SLH-DSA 9, 160, 198
 hardware security 215
 parameters 161
 s vs f tradeoff 162
 smoothing parameter 24, 93
 SNARK 250
 Solovay–Kitaev theorem 52
 SPA 186, 201
 SPHINCS+ 160
 sponge construction 35
 STARK 250
 state management 158
 stateless signing 161
 statistical distance 21
 Stern protocol 249
 successive minima 83
 supersingular curve 171
 support 21
 surface code 59
 SVP 86
 approximate 87
 quantum complexity 262
 syndrome 134
 syndrome decoding problem 138
 NP-completeness 138
 systematic form 134

T

temporal redundancy 208
 threshold signature 254
 timing side channel 183
 TLS 3, 43
 1-RTT handshake 43
 agility 267
 TLS 1.3
 named groups 230
 transference theorem 83
 tweakable hash function 164

U

unimodular matrix 81
universal hash family 22
unstructured search 56
UOV 170

V

Vélu's formulae 171
vectorisation problem 174

W

Watrous rewinding lemma 30
Winternitz OTS 153
Winternitz parameter
tradeoff 153

worst-case to average-case reduction 92

WOTS+ 153
chain computation 216

X

X.509 42
hybrid certificates 229
X25519Kyber768 230
XMSS 157, 232
multi-tree 157

Z

zero-knowledge proof 247
zeroisation 209
 $\mathbb{Z}_q$ 17